Lecture notes FYS3150 — Computational Physics, Fall 2026

Introduction

Welcome to this course!

- About me:
  - Anders Kvellestad
  - Researcher in the Section for Theoretical Physics
  - Background: Bergen → Oslo → Stockholm → Oslo → London → Oslo
  - Work on exploring new theories in particle physics
  - Keywords: LHC, supersymmetry, dark matter, Higgs, statistics, coding (Python, C++, …), supercomputers, causing and fixing bugs, responsible for coffee supplies in the theory group
- The teaching team this semester:
  - Carl Martin Fevang
  - Alv Skarpeid
  - Fredrik Nyseter Kortgaard
  - Elise Harila Christensen
  - Henrik Fossum Moen
- **Question:** Who are you?
  - Study programmes?
  - Level of coding experience?
  - Main motivation for this course?
    * Solve those pesky equations
    * Learn C++ and other tools
    * I just like working with computers
    * …?
- **Question:** What operating system are you using?
  - Linux?
  - macOS?
  - Windows?
- I will need at least two student representatives for the course evaluation

  - You'll join a meeting (~1 hour) with us teachers at the end of the semester, where we discuss what worked and what we can improve in the course
  - If you are willing to do this, just send me an email

About the course

- Course resources
  - Official UiO course page
  - Our own page, with course material
  - Our Git repo
  - Our discussion forum
  - Canvas, for handing in project reports
- Teaching language: English
- Programming languages:
  - Main focus on C++
  - Python for data analysis, making plots, etc.
  - Bash for terminal examples, short scripts, etc.
- This course has been taught by CompPhys guru Morten Hjorth-Jensen for many years
- I took over this course in 2021
- Based on Morten's old course, but with a number of changes from my side
- Main curriculum:
  - What we discuss in lectures (my lecture notes), plus what we work on in the projects
- Course background material:
  - Morten's lecture notes / book draft, available via the UiO course page
  - I might point you to relevant parts of Morten's notes
- Course philosophy:
  - Pragmatic, learning by doing (and learning by failing)
  - Will try to focus on concrete examples
  - Computational physics is a *huge* and active field — this course is just a first introduction
- Lectures:

  - Thursdays and Fridays, 10.15 – 12.00
  - I try to lecture in ~30 minute sessions, with two short breaks

- Group sessions:
  - Schedule
    * Wednesday, 10.15–12.00 (room Ø397)
    * Thursday, 08.15–10.00 (room Ø397)
    * Thursday, 12.15–14.00 (room Ø434)
    * Friday, 12.15–14.00 (room Ø434)
  - Probably the most important arena for learning in this course!
    * Great opportunity to meet and chat with each other and the teachers about anything related to the course
  - You can come to any group session you want, and as many as you want
    * Be patient with our group teachers — some have taught this course for several years, others are doing it for the first time
  - We start the group sessions already the first week

- Formal requirements
  - Two **problem sets**: Must be passed
  - Two **projects**: Scored from 0–100
  - A final written exam
  - Final grade based on weighted score:
    * Project weights: 20%, 40%
    * Exam weight: 40%

- For simplicity, we'll just refer to both the problem sets and projects as "projects", i.e. we'll talk about projects 1–4, and the grade is based on projects 3 and 4, plus the exam.
- The projects are closely connected with the learning outcomes listed on the course page
- *Tentative* deadlines:
  - Project 1: September 9
  - Project 2: September 23
  - Project 3: October 21
  - Project 4: November 18

- Exam date: December 7
- Policy on deadlines: **friendly, but strict**

  - Need to be strict on deadlines, both to keep things fair and to keep up with our time schedule
  - There are **no second attempts**
  - Substantial deadline extensions due to illness require a doctor's note

- **Collaboration is encouraged!**
  - We *strongly* encourage you to collaborate in small groups of **2–3** people.
  - A group hands in a *joint* project report and code
  - By working together *you will learn more*, and we get more time for grading per project report → *more detailed feedback from us*

- Asking questions:
  - **Please ask questions!**
  - Any time during lectures — just cut in and ask!
  - For help with your specific project/code:
    * Primary forum: group sessions
    * Secondary forum: our online discussion forum
  - *Think* and *try yourself* before you ask for help
  - When writing questions:
    * Keep it short and concise
    * Have respect for other people's time (your fellow students, the teachers, …)
  - *Almost* all questions are welcome!
    * The only type of questions I don't like are the "questions" that aren't really questions at all, but just someone trying to show off how much they know.
  - Any personal or procedural issues: Send me an email. (We can also set up a meeting.)

- The broad topics of this course:
  - Learn basic C++, with focus on numerics
  - Matrix operations, eigenvalue problems
  - Solve ordinary and partial differential equations
  - Numerical integration
  - Monte Carlo methods, simulation of stochastic systems
  - Proper presentation of results
  - Debugging :)

- This course is good for your CV (beyond just the grade you get)
  - We're at a university, so hopefully our main motivation for following/teaching a course should be that learning new stuff is interesting and valuable in itself — that's at least my main motivation when teaching this!

  - Having said that, after completing this course you can probably also add some new points to your CV:
    * Experience with C++
    * Experience with the Unix terminal
    * Experience with git and GitHub
    * Experience with writing technical reports in LaTeX
    * …

- In order to learn what you should in this course we expect you to:
  - attend the lectures
  - attend at least one group session per week
  - read/work through the content on the course web page
  - put significant effort into the projects, both coding and writing

The most useful advice you'll get all year

- Something you don't understand?
  - *Read* and *think*
  - *Discuss* with your fellow students
  - *Ask* us
- Code isn't working?
  - Don't just try stuff at random!
    * This rarely works, and when it does you typically still can't trust the results…
  - *Read the documentation* for the command/tool you are using
  - *Search online for the error message*, after removing things that are specific to your code (variable names, file names, etc.)
    * *Read* the explanations you find, don't just copy code
  - Try to isolate and reproduce the problem in a small, separate example code. (A *minimal working example*.)
  - Read the course pages on debugging
  - We'll also discuss debugging in the lectures
- How you present your results **really matters**.
  - Quality of language
  - Quality of figures

  - Layout
  - Report structure
  - Referencing
  - Code comments and documentation

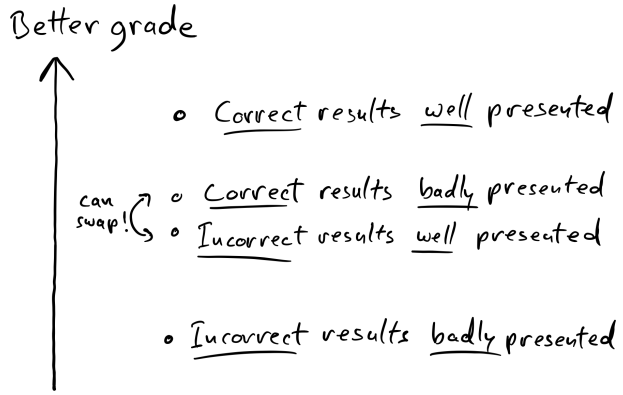


Figure 1: Presentation quality matters

- Spend time with pen and paper before you start coding
  - Make a rough sketch of program parts and flow
  - Discuss this sketch with your collaborators and/or the group teachers
  - Write an outline of your program with code comments first, then start filling in the code
  - Make a sketch of discretisations, to avoid mistakes with indices

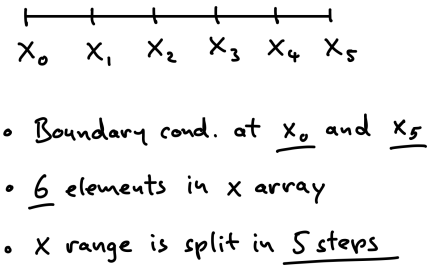


Figure 2: Sketch discretisations

- Make sure you understand the quantities you present in plots and tables
  - Makes it much easier to spot mistakes
  - Pay attention to units!
  - *Tip*: Always set axis ranges manually
- Read the report template we provide, plus the example student reports
- And read the *Checklist for reports* page on our webpage, to avoid many common mistakes

Plagiarism

- Plagiarism is **very serious**
- Have seen a few cases in the past
- Can have very serious consequences, e.g. losing the right to study at UiO
- You must:
  - Write your own text — never copy text from others (unless it is marked as a direct quote)
  - Write your own code, unless it's code we have provided to help
  - Always acknowledge contributions from others
  - Properly cite articles, books, webpages, …
    * We'll discuss this more in detail when you start writing project reports

Use of AI tools

- Generative AI tools like large language models (LLMs) can, like any tool, be used in wise ways and not-so-wise ways
- Learning something new is like exercising – you actually have to do the hard work yourself for it to help
- We want you to really **write your own code** and **write your own project reports**
  - It's the best way to actually learn the things you are supposed to learn in this course
    * The most "painful" moments in your work — when you work through the math yourself, when you try to formulate a correct and good sentence for your report, when you systematically go through your code line by line to find that one strange bug, or when you think carefully about whether a given result makes sense — these are the moments when you actually learn stuff!
- Also, in my experience: colleagues who really know topic X are also the ones who are able to get the best results when they apply AI tools to topic X – others mostly generate slop
- So, the policy on AI use in our course is:

  > **OK** for debugging, getting help understanding technical concepts, etc. **Not OK** for simply generating code/text.

  - By "debugging" we mean both help with debugging your C++ code, and help with correcting grammar etc. in your project reports

  - *If* you use AI tools in your work, you need to add to your project report a description of what you used these tools for. This would be part of the *Methods* section of your report. (We'll discuss report writing in detail later in the course.)
    * In the report template we have included a dedicated *Tools* subsection in the *Methods* section, where you mention the key tools you have used, and what you have used them for, e.g. sentences like "All figures in this report have been made using the Python package `matplotlib`."

- If we find it necessary, we will ask you to come present/discuss your work with us in person, so we can make sure that it really is your work we are assessing.
- **Important**: When you hand in a report or code, you take full responsibility for all the content. That is, you can never put the blame for anything on some AI tool.
  - Example: If an AI tool has generated a paragraph of text that is identical to an existing text, and you use this AI-generated text, then you are responsible for this plagiarism.
- For more on this topic, here's an interesting opinion piece by our colleague Anders Malthe-Sørenssen, published in Dagens Næringsliv: www.dn.no/kronikk/kunstig-intelligens/ki/ai/gjor-kunstig-intelligens-oss-dummere/2-1-1694498
  - The introduction might be of special interest to you. It describes a study from MIT which investigated how the use of AI tools (as opposed to more old-fashioned web search) impacted the learning outcomes for students who were tasked with solving a problem using a programming language they did not know from before — exactly the situation most of you are in in this course! The short summary was that the more heavily the students relied on AI tools in solving the problem the first time, the less they remembered about how they solved it when they were asked to solve *the same problem* again a little later. In short, these students hadn't learned much.
- Have a look at the KURT pages on AI: www.mn.uio.no/kurt/universitet/kunstig-intelligens/

In-lecture code discussion #1

- We have two pages with coding resources
  - anderkve.github.io/FYS3150
  - github.com/anderkve/FYS3150/tree/master/code_examples
- All code examples I discuss in the lectures can be found in one of these places
- Long code examples, e.g. example programs involving multiple files, are typically found in the `code_examples` directory of our Git repo.
- Make sure to explore these pages on your own! There's lots of help and hints to be found there!
  - In the first group sessions, spend some time going through the different introductory material on anderkve.github.io/FYS3150 before you start on project 1.
- Now let's introduce C++!
  - (Note that we won't have time in the lectures to talk about all C++ details you need for the projects.)
  - Intro:
    anderkve.github.io/FYS3150/book/introduction_to_cpp/intro
  - Hello World:
    anderkve.github.io/FYS3150/book/introduction_to_cpp/hello_world
  - Compiling and linking:
    anderkve.github.io/FYS3150/book/introduction_to_cpp/compiling_and_linking_take_1
  - Source files and header files:
    anderkve.github.io/FYS3150/book/introduction_to_cpp/source_files_and_header_files
  - Code structure:
    anderkve.github.io/FYS3150/book/introduction_to_cpp/code_structure
    * See also this example:
      github.com/anderkve/FYS3150/tree/master/code_examples/code_structure/example_1
  - Compilation and linking example with multiple files:
    github.com/anderkve/FYS3150/tree/master/code_examples/compilation_linking/example_1
  - *Strongly typed* languages (e.g. C++) vs *weakly typed* languages (e.g. Python).
    * anderkve.github.io/FYS3150/book/introduction_to_cpp/variables

- Write to file:
  anderkve.github.io/FYS3150/book/introduction_to_cpp/write_to_file
  * Also, remember that in cases with small output, simply *redirecting* terminal output into a file can be an easy and quick way to store output to a file — see the *Write terminal output to file* section of anderkve.github.io/FYS3150/book/using_the_terminal/basics

Topics in project 1

Some things are covered in the lectures, other things via examples on the webpage

- Discretisation of a continuous problem, in this case the following boundary value problem (BVP):

$$
\begin{aligned}
-\frac{d^2u}{dx^2} &= f(x) \\
x &\in [0,1] \\
f(x) &\text{ is known} \\
u(0) &= 0 \\
u(1) &= 0
\end{aligned}
$$

- Mathematical approx. to second derivative (suitable for discretisation)
- Connection between a BVP and a standard matrix equation ($\mathbf{A}\vec{x} = \vec{b}$), and approaches to solve this
  - Gaussian elimination
  - LU decomposition
- Errors!
  - Truncation error (purely math)
  - Numerical roundoff error (can't represent numbers with infinite precision on computers)
    - → *loss of numerical precision*
- Counting floating-point operations (FLOPs)
- Coding:
  - Working with arrays/vectors and matrices
  - Input/output (nicely formatted output)
  - Timing the code
  - Compilation and linking, basic code design

Discretisation of continuous functions

- Computers can't represent all possible numbers (finite range and "resolution")
  → Need to discretise!
- Take some function $u(x)$, with $x \in [x_{\min}, x_{\max}]$. ($u(x)$ might e.g. be the solution of our diff. eq. in project 1.)
- u and x are *continuous* quantities

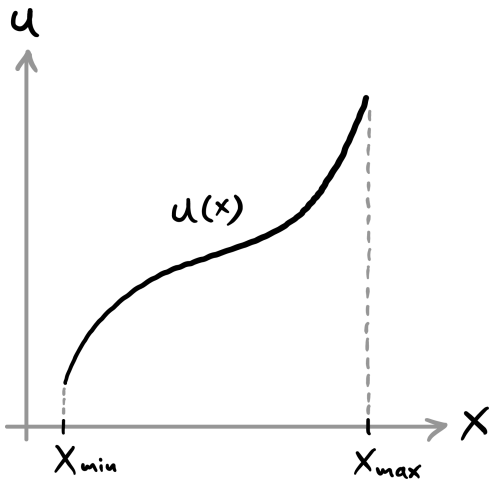


Figure 3: Continuous function

- Discretised representation

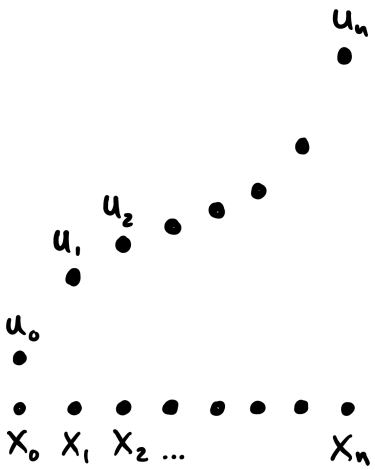


Figure 4: Discretised representation

Tip: When testing and debugging your code or trying to understand your results, it's often useful to work with a low number of points (coarse discretisation) and make plots that display your raw data points, i.e. not just directly draw lines between the points.

My notation

$$x \to x_i$$
$$u(x) \to u(x_i) \equiv u_i$$
$$u(x \pm h) \to u(x_i \pm h) \equiv u_{i\pm 1}$$

- So far u_i is the exact $u(x)$ at point $x = x_i$
- Our numerical methods will find an *approximation to the exact* u_i
- We will sometimes call this approximation v_i, to highlight that this approximation is not the same as the exact u_i

Basic relations

- $x_i = x_0 + ih$, with $i = 0, 1, 2, \ldots, n$
- step size: $h = x_1 - x_0 = \frac{x_2 - x_0}{2} = \ldots = \frac{x_n - x_0}{n}$.
  ($x_0 = x_{\min}$, $x_n = x_{\max}$)
- Will sometimes use notation Δx for h
- *Remember*: n **steps** corresponds to $n + 1$ **points**
- Always make a sketch if you are unsure about the discretisation

Numerical differentiation

See Chapter 3.1 in Morten's notes.

Main results

First derivative:

$$\left.\frac{du}{dx}\right|_{x_i} = u_i' = \frac{u_{i+1} - u_i}{h} + \mathcal{O}\left(h\right), \quad \text{(two-point, forward difference)}$$

$$\left.\frac{du}{dx}\right|_{x_i} = u_i' = \frac{u_i - u_{i-1}}{h} + \mathcal{O}\left(h\right), \quad \text{(two-point, backward difference)}$$

$$\left.\frac{du}{dx}\right|_{x_i} = u_i' = \frac{u_{i+1} - u_{i-1}}{2h} + \mathcal{O}\left(h^2\right) \quad \text{(three-point, central difference)}$$

Second derivative:

$$\left.\frac{d^2u}{dx^2}\right|_{x_i} = u_i'' = \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + \mathcal{O}\left(h^2\right)$$

Derivation

- Starting point: Taylor expansion of u around a point x

$$\begin{aligned} u(x+h) &= \sum_{n=0}^{\infty} \frac{1}{n!} u^{(n)}(x) h^n \\ &= u(x) + u'(x)h + \tfrac{1}{2}u''(x)h^2 + \tfrac{1}{6}u'''(x)h^3 + \mathcal{O}\left(h^4\right) \end{aligned}$$

> An aside on notation:
>
> - $u(x+h) = u(x) + u'(x)h + \mathcal{O}\left(h^2\right)$, (exact)
> - $u(x+h) \approx u(x) + u'(x)h$, (approximation, with truncation error $\mathcal{O}\left(h^2\right)$)

- Can get expression for $u'(x)$:

$$u(x+h) = u(x) + u'(x)h + \mathcal{O}\left(h^2\right)$$
$$\Rightarrow u'(x) = \frac{u(x+h) - u(x) - \mathcal{O}\left(h^2\right)}{h}$$

$$u'(x) = \frac{u(x+h) - u(x)}{h} + \mathcal{O}\left(h\right), \quad \text{(note power of } h\text{)}$$

Discretise:

$$u(x) \rightarrow u_i$$
$$\Rightarrow u_i' = \frac{u_{i+1} - u_i}{h} + \mathcal{O}\left(h\right)$$

(Two-point, forward difference)

- Compare to definition of the first derivative:

$$u'(x) \equiv \lim_{h \rightarrow 0} \frac{u(x+h) - u(x)}{h}$$

- We could have used the points x and $x - h$, which would have given us

$$u'(x) = \frac{u(x) - u(x-h)}{h} + \mathcal{O}\left(h\right)$$

Discretise:

$$u(x) \rightarrow u_i$$
$$\Rightarrow u_i' = \frac{u_i - u_{i-1}}{h} + \mathcal{O}\left(h\right)$$

(Two-point, backward difference)

- Quick illustration of forward difference method:
  - Example: $u(x) = a_0 + a_1 x + a_2 x^2$

- Exact: $u'(x) = a_1 + 2a_2x$
- Approximation:

$$\begin{aligned} u'(x) &\approx \frac{u(x+h) - u(x)}{h} \\ &= \frac{[a_0 + a_1(x+h) + a_2(x+h)^2] - [a_0 + a_1x + a_2x^2]}{h} \\ &= \frac{a_1h + a_2x^2 + 2a_2xh + a_2h^2 - a_2x^2}{h} \\ &= a_1 + 2a_2x + a_2h \end{aligned}$$

- Compare to the exact expression: our approximation is wrong by an $\mathcal{O}(h)$ term, as expected
- This **truncation error** gets smaller when we take $h \to 0$
- But doing this can lead to **roundoff errors** in the subtraction $u(x+h) - u(x)$, causing a **loss of precision**
- We will return to this topic later

- We can use more than two points to compute $u'(x)$:
  - Starting point: Taylor expansions for $u(x+x)$ and $u(x-h)$:

$$\begin{aligned} u(x+h) &= u(x) + u'(x)h + \tfrac{1}{2}u''(x)h^2 + \tfrac{1}{6}u'''(x)h^3 + \mathcal{O}\left(h^4\right) \\ u(x-h) &= u(x) - u'(x)h + \tfrac{1}{2}u''(x)h^2 - \tfrac{1}{6}u'''(x)h^3 + \mathcal{O}\left(h^4\right) \end{aligned}$$

  - Subtract:

$$u(x+h) - u(x-h) = 2u'h + \tfrac{2}{6}u'''h^3 + \mathcal{O}\left(h^5\right) \quad \text{(note power } h^5\text{)}$$

  - Rearrange:

$$u' = \frac{u(x+h) - u(x-h)}{2h} - \tfrac{1}{6}u'''h^2 - \mathcal{O}\left(h^4\right)$$

$$\boxed{u'(x) = \frac{u(x+h) - u(x-h)}{2h} + \mathcal{O}\left(h^2\right)}$$

Discretise:

$$u_i' = \frac{u_{i+1} - u_{i-1}}{2h} + \mathcal{O}\left(h^2\right)$$

(Three-point expression)

- Accuracy-efficiency trade-off:
  - The three-point expression for u' is more accurate than the two-point expressions
  - Price to pay: Need to evaluate (and keep track of) the function at more points
  - Could go further: A *five-point* expression for u' would have $\mathcal{O}\left(h^4\right)$ error, etc.
  - Trade-off between accuracy and number of function evaluations

- The second derivative
  - Add Taylor expansions for $u(x+h)$ and $u(x-h)$

$$u(x+h) + u(x-h) = 2u(x) + u''(x)h^2 + \mathcal{O}\left(h^4\right)$$

  - Rearrange to isolate $u''(x)$

$$u''(x) = \frac{u(x+h) - 2u(x) + u(x-h)}{h^2} + \mathcal{O}\left(h^2\right)$$

Discretise:

$$u_i'' = \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + \mathcal{O}\left(h^2\right)$$

In-lecture code discussion #2

- Hidden files on Unix systems
  - Hidden files have file names starting with a dot
  - Some relevant examples:
    * `.bashrc` and/or `.profile` in your home directory
    * `.gitignore` in your git repositories
  - Use the `-a` option to see hidden files in your file listings: `ls -a`
- Terminal-based text editors
  - Useful when you want to make quick file edits
  - Useful when you are logged into another system via a Unix terminal, e.g. if you are working on a supercomputer
  - Some people use the terminal-based editors as their main editors — can become very powerful and efficient tools
  - Two popular examples: `vim` and `nano`
- Short discussion of the `std::vector` class: anderkve.github.io/FYS3150/book/introduction_to_cpp/containers
  - Note the use of `my_vector.at(10)` as a safe alternative to `my_vector[10]` for accessing vector elements.
- How (not) to use `using namespace` in C++ programs: anderkve.github.io/FYS3150/book/introduction_to_cpp/source_files_and_header_files
- Integer vs floating-point division:
  - In Python, the statement `x = 7/10` will by default evaluate to `x = 0.7`
  - However, in C++ the statement `x = 7/10` will evaluate to `x = 0`
  - Since `7` and `10` are written as integers, C++ will do **integer division**
  - In integer division, it is correct that $7/10 = 0$
  - If we instead write `7.` and `10.`, C++ will treat these as floating-point numbers and perform floating-point division
  - So `x = 7./10.` will give the result `x = 0.7`
  - (The combinations `x = 7./10` and `x = 7/10.` will also give `x = 0.7`)
  - *Question*: Given the variable assignment `double x = 7/10;`, what value will `x` get?
  - *Answer*: `x` will be set to `x = 0.0`, since the assignment is evaluated as `double x = 0;`
- Function arguments: a first example of **pass-by-reference** versus **pass-by-value** github.com/anderkve/FYS3150/tree/master/code_examples/function_arguments/example_1

Boundary value problems (BVPs)

- Our case in project 1:

$$-\frac{d^2u}{dx^2} = f(x)$$

  - $u(x)$ is an *unknown* function → what we want to find
  - $f(x)$ is some *known* function
  - $x \in [0, 1]$
  - Boundary values: $u(0) = 0$ and $u(1) = 0$ (Dirichlet)

- Special case of:

$$\alpha\frac{d^2u}{dx^2} + \beta\frac{du}{dx} + \gamma u(x) = f(x)$$

  - *Ordinary* diff. eq., since there is only one independent variable (x)
  - *Linear* diff. eq., since each term has maximum one power of u, u', u'', …
  - *Second order* diff. eq., since the highest-order derivative is u''
  - *Inhomogeneous* diff. eq., when $f(x) \neq 0$

- Many diff. eqs. in physics are linear
  - Then the sum of two solutions is a new, valid solution! (*superposition*)
  - Famous example: The Schrödinger eq. in quantum mechanics is linear → superposition of quantum states!

- Many approaches to finding a solution
  - **Shooting methods** (described quickly below)
  - **Finite difference methods** (project 1, described below)
  - **Finite element methods** (not covered)

Quick description of shooting methods

- We want to solve a boundary value problem (BVP), where we start with known $u(x_{\min})$ and $u(x_{\max})$

- We'll do this by instead repeatedly solving an *initial value problem* (IVP), where we start with known $u(x_{\min})$ and $u'(x_{\min})$:
  - Start from the known $u(x_{\min})$
  - Guess a value for $u'(x_{\min})$
  - Solve the corresponding IVP forward ("shoot"). (We will discuss IVPs later in the course.)
  - Repeat the previous two steps (in some clever way) until we find a solution $u(x)$ that hits the known boundary condition at $u(x_{\max})$
  - This solution $u(x)$ is then a solution to our original BVP
- Things are easier when our diff. eq. is *linear*:
  - Guess a value for $u'(x_{\min})$, solve the IVP
    → let's call this solution $u_{(1)}(x)$
  - Guess another $u'(x_{\min})$, solve the IVP
    → let's call this solution $u_{(2)}(x)$
  - Since we have a linear diff. eq., a sum of solutions is a new solution:
    $u_c(x) = cu_{(1)}(x) + (1-c)u_{(2)}(x)$
  - Require that $u_c(x_{\max})$ should equal the known $u(x_{\max})$ (the second boundary condition)
  - Use this condition to determine a value for the free parameter c
    → this $u_c(x)$ is then the solution $u(x)$ to our BVP
- A drawback: Need to solve multiple IVPs to find the single solution to our BVP

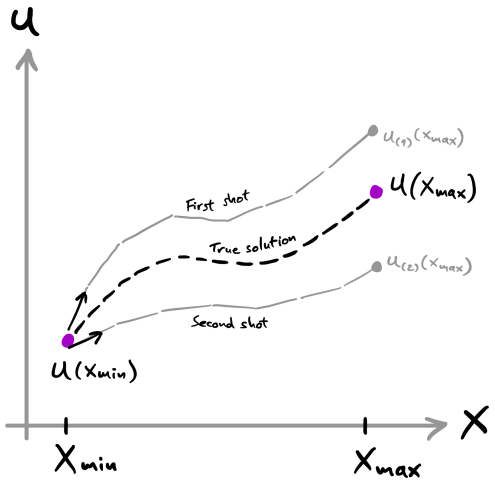


Figure 5: Sketch of the shooting method

Finite difference method

- Our problem: Find the function $u(x)$ that solves this diff. eq.:

$$-\frac{d^2u}{dx^2} = f(x)$$

- We know $u(0)$, $u(1)$, $f(x)$ and that $x \in [0, 1]$
- Strategy:
  - **Step 1**: Express problem as a matrix eq.
  - **Step 2**: Solve the matrix eq.

Step 1: Express as matrix eq.

- Discretise equation:

$$-\left[\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + \mathcal{O}\left(h^2\right)\right] = f_i, \quad f_i \equiv f(x_i)$$

- Approximate (leave out the $\mathcal{O}\left(h^2\right)$ terms) and change notation: $v_i \approx u_i$
- Arrange terms:

$$\boxed{-v_{i-1} + 2v_i - v_{i+1} = h^2 f_i}$$

- Note: this is a collection of multiple equations, one for each value we can insert for i
- New goal: Determine $v_1, v_2, \ldots, v_{n-1}$
- We know: v_0, v_n and all the f_i
- Below we'll consider the special case with $n_{\text{steps}} = 5$
  - $v_0, v_1, v_2, v_3, v_4, v_5$: 6 points
  - v_0 and v_5 are known boundary points
  - 4 unknowns: v_1, v_2, v_3, v_4
  - $h = \frac{v_5 - v_0}{n_{\text{steps}}} = 0.2$ (very large, just for illustration)

- The boxed expression represents a set of four equations. Let's write them out in a suggestive manner…

$$
\begin{array}{lccccccc}
(i=1) & -v_0 & +2v_1 & -v_2 & & & = & h^2 f_1 \\
(i=2) & & -v_1 & +2v_2 & -v_3 & & = & h^2 f_2 \\
(i=3) & & & -v_2 & +2v_3 & -v_4 & = & h^2 f_3 \\
(i=4) & & & & -v_3 & +2v_4 \quad -v_5 & = & h^2 f_4
\end{array}
$$

- v_0 and v_5 are known — let's move them over to the right-hand side and define some simpler notation g_1, g_2, g_3, g_4:

$$
\begin{array}{rrrrcrcl}
+2v_1 & -v_2 & & & = & h^2 f_1 + v_0 & \equiv & g_1 \\
-v_1 & +2v_2 & -v_3 & & = & h^2 f_2 & \equiv & g_2 \\
& -v_2 & +2v_3 & -v_4 & = & h^2 f_3 & \equiv & g_3 \\
& & -v_3 & +2v_4 & = & h^2 f_4 + v_5 & \equiv & g_4
\end{array}
$$

- This can be written as

$$
\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix}
$$

$$
\boxed{\mathbf{A}\vec{v} = \vec{g}}
$$

- $\mathbf{A}$ and $\vec{g}$ are known, we want to solve for $\vec{v}$
- Note that $\mathbf{A}$ is a **tridiagonal matrix**.
- The *diagonal* has only 2's, while the *superdiagonal* and *subdiagonal* contain only -1's.
- Note that the vector $\vec{v} = [v_1, v_2, v_3, v_4]$ in this equation only contains the *unknown* v_i. The known values at the boundaries, v_0 and v_5, are *not* included in $\vec{v}$.

Step 2: Solve the matrix eq.

- Overview of things we'll discuss:
  1. *Later in the course*: Methods for solving a *general* matrix equation $\mathbf{A}\vec{x} = \vec{b}$
     - Gaussian elimination
     - LU decomposition
     - Iterative methods
  2. *Now*: Method for solving $\mathbf{A}\vec{v} = \vec{g}$ when $\mathbf{A}$ is a *general*, *tridiagonal* matrix
     - Gaussian elimination turns into the Thomas algorithm
  3. *A task for you in Project 1*: Method for solving $\mathbf{A}\vec{v} = \vec{g}$ when $\mathbf{A}$ is the *special, tridiagonal* matrix above, with only -1's and 2's along the diagonals

Matrix equations: Gaussian elimination and the Thomas algorithm

Introduction

- A matrix equation $\mathbf{A}\vec{x} = \vec{b}$ ($\mathbf{A}$ and $\vec{b}$ known, $\vec{x}$ unknown) represents a set of linear equations

$$\begin{array}{rccccccccc}
(\text{eq. } 1) & a_{11}x_1 & + & a_{12}x_2 & + & \ldots & + & a_{1n}x_n & = & b_1 \\
(\text{eq. } 2) & a_{21}x_1 & + & a_{22}x_2 & + & \ldots & + & a_{2n}x_n & = & b_2 \\
(\ldots) & & & & & & & & & \\
(\text{eq. } m) & a_{m1}x_1 & + & a_{m2}x_2 & + & \ldots & + & a_{mn}x_n & = & b_m
\end{array}$$

- m equations, each with n terms — one for each unknown variable $(x_1, \ldots, x_n)$

$$\underset{(m\times n)}{\mathbf{A}} \; \underset{(n\times 1)}{\vec{x}} = \underset{(m\times 1)}{\vec{b}}$$

- We will focus on the case of a *square* matrix, i.e. when $m = n$
- This means we have n equations and n unknowns
- If all our equations are *linearly independent*, i.e. when each equation represents information not contained in the other equations, we should be able to solve for all our n unknowns $(x_1, \ldots, x_n)$
- Some equivalent statements:
  - All the equations are linearly independent
  - $\mathbf{A}$ is *not* singular (all eigenvalues of $\mathbf{A}$ are non-zero)

  - $\det \mathbf{A} \neq 0$

> *Side note:* When we have *more* equations (constraints) than unknowns, there is generally *no exact solution*. But we can *fit* our unknowns such that all our equations are as close to solved as possible. This is the typical case in science: you have some model with a few free parameters and the model needs to match many observations (constraints) as closely as possible.

Gaussian elimination, overview

- Start from general matrix equation

$$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet \end{bmatrix} \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix}$$

- **Step 1**: Forward substitution/elimination
  - Turn matrix into upper-triangular form

$$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \\ & \bullet & \bullet & \bullet \\ & & \bullet & \bullet \\ & & & \bullet \end{bmatrix} \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix}$$

  - Can then read off solution for x_m (last row)
- **Step 2**: Back substitution/elimination
  - Use the now known x_m to find x_{m-1}, then use these to find x_{m-2}, and so on
  - End up with this

$$\begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & & 1 \end{bmatrix} \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix}$$

Thomas algorithm: Gaussian elimination on a tridiagonal matrix

- Let's go back to the notation of project 1: $\mathbf{A}\vec{v} = \vec{g}$

- Let $\mathbf{A}$ be a *general* tridiagonal matrix
  - For concreteness we look at the case with a 4×4 matrix:

$$\begin{bmatrix} b_1 & c_1 & 0 & 0 \\ a_2 & b_2 & c_2 & 0 \\ 0 & a_3 & b_3 & c_3 \\ 0 & 0 & a_4 & b_4 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix}$$

- Note that we have used indices that correspond to the row numbers:
  - Subdiagonal: $\vec{a} = [a_2, a_3, a_4]$
  - Diagonal: $\vec{b} = [b_1, b_2, b_3, b_4]$
  - Superdiagonal: $\vec{c} = [c_1, c_2, c_3]$
- Now let's do **step 1**, the *forward substitution*
  - We start from the *augmented matrix*:

$$\begin{array}{lcccc|c} R_1: & b_1 & c_1 & 0 & 0 & g_1 \\ R_2: & a_2 & b_2 & c_2 & 0 & g_2 \\ R_3: & 0 & a_3 & b_3 & c_3 & g_3 \\ R_4: & 0 & 0 & a_4 & b_4 & g_4 \end{array}$$

  - To move towards an upper-triangular form, we want to set the a_2 entry in R_2 to 0
  - Use a row operation with R_1 to achieve this: $R_2 \to R_2 - \frac{a_2}{b_1} R_1$
  - This turns the a_2 entry into $a_2 - \frac{a_2}{b_1} b_1 = 0$

$$\begin{array}{lcccc|c} R_1: & b_1 & c_1 & 0 & 0 & g_1 \\ R_2: & 0 & (b_2 - \frac{a_2}{b_1} c_1) & c_2 & 0 & (g_2 - \frac{a_2}{b_1} g_1) \\ R_3: & 0 & a_3 & b_3 & c_3 & g_3 \\ R_4: & 0 & 0 & a_4 & b_4 & g_4 \end{array}$$

  - Introduce shorthand notation:
    - $\tilde{b}_1 = b_1$
    - $\tilde{b}_2 = b_2 - \frac{a_2}{\tilde{b}_1} c_1$
    - $\tilde{g}_1 = g_1$
    - $\tilde{g}_2 = g_2 - \frac{a_2}{\tilde{b}_1} \tilde{g}_1$

- We then have

$$\begin{array}{lcccc|c} R_1: & \tilde{b}_1 & c_1 & 0 & 0 & \tilde{g}_1 \\ R_2: & 0 & \tilde{b}_2 & c_2 & 0 & \tilde{g}_2 \\ R_3: & 0 & a_3 & b_3 & c_3 & g_3 \\ R_4: & 0 & 0 & a_4 & b_4 & g_4 \end{array}$$

- Now continue in the same way to turn the a_3 entry to zero
- Row operation: $R_3 \rightarrow R_3 - \frac{a_3}{\tilde{b}_2} R_2$
- Define notation:
  - $\tilde{b}_3 = b_3 - \frac{a_3}{\tilde{b}_2} c_2$
  - $\tilde{g}_3 = g_3 - \frac{a_3}{\tilde{b}_2} \tilde{g}_2$
- We then get

$$\begin{array}{lcccc|c} R_1: & \tilde{b}_1 & c_1 & 0 & 0 & \tilde{g}_1 \\ R_2: & 0 & \tilde{b}_2 & c_2 & 0 & \tilde{g}_2 \\ R_3: & 0 & 0 & \tilde{b}_3 & c_3 & \tilde{g}_3 \\ R_4: & 0 & 0 & a_4 & b_4 & g_4 \end{array}$$

- And once more, with feeling…
- Row operation: $R_4 \rightarrow R_4 - \frac{a_4}{\tilde{b}_3} R_3$
- Define notation:
  - $\tilde{b}_4 = b_4 - \frac{a_4}{\tilde{b}_3} c_3$
  - $\tilde{g}_4 = g_4 - \frac{a_4}{\tilde{b}_3} \tilde{g}_3$

$$\begin{array}{lcccc|c} R_1: & \tilde{b}_1 & c_1 & 0 & 0 & \tilde{g}_1 \\ R_2: & 0 & \tilde{b}_2 & c_2 & 0 & \tilde{g}_2 \\ R_3: & 0 & 0 & \tilde{b}_3 & c_3 & \tilde{g}_3 \\ R_4: & 0 & 0 & 0 & \tilde{b}_4 & \tilde{g}_4 \end{array}$$

- The forward substitution is now done! Here's the summary:

Forward substitution:

$$\tilde{b}_1 = b_1$$
$$\tilde{b}_i = b_i - \frac{a_i}{\tilde{b}_{i-1}} c_{i-1} \qquad i = 2, 3, 4$$
$$\tilde{g}_1 = g_1$$
$$\tilde{g}_i = g_i - \frac{a_i}{\tilde{b}_{i-1}} \tilde{g}_{i-1} \qquad i = 2, 3, 4$$

- Now let's do **step 2**, the *back substitution*
  - Starting point

$$\begin{array}{lcccc|c} R_1: & \tilde{b}_1 & c_1 & 0 & 0 & \tilde{g}_1 \\ R_2: & 0 & \tilde{b}_2 & c_2 & 0 & \tilde{g}_2 \\ R_3: & 0 & 0 & \tilde{b}_3 & c_3 & \tilde{g}_3 \\ R_4: & 0 & 0 & 0 & \tilde{b}_4 & \tilde{g}_4 \end{array}$$

  - We now want to get to an identity matrix form, starting from the bottom row
  - Row operation: $R_4 \rightarrow \frac{R_4}{\tilde{b}_4}$

$$\begin{array}{lcccc|cl} R_1: & \tilde{b}_1 & c_1 & 0 & 0 & \tilde{g}_1 & \\ R_2: & 0 & \tilde{b}_2 & c_2 & 0 & \tilde{g}_2 & \\ R_3: & 0 & 0 & \tilde{b}_3 & c_3 & \tilde{g}_3 & \\ R_4: & 0 & 0 & 0 & 1 & \frac{\tilde{g}_4}{\tilde{b}_4} & \rightarrow v_4 \end{array}$$

  - We now have the solution for v_4:

$$\boxed{v_4 = \frac{\tilde{g}_4}{\tilde{b}_4}}$$

  - Now we want to get R_3 on the form (0,0,1,0)
  - We can subtract $c_3 R_4$ to get rid of the c_3 entry in R_3, and then divide by $\tilde{b}_3$ to set the third element to 1

- Row operation: $R_3 \rightarrow \frac{R_3 - c_3 R_4}{\tilde{b}_3}$

$$\begin{array}{lcccc|c} R_1: & \tilde{b}_1 & c_1 & 0 & 0 & \tilde{g}_1 \\ R_2: & 0 & \tilde{b}_2 & c_2 & 0 & \tilde{g}_2 \\ R_3: & 0 & 0 & 1 & 0 & \frac{\tilde{g}_3 - c_3 v_4}{\tilde{b}_3} \quad \rightarrow v_3 \\ R_4: & 0 & 0 & 0 & 1 & v_4 \end{array}$$

- This gives us the solution for v_3:

$$\boxed{v_3 = \frac{\tilde{g}_3 - c_3 v_4}{\tilde{b}_3}}$$

- We can continue upwards like this to find all the remaining v_i. In summary:

Back substitution:

$$v_4 = \frac{\tilde{g}_4}{\tilde{b}_4}$$

$$v_i = \frac{\tilde{g}_i - c_i v_{i+1}}{\tilde{b}_i} \qquad i = 3, 2, 1$$

- Let's summarise what we've done:
  - Given a general tridiagonal matrix $\mathbf{A}$ and a vector $\vec{g}$, we have found the vector $\vec{v}$ that solves the equation $\mathbf{A}\vec{v} = \vec{g}$.
  - We used Gaussian elimination, which has two steps:
    - *forward substitution*
    - *back substitution*
  - Because $\mathbf{A}$ was tridiagonal, the Gaussian elimination procedure resulted in a fairly simple algorithm, which is known as the **Thomas algorithm** (Llewellyn Thomas, 1903–1992)

Coding tip: Note that we don't need to work with an entire matrix in memory here. To implement the Thomas algorithm above, we just need some arrays/vectors $\vec{a}$, $\vec{b}$, $\vec{c}$, $\vec{g}$, $\tilde{\vec{b}}$, $\tilde{\vec{g}}$ and $\vec{v}$.

Back to our boundary value problem

- We now have the tools we need to use a **finite difference method** to solve a boundary value problem like

$$-\frac{d^2u}{dx^2} = f(x)$$

  where $f(x)$ is some known function, and we know $u(x_{\min})$ and $u(x_{\max})$

  - Discretise the problem, using a discretised approximation for the second derivative
    - At this step we changed notation $u_i \to v_i$
  - Formulate the resulting set of equations as a matrix equation $\mathbf{A}\vec{v} = \vec{g}$
    - The second derivative in the diff. eq. → the matrix $\mathbf{A}$ will be tridiagonal, with a simple (-1,2,-1) form
  - Use the Thomas algorithm to solve the matrix equation
    - **However**, the Thomas algorithm is a method that can solve *any* tridiagonal matrix equation, but in the case of our BVP we are only interested in the case of a particularly simple, tridiagonal matrix. This means that we can simplify the Thomas algorithm for our use case — something you will do in project 1.

- *A reasonable question*: Why are we doing all this? Why not rather find $\mathbf{A}^{-1}$ and solve the equation as $\vec{v} = \mathbf{A}^{-1}\vec{g}$?

  - Finding $\mathbf{A}^{-1}$ numerically takes $\mathcal{O}\left(n^3\right)$ operations for an $n \times n$ matrix. This approach becomes useful if we need to solve *many* different equations ($\mathbf{A}\vec{v}_1 = \vec{g}_1, \mathbf{A}\vec{v}_2 = \vec{g}_2, \ldots$) that all involve the same matrix $\mathbf{A}$. But for solving a single equation $\mathbf{A}\vec{v} = \vec{g}$, other methods are quicker.

Counting floating-point operations (FLOPs)

- Floating-point numbers, *floats*: finite, inexact machine representation of the real numbers ($\mathbb{R}$)
- Floats are numbers where the decimal point (base 10) – or the *binary point* (base 2) – can be placed anywhere (it can "float") in a given string of digits, depending on which number we need to represent
  - Example, using base 10: The digits `112358` can represent `11.2358` or `1123.58`, depending on the placement of the decimal point.
- More technically: we represent the float as
$$[\text{sign}][\text{mantissa}] \times [\text{base}]^{[\text{exponent}]}$$
  - For base 10, this is just the usual scientific notation
  - On computers: base 2 (the binary number system)
    * We spend our given set of bits (e.g. 64 bits) to represent the numbers in the mantissa and exponent, plus one bit for the sign.
    * We'll discuss this in more detail in the next chapter
- Floating-point operations: $\{+, -, \times, \div\}$ with floats
- Generally a floating-point operation is computationally slower than an integer operation
  - True historically, but relative speed compared to integer operation depends on details of hardware, compiler optimisations, etc.
- Counting FLOPs is a way of estimating the *efficiency of an algorithm*
  - *Only a rough estimate* – the actual runtime of an algorithm also depends on many other factors, like hardware, memory layout, compiler optimisations, …
- *Note*: **FLOPs** (FLoating-point OPerations) vs **FLOPS** (FLoating-point OPerations per Second). FLOPS is a measure of *computer performance*, which we will not discuss in this course.
  - So how long a given task will take on a given computer will depend both on the number of **FLOPs** required for the task *and* the number of **FLOPS** for the computer — and a bunch of other things…
- Not all FLOPs are equal! Two examples:
  - Division depends on iterative algorithms
    * Several times slower than addition, subtraction or multiplication

  - Many CPUs have a dedicated instruction to perform the common operation $a \times b + c$ faster than two separate FLOPs. (This is called *fused multiply-add*, FMA)
    * We will follow the old convention and count $a \times b + c$ as two FLOPs

- Other common operations that require many FLOPs:
  - *Square root*: Depends on iterative algorithms, sometimes with optimised hardware support. Typically a bit slower than division
  - *Transcendental functions* (exp, log, sin, cos, etc.): Much slower, e.g. due to being based on some form of series expansion

Examples

- **Example 1:**

$$y = ab + c, \quad \text{1 mult., 1 add.} \rightarrow \text{2 FLOPs}$$

- **Example 2:**

$$\begin{aligned} &\text{for } i = 1, \ldots, n: && n \text{ repetitions} \\ &\quad y_i = a y_{i-1} + i && \text{2 FLOPs} \\ & && \rightarrow 2n \text{ FLOPs} \end{aligned}$$

- **Example 3:**

$$\begin{aligned} &\text{for } i = 1, \ldots, n: && n \text{ repetitions} \\ &\quad y_i = \frac{a}{b} y_{i-1} + i && \text{3 FLOPs} \\ & && \rightarrow 3n \text{ FLOPs (silly!)} \end{aligned}$$

- A more efficient version of example 3:

$$\begin{aligned} &c = \frac{a}{b} && \text{1 FLOP} \\ &\text{for } i = 1, \ldots, n: && n \text{ repetitions} \\ &\quad y_i = c y_{i-1} + i && \text{2 FLOPs} \\ & && \rightarrow (2n+1) \text{ FLOPs} \approx 2n \text{ FLOPs} \end{aligned}$$

> *Tip:* When code speed is important, avoid recomputing constants within a loop.

- **Example 4:** matrix-vector multiplication $\vec{y} = \mathbf{A}\vec{x}$, where A is $n \times n$:

$$y_i = \sum_{j=1}^{n} a_{ij} x_j \quad i = 1, \ldots, n$$

  - For each i we have n multiplications and $n-1$ additions, so $2n-1$ FLOPs
  - Repeat for all i: $\rightarrow n(2n-1) = 2n^2 - n$ FLOPs
  - So we say matrix-vector multiplication has $\mathcal{O}\left(n^2\right)$ computational complexity

- **Example 5:** standard matrix-matrix multiplication $\mathbf{A} = \mathbf{BC}$, with $n \times n$ matrices:

$$a_{ij} = \sum_{k=1}^{n} b_{ik} c_{kj} \quad i, j = 1, \ldots, n$$

  - For each (i, j) we have n multiplications and $n-1$ additions, so $2n-1$ FLOPs
  - Repeat for each of the n^2 different (i, j) pairs: $\rightarrow n^2(2n-1) = 2n^3 - n^2$ FLOPs
  - So we say matrix-matrix multiplication using the "schoolbook" algorithm has $\mathcal{O}\left(n^3\right)$ computational complexity
  - *But, there exist faster algorithms for matrix-matrix multiplication!*
    * Matrix multiplication is an active field of research
    * If you are interested, read about *Strassen's algorithm* and more recent advances

Binary representation

- *In short*: How to represent numbers using only two different symbols
- Basic element: a **bit**
  - 1/0, on/off, true/false, yes/no, hole/not-hole (punched cards), red/blue, …
  - The term *bit* is originally a contraction of *binary information digit*
- A bit doesn't have to be related to computers — it's a basic concept from information theory
  - A bit is the expected amount of *information* or *surprise* contained in the outcome of a 50/50 random draw. (The more surprising a result/message/signal is, the more information it contains — look up literature on *Shannon entropy* for more on this.)
- In principle, any physical system with two possible states can be used to represent the digits 0 and 1
- So we better use a numeral system that only needs two different digits to represent any number
  → the **binary system** or the **base 2 system**
- In base 10, we have ten different symbols (0–9) that can be used per position
- In base 2, we only have two different symbols per position
  → need to use more positions to express numbers
  → longer strings of symbols compared to the decimal system
- *Side note*: In computing and mathematics we also sometimes encounter the *hexadecimal* (base 16) system. In this case there are 16 different symbols (0–9 and A–F) per position.

Integers

- Example: 137 in base 10 and base 2: $(137)_{10} = (10001001)_2$
- Representation in the decimal (base 10) system:

$$\begin{aligned}(137)_{10} = \begin{array}{ccc} 10^2 & 10^1 & 10^0 \\ \hline 1 & 3 & 7 \end{array} &= (1 \times 10^2) + (3 \times 10^1) + (7 \times 10^0) \\ &= 100 + 30 + 7 \\ &= 137\end{aligned}$$

- Representation in the binary (base 2) system:

$$
\begin{aligned}
(10001001)_2 = & \begin{array}{cccccccc} 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ \hline 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{array} \\
&= (1 \times 2^7) + (0 \times 2^6) + \ldots + (1 \times 2^3) + \ldots + (1 \times 2^0) \\
&= 128 + 0 + 0 + 0 + 8 + 0 + 0 + 1 \\
&= 137
\end{aligned}
$$

- How can we find the correct binary string of 0's and 1's for a given number?
  - The same way we (without thinking about it) identify the correct string of digits in the decimal system: by doing *integer division and keeping track of remainders*

	Remainder	Position
$137 \setminus 10 = 13$	7	10^0
$13 \setminus 10 = 1$	3	10^1
$1 \setminus 10 = 0$	1	10^2

Table 1: Repeated integer division with 10 produces the base 10 representation of 137.

	Remainder	Position
$137 \setminus 2 = 68$	1	2^0
$68 \setminus 2 = 34$	0	2^1
$34 \setminus 2 = 17$	0	2^2
$17 \setminus 2 = 8$	1	2^3
$8 \setminus 2 = 4$	0	2^4
$4 \setminus 2 = 2$	0	2^5
$2 \setminus 2 = 1$	0	2^6
$1 \setminus 2 = 0$	1	2^7

Table 2: Repeated integer division with 2 produces the base 2 representation of 137.

- The more bits we have available, the longer the integer we can store
- If we are working with *signed* integers, we need one additional bit to represent the sign: $(-1)^0$ or $(-1)^1$

Floating-point numbers

- How to represent the real numbers ($\mathbb{R}$) in binary?
- Strategy: use *normalised, scientific notation in base 2*
- Example in decimal:

$$-9.90625 \times 10^0, \text{ or}$$
$$-0.990625 \times 10^1$$

- The latter convention, where the first digit is always zero, is often used in computing
- General form:

$$\pm \left[\text{number in } \left(\tfrac{1}{10}, 1\right)\right] \times 10^{[\text{integer exponent}]}$$

- In binary (base 2):

$$\pm \left[\text{number in } \left(\tfrac{1}{2}, 1\right)\right] \times 2^{[\text{integer exponent}]}$$

- Terminology:

$$[\text{sign}][\text{mantissa}] \times 2^{[\text{exponent}]}$$

- Whether the mantissa should be a number within $(\frac{1}{2}, 1)$ or within $(1, 2)$ is a matter of convention
- Another common term for the mantissa is the **significand**
- We already know how to represent the integer exponent and the sign bit in binary
- Binary representation of the mantissa:
  - Example: $(0.5625)_{10}$

$$\begin{aligned}(0.1001)_2 &= \frac{2^0 \quad 2^{-1} \quad 2^{-2} \quad 2^{-3} \quad 2^{-4}}{0 \quad\ 1 \quad\ 0 \quad\ 0 \quad\ 1} \\ &= (0 \times 2^0) + (1 \times 2^{-1}) + (0 \times 2^{-2}) + (0 \times 2^{-3}) + (1 \times 2^{-4}) \\ &= 0 + 0.5 + 0 + 0 + 0.0625 \\ &= 0.5625\end{aligned}$$

- **Single precision**: Using 32 bits (4 bytes) to represent a floating-point number
  - Sign: 1 bit
  - Exponent: 8 bits
  - Mantissa: 23 bits
- Example: The number -3.25

  - In normalised, scientific notation in base 2: -0.8125×2^2
    * Sign: -1 (so the *sign bit* will be 1 since $-1 = (-1)^1$)
    * Exponent: 2
    * Mantissa: 0.8125

- In memory, something like this:

sign bit	8-bit exponent (2)	23-bit mantissa (0.8125)
1	00000010	01101000 ... 000

- On most computer systems, the type **`float`** in C++ will correspond to a 32-bit number
- **Double precision**: Using 64 bits (8 bytes) to represent a floating-point number
  - Sign: 1 bit
  - Exponent: 11 bits
  - Mantissa: 52 bits
- On most systems, the type **`double`** in C++ will correspond to a 64-bit number
- An 11-bit exponent gives an exponent range of $(-1024, 1024)$, since $2^{11} = 2048$
  - Since $2^{1024} \approx 10^{308}$, the range of numbers that can be represented in double-precision is roughly $(10^{-308}, 10^{308})$
  - You can test this quickly in your Python terminal, since a floating-point number in Python (the **`float`** type in Python) by default will be a 64-bit number on most systems:

```
>>> 2.**1023
8.98846567431158e+307
>>>
>>> 2.**1024
OverflowError: (34, 'Numerical result out of range')
```

- Finite number of bits → unavoidable problems with range and accuracy
- Limited number of bits for the **exponent**:
  → a limited **range** of $\mathbb{R}$ can be represented
  - With 11 bits for the exponent, we get a range of $\sim (10^{-308}, 10^{308})$
- Limited number of bits for the **mantissa**:
  → a limited **resolution**/**precision** in our representation of the continuous $\mathbb{R}$
  - With 52 bits for the mantissa, we get a precision of around **15 digits** in the decimal system ($\log_{10}(2^{52}) \approx 15.654$)

Hidden bit: When using normalised, scientific notation in base 2, we know that the most significant digit, i.e. the first digit of the mantissa, will always be 0 (if the $(\frac{1}{2}, 1)$-convention is used), or always be 1 (if the $(1, 2)$-convention is used). So we don't need to explicitly store this bit in memory. This trick is referred to as the *hidden bit*, and it effectively increases the mantissa precision by one bit, e.g. from 52 bits to 53 bits for a double-precision number.

- *A silly example to illustrate the effect of limited range and precision*:
  - Let's work in base 10
  - Assume we only had memory for *one digit in the exponent* and *one digit in the mantissa*
  - We could then only represent these numbers:

$$\ldots, 1 \times 10^{-1}, 2 \times 10^{-1}, \ldots, 1 \times 10^{0}, 2 \times 10^{0}, \ldots, 1 \times 10^{1}, 2 \times 10^{1}, \ldots$$

  - We would have a range of $\sim (10^{-5}, 10^{5})$
  - The only numbers we would be able to use would be

$$\ldots, 0.1, 0.2, 0.3, \ldots, 1, 2, 3, \ldots, 10, 20, 30, \ldots, 100, 200, 300, \ldots$$

  - So a number 17 would just end up as 10, and a computation like 100 + 80 would just give the result 100

Errors

Truncation errors

- Truncation errors are purely mathematical in origin
- Typical case: we cut off a series expansion at some point
- Example: leaving out the $\mathcal{O}\left(h^2\right)$ terms in

$$u_i'' = \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + \mathcal{O}\left(h^2\right)$$

- *Note*: here a smaller step size h will give a smaller truncation error

Roundoff errors

- Numbers can only be stored with limited accuracy
- For *doubles*, our precision is ~15 decimal digits
  - We will often refer to this as our *machine precision*
- So *almost* all numbers we store are **approximations** to the true number we intended to store
  - True number: a
  - Floating-point representation of a: $fl(a)$
  - Given a true a, your $fl(a)$ will be in the range

    $$a(1 - \delta_m) < fl(a) < a(1 + \delta_m)$$

    where δ_m is the machine precision (e.g. $\delta_m \sim 10^{-15}$)
  - So given $fl(a)$, all you know is that the true number a is in the range

    $$fl(a)(1 - \delta_m) < a < fl(a)(1 + \delta_m)$$

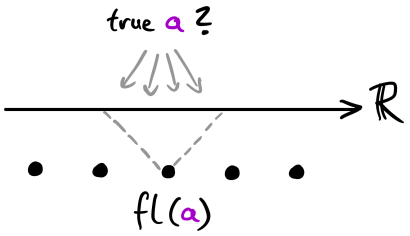


Figure 6: The continuous number line and the discretised floating-point representation

Loss of numerical precision

- Also known as **loss of significance**
- Typical case: subtraction with similar numbers
  → we lose the most significant digits, left with digits that are more affected by roundoff errors
- Example:
  - True values: $a = 1.0054321$, $b = 1.0040001$
  - Assume a machine precision of $\delta_m \sim 10^{-4}$ (just for illustration)
  - Approximate floating-point representations: $fl(a) = 1.005$, $fl(b) = 1.004$
  - 4 significant digits
  - Relative errors in the approximations:

$$\left|\frac{a - fl(a)}{a}\right| \approx 10^{-4}$$

$$\left|\frac{b - fl(b)}{b}\right| \approx 10^{-7}$$

  - So $fl(a)$ and $fl(b)$ are clearly very reasonable approximations to a and b, given our assumed machine precision
  - Now perform a subtraction:
  - True value: $a - b = 0.0014320$
  - Approximate: $fl(a) - fl(b) = 1.005 - 1.004 = 0.001$
  - Now we only have 1 significant digit!
  - Relative error:

$$\left|\frac{0.0014320 - 0.001}{0.0014320}\right| \approx 3 \times 10^{-1}$$

  - Suddenly we have a **30% error**, even though our input numbers were reasonable representations of the true values
  - Such **loss of precision** can easily happen in the middle of some long, complicated computation, and then all subsequent computations will end up with a large error.
  - Another common term for this is **catastrophic cancellation**
- Note that when discussing errors, we are usually most interested in the *relative* error:

- Example from project 1:
  - Error: $\Delta = |v_i - u_i|$
  - Relative error: $\epsilon = |\frac{v_i - u_i}{u_i}|$
  - It may be useful to e.g. study plots or tables of $\log_{10}(\epsilon)$ vs $\log_{10}(h)$
- Typical case for us:
  - If the step size is large: truncation error dominates
  - If the step size is tiny: roundoff errors lead to loss of precision → garbage results
  - So we expect that there is some optimal, intermediate step size that gives the smallest overall error

An example error analysis

> This topic includes a small in-lecture code discussion, using the `error_analysis` code example.

- Consider the function $u(x) = e^{2x}$
  - (We choose this example function just because it's trivial to differentiate many times, and the 2 in the exponent ensures that all the derivatives are not exactly equal.)
- We will use our familiar expression

$$\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$$

  to implement a computer program that computes (an approximation to) the true second derivative u_i'' at a given point x_i
- *Our question*: How do we expect that the *relative error* of our code output will depend on our choice of step size h?
- First of all, we know the exact answer for u_i'':

$$u_i'' = 4e^{2x_i}$$

- In what follows we will first consider the **error** (in absolute terms), and then later the **relative error**

- Error (signed):

$$\Delta(h) \equiv \text{approx.} - \text{true} = \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} - u_i''$$

- Relative error (signed):

$$\epsilon(h) \equiv \frac{\text{approx.} - \text{true}}{\text{true}} = \frac{\Delta(h)}{u_i''}$$

- Now, as a first step towards answering our question, let's construct a simple *model* for the total error $\Delta(h)$
- We will assume that $\Delta(h)$ is the sum of two contributions, namely a truncation error $\Delta_{\text{tr}}(h)$ and a roundoff error $\Delta_{\text{ro}}(h)$:

$$\Delta(h) = \Delta_{\text{tr}}(h) + \Delta_{\text{ro}}(h)$$

> *Side note:* Why are we not using the common "combine errors in quadrature' ' recipie, e.g. $\sigma_{x+y}^2 = \sigma_x^2 + \sigma_y^2$? That expression arises from the rule for *adding variances in probability theory*, assuming no covariance between x and y. It would be appropriate to use this if we used probability distributions to model our ignorance about each of the error contributions, and then wanted a combined probability distribution to express our ignorance of the combined error. So for instance, if we had said that the truncation error Δ_{tr} is a stochastic/uncertain variable with a Gaussian probability distribution $\mathcal{N}(0, \sigma_{\text{tr}}^2)$, and similarly, that Δ_{ro} is a stochastic/uncertain variable with a Gaussian probability distribution $\mathcal{N}(0, \sigma_{\text{ro}}^2)$, then we would have that our resulting probability distribution for the combined error Δ is a Gaussian distribution $\mathcal{N}(0, \sigma^2)$ with variance $\sigma^2 = \sigma_{\text{tr}}^2 + \sigma_{\text{ro}}^2$.
>
> However, here we are not modelling these errors probabilistically. Instead we are interested in finding an upper bound on the error we can make (or rather, an estimated upper bound).

- We will not care about the sign of the error, just the magnitude $|\Delta(h)|$. We can use the triangle inequality to express an upper bound (a worst-case bound) on $|\Delta(h)|$ using the magnitudes of $\Delta_{\text{tr}}(h)$ and $\Delta_{\text{ro}}(h)$:

$$|\Delta(h)| = |\Delta_{\text{tr}}(h) + \Delta_{\text{ro}}(h)| \leq |\Delta_{\text{tr}}(h)| + |\Delta_{\text{ro}}(h)|$$

- Let's first look at the truncation error:

$$\begin{aligned}\Delta_{\text{tr}}(h) &= \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} - u_i'' \\ &= \left(\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}\right) - \left(\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + \mathcal{O}\left(u_i^{(4)} h^2\right)\right) \\ &= \mathcal{O}\left(u_i^{(4)} h^2\right)\end{aligned}$$

  - Note that here we have included in our big-O notation that the leading term is not only proportional to h^2, but also to the fourth derivative, $u_i^{(4)}$. (To see this, go back to our derivation of the discretised expression for the second derivative.)
  - It is useful here to keep track of this dependence on the fourth derivative, since for other choices of the example function $u(x)$ the different-order derivatives at x_i could have vastly different values — or indeed be zero, if our $u(x)$ was a low-order polynomial.

- Now let's look at the roundoff error:

  - What we *want* to compute is

$$\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$$

    or to rewrite it slightly,

$$\frac{(u_{i+1} - u_i) - (u_i - u_{i-1})}{h \times h}$$

  - However, the computation we *actually* end up performing on the computer is something like this:

$$fl\left[\frac{fl\Big[fl\Big(fl(u_{i+1}) - fl(u_i)\Big) - fl\Big(fl(u_i) - fl(u_{i-1})\Big)\Big]}{fl\Big(fl(h) \times fl(h)\Big)}\right]$$

  - Consider the limit of small h and focus on the subtractions of near identical numbers

$$fl(u_{i+1}) - fl(u_i)$$

    and similar for $fl(u_i) - fl(u_{i-1})$

  - Recall:

$$a(1 - \delta_m) < fl(a) < a(1 + \delta_m)$$

- We can then estimate an upper bound for the result of the subtraction

$$\begin{aligned} fl(u_{i+1}) - fl(u_i) &\leq u_{i+1}(1+\delta_m) - u_i(1-\delta_m) \\ &= (u_{i+1} - u_i) + (u_{i+1} + u_i)\delta_m \end{aligned}$$

- In the limit $h \to 0$, i.e. when $u_{i+1} \to u_i$, the first parenthesis vanishes, but the second parenthesis does not
- So we are left with

$$fl(u_{i+1}) - fl(u_i) \leq \mathcal{O}\left(u_i \delta_m\right)$$

- *What this means*: While we know that the *true* value of $u_{i+1} - u_i$ goes to 0 when $h \to 0$, we have no guarantee that our actual computation $fl(u_{i+1}) - fl(u_i)$ will go to exactly 0 in this limit.
- Assuming that this subtraction is the most "dangerous" part in our computation of

$$\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$$

  we can estimate the roundoff error Δ_{ro} to be

$$\Delta_{\text{ro}}(h) = \mathcal{O}\left(\frac{u_i \delta_m}{h^2}\right)$$

- We can now put everything together in our bound for the combined error:

$$\begin{aligned} |\Delta(h)| &\leq |\Delta_{\text{tr}}(h)| + |\Delta_{\text{ro}}(h)| \\ &= \left|\mathcal{O}\left(u_i^{(4)} h^2\right)\right| + \left|\mathcal{O}\left(\frac{u_i \delta_m}{h^2}\right)\right| \end{aligned}$$

- Our model for the magnitude of the *relative* error ϵ in our computation of u_i'' then becomes

$$\begin{aligned} |\epsilon(h)| &= \left|\frac{\Delta(h)}{u_i''}\right| \\ &\leq \left|\mathcal{O}\left(\frac{u_i^{(4)}}{u_i''} h^2\right)\right| + \left|\mathcal{O}\left(\frac{u_i \delta_m}{u_i''} \frac{1}{h^2}\right)\right| \end{aligned}$$

- The first term grows when h *increases*, while the second term grows when h *decreases*
- For our choice of example function we also know that $\mathcal{O}\left(u_i\right) \approx \mathcal{O}\left(u_i''\right) \approx \mathcal{O}\left(u_i^{(4)}\right)$, so the

factors $\frac{u_i^{(4)}}{u_i''}$ and $\frac{u_i}{u_i''}$ won't suppress or enlarge the error terms much

- Let's now look at $\log_{10}|\epsilon(h)|$
  - On the right-hand side:
    * The truncation error $\mathcal{O}\left(\frac{u_i^{(4)}}{u_i''}h^2\right)$ hides a series in even powers of h. We will just keep the largest term, which is the h^2 term. (The step size is a number in $(0,1)$, that is, we assume we have scaled away any units in the original problem.)
    * Let's then collect all the factors except h in two positive constants C_1 and C_2.
    * Then we can write

$$\log_{10}|\epsilon(h)| \leq \log_{10}\left(C_1 h^2 + C_2 h^{-2}\right)$$

  - Look at the behaviour in the limits $h \to \infty$ (first term dominates) and $h \to 0$ (second term dominates)

$$\log_{10}|\epsilon(h)| \leq \begin{cases} -2\log_{10} h + \log_{10} C_2 & \text{for } h \to 0, \text{ i.e. } \log_{10} h \to -\infty \\ 2\log_{10} h + \log_{10} C_1 & \text{for } h \to \infty, \text{ i.e. } \log_{10} h \to \infty \end{cases}$$

  - Note that these are the equations for two straight lines (slopes 2 and -2) in a plot of $\log_{10}\epsilon(h)$ vs $\log_{10} h$
  - So we see that our model for the error suggests the qualitative behaviour we expected, namely that there should be some *optimal, intermediate choice for the step size* that gives the smallest overall error
  - We also see that we get a quantitative prediction for how quickly the error will grow when we move far away from the optimal step size choice

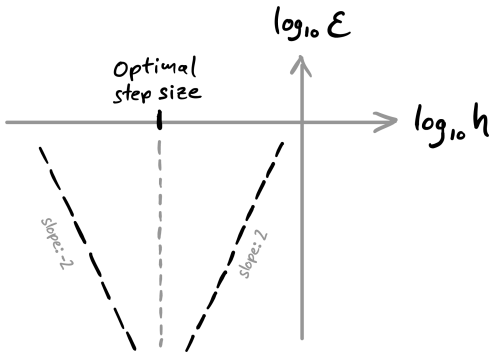


Figure 7: Sketch of a $\log_{10}\epsilon(h)$ vs $\log_{10} h$ plot, as suggested by our simple error model

Recap: solving matrix equations

- We have discussed how to solve matrix equations $\mathbf{A}\vec{x} = \vec{b}$
  - (We used the notation $\mathbf{A}\vec{v} = \vec{g}$ in project 1)
- **Gaussian elimination**:
  - Can be used to solve $\mathbf{A}\vec{x} = \vec{b}$ for a *general* (dense) $\mathbf{A}$
  - In that case it requires $\mathcal{O}\left(n^3\right)$ FLOPs, or more accurately $\mathcal{O}\left(\frac{2}{3}n^3\right)$
  - We have only looked at the special case for a *tridiagonal* $\mathbf{A}$ (more efficient)
- Next up: **LU decomposition**
- Later: **Iterative methods**

Classification of methods for solving matrix equations

Direct methods

- Examples:
  - Gaussian elimination
  - LU decomposition
- In theory, these methods give the *exact* answer in a *finite number of steps*
- In practice, these methods can suffer from numerical instabilities
- They typically work with the entire matrix at once
  → keeps the full matrix stored in memory

Indirect methods

- Examples:
  - Jacobi's iterative method
  - Gauss-Seidel
  - Relaxation methods
- *Iterate* closer and closer to the exact answer, but *will never get there exactly*
- Can often work without keeping the full matrix in memory
- Are often less susceptible to roundoff errors

Lower-upper (LU) decomposition

- Also commonly known as **lower-upper (LU) factorisation**
- We will introduce LU decomposition as an approach for solving $\mathbf{A}\vec{x} = \vec{b}$
- Actually a starting point for several different matrix tasks, as we will see
- Our plan:

  1. What is LU decomposition?
  2. What is it good for? (And what's the difficulty?)
  3. An algorithm for LU decomposition

1. What is LU decomposition?

- We will only consider *square* matrices
- A matrix $\mathbf{A}$ is said to admit an **LU decomposition** if it can be written as a product of a lower-triangular matrix ($\mathbf{L}$) and an upper-triangular matrix ($\mathbf{U}$)

$$\mathbf{A} = \mathbf{LU}$$

$$\begin{bmatrix} \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet \end{bmatrix} = \begin{bmatrix} \bullet & & \\ \bullet & \bullet & \\ \bullet & \bullet & \bullet \end{bmatrix} \begin{bmatrix} \bullet & \bullet & \bullet \\ & \bullet & \bullet \\ & & \bullet \end{bmatrix}$$

- Consider 3×3 example:
  - $\mathbf{A}$ contains 9 elements a_{ij}
  - $\mathbf{L}$ contains 6 elements l_{ij} and $\mathbf{U}$ contains 6 elements u_{ij}
  - So the relation $\mathbf{A} = \mathbf{LU}$ implies 9 equations (one for each known element a_{ij}) involving 12 unknowns (the l_{ij}'s and u_{ij}'s)
  - This is an *underdetermined* (underconstrained) set of equations (infinitely many solutions)
  - We can choose 3 elements to get a unique solution
  - Common to set the diagonal elements of $\mathbf{L}$ to 1

$$\mathbf{L} = \begin{bmatrix} 1 & & \\ \bullet & 1 & \\ \bullet & \bullet & 1 \end{bmatrix}$$

- If $\mathbf{A} = \mathbf{LU}$ and all the diagonal elements of $\mathbf{L}$ are 1's, the matrix $\mathbf{A}$ can also be factorised in the form $\mathbf{A} = \mathbf{LDU}'$, where
  - $\mathbf{D}$ is a diagonal matrix with the diagonal elements u_{ii} of the original $\mathbf{U}$ matrix
  - $\mathbf{U}'$ is the matrix generated by taking $\mathbf{U}$ and multiplying each row i with $\frac{1}{u_{ii}}$, so that both $\mathbf{L}$ and $\mathbf{U}'$ have only 1's along their diagonals

$$\mathbf{A} = \mathbf{LDU}'$$

$$\begin{bmatrix} \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet \end{bmatrix} = \begin{bmatrix} 1 & & \\ \bullet & 1 & \\ \bullet & \bullet & 1 \end{bmatrix} \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} \begin{bmatrix} 1 & \bullet & \bullet \\ & 1 & \bullet \\ & & 1 \end{bmatrix}$$

    * This is (unsurprisingly) called **LDU decomposition** or **LDU factorisation**

- Computational complexity:
  - It takes $\mathcal{O}\left(\frac{2}{3}n^3\right)$ operations to determine $\mathbf{L}$ and $\mathbf{U}$ for a given $\mathbf{A}$ $(n \times n)$
  - So the computational complexity of performing the decomposition $\mathbf{A} = \mathbf{LU}$ is the same as that of solving $\mathbf{A}\vec{x} = \vec{b}$ with Gaussian elimination
  - LU decomposition can be seen as the matrix representation of Gaussian elimination

- Existence:
  - If a square matrix $\mathbf{A}$
    * is *non-singular* (invertible), and
    * *all* its *leading principal minors* are non-zero (see note below)

    then it admits an LU (or LDU) decomposition
  - If a square matrix $\mathbf{A}$
    * is *singular* (not invertible),
    * has rank k, and
    * *the first* k of the leading principal minors are non-zero

    then it admits an LU (or LDU) decomposition
  - (For more details about this, see textbooks on linear algebra)

> *Side note*: The *leading principal minors* of $\mathbf{A}$ are the *determinants* of the square submatrices you get from $\mathbf{A}$ if you start in the upper left-hand corner and grow the submatrix by one row and column at a time

- Example: The leading principal minors of the matrix

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

are the determinants

$$\det \begin{bmatrix} a \end{bmatrix}, \det \begin{bmatrix} a & b \\ d & e \end{bmatrix} \text{ and } \det \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

2. What is it good for?

- Assume we *have performed* the LU decomposition
- We can now
  - solve **matrix equations**, $\mathbf{A}\vec{x} = \vec{b}$, at $\mathcal{O}(n^2)$ cost
  - easily compute **the determinant**, $\det \mathbf{A}$, at $\mathcal{O}(n)$ cost
  - find **the inverse**, $\mathbf{A}^{-1}$, at $\mathcal{O}(n^3)$ cost
    * Finding $\mathbf{A}^{-1}$ would have cost $\mathcal{O}(n^4)$ if we did it by treating $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ as n equations of the form $\mathbf{A}\vec{x} = \vec{b}$ and solved each one with Gaussian elimination

Solving matrix equations after LU decomposition

- We have $\mathbf{A} = \mathbf{L}\mathbf{U}$
- Want to solve $\mathbf{A}\vec{x} = \vec{b}$ for $\vec{x}$
- We will solve $\mathbf{A}\vec{x} = \mathbf{L}\mathbf{U}\vec{x} = \vec{b}$ in two steps
  - First we define some notation: $\vec{w} \equiv \mathbf{U}\vec{x}$.
    * Since $\vec{x}$ is unknown, $\vec{w}$ is unknown.
    * We can now write $\mathbf{L}\mathbf{U}\vec{x} = \mathbf{L}\vec{w} = \vec{b}$

  1. Solve $\mathbf{L}\vec{w} = \vec{b}$ for $\vec{w}$
  2. Solve $\mathbf{U}\vec{x} = \vec{w}$ for $\vec{x}$

- **Step 1**: Solve $\mathbf{L}\vec{w} = \vec{b}$ for $\vec{w}$

- Consider example with 4×4 matrices

$$\begin{bmatrix} l_{11} & 0 & 0 & 0 \\ l_{21} & l_{22} & 0 & 0 \\ l_{31} & l_{32} & l_{33} & 0 \\ l_{41} & l_{42} & l_{43} & l_{44} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

- Solve by forward substitution:
  - From $l_{11}w_1 = b_1$ we immediately get the solution for w_1:

$$w_1 = \frac{1}{l_{11}} b_1$$

  - From $l_{21}w_1 + l_{22}w_2 = b_2$, and using the now known w_1, we get

$$w_2 = \frac{1}{l_{22}} \Big[b_2 - l_{21}w_1 \Big]$$

  - Continuing the same way, we get

$$\begin{aligned} w_3 &= \frac{1}{l_{33}} \Big[b_3 - l_{31}w_1 - l_{32}w_2 \Big] \\ w_4 &= \frac{1}{l_{44}} \Big[b_4 - l_{41}w_1 - l_{42}w_2 - l_{43}w_3 \Big] \end{aligned}$$

  - In general, when $\mathbf{A}$ is $n \times n$:

$$\boxed{w_i = \frac{1}{l_{ii}} \Big[b_i - \sum_{j=1}^{i-1} l_{ij}w_j \Big]}$$

  - Counting FLOPs (here we assume $l_{ii} = 1$):

$$\sum_{i=1}^{n} (2i - 1) = n^2$$

    (which is less than the $\mathcal{O}\left(n^3\right)$ cost of doing the LU decomposition in the first place)
- Now we know $\vec{w}$ and can do step 2

- **Step 2**: Solve $\mathbf{U}\vec{x} = \vec{w}$ for $\vec{x}$

$$\begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}$$

- Solve for $\vec{x}$ by back substitution
  - From $u_{44}x_4 = w_4$ we immediately get the solution for x_4:

$$x_4 = \frac{1}{u_{44}} w_4$$

  - From $u_{33}x_3 + u_{34}x_4 = w_3$ we then get

$$x_3 = \frac{1}{u_{33}} \Big[w_3 - u_{34}x_4 \Big]$$

  - And so on…

$$x_2 = \frac{1}{u_{22}} \Big[w_2 - u_{23}x_3 - u_{24}x_4 \Big]$$
$$x_1 = \frac{1}{u_{11}} \Big[w_1 - u_{12}x_2 - u_{13}x_3 - u_{14}x_4 \Big]$$

  - In general, when $\mathbf{A}$ is $n \times n$:

$$\boxed{\begin{aligned} x_n &= \frac{1}{u_{nn}} w_n \\ x_i &= \frac{1}{u_{ii}} \Big[w_i - \sum_{j=i+1}^{n} u_{ij}x_j \Big], \quad i = (n-1), (n-2), \ldots, 1 \end{aligned}}$$

  - This also takes $\mathcal{O}\left(n^2\right)$ FLOPs, so the combined task of forward + back substitution to find $\vec{x}$ has an $\mathcal{O}\left(n^2\right)$ cost.

- In summary:
  - If we already have $\mathbf{A} = \mathbf{LU}$, we can solve $\mathbf{A}\vec{x} = \vec{b}$ at a total $\mathcal{O}\left(n^2\right)$ cost as follows:
    1. From $\mathbf{L}\vec{w} = \vec{b}$, find $\vec{w}$ by forward substitution
    2. From $\mathbf{U}\vec{x} = \vec{w}$, find $\vec{x}$ by back substitution

A difficulty

- We need to store the full matrix $\mathbf{A}$ $(n \times n)$ in memory for the LU decomposition
  - That's n^2 floating-point numbers
  - At double precision (64 bits = 8 bytes per number), this requires $n^2 \times 8$ bytes of memory
  - Example:
    * Assume $n = 10^4$
    * We then need 8×10^8 bytes $\approx 10^9$ bytes $= 1$ GB of memory
    * So we can quite quickly run out of memory
  - Also, since the decomposition is an $\mathcal{O}\left(n^3\right)$ operation, it will be slow when n is large

Finding the determinant after LU decomposition

- Once we have the decomposition $\mathbf{A} = \mathbf{LU}$, computing the determinant of $\mathbf{A}$ is trivial:

$$\begin{aligned}\det(\mathbf{A}) &= \det(\mathbf{LU}) \\ &= \det(\mathbf{L})\det(\mathbf{U}) \\ &= (1)(u_{11}u_{22}\dots u_{nn})\end{aligned}$$

  where we have assumed that $\mathbf{L}$ is on the standard form with 1's on the diagonal

- So in summary:

$$\boxed{\det(\mathbf{A}) = \prod_{i=1}^{n} u_{ii}}$$

  or equivalently

$$\boxed{\log\Big(\det(\mathbf{A})\Big) = \sum_{i=1}^{n} \log u_{ii}}$$

- For large matrices it is often useful to work numerically with the logarithm of the determinant, rather than the determinant itself

Finding the inverse after LU decomposition

- Once we have $\mathbf{A} = \mathbf{LU}$, we can find $\mathbf{A}^{-1}$ at $\mathcal{O}\left(n^3\right)$
- We know

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I} = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & & 1 \end{bmatrix} = \mathbf{A}\mathbf{A}^{-1}$$

- Write $\mathbf{A}^{-1}$ as column vectors

$$\mathbf{A}^{-1} = \begin{bmatrix} \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix} \dots \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix} \end{bmatrix} = \begin{bmatrix} \vec{\alpha}_1 & \vec{\alpha}_2 & \vec{\alpha}_3 & \vec{\alpha}_4 \end{bmatrix}$$

  where we have used a notation where e.g. $\vec{\alpha}_1$ contains the first column of $\mathbf{A}^{-1}$

$$\vec{\alpha}_1 \equiv \begin{bmatrix} (\mathbf{A}^{-1})_{11} \\ (\mathbf{A}^{-1})_{21} \\ (\mathbf{A}^{-1})_{31} \\ (\mathbf{A}^{-1})_{41} \end{bmatrix}$$

- Using $\mathbf{A} = \mathbf{LU}$ and our notation for $\mathbf{A}^{-1}$ we then have

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{LU}\begin{bmatrix} \vec{\alpha}_1 & \vec{\alpha}_2 & \vec{\alpha}_3 & \vec{\alpha}_4 \end{bmatrix} = \mathbf{I} = \begin{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \end{bmatrix}$$

- We can read this as four matrix equations, $(\mathbf{LU})\vec{\alpha}_i = \hat{e}_i$, where $\hat{e}_i$ are the unit vectors and $\vec{\alpha}_i$ are the unknown vectors we want to determine:

$$(\mathbf{LU})\vec{\alpha}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad \dots \quad (\mathbf{LU})\vec{\alpha}_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

- Note that each matrix equation is on the familiar form $(\mathbf{LU})\vec{x} = \vec{b}$, which we have already seen how to solve
- This was for a 4×4 example. In the general case, with $\mathbf{A}$ being $n \times n$:
  - If we have $\mathbf{A} = \mathbf{LU}$, then finding $\mathbf{A}^{-1}$ requires solving n matrix equations on the form $(\mathbf{LU})\vec{x} = \vec{b}$
  - Solving each equation has an $\mathcal{O}\left(n^2\right)$ cost
  - Thus, we can find $\mathbf{A}^{-1}$ in $\mathcal{O}\left(n^3\right)$ FLOPs
  - Finding the decomposition $\mathbf{A} = \mathbf{LU}$ in the first place also had an $\mathcal{O}\left(n^3\right)$ cost
  - So the **total cost of LU decomposition + finding $\mathbf{A}^{-1}$ is** $\mathcal{O}\left(n^3\right)$
  - (Recall that finding $\mathbf{A}^{-1}$ would have required $\mathcal{O}\left(n^4\right)$ FLOPs if we had naively solved each of the n equations $\mathbf{A}\vec{\alpha}_i = \hat{e}_i$ "from scratch" using Gaussian elimination)

Finding the inverse of a large matrix can be the main bottleneck in many applications, e.g. in various methods in machine learning. In such cases matrix decomposition techniques like LU decomposition, or the related *Cholesky decomposition* $\mathbf{A} = \mathbf{LL}^*$, typically play a key role.

3. An algorithm for LU decomposition

- How to determine the elements of $\mathbf{L}$ and $\mathbf{U}$ such that $\mathbf{A} = \mathbf{LU}$?
- Consider a 4×4 case:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & \bullet & \bullet & \bullet \\ a_{31} & \bullet & \bullet & \bullet \\ a_{41} & \bullet & \bullet & \bullet \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ l_{21} & 1 & 0 & 0 \\ l_{31} & l_{32} & 1 & 0 \\ l_{41} & l_{42} & l_{43} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{bmatrix}$$

- Let's look at the first column of $\mathbf{A}$, i.e. the elements a_{i1}:
  - From a_{11} we get:

$$a_{11} = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_{11} \\ 0 \\ 0 \\ 0 \end{bmatrix} = u_{11}$$

$$\boxed{u_{11} = a_{11}}$$

- From a_{21} we get:

$$a_{21} = \begin{bmatrix} l_{21} & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_{11} \\ 0 \\ 0 \\ 0 \end{bmatrix} = l_{21} u_{11}$$

$$\boxed{l_{21} = \frac{a_{21}}{u_{11}} \quad \text{(where } u_{11} \text{ is now known)}}$$

- Similarly, from the equations $a_{31} = l_{31} u_{11}$ and $a_{41} = l_{41} u_{11}$ we get

$$\boxed{l_{31} = \frac{a_{31}}{u_{11}}}$$

$$\boxed{l_{41} = \frac{a_{41}}{u_{11}}}$$

- Now for the second column $\mathbf{A}$ (elements a_{i2}):
  - From a_{12}:

$$a_{12} = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_{12} \\ u_{22} \\ 0 \\ 0 \end{bmatrix} = u_{12}$$

$$\boxed{u_{12} = a_{12}}$$

  - From a_{22}:

$$a_{22} = \begin{bmatrix} l_{21} & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_{12} \\ u_{22} \\ 0 \\ 0 \end{bmatrix} = l_{21} u_{12} + u_{22} \quad \text{(where } u_{22} \text{ is the unknown)}$$

$$\boxed{u_{22} = a_{22} - l_{21}u_{12}}$$

- From a_{32}:

$$a_{32} = \begin{bmatrix} l_{31} & l_{32} & 1 & 0 \end{bmatrix} \begin{bmatrix} u_{12} \\ u_{22} \\ 0 \\ 0 \end{bmatrix} = l_{31}u_{12} + l_{32}u_{22} \quad \text{(where } l_{32} \text{ is the unknown)}$$

$$\boxed{l_{32} = \frac{a_{32} - l_{31}u_{12}}{u_{22}}}$$

- Similarly, from $a_{42} = l_{42}u_{22} + l_{41}u_{12}$:

$$\boxed{l_{42} = \frac{a_{42} - l_{41}u_{12}}{u_{22}}}$$

- We can continue like this for the third and fourth columns
- The general pattern that appears gives us the following recipe:

$$\boxed{\begin{aligned} l_{ij} &= \frac{1}{u_{jj}} \left[a_{ij} - \sum_{k=1}^{j-1} l_{ik}\, u_{kj} \right] \quad \text{with } i > j \\ u_{ij} &= a_{ij} - \sum_{k=1}^{i-1} l_{ik}\, u_{kj} \quad \text{with } i \leq j \end{aligned}}$$

Pivoting

- For numerical stability we need to avoid $u_{11} \approx 0$
- Use a permutation matrix to interchange rows
- Instead of $\mathbf{A} = \mathbf{LU}$ we will then have

$$\mathbf{A} = \mathbf{PLU}$$

or equivalently

$$\mathbf{P}^T\mathbf{A} = \mathbf{LU}$$

- Here $\mathbf{P}$ is called the *pivot matrix*
- It satisfies the relation $\mathbf{P}^T = \mathbf{P}^{-1}$, or equivalently, $\mathbf{P}\mathbf{P}^T = \mathbf{I}$

Topics in project 2

- Scaling equations
- Physics case: “the buckling beam”
  - Will lead to a two-point boundary value problem of the form

$$\begin{aligned} \frac{d^2u}{dx^2} &= -\lambda u(x) \\ x &\in [0,1] \\ u(0) &= 0 \\ u(1) &= 0 \end{aligned}$$

- Eigenvalue problems, connection to two-point boundary value problem
- Jacobi’s rotation method for eigenvalue problems
- Coding:
  - Unit testing
  - Using the Armadillo library

Scaling equations

- Also known as **nondimensionalisation** or **using natural units** (but don't confuse this with the term *natural units* as used in e.g. particle physics)
- We can only represent a limited range of numbers on a computer
- We therefore want to avoid too large/small numbers in our codes.
  - First, simple approach: choose some sensible units
  - Better approach: scale away units, i.e. work with **dimensionless** variables
  - This is also useful for debugging, since it's then easier to notice if a result is surprisingly small/large

Example 1: A simulation of the solar system

- Silly choice of units: kg, m
  - In these units we would get e.g.
    $m_{\text{sun}} \approx 1.989 \times 10^{30}$ kg
    $r_{\text{earth-sun}} \approx 1.496 \times 10^{11}$ m
- Sensible choice of units: M_{sun}, au
  - Then we would have
    $m_{\text{sun}} = 1\ \text{M}_{\text{sun}}$
    $r_{\text{earth-sun}} = 1$ au

Example 2: Exponential decay of radioactive nuclei

- Consider the differential equation

$$\frac{dN(t)}{dt} = -\lambda N(t)$$

  or, written on a standard form

$$\frac{dN(t)}{dt} + \lambda N(t) = 0$$

- Initial value: $N(t=0) = N_0$

- Units:
  - N: number of nuclei, $[N] = 1$
  - t: time, $[t] = \text{s}$
  - λ: decay constant, $[\lambda] = \text{s}^{-1}$

> For this simple example we know that the analytical solution is $N(t) = N_0 e^{-\lambda t}$

- Define a scaled, dimensionless time variable (the **independent** variable):

$$\hat{t} \equiv \lambda t$$

- We can then rewrite the time derivative as

$$\frac{d}{dt} = \frac{d\hat{t}}{dt}\frac{d}{d\hat{t}} = \lambda \frac{d}{d\hat{t}}$$

- We insert this in our original differential equation (and remember that λ is just a constant)

$$\lambda \frac{dN}{d\hat{t}} + \lambda N = 0$$

$$\frac{dN}{d\hat{t}} + N = 0$$

> - Note that once we have changed variable from t to $\hat{t}$, we should think of N as a new function, compared to the original $N(t)$.
>   - Technically we should have indicated this with some new notation, e.g. $N_{\hat{t}}(\hat{t})$
>   - This new function $N_{\hat{t}}$ is defined by the requirement $N_{\hat{t}}(\hat{t}(t)) = N(t)$
>   - So our differential equation above should more properly be written out as something like this:
>
> $$\frac{dN_{\hat{t}}(\hat{t})}{d\hat{t}} + N_{\hat{t}}(\hat{t}) = 0$$
>
>   - However, this is often not written out so explicitly
>   - It is common to just let the argument in $N(\hat{t})$ tell us that we should read this as $N_{\hat{t}}(\hat{t})$
>   - This is similar to how we often use the simple notation $p(x)$ and $p(y)$ for two different

> probability distributions, when we technically should have written something like $p_x(x)$ and $p_y(y)$, or $f(x)$ and $g(y)$

Since $\hat{t}$ is dimensionless and defined using the typical time scale $(1/\lambda)$ of our problem, we know

- that a step size $h \ll 1$ along the $\hat{t}$ axis will actually correspond to a *small* step size for this problem
- that solving our differential equation for a time span $\hat{t} \in [0, \text{a few}]$ will be a reasonable starting point for studying the problem, rather than starting with a time span of say $\hat{t} \in [0, 0.1]$ or $\hat{t} \in [0, 100]$

- In the differential equation above, both N and $\hat{t}$ are dimensionless numbers
- But the typical value for N may still be impractical, e.g. some very large number
- So we may also consider scaling the **dependent** variable (N)
- Sensible choice here: use the initial value N_0 to set a scale for N

$$\hat{N} \equiv \frac{N}{N_0}$$

- The initial value in terms of our new variable $\hat{N}$ (let's call the initial value $\hat{N}_0$) is then simply

$$\hat{N}_0 = 1$$

> In the not-so-sloppy notation, the variable change would be
>
> $$\hat{N}_{\hat{t}} \equiv \frac{N_{\hat{t}}}{N_0}$$
>
> and the initial value relation would be
>
> $$\hat{N}_0 \equiv \frac{N_{\hat{t}}(\hat{t}(t=0))}{N_0} = \frac{N(t=0)}{N_0} = \frac{N_0}{N_0} = 1$$

- If we insert $N = N_0\hat{N}$ in our differential equation, we get

$$N_0 \frac{d\hat{N}}{d\hat{t}} + N_0\hat{N} = 0$$

or simply

$$\boxed{\frac{d\hat{N}}{d\hat{t}} + \hat{N} = 0}$$

In this final form of the differential equation, all quantities are dimensionless, and both the *independent* variable $\hat{t}$ and the *dependent* variable $\hat{N}$ have typical values of $\mathcal{O}(1)$.

- The analytical solution for our scaled differential equation is simply

$$\hat{N}(\hat{t}) = \hat{N}_0 e^{-\hat{t}} = e^{-\hat{t}} \quad \text{(recall that } \hat{N}_0 = 1\text{)}$$

- Using our definitions $\hat{N} = \frac{N}{N_0}$ and $\hat{t} = \lambda t$, we can of course get the solution in terms of our original variables N and t:

$$\frac{N}{N_0} = e^{-\lambda t}$$

$$N(t) = N_0 e^{-\lambda t}$$

> In the more careful notation, we get back to our original variables N and t by using
>
> $$\hat{t} = \lambda t$$
> $$\hat{N}_{\hat{t}}(\hat{t}(t)) = \frac{N_{\hat{t}}(\hat{t}(t))}{N_0} = \frac{N(t)}{N_0}$$
>
> which again leads to the solution
>
> $$N(t) = N_0 e^{-\lambda t}$$
>
> in terms of the original variables

Another reason why it is useful to do our numerical work in terms of scaled equations, is that it makes it easier to notice potential code bugs simply by looking out for surprisingly large (or small) outputs. If you know that all the quantities in your differential equation are $\mathcal{O}(1)$ numbers, but your code suddenly outputs some huge value, like $\mathcal{O}(10^{10})$, you immediately recognise that there may be a bug somewhere.

Presenting results

- When presenting our results, we should either *use the original, dimensionful variables*, or *specify the natural units used*

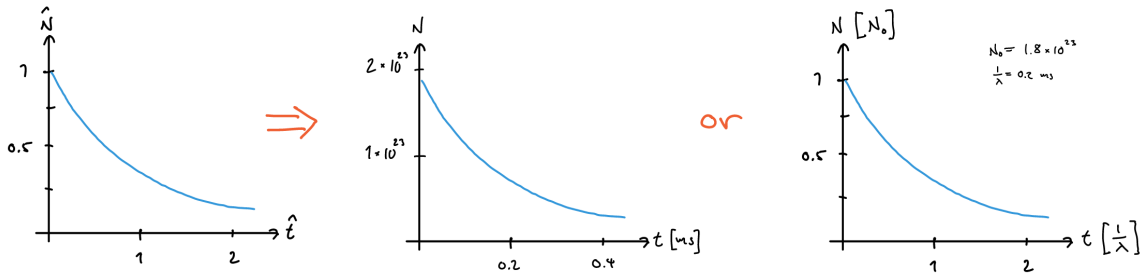


Figure 8: Our raw numerical solution will be like the left-hand plot, but the result we present should be like the middle or right-hand plot

Physics case for project 2: the buckling beam

- Description of the setup:
  - A beam of some material is extended between two points $x = 0$ and $x = L$
  - The function $u(x)$ denotes the shape of the beam (displacement from the x axis)
  - A force F is applied at the endpoint $x = L$, directed *into* the beam (i.e. along the x axis)
  - The material properties of the beam are collected in a constant γ
  - The beam can not be displaced from the x axis at the end points:
    - Boundary conditions: $u(0) = 0$ and $u(L) = 0$
  - We'll consider a so-called *pin endpoints* case:
    - The beam shape $u(x)$ can have non-zero derivative $u'(x)$ at $x = 0$ and $x = L$
- If the force F is large enough, the configuration is *unstable*
  - Any tiny perturbation will cause the beam to *buckle* (bend) into some shape
- **Our question**: What beam shapes (i.e. what function forms $u(x)$) can arise?
- We are considering this as a *static* problem — there is no time dependence here
  - We are looking for what static beam shapes are theoretically allowed under the conditions described above
- Differential equation:

$$\boxed{\gamma \frac{d^2u}{dx^2} = -Fu(x)}$$

- Do the following steps:
  - Scale the equation, to use dimensionless position $\hat{x} \equiv \frac{x}{L}$
  - Define new notation: $\lambda_c \equiv \frac{FL^2}{\gamma}$
  - Discretise with n *steps* (so we get $n + 1$ points *including endpoints*, which means we have $n - 1$ *interior* points)
- As you will see when you work on the project, after the above steps we end up with the following **eigenvalue problem**:

$$\boxed{\mathbf{A}\vec{v} = \lambda\vec{v}}$$

- $\mathbf{A}$ has size $N \times N = (n-1) \times (n-1)$
- $\lambda = \lambda(h)$, which should go to λ_c in the limit $h \to 0$
- The elements v_i of $\vec{v}$ are approximations to the exact $u_i = u(\hat{x}_i)$
- $\vec{v}$ contains the *interior* points:

$$\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ \dots \\ v_N \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \\ \dots \\ v_{n-1} \end{bmatrix}$$

- Our complete, approximate solution to the BVP: $\vec{v}^* = [v_0, v_1, v_2, \dots, v_{n-1}, v_n]$
- The solutions we find for $\mathbf{A}\vec{v} = \lambda\vec{v}$ are eigenvector-eigenvalue pairs, $(\vec{v}^{(i)}, \lambda^{(i)})$
- In the continuous limit ($h \to 0$, i.e. $n \to \infty$) these correspond to eigenvalues $\lambda_c^{(i)}$ and *eigenfunctions* $u^{(i)}(\hat{x})$
- Like in project 1 we have that $\mathbf{A}$ is a tridiagonal matrix (due to the second derivative), but this time we keep the factor $1/h^2$ from the second derivative as part of $\mathbf{A}$
- 4×4 example:

$$\mathbf{A} = \begin{bmatrix} \frac{2}{h^2} & -\frac{1}{h^2} & 0 & 0 \\ -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} & 0 \\ 0 & -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} \\ 0 & 0 & -\frac{1}{h^2} & \frac{2}{h^2} \end{bmatrix} = \frac{1}{h^2}\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}$$

- Keeping $\frac{1}{h^2}$ on the left-hand side makes it easier to see how our approach would work also for a more general case, where we had a more complicated differential equation:

$$\Big[\text{some general operator}\Big]u(x) = \lambda u(x)$$

$$\left[\frac{d^2u}{dx^2} + \dots\right]u(x) = \lambda u(x)$$

Eigenvalue problems

- See Chapters 7.1 – 7.4 in Morten's notes.
- Actually **eigenvalue and eigenvector** problems, or **eigensystems**
- Find pairs of $(\lambda, \vec{x})$ that satisfy

$$\mathbf{A}\vec{x} = \lambda\vec{x}$$

- Sometimes only interested in one of them
  - Example from quantum mechanics: might be interested in only the allowed energies (eigenvalues), not the wavefunctions (eigenvectors)
- First consider the standard analytical approach:

$$\begin{aligned}\mathbf{A}\vec{x} &= \lambda\vec{x} \\ (\mathbf{A} - \lambda\mathbf{I})\vec{x} &= 0\end{aligned}$$

- A matrix eq. $\mathbf{M}\vec{x} = 0$ has a non-trivial solution iff $\det(\mathbf{M}) = 0$
- So look at $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$
- $\det(\mathbf{A} - \lambda\mathbf{I})$ is a polynomial of degree N in λ
  - The *characteristic polynomial*:

$$P(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I}) = (\lambda_1 - \lambda)(\lambda_2 - \lambda)\dots$$

  - The eigenvalues are the N roots of $P(\lambda)$
  - The set of roots (eigenvalues) of $\mathbf{A}$ is called *the spectrum*, denoted as

$$\lambda(\mathbf{A}) = \{\lambda_1, \lambda_2, \dots, \lambda_N\}$$

    * The determinant of $\mathbf{A}$ is the product of the eigenvalues:

$$\det(\mathbf{A}) = \lambda_1\lambda_2\dots\lambda_N$$

- This is all nice analytically, but inefficient to compute $\det(\mathbf{A} - \lambda\mathbf{I})$ when N is large
- We will instead focus on **Jacobi's rotation method**

  - Classic algorithm
  - Reasonably intuitive
  - Suitable for symmetric, real matrices
  - Gives us an example of an iterative method
  - Not very efficient, especially for large matrices
  - But well-suited for parallelisation

- Many other methods exist, with different strengths and weaknesses. A few examples:
  - *QR algorithm* (general purpose)
  - *Power iteration* (largest eigenvalue)
  - *Inverse iteration* (smallest eigenvalue)
  - *Shift method* (eigenvalue closest to a given value)

- A basis for Jacobi's rotation method (and other methods) is **similarity transformations**

Similarity transformations

- Transformations with an orthogonal matrix $\mathbf{S}$:

$$\mathbf{S}^T = \mathbf{S}^{-1}$$
$$\mathbf{S}^T\mathbf{S} = \mathbf{S}\mathbf{S}^T = \mathbf{I}$$

- We will assume $\mathbf{A}$ is **real** and **symmetric**
- A is $N \times N$, with N eigenvalues: $\lambda^{(1)}, \lambda^{(2)}, \ldots, \lambda^{(N)}$
  - The eigenvalues are not necessarily distinct
- From **the spectral theorem**: Then there exists an orthogonal matrix $\mathbf{S}$ such that

$$\mathbf{S}^T\mathbf{A}\mathbf{S} = \mathbf{D} \quad \text{(diagonal matrix)}$$

- So $\mathbf{S}$ diagonalises $\mathbf{A}$, but how is this connected to our eigenvalue-eigenvector task?
  - Apply $\mathbf{S}$ from the left

$$\mathbf{A}\mathbf{S} = \mathbf{S}\mathbf{D}$$

  - Express $\mathbf{S}$ as column vectors:

$$\mathbf{S} = \begin{bmatrix} \vec{s}_1 & \vec{s}_2 & \dots & \vec{s}_N \end{bmatrix}$$

  - Then $\mathbf{AS} = \mathbf{SD}$ becomes

$$\begin{aligned} \mathbf{A}\begin{bmatrix} \vec{s}_1 & \vec{s}_2 & \dots & \vec{s}_N \end{bmatrix} &= \begin{bmatrix} \vec{s}_1 & \vec{s}_2 & \dots & \vec{s}_N \end{bmatrix}\mathbf{D} \\ &= \begin{bmatrix} d_{11}\vec{s}_1 & d_{22}\vec{s}_2 & \dots & d_{NN}\vec{s}_N \end{bmatrix} \end{aligned}$$

  - This is a set of N equations of a familiar form:

$$\begin{aligned} \mathbf{A}\vec{s}_1 &= d_{11}\vec{s}_1 \\ \mathbf{A}\vec{s}_2 &= d_{22}\vec{s}_2 \\ &\dots \\ \mathbf{A}\vec{s}_N &= d_{NN}\vec{s}_N \end{aligned}$$

  - We see that:
    * The elements of $\mathbf{D}$ are the **eigenvalues** of $\mathbf{A}$
    * The column vectors in $\mathbf{S}$ are the **eigenvectors** of $\mathbf{A}$

- Idea behind Jacobi's rotation method:
  - Find $\mathbf{S}$ by iteratively applying similarity transformations (rotations) $\mathbf{S}_1, \mathbf{S}_2, \dots$, to $\mathbf{A}$

$$\mathbf{A} \to \mathbf{S}_1^T\mathbf{A}\mathbf{S}_1 \to \mathbf{S}_2^T\mathbf{S}_1^T\mathbf{A}\mathbf{S}_1\mathbf{S}_2 \to \dots$$

    such that each new transformation makes the resulting matrix a little more diagonal-like

  - Once our transformed matrix is sufficiently close to being diagonal, the product of all our applied transformations is our approximation to $\mathbf{S}$

$$\mathbf{S} \approx \mathbf{S}_1\mathbf{S}_2 \dots$$

> *Side note*: Similarity transformations preserve eigenvalues. If we apply some similarity transformation $\mathbf{Q}$ to the equation $\mathbf{A}\vec{x} = \lambda\vec{x}$, we transform the equation into an equation with
>
> - a transformed matrix, $\mathbf{A} \to \mathbf{Q}^T\mathbf{A}\mathbf{Q}$
> - transformed eigenvectors, $\vec{x} \to \mathbf{Q}^T\vec{x}$

> - but the *same* eigenvalues, λ
>
> To see this, just left-multiply both sides with $\mathbf{Q}^T$ and remember that $\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$:
>
> $$\begin{aligned}\mathbf{A}\vec{x} &= \lambda\vec{x}\\ \mathbf{Q}^T\mathbf{A}(\mathbf{Q}\mathbf{Q}^T)\vec{x} &= \mathbf{Q}^T\lambda\vec{x}\\ (\mathbf{Q}^T\mathbf{A}\mathbf{Q})(\mathbf{Q}^T\vec{x}) &= \lambda(\mathbf{Q}^T\vec{x})\end{aligned}$$

Jacobi's rotation method

- We want to diagonalise $\mathbf{A}$ using a similarity transformation
- That is, we want to find $\mathbf{S}$ such that $\mathbf{S}^T\mathbf{A}\mathbf{S} = \mathbf{D}$
- Will do this by applying a series of similarity transformations (rotation matrices) $\mathbf{S}_1, \mathbf{S}_2, \ldots$

$$\begin{aligned}\mathbf{A} &\equiv \mathbf{A}^{(1)}\\ \mathbf{S}_1^T\mathbf{A}\mathbf{S}_1 &\equiv \mathbf{A}^{(2)}\\ \mathbf{S}_2^T\mathbf{S}_1^T\mathbf{A}\mathbf{S}_1\mathbf{S}_2 &\equiv \mathbf{A}^{(3)}\\ \ldots\end{aligned}$$

  until we get a matrix $\mathbf{A}^{(M)}$ that is *close enough* to being diagonal:

$$\mathbf{S}_{M-1}^T\ldots\mathbf{S}_1^T\mathbf{A}\mathbf{S}_1\ldots\mathbf{S}_{M-1} \equiv \mathbf{A}^{(M)} \approx \mathbf{D}$$

- Then we will have $\mathbf{S} \approx \mathbf{S}_1\mathbf{S}_2\ldots\mathbf{S}_{M-1}$ and we are done:
  - $\mathbf{S}$ contains the eigenvectors of $\mathbf{A}$
  - $\mathbf{D}$ contains the eigenvalues of $\mathbf{A}$
- Notation-wise, each new transformation in this procedure can be written as

$$\boxed{\mathbf{A}^{(m+1)} = \mathbf{S}_m^T\mathbf{A}^{(m)}\mathbf{S}_m}$$

- For bookkeeping, it is also useful to have a matrix $\mathbf{R}$ that encodes the combined transformation so far, i.e.

$$\boxed{\mathbf{R}^{(m+1)} = \mathbf{R}^{(m)}\mathbf{S}_m}$$

- Starting from $\mathbf{R}^{(1)} = \mathbf{I}$ this means

$$\begin{aligned}
\mathbf{R}^{(2)} &= \mathbf{R}^{(1)}\mathbf{S}_1 = \mathbf{S}_1 \\
\mathbf{R}^{(3)} &= \mathbf{R}^{(2)}\mathbf{S}_2 = \mathbf{S}_1\mathbf{S}_2 \\
&\dots \\
\mathbf{R}^{(M)} &= \mathbf{R}^{(M-1)}\mathbf{S}_{M-1} = \mathbf{S}_1\mathbf{S}_2\dots\mathbf{S}_{M-1}
\end{aligned}$$

  where the final $\mathbf{R}^{(M)}$ is our approximation to $\mathbf{S}$

- Now it's time to see how the individual matrices $\mathbf{S}_i$ are constructed
- Jacobi's rotation method is (unsurprisingly) based on *rotation matrices*
- Let's use a 10×10 matrix to illustrate the general structure

$$\mathbf{S}_m = \begin{bmatrix}
1 & & & & & & & & & \\
& 1 & & & & & & & & \\
& & 1 & & & & & & & \\
& & & c_\theta & & & s_\theta & & & \\
& & & & 1 & & & & & \\
& & & & & 1 & & & & \\
& & & -s_\theta & & & c_\theta & & & \\
& & & & & & & 1 & & \\
& & & & & & & & 1 & \\
& & & & & & & & & 1
\end{bmatrix}$$

- Here we use the shorthand notation $c_\theta \equiv \cos\theta$ and $s_\theta \equiv \sin\theta$
- All empty elements are 0's
- This example matrix represents a clockwise rotation in the (x_4, x_7) plane, by an angle θ
  - If we flip the signs of the two s_θ entries we would have a counterclockwise rotation. (Here we use a clockwise rotation to be consistent with the notation in Morten's notes.)
- Notation:
  - k: row index of the upper s_θ entry (our example: $k = 4$)
  - l: column index of the upper s_θ entry (our example: $l = 7$)
- If we know (k, l) we also know the positions of the three other c_θ and s_θ entries
- **Note**: The matrix $\mathbf{S}_m$ is fully specified by the three numbers (k, l, θ)
- In Jacobi's rotation method the next rotation matrix $\mathbf{S}_m$ is constructed as follows:

1. In the current matrix $\mathbf{A}^{(m)}$, find the indices (k, l) of the largest (absolute value) off-diagonal element in the upper half of the matrix
2. Determine the θ needed to "rotate away" this off-diagonal element, i.e. find the θ such that the element $a_{kl}^{(m+1)}$ in $\mathbf{A}^{(m+1)} = \mathbf{S}_m^T \mathbf{A}^{(m)} \mathbf{S}_m$ will be zero

An example with a two-by-two matrix

- Let's see how one such rotation matrix is constructed using a 2×2 example
- Start from

$$\mathbf{A} = \mathbf{A}^{(1)} = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} \\ a_{21}^{(1)} & a_{22}^{(1)} \end{bmatrix}$$

- We also assume $\mathbf{A}$ is symmetric: $a_{21}^{(1)} = a_{12}^{(1)}$
- What we *want* our similarity transformation to achieve is

$$\mathbf{A}^{(2)} = \begin{bmatrix} a_{11}^{(2)} & a_{12}^{(2)} \\ a_{21}^{(2)} & a_{22}^{(2)} \end{bmatrix} = \begin{bmatrix} a_{11}^{(2)} & 0 \\ 0 & a_{22}^{(2)} \end{bmatrix}$$

- So we need a transformation $\mathbf{S}_1$ that gives $a_{12}^{(2)} = 0$
- This is a 2×2 example, so we have $k = 1$ and $l = 2$
- Our rotation matrix is

$$\mathbf{S}_1 = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \equiv \begin{bmatrix} c_\theta & s_\theta \\ -s_\theta & c_\theta \end{bmatrix}$$

- Recall that $\mathbf{A}^{(1)}$ is symmetric. If we write out the transformation $\mathbf{A}^{(2)} = \mathbf{S}_1^T \mathbf{A}^{(1)} \mathbf{S}_1$

$$\begin{bmatrix} a_{11}^{(2)} & a_{12}^{(2)} \\ a_{21}^{(2)} & a_{22}^{(2)} \end{bmatrix} = \begin{bmatrix} c_\theta & -s_\theta \\ s_\theta & c_\theta \end{bmatrix} \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} \\ a_{12}^{(1)} & a_{22}^{(1)} \end{bmatrix} \begin{bmatrix} c_\theta & s_\theta \\ -s_\theta & c_\theta \end{bmatrix}$$

we get the following equations for the four elements of $\mathbf{A}^{(2)}$:

$$
\begin{aligned}
a_{11}^{(2)} &= a_{11}^{(1)}c_\theta^2 - 2a_{12}^{(1)}c_\theta s_\theta + a_{22}^{(1)}s_\theta^2 \\
a_{22}^{(2)} &= a_{22}^{(1)}c_\theta^2 + 2a_{12}^{(1)}c_\theta s_\theta + a_{11}^{(1)}s_\theta^2 \\
a_{12}^{(2)} &= (a_{11}^{(1)} - a_{22}^{(1)})c_\theta s_\theta + a_{12}^{(1)}(c_\theta^2 - s_\theta^2) \\
a_{21}^{(2)} &= (a_{11}^{(1)} - a_{22}^{(1)})c_\theta s_\theta + a_{12}^{(1)}(c_\theta^2 - s_\theta^2)
\end{aligned}
$$

- We see explicitly that the transformation preserves the symmetry: $a_{21}^{(2)} = a_{12}^{(2)}$
- To determine θ we now apply our requirement: $a_{12}^{(2)} = 0$. This gives us the equation

$$
\left(\frac{a_{11}^{(1)} - a_{22}^{(1)}}{a_{12}^{(1)}}\right) c_\theta s_\theta + c_\theta^2 - s_\theta^2 = 0 \tag{$\star$}
$$

- Define some shorthand notation:

$$
\begin{aligned}
\tan\theta \equiv t_\theta &= \frac{s_\theta}{c_\theta} \\
\tau &\equiv \frac{a_{22}^{(1)} - a_{11}^{(1)}}{2a_{12}^{(1)}}
\end{aligned}
$$

- Divide Eq. $(\star)$ by c_θ^2 and use the new notation to get

$$
\boxed{t_\theta^2 + 2\tau t_\theta - 1 = 0}
$$

- This second order equation for t_θ has solutions

$$
t_\theta = -\tau \pm \sqrt{1+\tau^2}
$$

- So the two choices for θ that correspond to the two t_θ values above will ensure $a_{12}^{(2)} = 0$
- In the Jacobi rotation method, choosing the smaller of the two angles (the smaller t_θ value) typically gives faster convergence
- With t_θ known we can compute c_θ and s_θ

$$
\begin{aligned}
c_\theta &= \frac{1}{\sqrt{1+t_\theta^2}} \\
s_\theta &= c_\theta t_\theta
\end{aligned}
$$

- Now we can compute the new matrix elements $a_{11}^{(2)}$ and $a_{22}^{(2)}$ using the equations above
  - No need to compute $a_{12}^{(2)}$ or $a_{21}^{(2)}$ since these are 0 by our choice of θ
- We now have our new $\mathbf{A}^{(2)}$:

$$\mathbf{A}^{(2)} = \begin{bmatrix} a_{11}^{(2)} & 0 \\ 0 & a_{22}^{(2)} \end{bmatrix}$$

- So, using a similarity transformation of the original $\mathbf{A}$ we have reached a diagonal matrix
- We then know that $a_{11}^{(2)}$ and $a_{22}^{(2)}$ are the **eigenvalues** of $\mathbf{A}$
- The **eigenvectors** of $\mathbf{A}$ will be the column vectors contained in the combined transformation matrix $\mathbf{R}$ (which in this case consists of only one rotation)

$$\mathbf{R}^{(2)} = \mathbf{R}^{(1)}\mathbf{S}_1 = \mathbf{I}\mathbf{S}_1 = \begin{bmatrix} \begin{bmatrix} c_\theta \\ -s_\theta \end{bmatrix} & \begin{bmatrix} s_\theta \\ c_\theta \end{bmatrix} \end{bmatrix}$$

- Note:
  - Because this was a 2×2 example, we got an *exactly* diagonal matrix
  - In the general $N \times N$ case we will need many transformations $\mathbf{S}_m$, and we will in general never reach an exactly diagonal matrix
  - The reason is that each transformation $\mathbf{S}_m$ also affects the other elements along rows/-columns k and l
  - So, an off-diagonal element a_{ij} that had previously been rotated to $a_{ij} = 0$ can be transformed back to $a_{ij} \neq 0$ by a later rotation
  - Still, the method is guaranteed to converge towards a diagonal matrix

A general algorithm for Jacobi's rotation method

- We have seen how the equation for determining θ arises
- Let us now write down a general algorithm for Jacobi's rotation method

Step 1:

- Choose a tolerance ϵ, e.g. $\epsilon = 10^{-8}$
- Let $\mathbf{A}^{(1)} = \mathbf{A}$

- Let $\mathbf{R}^{(1)} = \mathbf{I}$

Step 2:

- Find indices (k, l) of the maximum (absolute value) off-diagonal element in $\mathbf{A}^{(1)}$

Step 3:

- While $|a_{kl}^{(m)}| > \epsilon$:

  Step 3.1:

  - Compute τ:

$$\tau = \frac{a_{ll}^{(m)} - a_{kk}^{(m)}}{2a_{kl}^{(m)}}$$

  Step 3.2:

  - Compute t_θ:

$$t_\theta = -\tau \pm \sqrt{1 + \tau^2}$$

    * Choose the solution that gives the smallest t_θ:

$$\text{If } \tau > 0: \quad t_\theta = -\tau + \sqrt{1 + \tau^2} = \frac{1}{\tau + \sqrt{1 + \tau^2}}$$
$$\text{If } \tau < 0: \quad t_\theta = -\tau - \sqrt{1 + \tau^2} = \frac{1}{\tau - \sqrt{1 + \tau^2}}$$

    * Using the last versions of these expressions reduces the chance of loss of numerical precision in cases where τ is large

  - Compute c_θ and s_θ:

$$c_\theta = \frac{1}{\sqrt{1 + t_\theta^2}}$$
$$s_\theta = c_\theta t_\theta$$

  Step 3.3:

  - Transform the current $\mathbf{A}$ matrix as $\mathbf{A}^{(m)} \to \mathbf{A}^{(m+1)} = \mathbf{S}_m^T \mathbf{A}^{(m)} \mathbf{S}_m$ by updating elements.

Start with the four elements given by k and l:

$$
\begin{aligned}
a_{kk}^{(m+1)} &= a_{kk}^{(m)} c_\theta^2 - 2a_{kl}^{(m)} c_\theta s_\theta + a_{ll}^{(m)} s_\theta^2 \\
a_{ll}^{(m+1)} &= a_{ll}^{(m)} c_\theta^2 + 2a_{kl}^{(m)} c_\theta s_\theta + a_{kk}^{(m)} s_\theta^2 \\
a_{kl}^{(m+1)} &= 0 \\
a_{lk}^{(m+1)} &= 0
\end{aligned}
$$

- Next, update all other elements along these rows and columns.
  For all i where $i \neq k$ **and** $i \neq l$:

$$
\begin{aligned}
a_{ik}^{(m+1)} &= a_{ik}^{(m)} c_\theta - a_{il}^{(m)} s_\theta && \text{(a)} \\
a_{ki}^{(m+1)} &= a_{ik}^{(m+1)} && \text{(b)} \\
a_{il}^{(m+1)} &= a_{il}^{(m)} c_\theta + a_{ik}^{(m)} s_\theta && \text{(c)} \\
a_{li}^{(m+1)} &= a_{il}^{(m+1)} && \text{(d)}
\end{aligned}
$$

  * *Note*: When evaluating Eq. (c) above, make sure to use the old element $a_{ik}^{(m)}$, not the new element $a_{ik}^{(m+1)}$ computed in Eq. (a)

Step 3.4:

- Update the overall rotation matrix, $\mathbf{R}^{(m)} \rightarrow \mathbf{R}^{(m+1)} = \mathbf{R}^{(m)}\mathbf{S}_m$, by updating elements.
  For all i:

$$
\begin{aligned}
r_{ik}^{(m+1)} &= r_{ik}^{(m)} c_\theta - r_{il}^{(m)} s_\theta && \text{(a)} \\
r_{il}^{(m+1)} &= r_{il}^{(m)} c_\theta + r_{ik}^{(m)} s_\theta && \text{(b)}
\end{aligned}
$$

  * As above, make sure to use the old element $r_{ik}^{(m)}$ when evaluating Eq. (b) here

Step 3.5:

- Find the (k, l) indices of the maximum off-diagonal element in the new matrix $\mathbf{A}^{(m+1)}$
- (While loop returns to **Step 3**)

Step 4:

- Read out eigenvalues from the diagonal of the final $\mathbf{A}^{(m+1)}$ matrix
- Read out eigenvectors from the columns of the final $\mathbf{R}^{(m+1)}$ matrix

In-lecture code discussion #3

- Debugging: Examples of common bugs and some tips for debugging
  github.com/anderkve/FYS3150/tree/master/code_examples/fantastic_bugs_and_where_to_find_them

Iterative methods for solving matrix equations

- We now return to the topic of how to solve matrix equations of the general form $\mathbf{A}\vec{x} = \vec{b}$

Direct vs iterative methods

- We have previously looked at **direct methods** for solving $\mathbf{A}\vec{x} = \vec{b}$:
  - Gaussian elimination
    - In the case of a tridiagonal matrix $\mathbf{A}$ this became the Thomas algorithm
  - LU decomposition
    - A starting point for many matrix tasks, including finding the inverse $\mathbf{A}^{-1}$
    - When we have $\mathbf{A}^{-1}$, we get $\vec{x}$ from matrix-vector multiplication: $\vec{x} = \mathbf{A}^{-1}\vec{b}$
- The direct methods in principle give **the exact answer** in a **finite number of steps**
- But due e.g. to their reliance on exact equalities/cancellations in the mathematics, these methods can be susceptible to numerical problems
- Alternative class of methods: **iterative methods**
- Iterative methods iterate closer and closer to the true solution $\vec{x}$, but will generally never get there exactly
- Need some convergence criteria for deciding when to stop the iterations
- For large, *sparse* matrices (mostly zeros), iterative methods are often much faster and use much less memory than direct methods
  - Each iteration only involves the non-zero elements of $\mathbf{A}$, while direct methods like Gaussian elimination and LU decomposition cost $\mathcal{O}(N^3)$ operations and typically fill in many of the zeros
  - For small or dense matrices, direct methods are usually the better choice: In this case each iteration of an iterative algorithm costs $\mathcal{O}(N^2)$, so with many iterations the cost can become larger than the $\mathcal{O}(N^3)$ cost of LU decomposition or Gaussian elimination, and the direct methods also work for more matrices.
- Added bonus: the simplest iterative methods are often very easy to implement in code
  - As we will see, when an iterative method is expressed in matrix form it can look very complicated

  - But often it boils down to some very simple set of equations for the individual components x_i of the solution vector, and it is usually these component-level equations we would implement in our code

> At this point you may be thinking: *Why didn't we use an iterative method to solve the matrix equation in project 1?* The answer is that we definitely could have done that — coding-wise it would have been very easy! But since Gaussian elimination is such an important stepping stone for many more advanced algorithms, it's an important algorithm to get some hands-on experience with, and that's why we focused on that (in the form of the Thomas algorithm) in project 1.

- In the following we will look at three examples of iterative methods:
  - **The Jacobi method** (not to be confused with Jacobi's rotation method for eigenvalue problems $\mathbf{A}\vec{x} = \lambda\vec{x}$)
  - **Gauss-Seidel**
  - **Successive over-relaxation**
- But first we need to discuss how to check for convergence when using an iterative approach

Checking convergence for an iterative method

- An iterative method has the conceptual form of a `while` loop in a program: we keep doing some steps over and over again until we have reached some stopping criterion
- Below are two approaches to how that stopping criterion can be formulated
- Best practice is often to use both

Alternative 1: monitor the relative change in our estimate for $\vec{x}$

- We let $\vec{x}^{(m)}$ denote our estimate for the true $\vec{x}$ after m steps of our iterative method
- Let ϵ denote the *the norm of the difference vector between the estimates at steps m and $m+1$, relative to the norm of the new estimate*:

$$\epsilon = \frac{|\vec{x}^{(m+1)} - \vec{x}^{(m)}|_l}{|\vec{x}^{(m+1)}|_l}$$

- Here the subscript l indicates that we are using some particular l-*norm* to measure the length of a vector:

$$|\vec{x}|_l \equiv \left[\sum_{i=1}^{N} |x_i|^l\right]^{\frac{1}{l}}$$

  where N is the number of elements x_i in the vector $\vec{x}$

- Some common choices for l:

$$\begin{aligned} l = 1: &\quad |\vec{x}|_1 = |x_1| + |x_2| + \ldots + |x_N| \quad \text{(sum of absolute values of the vector elements)} \\ l = 2: &\quad |\vec{x}|_2 = \sqrt{x_1^2 + x_2^2 + \ldots + x_N^2} \quad \text{(the familiar Euclidean vector length)} \\ l = \infty: &\quad |\vec{x}|_\infty = \max_i |x_i| \quad \text{(the largest vector element in absolute value)} \end{aligned}$$

> To see why $|\vec{x}|_\infty$ ends up picking out the largest vector element, just check what happens in the general expression for $|\vec{x}|_l$ when you set l to some very large value, say $l = 1000$. In that case, the sum $|x_1|^{1000} + |x_2|^{1000} + \ldots + |x_N|^{1000}$ will be highly dominated by the term containing the largest vector element. For example, consider the vector $\vec{x} = [0.9, 1.2, 1.19, 0.7]$. Then we would have that $0.9^{1000} + 1.2^{1000} + 1.19^{1000} + 0.7^{1000} \approx 1.2^{1000}$. So the 1000-norm of this vector $\vec{x}$ would be $|\vec{x}|_{1000} = [0.9^{1000} + 1.2^{1000} + 1.19^{1000} + 0.7^{1000}]^{\frac{1}{l}} \approx [1.2^{1000}]^{\frac{1}{l}} = 1.2 = \max_i |x_i|$. As we increase l towards $l = \infty$ the approximation in the previous expression becomes more and more precise.

- When we have chosen a particular l-norm, we can decide on some small threshold value such that when ϵ falls below this value, we stop the iterations
- In other words, we stop when $\vec{x}^{(m+1)}$ is sufficiently similar to our previous $\vec{x}^{(m)}$, so that we don't think we will gain much from running more iterations
- *Pro*: Computing the l-norm of the new vector $\vec{x}^{(m+1)}$ is a quick operation
- *Con*: We are here just checking whether or not our estimate for $\vec{x}$ has (almost) stopped changing — we are *not* measuring how close our estimate $\vec{x}^{(m)}$ is to actually solving the equation $\mathbf{A}\vec{x} = \vec{b}$

Alternative 2: monitor the residual

- Recall that a true solution $\vec{x}$ should satisfy $\mathbf{A}\vec{x} = \vec{b}$
- Let **the residual** $\vec{r}$ be the vector difference between $\vec{b}$ and $\mathbf{A}\vec{x}^{(m)}$ (our current estimate for $\mathbf{A}\vec{x}$)

$$\vec{r} = \mathbf{A}\vec{x}^{(m)} - \vec{b}$$

- At every iteration we can then check the ratio of the lengths of $\vec{r}$ and $\vec{b}$:

$$\frac{|\vec{r}|_l}{|\vec{b}|_l}$$

- We stop the iterations when this ratio falls below a threshold value that we have decided
- *Pro*: Compared to our previous approach to checking for convergence, this approach more directly monitors how far our estimate $\vec{x}^{(m)}$ is from satisfying the equation $\mathbf{A}\vec{x}^{(m)} = \vec{b}$
- *Con*: This approach is computationally more expensive than the previous approach, since it requires a matrix-vector multiplication.

The Jacobi method

> Despite the similar name, do not confuse this iterative method for solving $\mathbf{A}\vec{x} = \vec{b}$ with *Jacobi's rotation method*, which is a method for solving an eigenvalue-eigenvector problem $\mathbf{A}\vec{x} = \lambda\vec{x}$.

- The starting point is to rewrite our matrix $\mathbf{A}$ as the sum $\mathbf{L} + \mathbf{D} + \mathbf{U}$, where $\mathbf{L}$ is a strictly lower-triangular matrix, $\mathbf{D}$ is a diagonal matrix and $\mathbf{U}$ is a strictly upper-triangular matrix
- Here is a 3×3 example:

$$\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{U}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} & & \\ a_{21} & & \\ a_{31} & a_{32} & \end{bmatrix} + \begin{bmatrix} a_{11} & & \\ & a_{22} & \\ & & a_{33} \end{bmatrix} + \begin{bmatrix} & a_{12} & a_{13} \\ & & a_{23} \\ & & \end{bmatrix}$$

  - Note that this simple decomposition is *not* the same as *LU decomposition*, which was the much more complicated task of writing $\mathbf{A}$ as a *product* $\mathbf{A} = \mathbf{LU}$ (or alternatively as $\mathbf{A} = \mathbf{LDU}$)

- We can now rewrite our matrix problem as follows

$$\begin{aligned} \mathbf{A}\vec{x} &= \vec{b} \\ (\mathbf{L} + \mathbf{D} + \mathbf{U})\vec{x} &= \vec{b} \\ \mathbf{D}\vec{x} &= -(\mathbf{L} + \mathbf{U})\vec{x} + \vec{b} \\ \vec{x} &= -\mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\vec{x} + \mathbf{D}^{-1}\vec{b} \end{aligned}$$

- This last expression has the form $\vec{x} = [\text{some matrix}]\vec{x} + [\text{stuff independent of } \vec{x}]$
- We will take this as inspiration and suggest the following iterative recipe for how to change some current estimate $\vec{x}^{(m)}$ to a new (and hopefully improved) estimate $\vec{x}^{(m+1)}$:

$$\boxed{\vec{x}^{(m+1)} = -\mathbf{D}^{-1}(\mathbf{L}+\mathbf{U})\vec{x}^{(m)} + \mathbf{D}^{-1}\vec{b}}$$

- Note that since $\mathbf{D}$ is diagonal, we already know that the inverse $\mathbf{D}^{-1}$ is simply the diagonal matrix with the reciprocal matrix elements

$$\mathbf{D}^{-1} = \text{diag}\left(\frac{1}{a_{11}}, \frac{1}{a_{22}}, \ldots, \frac{1}{a_{NN}}\right)$$

- The matrix $\mathbf{T} = -\mathbf{D}^{-1}(\mathbf{L}+\mathbf{U})$, which operates on $\vec{x}^{(m)}$ on the right-hand side of our iteration rule, is called the **iteration matrix** or the **update matrix**
- Our iterative matrix recipe above might look complicated, but it turns into some very simple update rules when expressed in terms of the individual vector components $x_i^{(m+1)}$
- To see this, let's write out the matrix multiplication above for a 4×4 example:

$$x_1^{(m+1)} = \frac{1}{a_{11}}\left[b_1 - a_{12}x_2^{(m)} - a_{13}x_3^{(m)} - a_{14}x_4^{(m)}\right]$$

$$x_2^{(m+1)} = \frac{1}{a_{22}}\left[b_2 - a_{21}x_1^{(m)} - a_{23}x_3^{(m)} - a_{24}x_4^{(m)}\right]$$

$$x_3^{(m+1)} = \frac{1}{a_{33}}\left[b_3 - a_{31}x_1^{(m)} - a_{32}x_2^{(m)} - a_{34}x_4^{(m)}\right]$$

$$x_4^{(m+1)} = \frac{1}{a_{44}}\left[b_4 - a_{41}x_1^{(m)} - a_{42}x_2^{(m)} - a_{43}x_3^{(m)}\right]$$

- So for a general case with a length-N vector, the iterative recipe on component form is simply

$$\boxed{x_i^{(m+1)} = \frac{1}{a_{ii}}\left[b_i - \sum_{\substack{j=1 \\ j\neq i}}^{N} a_{ij}x_j^{(m)}\right]}$$

- To start our iterative method, we need to make an **initial guess**, $\vec{x}^{(0)}$
- In general, the closer our initial guess $\vec{x}^{(0)}$ is to the true $\vec{x}$, the fewer iterations we need to perform

- Starting from *any* initial guess $\vec{x}^{(0)}$, the method will converge towards the true solution if the largest eigenvalue (in absolute value) of the update matrix $\mathbf{T}$ is less than 1
  - This is an example of a general result regarding convergence of iterative methods:
    * The **spectral radius** $\rho(M)$ of some matrix $\mathbf{M}$ is defined as $\rho(\mathbf{M}) = \max_i |\lambda_i|$
    * If the spectral radius of the update matrix $\mathbf{T}$ satisfies $\rho(\mathbf{T}) < 1$, the iterative method will converge starting from any initial guess
    * The smaller the spectral radius, the faster the convergence. (We will return to this later in the course.)
- A useful special case to remember is that the requirement $\rho(\mathbf{T}) < 1$ is always satisfied if the original matrix $\mathbf{A}$ is **strictly diagonally dominant** or **irreducibly diagonally dominant**
  - The matrix $\mathbf{A}$ is *strictly diagonally dominant* if
    * $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for *every* row i in $\mathbf{A}$
  - The matrix $\mathbf{A}$ is *irreducibly diagonally dominant* if
    * $|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|$ for *every* row i in $\mathbf{A}$, *and*
    * $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for *at least one* of the rows in $\mathbf{A}$, *and*
    * $\mathbf{A}$ is *irreducible*, meaning that the set of equations cannot be split into a smaller subset of equations that can be solved independently of the rest. (Formally: there is no reordering of the unknowns, applied to both rows and columns, that brings $\mathbf{A}$ to block-triangular form.)
  - *Example*: The tridiagonal matrix from Project 1, with the elements $(-1, 2, -1)$, is not *strictly diagonally dominant*, but it is *irreducibly diagonally dominant*, so the method converges.
    * Consider 4×4 case
    * Rows 2 and 3 have $|a_{ii}| = 2 = \sum_{j \neq i} |a_{ij}|$, so it is not strictly diagonally dominant
    * But all 4 rows satisfy $|a_{ii}| = 2 \geq \sum_{j \neq i} |a_{ij}|$, and rows 1 and 4 satisfy $|a_{ii}| = 2 > \sum_{j \neq i} |a_{ij}| = 1$
    * Further, the matrix is irreducible since every unknown x_i is coupled to its neighbours $x_{i \pm 1}$
    * So the matrix is irreducibly diagonally dominant, and the method will converge.

Gauss-Seidel

- The Gauss-Seidel method builds on the Jacobi method, but it immediately starts using each computed estimate $x_i^{(m+1)}$ in subsequent computations during the same iteration
- Let's see this in action for an example where $\mathbf{A}$ is 4×4:

$$x_1^{(m+1)} = \frac{1}{a_{11}}\left[b_1 - a_{12}x_2^{(m)} - a_{13}x_3^{(m)} - a_{14}x_4^{(m)}\right]$$

$$x_2^{(m+1)} = \frac{1}{a_{22}}\left[b_2 - a_{21}x_1^{(m+1)} - a_{23}x_3^{(m)} - a_{24}x_4^{(m)}\right]$$

$$x_3^{(m+1)} = \frac{1}{a_{33}}\left[b_3 - a_{31}x_1^{(m+1)} - a_{32}x_2^{(m+1)} - a_{34}x_4^{(m)}\right]$$

$$x_4^{(m+1)} = \frac{1}{a_{44}}\left[b_4 - a_{41}x_1^{(m+1)} - a_{42}x_2^{(m+1)} - a_{43}x_3^{(m+1)}\right]$$

- So we are effectively doing a form of forward substitution
- For the general case where $\mathbf{A}$ is $N \times N$, the update rule on component form is

$$\boxed{x_i^{(m+1)} = \frac{1}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(m+1)} - \sum_{j=i+1}^{N} a_{ij}x_j^{(m)}\right]}$$

  - Note that the elements $x_i^{(m+1)}$ must be computed *sequentially*, in the order $i = 1, 2, \ldots, N$
  - In contrast, for the Jacobi method we could have computed all the $x_i^{(m+1)}$ in parallel, since these computations were independent of each other

- On matrix form, the Gauss-Seidel method corresponds to starting from the matrix equation

$$\begin{aligned}
\mathbf{A}\vec{x} &= \vec{b} \\
(\mathbf{L} + \mathbf{D} + \mathbf{U})\vec{x} &= \vec{b} \\
(\mathbf{L} + \mathbf{D})\vec{x} &= -\mathbf{U}\vec{x} + \vec{b} \\
\vec{x} &= -(\mathbf{L} + \mathbf{D})^{-1}\mathbf{U}\vec{x} + (\mathbf{L} + \mathbf{D})^{-1}\vec{b}
\end{aligned}$$

  and from this suggest the update rule

$$\boxed{\vec{x}^{(m+1)} = -(\mathbf{L} + \mathbf{D})^{-1}\mathbf{U}\vec{x}^{(m)} + (\mathbf{L} + \mathbf{D})^{-1}\vec{b}}$$

- When comparing the last term of this expression to the component-wise update rule above, it may look like the b_i term is missing any contribution from the $\mathbf{L}$ matrix. (The b_i term looks identical to the same term in the Jacobi update rule.) However, this is only apparent – the terms that combine b_i with matrix elements from $\mathbf{L}$ show up if we write out the $x_j^{(m+1)}$ factor in the first sum of the component update rule.
- While we don't show it here, one way to see that the matrix form really corresponds to the component-wise update rule we presented above is the following:
  - Start from the matrix expression

$$(\mathbf{L} + \mathbf{D})\vec{x}^{(m+1)} = -\mathbf{U}\vec{x}^{(m)} + \vec{b}$$

  - View this as a regular matrix equation for an unknown vector $\vec{x}^{(m+1)}$ and solve it using forward substitution. (We only need forward substitution since the matrix $(\mathbf{L} + \mathbf{D})$ is a lower-triangular matrix.)
  - The solutions you get for the components of $\vec{x}^{(m+1)}$ correspond to the component-wise update rules we presented above
- The Gauss-Seidel method typically has faster convergence per iteration than the Jacobi method
- On the other hand, the Jacobi method is easier to parallelise, since each component update can be performed independently. (In Gauss-Seidel the component updates must be done sequentially.)

Successive over-relaxation

- Successive over-relaxation (SOR) is a modified version of the Gauss-Seidel method
- SOR can achieve faster convergence than Gauss-Seidel, but it comes with a free *weight parameter* ω, and the optimal choice for ω is unknown in most cases
- SOR can be seen as the following schematic generalisation of the Gauss-Seidel method:

$$[\text{new}] = [\text{old}] + [\text{weight}] * [\text{the change [new]-[old] from Gauss-Seidel}]$$

- If we choose the weight $\omega = 1$ we simply get back the Gauss-Seidel method
- The method can only converge for $0 < \omega < 2$. For $\omega \leq 0$ or $\omega \geq 2$ it will *not* converge
- If $\mathbf{A}$ is symmetric and positive definite (every eigenvalue is > 0), the method is guaranteed to converge for any $0 < \omega < 2$, but the optimal choice is problem-specific (and generally unknown)

  - Usually the optimal choice lies in the range $1 < \omega < 2$ (*over-relaxation*). The choice $0 < \omega < 1$ (*under-relaxation*) is typically not useful for improving convergence compared to e.g. Gauss-Seidel

- On component form, the update rule for SOR is

$$\boxed{x_i^{(m+1)} = x_i^{(m)} + \frac{\omega}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(m+1)} - \sum_{j=i+1}^{N} a_{ij}x_j^{(m)} - a_{ii}x_i^{(m)}\right]}$$

  - As with the Gauss-Seidel method, the vector components $x_i^{(m+1)}$ must be computed *sequentially*, in the order $i = 1, 2, \ldots, N$

- The update rule on matrix form is given by the following (rather confusing-looking) expression:

$$\boxed{\vec{x}^{(m+1)} = [\omega\mathbf{L} + \mathbf{D}]^{-1}\left[-[\omega\mathbf{U} + (\omega - 1)\mathbf{D}]\,\vec{x}^{(m)} + \omega\vec{b}\right]}$$

  - The best choice for ω would be the choice that gives the smallest spectral radius for the update matrix $\mathbf{T} = [\omega\mathbf{L} + \mathbf{D}]^{-1}[-[\omega\mathbf{U} + (\omega - 1)\mathbf{D}]]$

In-lecture code discussion #4

- Topic: Classes in C++

How to write a scientific report

- In what follows we will detail how to write good reports for this course, and point out some of the most common mistakes
- However, much of what follows may seem a bit dry and vague unless you have some concrete example to look at
- So, before reading further, I recommend you do the following:

1. Read the TeX template we have provided — it is written as an example report with lots of comments on structure, layout, etc.:
   anderkve.github.io/FYS3150/book/writing_reports/project_templates.html

2. Read the 2018 student report we have provided as an example of a really good report: github.com/anderkve/FYS3150/blob/master/book/writing_reports/example_report.pdf
3. Read our "Checklist for reports", to see the most common and easily avoidable mistakes (minor and major): https://anderkve.github.io/FYS3150/book/writing_reports/checklist.html

General remarks

- A report is a **self-contained document**
  - It should look and feel like a scientific article, not like an answer to some problem set
  - The correct way to think about your report: *I have carried out some numerical study and now I need to describe my work and findings to the outside world, through the format of a scientific paper and a code repository*
  - The incorrect way to think about your report: *I'm a student in a particular course who was given a list of questions, and now I'm writing down my answers for the teachers to grade*
  - In short, you should write about the project as if it was a small research project, not a course assignment
    * Assume your reader has never seen the project description, and does not know that you are a student taking a course on computational physics
    * So, for example, the title should not contain words like "FYS3150" or "Project 3", but should be something descriptive and interesting
    * And phrases like "Problem 6" should not appear anywhere
    * Note that our project descriptions are still structured as a set of enumerated problems. But this is just to help you structure your work, and to make it easier for you to discuss concrete problems with the teachers and your fellow students.
- The quality of the presentation really matters. Pay attention to
  - Grammar and typos
  - Figures and figure captions
  - Tables and table captions
- Remember to include a link to your GitHub repository. See the TeX template for an example of one way to do this.

Report format

- You have to use the TeX template we have provided
  - (Yes, you really have to use our template — it's not enough to write a TeX document that looks roughly like our template)
- Report format:
  - *Title*
  - *Abstract*
  - *Introduction*
  - *Methods* (or *Theory*, if you prefer that section title)
  - *Results and discussion*
  - *Conclusions*
  - If applicable: *Appendices*
  - *References*
- In the following we will discuss each of these sections

Title

- As mentioned above, do not use a title like "FYS3150: Project 3"
- Use a title that tells the reader what the report is about, e.g. something along the lines of "Numerical simulations of [some system]" or "Simulating [some system] using [some method]"

Abstract

- The abstract is a *short* summary of your report — probably no more than around ten sentences
- It should state what the report is about and what the main findings are
  - In other words, the abstract should not just be a long list of things you plan to do, e.g. "First we will do this. Then we will do that…". You have already performed the study, so tell the reader what the main results are.
- If you have arrived at some key numerical results, make sure to mention these numbers in the abstract

  - Examples:
    * "We found that [some algorithm] requires 3N-1 FLOPs."
    * "Using [method X], we found that the numerical error was minimised for a step size of [some number]."

- It is OK to use technical language in the abstract without introducing it or explaining it
- The abstract is a very important part of a report. In the real world, it is typically the title and abstract that determines whether a reader will bother to read the rest of the document, or move on to read another article, check their email, or watch fourteen online videos with cute kittens.

Introduction

- The introduction sets the stage for the rest of the work
- In the introduction the goal is to:
  1. Briefly introduce and motivate the general topic of the report
     - Tell the reader why this topic is interesting and important — make the reader interested in reading the rest of the document
     - Maybe point the reader to some resources where they can learn more about the science topic or algorithms discussed in this report?
  2. Tell the reader more specifically what your particular study is about
  3. Explain the structure of the rest of the report, so the reader knows what to expect in the coming sections
     - This overview is typically given in the last paragraph of the introduction. See the TeX template for an example.
- In some cases it may be natural to include a couple of equations in the introduction, but the introduction is not the place to go into a detailed discussion of the theoretical background or the relevant algorithms — that belongs in the *Methods*/*Theory* section

Methods/Theory

- (Don't use the *Methods/Theory* as section title — choose either *Methods* or *Theory*, depending on which you think is most suitable)
- This is where you introduce the physics topic and/or relevant algorithms in more detail

  - It may be useful to organise the content into separate subsections, with separate subsection titles. For instance, for Project 3 it may be natural to have one subsection on Penning traps and another subsection introducing the Runge-Kutta algorithm

- The *Methods*/*Theory* is usually the place where most of the notation is introduced
  - As a general rule, whenever you write an equation that uses some symbol that has not appeared earlier in the report, you should explain what that symbol represents. (Except symbols that are completely self-explanatory, like using (x, y, z) for the coordinates in a 3D space.)
  - Below is an excerpt from the project description of Project 3, to illustrate how the text surrounding two equations is used to explain what the variables in those equations mean:

    > We will consider an ideal Penning trap in which the electric field is defined by the electric potential
    >
    > $$V(x, y, z) = \frac{V_0}{2d^2}(2z^2 - x^2 - y^2).$$
    >
    > Here V_0 is the potential applied to the electrodes and d is called the *characteristic dimension*, which represents the length scale for the region between the electrodes. (Technically, d is defined as $d = \sqrt{z_0^2 + r_0^2/2}$, where z_0 is the distance from the center to one of the end caps and r_0 is the distance from the center to the ring.) The electric field traps the particle in the z-direction, but the particle can still escape in the radial direction (in the xy-plane). Therefore, a homogeneous magnetic field is imposed in the z-direction of the apparatus, i.e
    >
    > $$\mathbf{B} = B_0\hat{e}_z = (0, 0, B_0),$$
    >
    > where B_0 is the field strength, with $B_0 > 0$.

  - Notice how the equations are treated as any other part of the current sentence, meaning that some equations will end with a period (if the next line is a new sentence), and some will end with comma (if the sentence continues after the equation)

- Sometimes it may be natural to include the full derivation of a method or theoretical result in this section, but keep in mind that the *Method*/*Theory* section should not be so long that it completely disrupts the general flow and focus of the report
  - If you want to include a derivation or explanation that feels too long or detailed, it may be a good idea to rather put this in an appendix. Then you can focus on the main points/equa-

tions that are relevant for the rest of the report, and point the reader to the appendix for further details, e.g. with a sentence like "A detailed derivation of [some equation/method] is given in Appendix A."

- The *Methods*/*Theory* section is also a natural place to reference the main tools and software libraries you have used in your study
  - For this purpose, our TeX template includes a subsection titled *Tools* — you *must* include this subsection in your report
- It is not common to use the *Method*/*Theory* section to present results from your study
  - One possible exception might be if you have performed some validation tests (e.g. some unit test to check that a particularly complicated part of your code is working as expected), and you want to show the reader the results of this validation test
    * But remember: while validation tests are really *important* in order to debug your code and establish that it is working as expected, they typically don't make for very exciting reading… The typical reader is interested in your main simulations and results, not your debugging process.
    * A good alternative might be to just mention how you have validated your code, and then point the reader to an appendix and/or a part of the code repository where you provide more details on the validation tests

Results and discussion

- This is (unsurprisingly) the section where you present and discuss your results
- There are two different strategies for how to structure this section:
  - **Strategy 1**:
    * Split the section into two subsections, one called *Results* and one called *Discussion*
    * In the *Results* subsection, simply present each relevant figure and table, but do not go into long discussions or explanations of the results
    * In the *Discussion* subsection, refer back to each figure and table in the *Results* subsection and discuss the results
  - **Strategy 2**:
    * Do the presentation and discussion of the results in one go, i.e. discuss each new figure/table right after presenting it

- Use the strategy you prefer
  - Personal preference: I tend to follow *strategy 2* when I write the *Results and discussion* sections in my own papers, as I find that my text flows more naturally with that approach. (But again, you should do what you prefer.)
- *Important*: The *Results* section is not somewhere where you can simply "dump" every single plot and table you have generated. An important aspect of writing a report is selecting which figures/tables you want to present to the reader.
  - Every figure and table you include in the report *must* be explicitly referred to somewhere in the main text, i.e. with sentences like "In Figure 3 we show the result from [...]" and "The results from [...] are given in Table 2."
  - If a figure/table is not mentioned anywhere in the main text, it should not be included in your report
- Every figure/table should have descriptive figure/table caption
  - Often there will be some duplication of content between the figure caption and the main text, and that's perfectly fine! (But try to avoid repeating the exact same sentences.)
  - For example, it may be natural to mention key numbers like the step size both when introducing a result in the main text ("In Figure 3 we show the result from the single-particle simulation when using a step size $h = 10^{-3}$.") and in the figure caption ("Figure 3: Simulation of a single particle in an ideal Penning trap. The graph shows the particle trajectory in the xy plane, for the initial condition [...] and the simulation step size $h = 10^{-3}$".)
- Make sure that text appearing in figures (axis labels, legends, etc.) has a reasonable size when included in the document
  - The rule of thumb here is that the text appearing in figures should not look too much smaller or larger than the main text in the document
- When discussing the results, point out the main features that you want the reader to notice. Help the reader interpret the figures and tables.
  - A tip: A useful question to have in the back of your mind when considering how to discuss your results can be the question "Are these results as expected?"
  - If some result look like what you would expect, let the reader know, and ideally help the reader understand why you expected the result to look like this (e.g. connect it back to some known aspect of the theory/method)
  - If you think something in your results looks fishy, be honest about it in your report!

    * If you suspect that the reason is a bug, maybe add your thoughts on what the source of the problem may be
    * A discussion of results that are wrong can still give a very good and interesting report!
    * *Important:* If it is clear that something is wrong with your results, make sure to discuss this in the report. It looks very bad if the report simply does not mention something that is obviously a problem.

- If you have any thoughts or ideas of how your study could have been improved (e.g. if you had more time), it can be nice to briefly mention this in a paragraph towards the end of the *Results and discussion* section

Conclusion

- In the *Conclusion* section you give a brief summary of what you have done in the study, and repeat the key results and numbers
- In terms of content, the conclusion is typically very similar to the abstract
    - But make sure that you don't directly copy sentences from the abstract — that looks bad

Appendices

- If you have any appendices, they should go after the *Conclusion* section
- A common mistake: to simply dump a bunch of plots or equations in an appendix, without any description or context
    - Just like any other part of the report, an appendix should be written as a proper text, e.g. with proper explanation of the symbols appearing in equations (unless they have been explained earlier in the report), proper captions for any figures/tables, etc.

References

- Make sure to always have at least some references — an empty reference list is almost always the wrong decision
- Just as with figures/tables, every reference you include in your reference list should be referred to somewhere in the main text. If a reference is never cited in the main text, it should not be included in your reference list.

- In this course we don't require you to follow a specific referencing style, but each reference in your reference list should be nicely and consistently formatted, and contain all the necessary information
  - We recommend using BibTeX or similar to generate the reference list. You can see how this works in the TeX report template we have provided.
- You are allowed to use the project description page as a reference. If you do, reference it like you would any other webpage — see the TeX template for an example.
- Don't cite Wikipedia. Don't get me wrong, Wikipedia is an amazing resource that I do encourage you to use. But you should typically use it as a starting point to find more proper sources — that is, sources with more clearly defined authorship, and that have ideally gone through a publishing and/or review process (books, articles, etc.)

Final remarks

- By now you may have the impression that we are incredibly nitpicky when it comes to these reports…
- In some sense we are, but don't be discouraged by this! We of course don't expect your reports to be perfect — in any long, written document there will always be many ways in which it could be improved, and we know that you have limited time to complete these projects.
- We only focus on these details because learning to communicate your work in a clear and professional manner is an incredibly important skill for your future career, almost regardless of career path — and it is a skill that a university education is supposed to teach you.
- A report that looks professional and reads well communicates to the reader that *We pay attention to details, so you can trust our code/results/calculations to be correct.*
  - And if your readers trust your work, then they will be much more likely to use/cite your work, so that your work can have an actual impact.
- On the other hand, if a report is full of typos, sloppy mistakes and low-quality figures, the readers will think *These authors are really sloppy, I won't trust their results.*
  - And then your work will have much less impact, since others will prefer to redo the work themselves rather than trust your work.

Grading system for reports

- We judge the project reports on the following aspects, which each have a given importance weight

Aspect	Weight
Title and abstract	1
Introduction	2
Theory/methods	4
Code	4
Results and discussion	5
Conclusions	1
References	1
Overall impression	2

Table 3: Aspects and weights considered when grading reports

- When grading a report we first score each of these aspects using a 0–5 points scale, and then combine these scores in a weighted sum using the weights above
- So the maximum score is 100 points
- Each report is independently scored by two teachers
  - If the two scores are similar we take the average as the final score
  - If the two scores are very different we take a second look together to discuss the report and agree on a final score
- While each report is scored by two teachers, only one of the teachers will write the written feedback
- When we set your final grades we take as our starting point the system described here: uio.no/studier/eksamen/karakterer/fagspesifikk-karakterbeskrivelse/mn-math

  The short summary is that our guideline for converting between scores and grades is:

  - 92 - 100 points: A
  - 77 - 91 points: B
  - 58 - 76 points: C
  - 46 - 57 points: D
  - 40 - 45 points: E
  - 0 - 39 points: F

It's important to keep in mind that these score thresholds are just to be interpreted as **guidelines** for our grading. At the end of the course we make a final, overall assessment of all the projects to decide exactly where to draw the boundaries between the different grades.

Topics in project 3

- Methods for solving *initial value problems* (ODEs)
  - (In previous projects we have focused on *boundary value problems*)
- Main algorithm: Runge-Kutta, 4th order
- Coding:
  - Object-oriented programming, classes in C++
  - More use of the Armadillo library
- Writing a proper scientific report
- Physics case: simulating a *Penning trap*

Physics of project 3: the Penning trap

- A **Penning trap** is a device for storing charged particles in a vacuum, using static $\vec{E}$ and $\vec{B}$ fields
- Below is a brief, qualitative introduction. The details you need for the project (the trap potential, the equations of motion, etc.) are given on the Project 3 page: anderkve.github.io/FYS3150/book/projects/project3
- Good old classical physics (electromagnetism + Newton's 2nd law) is sufficient for us — no quantum mechanics needed

Why not just use an electric field?

- A naive (and unphysical!) idea: Ensure that a positive particle stays put by pushing in from all sides with a static $\vec{E}$-field, as sketched in panel (a) of the figure below

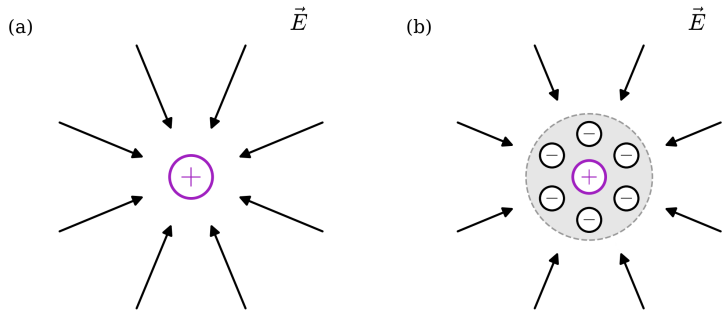


Figure 9: (a) The naive idea: confine a positive particle by having a static electric field pointing inwards from all sides. (b) By Gauss' law, an electric field pointing inwards from all sides requires a *net* negative charge at the centre — a *sink* for the field. So in addition to the positive particle we want to trap, there would have to be an even larger negative charge in the same region.

- This is not possible if we want the trap to be a vacuum!
  - Gauss' law:

    $$\nabla \cdot \vec{E} = \frac{\rho}{\epsilon_0}$$

    where ρ is the charge density
  - So the only way to get this $\vec{E}$-field configuration is to have a *sink* for the field, i.e. a *net* negative charge at the centre
  - Note that this is in addition to the positive particle we want to trap: to get the net charge negative, the region around our particle would have to contain an even larger negative charge (panel (b) in the figure)
  - $\Rightarrow$ Not a vacuum!
- We need a different solution $\Rightarrow$ the Penning trap!

The Penning trap

- Penning traps are used e.g. at CERN for antimatter research
- The trap consists of **three electrodes**: two *end caps* and one *ring*

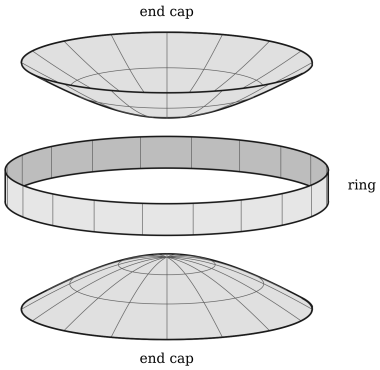


Figure 10: The three electrodes of a Penning trap: two end caps and a ring.

- Below is a cross section of the trap, viewed at right angles to the z axis

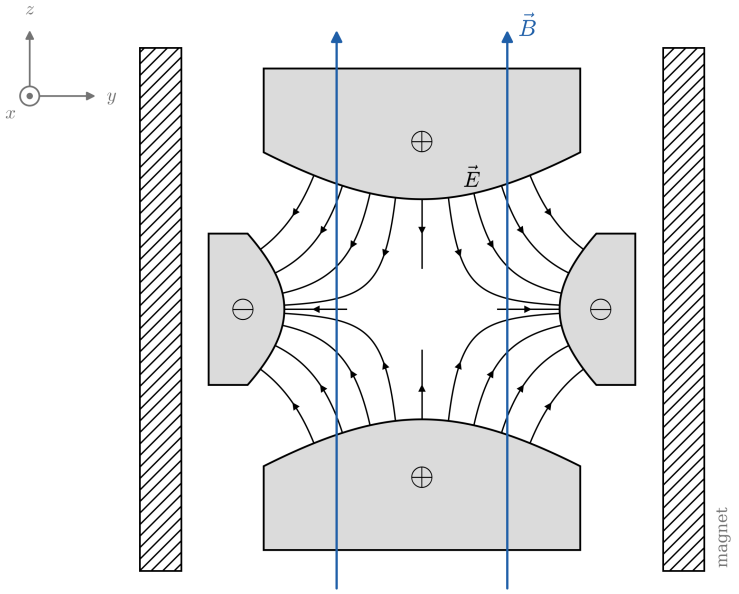


Figure 11: Cross section of a Penning trap. The end caps (top and bottom) are held at a positive potential and the ring (left and right) at a negative potential, giving the electric field lines shown in black. The whole trap sits inside a magnet, which sets up a homogeneous magnetic field $\vec{B}$ in the z direction.

- Assume a positive particle in the trap
- The $\vec{E}$-field *confines* the particle in the z direction, but *pushes it outwards* in the x and y directions
- Compensate for this with a $\vec{B}$-field in the z direction — i.e. put the trap in a magnet

  - Recall the Lorentz force: $\vec{F} = q\vec{E} + q\vec{v} \times \vec{B}$

- When a particle moves outwards in the xy direction, the $\vec{B}$-field deflects it and puts it into some orbital path
- Penning traps can be used to store / cool down charged particles
- Also used for super-precise measurements, e.g. determination of the masses of ions (by studying their movement), magnetic moments, …
- Drawback: Lots of material! Difficult to access the particles, and difficult to get particles into the trap
- One approach to filling the trap:
  - Put a neutral gas inside
  - Irradiate it with ionizing radiation: atoms → electrons + ions
  - The electrons fly away, the positive ions are trapped inside
- Another approach: Synchronize the $\vec{E}$- and $\vec{B}$-fields with a beam of particles into the trap, so that the fields are off just as the particles enter

Initial value problems

- Previously we have worked on solving *boundary value problems*, i.e. ODEs where we know the solution at the boundaries of our domain
  - We did this in both projects 1 and 2
  - After discretising our ODE using a finite difference scheme, we ended up with some matrix problem
- Now we will turn to *initial value problems*, i.e. ODEs where we know how the solution starts, and we need to evolve it forwards
  - This will be our focus in project 3
  - For initial value problems the independent variable is often time — but it doesn't have to be!
- Later in the course (project 5) we will look at *partial differential equations* (PDEs)
- We will discuss the following methods for solving initial value problems:
  - Forward Euler
  - Predictor-Corrector
  - Runge-Kutta
  - Leapfrog
  - Verlet, including *velocity Verlet*
- But before we look at the different methods, let's recap some basics about ODEs

Classification

- Since different methods work well for different types of differential equations, it's useful to remind ourselves of some basic ODE classification
- First order ODE:

$$\frac{dy}{dt} = f(t, y)$$

  - This equation is *first order*, since the highest-order derivative of y is $\frac{dy}{dt}$
  - In an initial value problem we would typically know $y(t_0)$

- Second order ODE:

$$\frac{d^2y}{dt^2} = f\left(t, y, \frac{dy}{dt}\right)$$

  - This equation is *second order*, since the highest-order derivative of y is $\frac{d^2y}{dt^2}$
  - In an initial value problem we would typically know both $y(t_0)$ and $y'(t_0)$
  - A typical example from physics would be Newton's second law:

$$\frac{d^2x}{dt^2} = \frac{1}{m}F\left(t, x, \frac{dx}{dt}\right)$$

- Linear versus non-linear ODEs:
  - Example of a *linear*, first-order ODE:

$$\frac{dy}{dt} = g^3(t)y(t)$$

    * Linear, since the highest power of y in any term is 1
  - Example of a *non-linear*, first-order ODE:

$$\frac{dy}{dt} = g^3(t)y(t) - k(t)y^2(t)$$

    * Non-linear, since the dependence on y is more complicated than a simple linear dependence (note the y^2 term)
  - In project 3, the Coulomb interaction between the charged particles in the Penning trap gives rise to a set of non-linear differential equations
  - Non-linear problems typically require a numerical approach. (Often a closed-form analytical solution does not exist.)

From a second-order equation to coupled first-order equations

- If we have to solve an mth-order ODE, we can always rewrite it as a set of m first-order ODEs
- The resulting first-order equations will typically be *coupled*
- Let's take Newton's second law in one dimension as example:

$$\frac{d^2x}{dt^2} = \frac{1}{m}F\left(t, x, \frac{dx}{dt}\right)$$

- This is a second-order differential equation for $x(t)$
- We can turn this into two first-order differential equations as follows:
  - First we *define* a new variable with a rather suggestive name: $v \equiv \frac{dx}{dt}$
  - We can now write $\frac{d^2x}{dt^2}$ as $\frac{dv}{dt}$
  - We are then left with the following two first-order differential equations:

$$\frac{dx}{dt} = v(t) \quad \text{(from the definition of } v\text{)}$$

$$\frac{dv}{dt} = \frac{1}{m}F(t, x, v) \quad \text{(from the original diff. eq.)}$$

  - These equations are coupled, since the unknown $v(t)$ appears in the diff. eq. for $x(t)$, and the unknown $x(t)$ appears in the diff. eq. for $v(t)$
- In project 3 you will use this approach to obtain a set of coupled, first-order equations that your numerical programs will solve

Local versus global errors

- Before we work our way through a series of methods for solving initial value problems, we should quickly discuss the difference between the **local error** and the **global error** of a method
- Our standard expressions for numerically computing a derivative introduce some **truncation error**
- The reason is typically that we truncate a Taylor expansion at some point
- Let's say our method for solving an initial value problem involves truncating a Taylor expansion starting from the $\mathcal{O}\left(h^k\right)$ terms in the expansion (h is step size)
- Thus, at *every time step* of our method we expect to introduce an $\mathcal{O}\left(h^k\right)$ error
  - This is referred to as the **local error**
- Now, say that we evolve our solution for a total of n time steps
- Since h is the step size, we have that $n \propto 1/h$
- Let's assume that the local errors at each step simply accumulate
- We then expect the **global error** (the final error) to be of order $\mathcal{O}\left(nh^k\right) = \mathcal{O}\left(\frac{1}{h}h^k\right) = \mathcal{O}\left(h^{k-1}\right)$

- Remember that h is a small number ($h < 1$), so the $\mathcal{O}\left(h^{k-1}\right)$ global error is larger than the $\mathcal{O}\left(h^k\right)$ local error — as expected
- We categorise different methods by referring to how their global error depends on the step size:
  - A **first-order** method has a $\mathcal{O}\left(h^1\right)$ global error
  - A **second-order** method has a $\mathcal{O}\left(h^2\right)$ global error
  - etc.
- Note that in some cases, the relationship between the local and global error will be more complicated than simply subtracting 1 from the power of the local error

Forward Euler

- Consider a first-order, ordinary differential equation:

$$\frac{dy}{dt} = f(t, y)$$

  - We seek the unknown function $y(t)$
  - We know the starting value $y(t_0)$
  - The right-hand side $f(t, y)$ is some known function of t and $y(t)$
- The **Forward Euler** algorithm for solving this is simply

$$\boxed{y_{i+1} = y_i + hf_i}$$

  - Notation: $f_i \equiv f(t_i, y_i)$
  - Local error: $\mathcal{O}\left(h^2\right)$
  - Global error: $\mathcal{O}\left(h\right)$
    - So Forward Euler is a **first-order** method
  - Forward Euler is a **single-step** method, since we only need the current point y_i to compute our next point y_{i+1}
- Forward Euler is a very simple method, and a basic building block in many other algorithms
- Alternative formulation, to more easily see the connection with later methods:

$$\boxed{\begin{aligned} k &= hf_i = hf(t_i, y_i) \\ y_{i+1} &= y_i + k \end{aligned}}$$

Derivation of Forward Euler

- First, replace $\frac{dy}{dt}$ in the differential equation with our familiar forward-difference expression:

$$\frac{y(t+h) - y(t)}{h} + \mathcal{O}(h) = f(t, y)$$

- Rearrange for $y(t+h)$:

$$y(t+h) = y(t) + hf(t, y) + \mathcal{O}\left(h^2\right)$$

- Discretise: $y(t) \to y_i$, $f(t, y) \to f_i$
- Approximate: leave out the $\mathcal{O}(h^2)$ terms
- Result:

$$y_{i+1} = y_i + hf_i$$

Forward Euler and Euler-Cromer for coupled equations

- Consider a second-order initial value problem for the unknown function $x(t)$:

$$\frac{d^2x}{dt^2} = f\left(t, x, \frac{dx}{dt}\right)$$

- As usual, we can turn this into two, coupled first-order equations:
  - Define $v \equiv \frac{dx}{dt}$
  - Then we get the two coupled equations:

$$\frac{dv}{dt} = f(t, x, v)$$

$$\frac{dx}{dt} = v(t)$$

- The Forward Euler method is then simply

$$\boxed{\begin{aligned} v_{i+1} &= v_i + hf_i \\ x_{i+1} &= x_i + hv_i \end{aligned}}$$

- A very simple improvement of this method is called **Euler-Cromer**
- It consists of simply using the new v_{i+1} in the computation of x_{i+1}

$$\boxed{\begin{aligned} v_{i+1} &= v_i + hf_i \\ x_{i+1} &= x_i + hv_{i+1} \end{aligned}}$$

- Like Forward Euler, the Euler-Cromer method is a first-order method
- But the Euler-Cromer method is also a **symplectic** method (when the force depends only on position, and possibly time, not on velocity)
  - For physics applications, this in practice means that solutions found with Euler-Cromer will nearly satisfy energy conservation
  - In contrast, a solution found with Forward Euler will typically exhibit **energy drift**, that is, the numerical errors will add up in such a way that the energy of the solution keeps increasing or decreasing
  - Symplectic methods are therefore particularly useful when simulating physics systems over long time spans. (More on symplectic methods later, when we discuss the Leapfrog and Verlet methods.)

Predictor-Corrector

- We return to our example of a first-order initial value problem:

$$\frac{dy}{dt} = f(t, y)$$

  - We seek the unknown function $y(t)$
  - We know the starting value $y(t_0)$
  - The right-hand side $f(t, y)$ is some known function of t and $y(t)$

- The **Predictor-Corrector** method can be seen as a slightly more advanced variant of Forward Euler
  - *Note:* Here we only discuss a specific, simple variant of Predictor-Corrector and refer to it as "the Predictor-Corrector method". In reality Predictor-Corrector is a *class* of methods, where different variants can e.g. achieve different order of the global error.
- Recall Forward Euler:
  - When we "shoot" our way across the time interval (t_i, t_{i+1}), we only use the gradient (i.e. the right-hand side of our differential equation) evaluated at the start point of the interval:

$$y_{i+1} = y_i + hf_i$$

- Predictor-Corrector:
  - We can get a better estimate for the true, average gradient across the time interval (t_i, t_{i+1}) if we combine the gradients at the start point (f_i) and end point (f_{i+1})
  - That is, what we *want* is a method like this:

$$y_{i+1} = y_i + h\left(\frac{f_i + f_{i+1}}{2}\right)$$

  - Complication:
    * The right-hand side $f(t, y)$ depends on $y(t)$, so to evaluate $f_{i+1} = f(t_{i+1}, y_{i+1})$ we need to know y_{i+1}
    * But y_{i+1} is exactly what we are trying to estimate in the first place…
  - Solution:
    * First, use a simple Forward Euler step to estimate (or *predict*) the unknown y_{i+1}. Notation: y^*_{i+1}
    * Then, use y^*_{i+1} to estimate f_{i+1}. Notation: f^*_{i+1}
    * Finally, use f^*_{i+1} to compute an improved estimate for y_{i+1} (or *correct* the estimate for y_{i+1})

1) Predict:

$$y^*_{i+1} = y_i + hf_i$$

$$f^*_{i+1} = f(t_{i+1}, y^*_{i+1})$$

2) Correct:

$$y_{i+1} = y_i + h\left(\frac{f_i + f^*_{i+1}}{2}\right)$$

  - Local error: $\mathcal{O}\left(h^3\right)$
  - Global error: $\mathcal{O}\left(h^2\right)$
    * So Predictor-Corrector is a **second-order** method
  - Predictor-Corrector requires *two* evaluations of the right-hand side $f(t, y)$ for each time step
  - Like Forward Euler, Predictor-Corrector is a single-step method, since it only requires keeping track of the current point y_i to get the next point y_{i+1}

- Predictor-Corrector in alternative notation, where we use two different k's to denote two different shifts in the y direction:

$$k_1 = hf_i = hf(t_i, y_i)$$

$$k_2 = hf^*_{i+1} = hf(t_i + h, y_i + k_1)$$

$$y_{i+1} = y_i + \tfrac{1}{2}(k_1 + k_2)$$

Derivation of local error for Predictor-Corrector

- Let y^{PC}_{i+1} denote the Predictor-Corrector approximation to the true y_{i+1}
- Let τ_i denote the local truncation error we introduce when going from t_i to t_{i+1}

$$\tau_i = y_{i+1} - y^{\mathsf{PC}}_{i+1}$$

- To show that the local truncation error for Predictor-Corrector is $\mathcal{O}\left(h^3\right)$ we Taylor expand y_{i+1} and y^{PC}_{i+1} in the expression above

- A note on notation: f_i' means the *total* time derivative of $f(t, y(t))$ along the solution, evaluated at t_i. Since f depends on t both directly and through $y(t)$, the chain rule gives

$$f_i' \equiv \frac{d}{dt} f(t, y(t))\bigg|_{t_i} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial y}\frac{dy}{dt} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial y} f_i$$

  where the partial derivatives are evaluated at (t_i, y_i)

- Taylor expansion for the true value y_{i+1}:

$$\begin{aligned} y_{i+1} &= y_i + h y_i' + \tfrac{1}{2} h^2 y_i'' + \mathcal{O}\left(h^3\right) \\ &= y_i + h f_i + \tfrac{1}{2} h^2 f_i' + \mathcal{O}\left(h^3\right) \end{aligned}$$

- Now let's look at y_{i+1}^{PC}

$$\begin{aligned} y_{i+1}^{\text{PC}} &= y_i + \frac{h}{2}\left(f_i + f_{i+1}^*\right) \\ &= y_i + \frac{h}{2}\left(f_i + f(t_i + h, y_i + h f_i)\right) \end{aligned}$$

- Here we need to Taylor expand $f(t_i + h, y_i + h f_i)$. This is a function of two variables (t, y), so we expand about the point (t_i, y_i) in both arguments:

$$\begin{aligned} f(t_i + h, y_i + h f_i) &= f_i + h\frac{\partial f}{\partial t} + h f_i \frac{\partial f}{\partial y} + \mathcal{O}\left(h^2\right) \\ &= f_i + h f_i' + \mathcal{O}\left(h^2\right) \end{aligned}$$

  where in the last step we recognised the total derivative f_i' from above

- Plug this into the expression for y_{i+1}^{PC}:

$$\begin{aligned} y_{i+1}^{\text{PC}} &= y_i + \frac{h}{2}\left(f_i + f_i + h f_i' + \mathcal{O}\left(h^2\right)\right) \\ &= y_i + h f_i + \tfrac{1}{2} h^2 f_i' + \mathcal{O}\left(h^3\right) \end{aligned}$$

- Now finally we can insert everything into our expression for the local truncation error τ_i and subtract:

$$\begin{aligned} \tau_i &= y_{i+1} - y_{i+1}^{\text{PC}} \\ &= \left[y_i + h f_i + \tfrac{1}{2} h^2 f_i' + \mathcal{O}\left(h^3\right)\right] - \left[y_i + h f_i + \tfrac{1}{2} h^2 f_i' + \mathcal{O}\left(h^3\right)\right] \\ &= \mathcal{O}\left(h^3\right) \end{aligned}$$

- So we see the local truncation error is indeed $\mathcal{O}\left(h^3\right)$: the h and h^2 terms cancel exactly, and the first surviving term is of order h^3
  - (If you carry the expansions one order further, the h^3 coefficient comes out as $-\frac{1}{12}f_i'' + \frac{1}{4}\frac{\partial f}{\partial y}f_i'$. We don't need it here, and it requires the second partial derivatives of f, so we stop at $\mathcal{O}\left(h^3\right)$.)

Runge-Kutta

- We stick with our example of a first-order initial value problem:

$$\frac{dy}{dt} = f(t, y)$$

- The **Runge-Kutta** methods can be seen as further generalisations of Predictor-Corrector
- Recall from above that Predictor-Corrector:
  - can be seen as a second-order Taylor expansion of y_{i+1}
  - achieved an $\mathcal{O}\left(h^2\right)$ global error
  - used multiple (two) estimates of the gradient, computed at different points in the (t_i, t_{i+1})
  - combined the gradient estimates in a weighted sum (a simple average) when computing the final y_{i+1}
- An *m-th order Runge-Kutta method*:
  - corresponds to an m-th order Taylor expansion of y_{i+1}
  - will have an $\mathcal{O}\left(h^m\right)$ global error
  - will use multiple estimates of the gradient, computed at different points in the (t_i, t_{i+1})
  - will combine these gradient estimates in a weighted sum when computing the final y_{i+1}
- For each choice of order m, there are different possible choices for:
  - where in the interval (t_i, t_{i+1}) the gradient should be evaluated
  - how to do the weighted combination of these gradient evaluations
- So the Predictor-Corrector method we considered corresponds to a specific *second-order Runge-Kutta* method, where the gradient is evaluated at the points t_i and t_{i+1} and these gradients are combined with equal weights

- How to choose the order m? Must strike a balance between a low number of evaluations of $f(t, y)$ (computation time) and a low global error (accuracy)
  - For $m \leq 4$ an m-th order Runge-Kutta method needs exactly m evaluations of f per step. For $m \geq 5$ it needs *more* evaluations than the order, so the cost grows faster than the accuracy. This is one reason why the fourth-order method is so popular.
- A very common choice: 4-th order Runge-Kutta method (**RK4**):
  - With RK4, reducing the step size h by a factor 10 will reduce the global truncation error by a factor 10,000
  - RK4 requires four f evaluations per step, but the high accuracy means we can use much larger step sizes h than with e.g. Predictor-Corrector or Forward Euler
  - So with the right step size, RK4 is typically more efficient than Predictor-Corrector or Forward Euler
- The RK4 algorithm:

$$
\begin{aligned}
k_1 &= hf(t_i, y_i) \\
k_2 &= hf(t_i + \tfrac{1}{2}h, y_i + \tfrac{1}{2}k_1) \\
k_3 &= hf(t_i + \tfrac{1}{2}h, y_i + \tfrac{1}{2}k_2) \\
k_4 &= hf(t_i + h, y_i + k_3) \\
y_{i+1} &= y_i + \tfrac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}
$$

  - Local error: $\mathcal{O}\left(h^5\right)$
  - Global error: $\mathcal{O}\left(h^4\right)$
  - Note the use of four different evaluations of $f(t, y)$:
    - One at the start point t_i
    - Two different estimates at the middle point $t_i + \frac{1}{2}h$
    - One estimate at the end point $t_{i+1} = t_i + h$
  - Like Forward Euler and Predictor-Corrector, Runge-Kutta is a single-step method

RK4 and Simpson's rule for integration

- The form of the weighted combination $\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ in RK4 can be seen as arising from Simpson's rule for numerical integration
- Simpson's rule:

$$\int_a^b f(x)\,dx \approx \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]$$

- To see this, start from the Fundamental Theorem of Calculus expressed with our unknown function $y(t)$:

$$\int_{t_a}^{t_b} \frac{dy}{dt}\,dt = y(t_b) - y(t_a)$$

- Choose the integration end points t_a and t_b to be the end points t_i and t_{i+1} for one step of the RK4 method
- Then $y(t_a) = y_i$ and $y(t_b) = y_{i+1}$
- Also, insert our differential equation $\frac{dy}{dt} = f(t, y(t))$ for the integrand
- We then get

$$\int_{t_i}^{t_{i+1}} f(t, y(t))\,dt = y_{i+1} - y_i$$

- Rearrange for y_{i+1}

$$y_{i+1} = y_i + \int_{t_i}^{t_{i+1}} f(t, y(t))\,dt$$

- We can solve the time step integral (approximately) using Simpson's rule

$$\begin{aligned} y_{i+1} &\approx y_i + \frac{t_{i+1} - t_i}{6}\left[f_i + 4f_{i+\frac{1}{2}} + f_{i+1}\right] \\ &= y_i + \frac{1}{6}\left[hf_i + 4hf_{i+\frac{1}{2}} + hf_{i+1}\right] \\ &= y_i + \frac{1}{6}\left[hf_i + 2hf_{i+\frac{1}{2}} + 2hf_{i+\frac{1}{2}} + hf_{i+1}\right] \end{aligned}$$

where the shorthand notation $f_{i+\frac{1}{2}}$ means $f_{i+\frac{1}{2}} = f(t_i + \frac{1}{2}h, y(t_i + \frac{1}{2}h))$

- The last line above has exactly the form of the RK4 algorithm, except that in RK4 the two occurrences of $f_{i+\frac{1}{2}}$ and the f_{i+1} are replaced by estimates

RK4 for sets of coupled equations

- In the Runge-Kutta algorithm we have to compute a series of k's
- Each new such k computation relies on estimating the right-hand side of the differential equation at a new half/whole time step away from the current time t_i
- Now assume we have a system of *coupled* differential equations, e.g. two first-order equations describing a position x and a velocity v:

$$\frac{dx}{dt} = v(t)$$

$$\frac{dv}{dt} = f(t, x, v)$$

- When solving these equations with RK4, we will for each time step compute two sets of k's:
  - One set for estimating x_{i+1}: $k_{x,1}, k_{x,2}, k_{x,3}, k_{x,4}$
  - One set for estimating v_{i+1}: $k_{v,1}, k_{v,2}, k_{v,3}, k_{v,4}$
- To get all these k evaluations correct, we need to perform them "in sync":

$$
\begin{aligned}
k_{x,1} &= hv_i \\
k_{v,1} &= hf(t_i, x_i, v_i) \\
\\
k_{x,2} &= h(v_i + \tfrac{1}{2}k_{v,1}) \\
k_{v,2} &= hf(t_i + \tfrac{1}{2}h, x_i + \tfrac{1}{2}k_{x,1}, v_i + \tfrac{1}{2}k_{v,1}) \\
\\
k_{x,3} &= h(v_i + \tfrac{1}{2}k_{v,2}) \\
k_{v,3} &= hf(t_i + \tfrac{1}{2}h, x_i + \tfrac{1}{2}k_{x,2}, v_i + \tfrac{1}{2}k_{v,2}) \\
\\
k_{x,4} &= h(v_i + k_{v,3}) \\
k_{v,4} &= hf(t_i + h, x_i + k_{x,3}, v_i + k_{v,3}) \\
\\
x_{i+1} &= x_i + \tfrac{1}{6}(k_{x,1} + 2k_{x,2} + 2k_{x,3} + k_{x,4}) \\
v_{i+1} &= v_i + \tfrac{1}{6}(k_{v,1} + 2k_{v,2} + 2k_{v,3} + k_{v,4})
\end{aligned}
$$

Example: many-particle simulation

- To see how this plays out in practice, consider a case similar to that in project 3:
- We are writing code to simulate a collection of interacting particles (in the classical physics sense) in three space dimensions
- For each particle we will have a position vector $\vec{r}$ and a velocity vector $\vec{v}$
- So, for each particle, we need to solve a set of differential equations on the form

$$\frac{d\vec{r}}{dt} = \vec{v}$$

$$\frac{d\vec{v}}{dt} = \frac{\vec{F}}{m}$$

  where $\vec{F}$ is the total force acting on the particle and m is the particle mass
- Let's say the particles interact via Coulomb interactions
- Therefore, the total force $\vec{F}$ acting on a particle depends on the current positions of all the other particles in the simulation
  - This means the equations above will not only be coupled, but also non-linear, due to the Coulomb interaction terms in $\vec{F}$

- Since we are working in three space dimensions, the equations above represent six differential equations for each particle (three position components and three velocity components)
- So to evolve one particle one time step will involve computing $6 \times 4 = 24$ different k's
- To keep track of all these k's it's useful to simply work at the level of 3-vectors, i.e. use vectors also for the k's:
  - For position: $\vec{k}_{\vec{r},1}, \vec{k}_{\vec{r},2}, \vec{k}_{\vec{r},3}, \vec{k}_{\vec{r},4}$
  - For velocity: $\vec{k}_{\vec{v},1}, \vec{k}_{\vec{v},2}, \vec{k}_{\vec{v},3}, \vec{k}_{\vec{v},4}$
- To evolve this simulation one time step using the RK4 algorithm, we would do something like the following:
  - Store a copy of the current state of the simulation (all the particle positions $\vec{r}_i$ and velocities $\vec{v}_i$). We will need it for all the steps below.
  - For each particle: compute $\vec{k}_{\vec{r},1}$ and $\vec{k}_{\vec{v},1}$ using the stored state
  - For each particle: set the temporary position and velocity to $\vec{r}_i + \frac{1}{2}\vec{k}_{\vec{r},1}$ and $\vec{v}_i + \frac{1}{2}\vec{k}_{\vec{v},1}$
  - For each particle: compute $\vec{k}_{\vec{r},2}$ and $\vec{k}_{\vec{v},2}$ using the temporary state
  - For each particle: set the temporary position and velocity to $\vec{r}_i + \frac{1}{2}\vec{k}_{\vec{r},2}$ and $\vec{v}_i + \frac{1}{2}\vec{k}_{\vec{v},2}$
  - For each particle: compute $\vec{k}_{\vec{r},3}$ and $\vec{k}_{\vec{v},3}$ using the temporary state
  - For each particle: set the temporary position and velocity to $\vec{r}_i + \vec{k}_{\vec{r},3}$ and $\vec{v}_i + \vec{k}_{\vec{v},3}$
  - For each particle: compute $\vec{k}_{\vec{r},4}$ and $\vec{k}_{\vec{v},4}$ using the temporary state
  - Final step: For each particle, perform the proper RK4 update of position and velocity using the stored $\vec{r}_i$ and $\vec{v}_i$, together with all the $\vec{k}_{\vec{r},j}$ and $\vec{k}_{\vec{v},j}$ computed above

Next algorithms: Leapfrog and Verlet

- We will look at two more methods for solving initial value problems:
  - The leapfrog method
  - The Verlet method
- Most common application: physics simulations based on Newton's second law
  - Second-order diff. eq. → two coupled first-order diff. eqs.
- Useful properties:
  - *Time-symmetric* (or *reversible*) methods
    * A time-symmetric method is one where if we first evolve the solution y_i forwards with a step h to get y_{i+1}, and then evolve y_{i+1} backwards with a step $-h$, we get back exactly to y_i
      · The methods we have considered so far (Forward Euler, Predictor-Corrector, RK4) are *not* time-symmetric. Here is a quick demonstration for Forward Euler:

$$\begin{aligned} y_{i+1} &= y_i + hf(t_i, y_i) \quad \text{(forward)} \\ \hat{y}_i &= y_{i+1} - hf(t_{i+1}, y_{i+1}) \quad \text{(backward)} \\ &= y_i + hf(t_i, y_i) - hf(t_{i+1}, y_{i+1}) \\ &= y_i + h\left[f(t_i, y_i) - f(t_{i+1}, y_{i+1})\right] \\ &\neq y_i \end{aligned}$$

    * Time-symmetry/reversibility is a good property for simulating physical systems where we know the underlying laws are reversible
  - *Symplectic* methods
    * Algorithms used for studying *Hamiltonian systems* (systems that can be described by Hamilton's equations from classical mechanics)
    * Conserve phase space volumes (e.g conserve area in a (x, p) plane), which means they conserve the statistical properties of Hamiltonian systems, even if each individual solution differs from the corresponding true solution.
    * Common property: (almost) respect energy conservation.
    * Technically: the almost exactly conserved quantity is a Hamiltonian that is slightly different (perturbed) from the original one.

        * Practical consequence: Our energy estimate will oscillate in a bounded way, around a central value that is slightly different from the true energy.
        * Example: Simulate planetary system
            · Get stable orbits.
            · Can run the simulation for a long time without having planets drift away.
            · See this GIF for an example where the Leapfrog method gives energy-conserving solutions while Forward Euler does not: commons.wikimedia.org/wiki/File:Euler_leapfrog_comparison.gif
    - Symplectic algorithms are typically used for physics simulations where we need to simulate a system over a long time period.
        * Examples: astrophysics, molecular dynamics
    - The symplectic property assumes that the step size h is kept fixed.
- Drawbacks:
    - The simplest algorithms assume that the force (or acceleration) does not depend on the velocity
        * OK for e.g. a gravitational force
        * Not OK for the Lorentz force in Project 3: $\vec{F} = q\vec{E} + q\vec{v} \times \vec{B}$
- Problem-specific algorithms can get around this (but may be more complicated to implement).
    - A famous example for handling the Lorentz force ($\vec{F} = q\vec{E} + q\vec{v} \times \vec{B}$) is the **Boris algorithm** or **Boris push**. It modifies the integration step to account for the rotational effect of the magnetic field.
    - A standard method e.g. in simulations of plasma

> If this was a course specifically focusing on many-particle simulations it would be natural to use a leapfrog/Verlet algorithm with a Boris push to simulate the Penning trap in Project 3. However, since RK4 is such an important general-purpose method for ODEs, we prioritise using Project 3 to get some hands-on experience with RK4.

Technical aside: what does "symplectic" actually mean?

> This aside is for the especially interested.

- **Hamiltonian systems**
  - A system is *Hamiltonian* if its equations of motion follow from a single function $H(x, p, t)$, the *Hamiltonian*, through *Hamilton's equations*:

$$\frac{dx}{dt} = \frac{\partial H}{\partial p}, \qquad \frac{dp}{dt} = -\frac{\partial H}{\partial x}$$

  - Here p is the *canonical momentum*. For a particle moving in a potential $V(x)$ it is simply $p = mv$.
  - Example: $H = \frac{p^2}{2m} + V(x)$ gives $\frac{dx}{dt} = \frac{p}{m}$ and $\frac{dp}{dt} = -\frac{dV}{dx} = F(x)$, i.e. Newton's second law.
  - In practice: a Hamiltonian system means mechanics without friction. A system with friction or other dissipation is not Hamiltonian.
  - If H does not depend explicitly on time, then H is conserved and equals the total energy. (Insert Hamilton's equations into $\frac{dH}{dt} = \frac{\partial H}{\partial x}\frac{dx}{dt} + \frac{\partial H}{\partial p}\frac{dp}{dt}$ to see that it vanishes.)
  - The state (x, p) is a point in *phase space*. With N coordinates, phase space has $2N$ dimensions.
- **Symplectic maps**
  - One time step of a numerical method is a map $(x_i, p_i) \to (x_{i+1}, p_{i+1})$ from phase space to phase space.
  - Let $\mathbf{M}$ be the Jacobian matrix of this map, i.e. the matrix of derivatives of (x_{i+1}, p_{i+1}) with respect to (x_i, p_i).
  - The method is *symplectic* if, for every point in phase space and every step size h,

$$\mathbf{M}^T \mathbf{J} \mathbf{M} = \mathbf{J}, \qquad \mathbf{J} = \begin{bmatrix} 0 & \mathbf{I} \\ -\mathbf{I} & 0 \end{bmatrix}$$

    where $\mathbf{I}$ is the $N \times N$ identity matrix.
  - In the language of classical mechanics: each time step is a *canonical transformation*, i.e. a change of variables that preserves the form of Hamilton's equations.
  - Let's look at what this condition means for a single coordinate x and momentum p, i.e. $N = 1$. Then $\mathbf{M}$ is a 2×2 matrix, and $\mathbf{J}$ is just

$$\mathbf{J} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

* Write $\mathbf{M}$ with general elements and compute $\mathbf{M}^T\mathbf{J}\mathbf{M}$:

$$\mathbf{M} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad \Rightarrow \quad \mathbf{M}^T\mathbf{J}\mathbf{M} = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$
$$= \begin{bmatrix} 0 & ad-bc \\ -(ad-bc) & 0 \end{bmatrix}$$
$$= (\det \mathbf{M})\, \mathbf{J}$$

* So for a single coordinate, the symplectic condition $\mathbf{M}^T\mathbf{J}\mathbf{M} = \mathbf{J}$ is simply $\det \mathbf{M} = 1$.
* Geometric meaning: a small rectangle in the (x, p) plane with sides Δx and Δp is mapped by one time step to a small parallelogram with area $|\det \mathbf{M}|\, \Delta x\, \Delta p$. So $\det \mathbf{M} = 1$ means that *one time step conserves area in the* (x, p) *plane*. Any larger region can be covered by small rectangles, so its area is conserved too.

– For more coordinates, the condition $\mathbf{M}^T\mathbf{J}\mathbf{M} = \mathbf{J}$ is stronger than conservation of the $2N$-dimensional phase space volume, but volume conservation follows from it.

– The *exact* time evolution of a Hamiltonian system is itself a symplectic map (a theorem by Poincaré). A symplectic method reproduces this property of the true dynamics, even though it reproduces the trajectory only approximately.

– Consequence: with a fixed step size h, a symplectic method of order p is, to very good approximation, the exact solution of a slightly *modified* Hamiltonian $\tilde{H} = H + \mathcal{O}(h^p)$. The method conserves $\tilde{H}$, so the computed energy H oscillates in a band of width $\mathcal{O}(h^p)$ around the true value instead of drifting away.

- **Example: Forward Euler versus Euler-Cromer**

– Take $H = \frac{p^2}{2m} + V(x)$ with a single coordinate, so $\frac{dp}{dt} = F(x) = -\frac{dV}{dx}$ and the symplectic condition is $\det \mathbf{M} = 1$. Below, all derivatives $\frac{dF}{dx}$ are evaluated at x_i.

– Forward Euler:

$$p_{i+1} = p_i + hF(x_i)$$
$$x_{i+1} = x_i + \tfrac{h}{m} p_i$$

$$\mathbf{M} = \begin{bmatrix} \frac{\partial x_{i+1}}{\partial x_i} & \frac{\partial x_{i+1}}{\partial p_i} \\ \frac{\partial p_{i+1}}{\partial x_i} & \frac{\partial p_{i+1}}{\partial p_i} \end{bmatrix} = \begin{bmatrix} 1 & \frac{h}{m} \\ h\frac{dF}{dx} & 1 \end{bmatrix}, \qquad \det \mathbf{M} = 1 - \tfrac{h^2}{m}\frac{dF}{dx} \neq 1$$

* So Forward Euler is *not* symplectic. For an attractive force ($\frac{dF}{dx} < 0$) the phase space

area grows by a factor $1 + \frac{h^2}{m}\left|\frac{dF}{dx}\right|$ every step. This is the energy drift.

- Euler-Cromer, i.e. use the updated p_{i+1} in the position step:

$$\begin{aligned} p_{i+1} &= p_i + hF(x_i) \\ x_{i+1} &= x_i + \tfrac{h}{m}p_{i+1} = x_i + \tfrac{h}{m}p_i + \tfrac{h^2}{m}F(x_i) \end{aligned}$$

$$\mathbf{M} = \begin{bmatrix} 1 + \frac{h^2}{m}\frac{dF}{dx} & \frac{h}{m} \\ h\frac{dF}{dx} & 1 \end{bmatrix}, \qquad \det \mathbf{M} = \left(1 + \tfrac{h^2}{m}\frac{dF}{dx}\right) - \tfrac{h}{m} \cdot h\frac{dF}{dx} = 1$$

  * So Euler-Cromer *is* symplectic. The tiny change of using p_{i+1} instead of p_i in the position update is exactly what makes the map area-preserving.

- Similarly, the Leapfrog and Verlet methods we introduce below are symplectic.

Leapfrog

- We will again consider our first-order initial value problem:

$$\frac{dy}{dt} = f(t, y)$$

- Recall that we have seen several ways to express the first derivative:

$$\begin{aligned} \left.\frac{dy}{dt}\right|_{t_i} &= y_i' = \frac{y_{i+1} - y_i}{h} + \mathcal{O}\left(h\right), && \text{(two-point, forward difference)} \\ \left.\frac{dy}{dt}\right|_{t_i} &= y_i' = \frac{y_i - y_{i-1}}{h} + \mathcal{O}\left(h\right), && \text{(two-point, backward difference)} \\ \left.\frac{dy}{dt}\right|_{t_i} &= y_i' = \frac{y_{i+1} - y_{i-1}}{2h} + \mathcal{O}\left(h^2\right), && \text{(three-point, central difference)} \end{aligned}$$

- Leapfrog idea: Use the central difference scheme for the derivative in our diff. eq.

$$\frac{y_{i+1} - y_{i-1}}{2h} + \mathcal{O}\left(h^2\right) = f_i$$

  - This is the reason why Leapfrog is a time-symmetric method: the central difference expression for y_i' is symmetric in that it uses both y_{i+1} and y_{i-1} (with equal weights)

- Rearrange for y_{i+1} to get the Leapfrog algorithm:

$$\boxed{y_{i+1} = y_{i-1} + 2hf_i}$$

- Local error $\mathcal{O}\left(h^3\right)$, global error $\mathcal{O}\left(h^2\right)$
- This is the same global error as for the Predictor-Corrector method we considered
- In contrast to PC, the Leapfrog method only requires one evaluation of $f(t, y)$ per step. (PC requires two evaluations per step.)
- But, for Leapfrog we must keep track of the previous step y_{i-1} in order to compute y_{i+1}. (It is not sufficient to know the current step y_i.)
  - This is an example of a *multi-step* (two-step) method
- The method can be generalised to use more of the past steps
- *A small drawback*: The Leapfrog method is not *self-starting*, since computing the first step y_1 would require the non-existing y_{-1}
  - *Solution*: Compute y_1 with a simple Forward Euler step ($y_1 = y_0 + hf_0$) or some higher-order self-starting method, then use the Leapfrog for y_2 and onwards

Leapfrog and velocity Verlet for coupled equations

- Leapfrog and Verlet are popular methods for physics simulations solving Newton's second law
- Assume the force does *not* depend on velocity:

$$\frac{d^2x}{dt^2} = \frac{1}{m}F(t, x) = a(t, x)$$

- As we know, this second-order diff. eq. gives us two, coupled first-order diff. eqs. to solve:

$$\frac{dv}{dt} = a(t, x)$$

$$\frac{dx}{dt} = v(t)$$

  - So the acceleration $a(t, x)$ plays the role of $f(t, y)$ from the previous section.
- We will solve this using a staggered scheme where position and velocity leap over each other in time

- First some notation:

$$t_{i-\frac{1}{2}} \equiv t_i - \frac{1}{2}h$$

such that

$$x_{i-\frac{1}{2}} \equiv x(t_i - \tfrac{1}{2}h)$$

$$v_{i-\frac{1}{2}} \equiv v(t_i - \tfrac{1}{2}h)$$

- The algorithm is then:

$$\boxed{\begin{aligned} v_{i+\frac{1}{2}} &= v_{i-\frac{1}{2}} + ha_i \\ x_{i+1} &= x_i + hv_{i+\frac{1}{2}} \end{aligned}}$$

- Note:
  - The computation of $v_{i+\frac{1}{2}}$ uses the acceleration at the midpoint of $(t_{i-\frac{1}{2}}, t_{i+\frac{1}{2}})$
  - The computation of x_{i+1} then uses the velocity at the midpoint of (t_i, t_{i+1})
  - Since we have assumed that a_i does not depend on the velocity, i.e. that $a_i = a(t_i, x_i)$, it's OK that we never evaluate v at the same times as x and a
- Local error $\mathcal{O}\left(h^3\right)$, global error $\mathcal{O}\left(h^2\right)$
- If we also want to know the final velocity, we can use the above method and then for the very last time step add the computation

$$\boxed{v_{i+1} = v_{i+\frac{1}{2}} + \tfrac{1}{2}ha_{i+1}}$$

- However, we would often like to know v at all the same time steps as x, e.g. to compute the energy or momentum as function of time
- For this we can use a synchronised form of the Leapfrog, which is also known as **velocity Verlet**:
  - The so-called "kick-drift-kick" formulation looks like this:

$$
\begin{aligned}
v_{i+\frac{1}{2}} &= v_i + \tfrac{1}{2}ha_i \quad \text{(kick)} \\
x_{i+1} &= x_i + hv_{i+\frac{1}{2}} \quad \text{(drift)} \\
v_{i+1} &= v_{i+\frac{1}{2}} + \tfrac{1}{2}ha_{i+1} \quad \text{(kick)}
\end{aligned}
$$

- If we simply insert the first equation into the second and third equations, we get another common formulation:

$$
\begin{aligned}
x_{i+1} &= x_i + hv_i + \tfrac{1}{2}h^2a_i \\
v_{i+1} &= v_i + h\left(\frac{a_i + a_{i+1}}{2}\right)
\end{aligned}
$$

- In both formulations, note how the last line makes use of our assumption that we don't need to know v_{i+1} in order to evaluate a_{i+1}
  * Recall the Predictor-Corrector method: There we had a similar expression for y_{i+1} in terms of f_i and f_{i+1}, but had to replace f_{i+1} with an approximation f^*_{i+1} to get around the fact that f_{i+1} depended on the unknown y_{i+1}
- In the synchronised form it looks like we have to evaluate the acceleration a twice per time step
  * (Remember, evaluating a can be computationally expensive, e.g. if you have to loop over the forces coming from all particles in a many-particle simulation)
  * However, it's not so bad: The a_{i+1} we evaluate for the v_{i+1} expression will be the same as the a_i we need when we move to the next time step. So we can store a_{i+1} in memory and avoid evaluating a_i in the next time step.
- Single-step or multi-step? Self-starting?
  * In the previous section we considered a single dependent variable y and saw that Leapfrog was a two-step method to find y_{i+1} from y_i and y_{i-1}. It was not self-starting.
  * Here, when considering a case with two coupled dependent variables (x, v) and using the velocity Verlet formulation (i.e. the synchronised version), we get a single-step method to find (x_{i+1}, v_{i+1}) from (x_i, v_i). So it is self-starting.

Derivation of the Leapfrog for coupled equations

- To derive the expressions for the coupled Leapfrog method is simple:

  1. In the equation

  $$\frac{dv}{dt} = a(t, x)$$

  use a central difference for the first derivative, centred on t_i and with a step size $\frac{1}{2}h$:

  $$\frac{v_{i+\frac{1}{2}} - v_{i-\frac{1}{2}}}{2(\frac{1}{2}h)} + \mathcal{O}\left(h^2\right) = a_i$$

  $$\rightarrow v_{i+\frac{1}{2}} = v_{i-\frac{1}{2}} + ha_i + \mathcal{O}\left(h^3\right)$$

  2. Similarly, for the equation

  $$\frac{dx}{dt} = v(t)$$

  use a central difference centred on $t_{i+\frac{1}{2}}$ and a step size $\frac{1}{2}h$:

  $$\frac{x_{i+1} - x_i}{2(\frac{1}{2}h)} + \mathcal{O}\left(h^2\right) = v_{i+\frac{1}{2}}$$

  $$\rightarrow x_{i+1} = x_i + hv_{i+\frac{1}{2}} + \mathcal{O}\left(h^3\right)$$

Verlet

- Leapfrog, Verlet, and velocity Verlet are all closely related
  - What names are used differ between fields
- By simply “Verlet” we typically mean **position Verlet**
- This is a method to solve a second-order ODE on the form

  $$\frac{d^2x}{dt^2} = a(t, x)$$

  where we *don’t* rewrite it as two first-order ODEs.
- So in the context of a mechanics problem, this means we don’t explicitly compute and store the velocity v at each time step.
  - Note that we are again assuming that $a(t, x)$ does not depend on velocity

- One way to derive the Verlet update expression: Insert our familiar expression for the second derivative, to obtain

$$\frac{x_{i+1} - 2x_i + x_{i-1}}{h^2} + \mathcal{O}\left(h^2\right) = a(t, x)$$
$$x_{i+1} - 2x_i + x_{i-1} = h^2 a(t, x) + \mathcal{O}\left(h^4\right)$$

- Rearrange for x_{i+1} and approximate:

$$\boxed{x_{i+1} = 2x_i - x_{i-1} + h^2 a_i}$$

  - Local error in position is $\mathcal{O}\left(h^4\right)$
  - But global error is still $\mathcal{O}\left(h^2\right)$
    * Cumulative error after n steps is $\frac{(n-1)n}{2}\mathcal{O}\left(h^4\right) = \mathcal{O}\left(h^2\right)$
    * Connected to the fact that we are here solving a *second-order* equation directly, not a first-order equation
      · In the continuous case you would do a double integral over time to get $x(t)$
      · In the discrete case this leads to a double sum in the expression for how the error accumulates
  - The velocity v is not used/needed in the update expression for x_{i+1}
  - A two-step method, not self-starting.

- The position Verlet algorithm is algebraically equivalent to velocity Verlet
  - To see this, start with the velocity Verlet expressions

$$x_{i+1} = x_i + hv_i + \tfrac{1}{2}h^2 a_i \quad (1)$$
$$v_{i+1} = v_i + \tfrac{1}{2}h\left(a_i + a_{i+1}\right) \quad (2)$$

  - And once more, but now for the previous time step:

$$x_i = x_{i-1} + hv_{i-1} + \tfrac{1}{2}h^2 a_{i-1} \quad (3)$$
$$v_i = v_{i-1} + \tfrac{1}{2}h\left(a_{i-1} + a_i\right) \quad (4)$$

  - We can now use (3) and (4) to replace hv_i in (1) with an expression just involving position and acceleration

- Multiply (4) by h to get

$$hv_i = hv_{i-1} + \tfrac{1}{2}h^2 a_{i-1} + \tfrac{1}{2}h^2 a_i$$

- Then use (3) to replace $hv_{i-1} + \frac{1}{2}h^2 a_{i-1}$ to get the expression

$$hv_i = x_i - x_{i-1} + \tfrac{1}{2}h^2 a_i$$

- If we insert this for hv_i in (1) we finally get

$$\begin{aligned} x_{i+1} &= x_i + \left[x_i - x_{i-1} + \tfrac{1}{2}h^2 a_i\right] + \tfrac{1}{2}h^2 a_i \\ x_{i+1} &= 2x_i - x_{i-1} + h^2 a_i \end{aligned}$$

  which is exactly the **position Verlet** expression

Algorithm classification

- We use several concepts when classifying methods for solving ODEs, and we have already encountered some of these
- Below we will briefly summarise
  - consistency
  - order of global error
  - single-step vs multi-step
  - stability
  - explicit vs implicit

Consistency

- We think of a finite difference method as a way to *approximately* solve our *original* differential equation
- A different perspective is that when $h \neq 0$, a finite difference method is actually solving a slightly *different* differential equation, and it is solving this equation *exactly*
- This equation is called *the modified differential equation* (MDE)
- A method is **consistent** if

  $$[\text{modified diff. eq.}] \to [\text{original diff. eq.}]$$

  when $h \to 0$
- Another way to phrase this: A method is **consistent** if the local truncation error τ of a single step vanishes *faster than* h when $h \to 0$, i.e. if $\tau/h \to 0$
  - Why faster than h? The local error is the error *per step*, but the number of steps needed to reach a fixed final time grows as $n \propto 1/h$. So the total error is roughly $n\tau \propto \tau/h$. For this to vanish we need $\tau/h \to 0$.
  - Example of an inconsistent method: $y_{i+1} = y_i$ (do nothing). The local error $y_{i+1} - y_i \approx hf_i$ does go to zero when $h \to 0$, but after $n \propto 1/h$ steps we are still at y_0, so the total error never improves.
- Let's look at an example of how to determine the modified differential equation
- Example: MDE for Forward Euler

- Original diff. eq.:

$$\frac{dy}{dt} = f(t, y)$$

- Assume there is a modified differential equation on the form

$$\frac{du}{dt} = f(t, u) + ha_1(t, u) + h^2 a_2(t, u) + \ldots \tag{$\star$}$$

  and that the Forward Euler method

$$u(t + h) = u(t) + hf(t, u)$$

  solves *this* differential equation *exactly*

- To find the modified equation we must now determine the unknown functions $a_1(t, u)$, $a_2(t, u)$, …in Eq. ($\star$). Let's focus on a_1 since this will be the leading term (lowest power of h)
- Our strategy is:
  - Start from a general Taylor-expansion for $u(t + h)$
  - Use Eq. ($\star$) to get an expansion that involves the $a_1(t, u)$, $a_2(t, u)$, …
  - Match this to the simple Forward Euler expression to get requirements that determine $a_1(t, u)$, $a_2(t, u)$, …
- Taylor-expansion for $u(t + h)$:

$$u(t + h) = u(t) + h\frac{du}{dt} + \tfrac{1}{2}h^2\frac{d^2u}{dt^2} + \mathcal{O}\left(h^3\right)$$

- For $\frac{du}{dt}$ we can insert Eq. ($\star$) directly
- For $\frac{d^2u}{dt^2}$ we can use Eq. ($\star$) to compute it:

$$\begin{aligned}
\frac{d^2u}{dt^2} &= \frac{d}{dt}\left[\frac{du}{dt}\right] \\
&= \frac{d}{dt}f(t, u(t)) + h\frac{d}{dt}a_1(t, u) + \mathcal{O}\left(h^2\right) \\
&= \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u}\frac{du}{dt} + h\left[\frac{\partial a_1}{\partial t} + \frac{\partial a_1}{\partial u}\frac{du}{dt}\right] + \mathcal{O}\left(h^2\right)
\end{aligned}$$

- Insert the expressions for $\frac{du}{dt}$ and $\frac{d^2u}{dt^2}$ into the Taylor expansion, only keeping terms up to $\mathcal{O}\left(h^2\right)$

- Insert this into the expansion for $u(t+h)$:

$$u(t+h) = u(t) + h\left[f(t,u) + ha_1(t,u) + \mathcal{O}\left(h^2\right)\right] + \tfrac{1}{2}h^2\left[\frac{\partial f}{\partial t} + \frac{\partial f}{\partial u}\frac{du}{dt} + h\left[\ldots\right] + \mathcal{O}\left(h^2\right)\right] + \mathcal{O}\left(h^3\right)$$

- Since we will only keep terms up to $\mathcal{O}\left(h^2\right)$, we can use Eq. ($\star$) to simply replace the $\frac{du}{dt}$ factor in the last line with $f(t,u)$
- We then get

$$u(t+h) = u(t) + hf(t,u) + h^2\left[a_1(t,u) + \frac{1}{2}\left(\frac{\partial f}{\partial t} + \frac{\partial f}{\partial u}f(t,u)\right)\right] + \mathcal{O}\left(h^3\right)$$

- Our starting point was that the function u is the solution to the differential equation that the Forward Euler method solves exactly. So our long expansion for $u(t+h)$ above must match the simple Forward Euler expression

$$u(t+h) = u(t) + hf(t,u)$$

- This means the unknown $a_1(t,u)$ must make the h^2 term vanish, i.e.

$$a_1(t,u) = -\frac{1}{2}\left(\frac{\partial f}{\partial t} + \frac{\partial f}{\partial u}f(t,u)\right)$$

- Similarly, we could have determined $a_2(t,u)$ by requiring that the h^3 terms must vanish, etc.
- So, for Forward Euler our modified differential equation is

$$\frac{du}{dt} = f(t,u) - \frac{1}{2}h\left(\frac{\partial f}{\partial t} + \frac{\partial f}{\partial u}f(t,u)\right) + \mathcal{O}\left(h^2\right)$$

- First we see, as expected, that when $h \to 0$, then the modified differential equation takes the same form as our original diff. eq.

$$\frac{dy}{dt} = f(t,y)$$

  so Forward Euler is a *consistent* method
- But the MDE is useful also in other ways. For a given original differential equation, i.e. a given right-hand side function $f(t,u)$ we can now compute the additional terms in the modified differential equation to learn what the Forward Euler method really is doing…

- Example:

$$\frac{dy}{dt} = \lambda y$$

- We then know that the true solution $y(t)$ is

$$y(t) = Ce^{\lambda t}$$

i.e. $y(t)$ describes exponential growth and the constant λ is the growth rate

- For the MDE we then have

$$f(t,u) = \lambda u, \quad \frac{\partial f}{\partial t} = 0, \quad \frac{\partial f}{\partial u} = \lambda$$

so the MDE becomes

$$\frac{du}{dt} = \lambda u - \tfrac{1}{2}h\left(0 + \lambda^2 u\right)$$

which we can write

$$\frac{du}{dt} = \lambda\left(1 - \tfrac{1}{2}h\lambda\right)u$$

- Here we see that Forward Euler will for this example really solve a differential equation describing exponential growth with a growth rate

$$\lambda\left(1 - \tfrac{1}{2}h\lambda\right)$$

which for $\lambda > 0$ is lower than the growth rate λ in the original differential equation. (For $\lambda < 0$ the solution decays, and the same expression shows that Forward Euler makes it decay slightly *faster* than the true solution.)

Order of global error

- The order of the global error describes how the **total accumulated error** over an interval depends on the step size h
  - How quickly does the error go to zero if we take $h \to 0$
- If the global error is $\mathcal{O}\left(h^k\right)$, we say the method is **order** k
- Higher order means faster error reduction when decreasing h

Single-step vs multi-step

- **Single-step methods:** Obtain the next value using only the *current* step
  - Examples: Forward Euler, RK4
- **Multi-step methods:** Need to reuse information from *several previous steps*
  - Examples: Leapfrog, position Verlet

Stability

- For a fixed step size h, how is the error we have accumulated so far propagated forwards by the subsequent time steps of the method?
- For a **stable** method the errors remain **bounded**, while an **unstable** method will have errors that grow exponentially
- Stability depends on method, step h and the type of differential equation
  - A method can be stable for some types of problems and unstable for other problems
- We will discuss stability analysis (matrix method) later
- Some examples:
  - Forward Euler and Predictor-Corrector are *conditionally stable* (stable if h is below a certain limit) for *decaying solutions*, i.e. solutions to

$$\frac{dy}{dt} = -\lambda y$$

  - Leapfrog is *unstable* for decaying solutions
- Note: If a method is both *consistent* and *stable*, then it will converge to the true solution in the limit $h \to 0$ (ignoring numerical round-off errors)

Explicit vs implicit

- **Explicit method:**
  - A method where we obtain the next time step (y_{i+1}) directly, i.e. without solving a system of equations

  - Conceptually:

$$y_{i+1} = G(y_i, \ldots)$$

    (Here G is just some function that depends on the given method.)
  - Explicit methods are often easy to implement and computationally cheap
  - All the methods we have discussed so far are explicit methods

- **Implicit method:**
  - A method where we need to solve (a system of) equations to find the next step y_{i+1}
  - Conceptually:

$$y_{i+1} = G(y_{i+1}, y_i, \ldots)$$

  - Implicit methods are typically more complex and expensive, but are often more stable
  - We will get back to this when we discuss partial differential equations

- Simple example: Forward Euler (explicit) and Backward Euler (implicit)
  - Consider the differential equation

$$\frac{dy}{dt} = f = \lambda y$$

  - Forward Euler:

$$\begin{aligned} y_{i+1} &= y_i + hf_i \\ y_{i+1} &= y_i + h\lambda y_i \\ y_{i+1} &= y_i \left[1 + h\lambda\right] \end{aligned}$$

    * The right-hand side does *not* depend on y_{i+1}, so we can simply evaluate it to get y_{i+1}
  - Backward Euler:
    * Backward Euler is based on the backward-difference expression for y_i'

$$y_i' = \frac{y_i - y_{i-1}}{h}$$

* Or, written in terms of $(i+1)$ and i:

$$y'_{i+1} = \frac{y_{i+1} - y_i}{h}$$

* So the Backward Euler ODE method gives us

$$y_{i+1} = y_i + hf_{i+1}$$
$$y_{i+1} = y_i + h\lambda y_{i+1}$$

* Here the right-hand side *does* depend on y_{i+1}, so we must solve this equation to get y_{i+1}
* In this example it is easy, because Backward Euler gives us a simple linear equation with solution

$$y_{i+1} = \frac{y_i}{1 - h\lambda}$$

* In general, for an implicit method, we may need to run e.g. some iterative algorithm to solve the equations

Topics in project 4

- Physics case: the Ising model (in 2D)
- Basics of probabilities and probability densities
- Expectation values
- Monte Carlo methods
  - Sampling from low-dimensional pdfs
    - Rejection sampling
    - Inverse transform sampling
  - Sampling from high-dimensional pdfs
    - Markov chain Monte Carlo
  - Random number generation
- Coding:
  - Parallelisation (with OpenMP)
- Related topic: numerical integration
  - High-dimensional integrals
  - Low-dimensional integrals

Introduction to probability: what does probability mean?

- [In-class discussion of a die throw]
  - Q: If I throw this die, what's the probability that I'll get a six?
  - [I throw the die, look at the result, but hide the result from everyone else]
  - Q: Now, what's the probability that the result is a six?
  - [Discuss if/when it makes sense for two different people to claim different probabilities]
- The big question: What does a probability statement like $P(X) = 10\%$ really *mean*?
- We don't know! Or at least, we don't agree…
- The *mathematics* of probabilities is well understood, but we don't know what a probability *is*

- The **interpretation of probability** is an open question in philosophy!
- Recommended reading: *Interpretations of Probability* in the Stanford Encyclopedia of Philosophy: plato.stanford.edu/entries/probability-interpret

> *Probability is the most important concept in modern science, especially as nobody has the slightest notion what it means.* — Bertrand Russell, 1929

- There are several different philosophical views on the interpretation of probability
- But in practical use, there are two classes of interpretations that dominate: **Frequentist probability** and **Bayesian probability**
- **Frequentist probability**

$$\boxed{P(X) \equiv \lim_{n\to\infty} \frac{n_X}{n}}$$

  - Here probability is *defined* to be related to long-run relative frequencies of events/outcomes
  - So frequentist probability is inherently connected to something **repeatable**
  - There must also be some source of variation (randomness?) between trials, otherwise all repeated trials would give the same result (in which case all frequentist probabilities would be 0 or 1)

- **Bayesian probability**

$$\boxed{P(X) \equiv \text{degree of belief/knowledge that } X \text{ is true}}$$

  - A broader definition of probability, compared to the frequentist definition
  - Here probability theory can be seen as a mathematical framework for *reasoning under uncertainty* or for *making consistent bets*
  - In the Bayesian philosophy, we can assign probabilities to any type of statement X, not only statements about something repeatable
    - (That of course means that we can *also* assign Bayesian probability to statements that concern long-run relative frequencies)
  - Two main schools of thought within the Bayesian camp:
    - *Subjective Bayesianism*: Typically refers to $P(X)$ as a degree of *belief*

* *Objective Bayesianism*: Typically refers to $P(X)$ as a degree of *knowledge/information*

> *Disclaimer*: I personally find the Bayesian approach to probability very convincing. My presentation of this topic is therefore probably (in the Bayesian sense!) coloured by my own view. So make sure to also discuss this philosophical question with your friendly neighbourhood frequentist!

- Frequentist and Bayesian probability ends up producing the same *mathematical rules* for probability theory, i.e.

$$P(X) + P(\bar{X}) = 1$$
$$P(X, Y) = P(X|Y)P(Y)$$
$$P(X|Y)P(Y) = P(Y|X)P(X)$$

- But philosophically, frequentists and (different schools of) Bayesians often point to different sets of axioms (e.g. Kolmogorov, Cox, Dutch-book coherence) as the logical starting point for probability theory

Consequences

- Below we will mention three topics where the interpretation of probability has important consequences, both practically and philosophically

1. Different approaches to statistics

- Different interpretations of probability give rise to different approaches to statistics
- That's the reason why you will often at universities find separate courses for frequentist (or *classical*) statistics and Bayesian statistics
- Within Bayesian statistics we can ask questions on the form $P(\text{hypothesis} \mid \text{data})$, since this just means *What is my/our degree of belief/knowledge that the hypothesis is true, given the data we have?*
  - *Example*: What is the probability that a die is not loaded (the hypothesis), given that we have gotten a six in nine out of ten die rolls (the data)
- From the frequentist point of view, $P(\text{hypothesis} \mid \text{data})$ is not a valid probability, since the *hypothesis* is not a repeatable experiment.

- But both frequentists and Bayesians are perfectly happy with probabilities on the form $P(\text{data} \mid \text{hypothesis})$
  - *Example*: What is the probability that we will get at least nine sixes in ten die rolls (data), assuming a probability of 1/6 to get a six in each roll (hypothesis)

2. Relation between probability and randomness

- Our interpretation of probability can also affect how we view the relation between probability and **randomness**
- If probability is just a degree of belief/knowledge (the Bayesian view), there is *no necessary* link to randomness
  - In this view, probability can just as well be used to discuss outcomes of deterministic processes, or outcomes that have already happened — the use of probability can simply express a lack of information/certainty
- In the frequentist view, a probability of e.g. $P(X) = 50\%$ means that if we repeat some experiment indefinitely, the fraction of trials that give outcome X will tend towards 0.5
  - But what is the source for the variation? Why don't we always get the same result?
- This forces us to think carefully about what we exactly mean by statements like "a **random** process" or "repeating the **same** experiment"
  - If we get different outcomes in different trials, have we really repeated **the same** experiment?
  - To what extent does this depend on whether there exists some **true randomness** in Nature?
  - [Discuss: in what sense is e.g. a coin flip a *random* process?]

3. The interpretation of quantum mechanics

- We know how to *use* probability theory, and we know that it *is useful*, even if we are not sure what probabilities most fundamentally *are*
- In an analogous way, we know how to *use* quantum mechanics, and we know that it *is immensely useful* as a physical theory, but physicists and philosophers are still wondering what quantum mechanics truly *means* (and in particular, what *quantum states* really represent)
  - Recommended reading: *Philosophical Issues in Quantum Theory* in the Stanford Encyclopedia of Philosophy: plato.stanford.edu/entries/qt-issues/

- In quantum mechanics, the experimental predictions we produce from our theory come in the form of probability distributions
- So it's not surprising that several established philosophical interpretations of quantum mechanics are closely tied to particular philosophical views on probabilities

Properties of probabilities and probability density functions

My notation

- In my notation $p(x)$ can mean two different things, depending on context
- If x is a **discrete variable**:
  - $p(x)$ is the **probability** (or **probability mass**) for x
  - Example: $p(a) = \mathrm{Prob}(X = a)$
  - Since $p(x)$ is a probability, it is unitless: $[p(x)] = 1$
- If x is a **continuous variable**:
  - $p(x)$ is the **probability density** at x
  - Example: $p(a)dx = \mathrm{Prob}(X \in [a, a + dx])$
  - Since $p(x)$ is a probability density, it has units $[p(x)] = \frac{1}{[x]}$

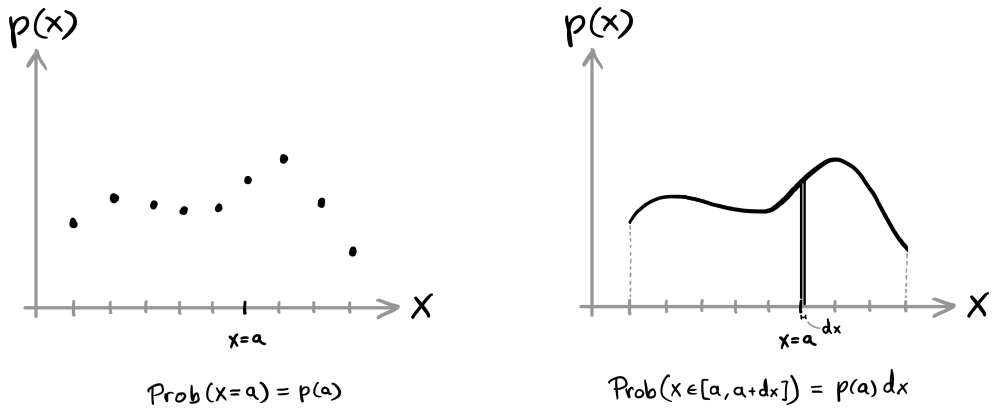


Figure 12: Examples of a probability function $p(x)$ for a discrete x (left), and a probability density $p(x)$ for a continuous x (right)

- When working with multiple variables, we *should* technically use a notation like this:

$$p_x(x), \quad p_y(y), \quad p_{x,y}(x,y), \quad p_{x|y}(x|y)$$

- Or alternatively, a notation where we give each probability (density) a completely new name:

$$p_x(x) = f(x), \quad p_y(y) = g(y), \quad p_{x,y}(x,y) = h(x,y), \quad p_{x|y}(x|y) = q(x|y)$$

- But personally I often find this verbose notation more confusing than helpful
- So I will be "sloppy" and use a simpler notation:

$$p(x), \quad p(y), \quad p(x,y), \quad p(x|y)$$

- In this notation, it is the argument in $p(\cdot)$ that clarifies which probability (density) function we are talking about
- In cases where this can be misunderstood, I will use a more verbose notation
- I will use the abbreviation **pdf** for *probability distribution function*, both for the continuous variable case (the pdf is a *probability density* function) and the discrete variable case (the pdf is a *probability mass* function)
  - In the statistics and probability literature, the abbreviation **pmf** is often used for *probability mass function*
- A pdf will, like any other function, have a given **domain** $\mathcal{D}$
  - When we think of x as an outcome of some experiment/sampling, $\mathcal{D}$ is often called the **sample space** or **outcome space**
    * *Example*: A die throw: $\mathcal{D} = \{1, 2, 3, 4, 5, 6\}$
    * *Example*: Survival time of a given unstable nucleus: $\mathcal{D} = [0, \infty)$
- Some terminology:
  - We will often say:
    * X *is a stochastic variable*
    * X *is a random variable*
  - But for a Bayesian it may be more natural to say:
    * X *is an unknown/uncertain variable*

  - We will also often use phrases like these:
    * X *has a pdf* $p(x)$
    * X *follows a pdf* $p(x)$
    * X *is distributed as* $p(x)$
  - A shorthand notation for this, much used in the statistics and probability literature:

$$X \sim p(x)$$

  - This notation can be confusing to physicists, because we sometimes use $\sim$ to mean *roughly equal to*, or *proportional to* — which it certainly doesn't mean in the above context

Some basics

- Probabilities are numbers in $[0, 1]$
  - Discrete case: $0 \leq p(x) \leq 1$
  - Continuous case: $0 \leq p(x)dx \leq 1$
  - *Note*: A probability *density* $p(x)$ can itself have *arbitrarily large, positive numerical value*
    * It's just the product $p(x)dx$ that must be in $[0, 1]$
    * The numerical value for $p(x)$ will for instance depend on what units we use for the variable x
- Pdfs are *normalised to unity*:
  - Discrete case:

$$\sum_{x \in \mathcal{D}} p(x) = 1$$

  - Continuous case:

$$\int_{x \in \mathcal{D}} p(x)dx = 1$$

- A *function of a random variable* is itself a random variable
  - Say y is some function $y(x)$, where x is a random variable with pdf $p(x)$
  - Then y is itself a random variable, with some other pdf $p(y)$
  - Example:

* Let y be the sum in a dice throw with two dice
* If you assign a flat probability function over the outcomes {1,2,3,4,5,6} for each die, the probability function for the *sum* takes on a triangle shape

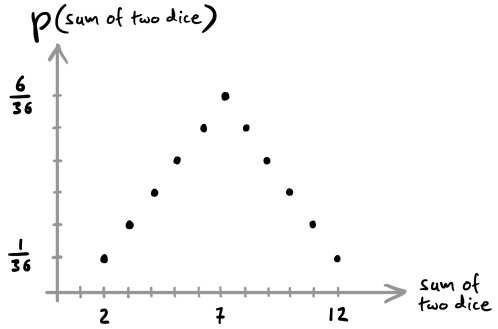


Figure 13: Probability function for the sum in a throw with two dice, if we assume a flat probability function for the outcome of each die

- For a given pdf we also have a **cumulative distribution function** (**cdf**)
  - Common notation for the cdf: $F(x)$
  - Definition: $F(x) \equiv \text{Prob}(X \leq x)$
  - So $F(x)$ is the total probability for the random variable X to take on a value *lower than or equal to some value* x
  - For a continuous variable x, assuming the domain of x is $\mathbb{R}$:

$$F(x) \equiv \text{Prob}(X \leq x) = \int_{-\infty}^{x} p(x')dx'$$

  - Note that this means that $F(-\infty) = 0$ and $F(\infty) = 1$
  - The pdf $p(x)$ is the derivative of the cdf $F(x)$:

$$p(x) = \frac{d}{dx}F(x)$$

    * This simply follows from the definition of $F(x)$ and the fundamental theorem of calculus (the derivative of an integral with respect to its upper limit):

$$\frac{d}{dx}F(x) = \frac{d}{dx}\int_{-\infty}^{x} p(x')dx' = p(x)$$

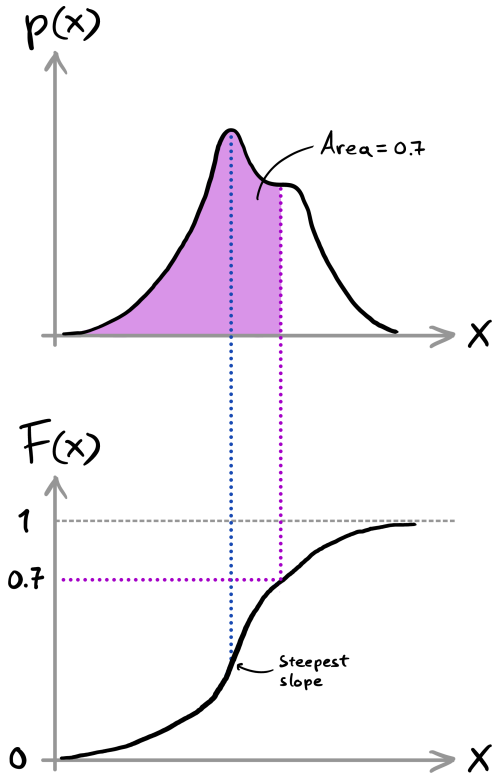


Figure 14: Illustration of the relation between a pdf $p(x)$ and the corresponding cdf $F(x)$

Probability distributions of many variables

- My notation:
  - General case: $p(x_1, x_2, x_3, \ldots)$ or $p(\vec{x})$
  - When only two variables: $p(x, y)$
- In the following we will consider the two-variable case for illustration
- Need to distinguish between three types of probability distributions:
  - The **joint distribution**: $p(x, y)$ (2D distribution)
  - The **conditional distributions**: $p(x|y), p(y|x)$ (1D distributions)
  - The **marginal distributions**: $p(x), p(y)$ (1D distributions)
- Below we will consider each type of distribution in turn

The joint distribution

- Two-variable example: $p(x, y)$
- Probability distribution over the (x, y) plane
- Meaning:

$$p(x, y)dxdy = \text{Prob}(X \in [x, x + dx] \text{ and } Y \in [y, y + dy])$$

- Normalisation:

$$\int_{x\in\mathcal{D}_x} \int_{y\in\mathcal{D}_y} p(x, y)\, dx\, dy = 1$$

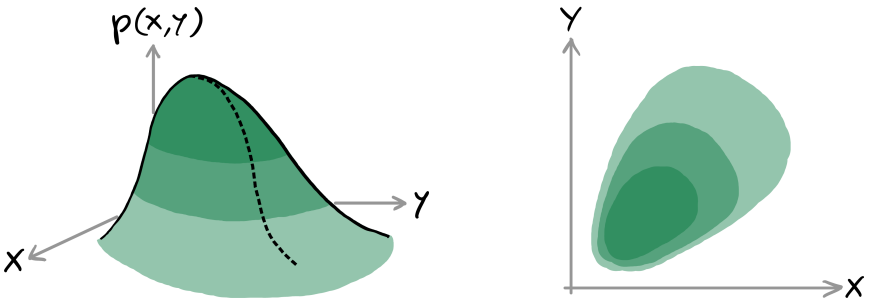


Figure 15: A two-variable joint probability distribution $p(x, y)$ visualised as a 3D figure (left) and as a 2D colour map (right)

Conditional distributions

- Two-variable examples: $p(x|y)$ and $p(y|x)$
- $p(x|y)$ is a 1D distribution over x, and similarly $p(y|x)$ is a 1D distribution over y
- Meaning:

$$p(x|y)dx = \text{Prob}(X \in [x, x + dx] \text{ given that } Y = y)$$
$$p(y|x)dy = \text{Prob}(Y \in [y, y + dy] \text{ given that } X = x)$$

- Each choice of the condition y gives a different $p(x|y)$
  - So in general, a single joint distribution $p(x, y)$ can give rise to infinitely many different $p(x|y)$ and $p(y|x)$ distributions.

- Being 1D pdfs, each $p(x|y)$ and $p(y|x)$ is normalised to integrate to unity

$$\int_{x\in\mathcal{D}_x} p(x|y)\,dx = 1, \qquad \int_{y\in\mathcal{D}_y} p(y|x)\,dy = 1$$

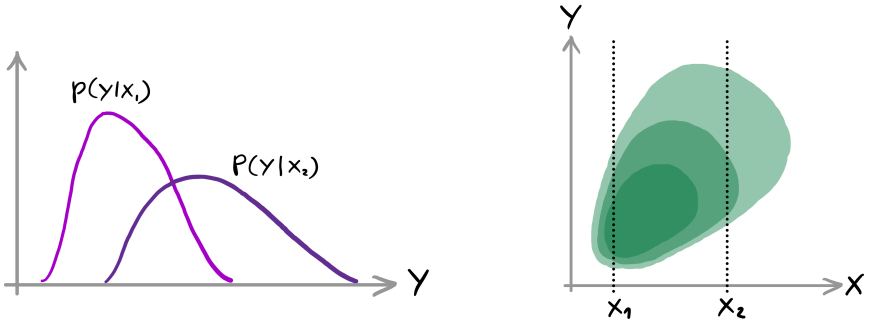


Figure 16: Two conditional probability distributions $p(y|x_1)$ and $p(y|x_2)$ (left), both derived from the joint distribution $p(x, y)$ (right)

The marginal distributions

- Two-variable example: $p(x)$ and $p(y)$, derived from the single, joint pdf $p(x, y)$
- Meaning:

$$p(x)dx = \text{Prob}(X \in [x, x + dx] \text{ regardless of } Y)$$
$$p(y)dy = \text{Prob}(Y \in [y, y + dy] \text{ regardless of } X)$$

- The marginal distributions are the distributions obtained by starting from the joint distribution and integrating out ("marginalising over") some variables:

$$p(x) = \int_{y\in\mathcal{D}_y} p(x, y)\,dy$$
$$p(y) = \int_{x\in\mathcal{D}_x} p(x, y)\,dx$$

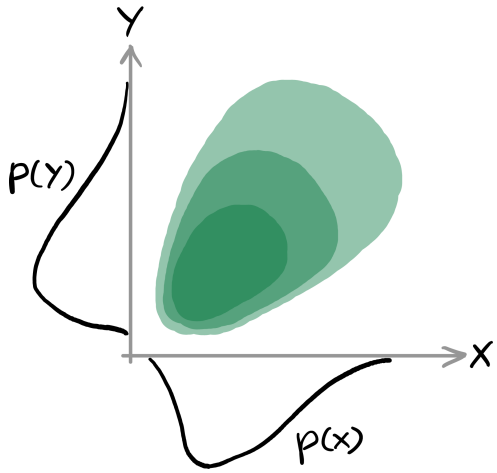


Figure 17: The two marginal probability distributions $p(x)$ and $p(y)$, derived from the joint probability distribution $p(x, y)$

Relation between the joint, conditional and marginal distributions

- The three types of distributions are connected through

$$p(x, y) = p(x|y)\, p(y) = p(y|x)\, p(x)$$

- So the conditional distribution is the joint distribution divided by the marginal distribution for the variable we condition on:

$$p(x|y) = \frac{p(x, y)}{p(y)}, \quad p(y|x) = \frac{p(x, y)}{p(x)}$$

  - Intuitively: $p(x|y)$ is the "slice" of $p(x, y)$ along the x direction at the fixed value y, renormalised to integrate to unity. The normalisation factor is exactly the marginal $p(y) = \int p(x, y)\, dx$
  - Combining the two expressions for $p(x, y)$ above gives $p(x|y) = p(y|x)\, p(x) \,/\, p(y)$, which is *Bayes' theorem*

Expectation values

- Let x be a random variable with pdf $p(x)$, and let y be some function of x
- The **expectation value** of $y(x)$ is:

$$\text{Continuous case:} \quad \mathrm{E}[y] \equiv \int_{x \in \mathcal{D}} y(x)p(x)dx$$

$$\text{Discrete case:} \quad \mathrm{E}[y] \equiv \sum_{x_i \in \mathcal{D}} y(x_i)p(x_i)$$

  - Remember that here $p(x)$ is the only pdf — $y(x)$ is just some arbitrary function of x
  - Two ways of thinking about $\mathrm{E}[y]$:
    1. *Compute the $y(x)$ value corresponding to every possible x value, and combine these y values in a weighted sum, where the weight for each $y(x)$ value is the probability $p(x)$ for the corresponding x value*
    2. *If we sample an infinite number of x values from $p(x)$ and compute $y(x)$ for each sample, then $E[y]$ will be the average of all these computed y values*

- Alternative notation, much used in physics:

$$\langle y \rangle \equiv \mathrm{E}[y]$$

  - We will use this notation in Project 4

- Example:
  - Let x be the outcome of a coin flip: $x \in \mathcal{D} = \{\text{heads}, \text{tails}\}$
  - Let's assign a flat probability function $p(x)$:

$$p(\text{heads}) = 0.5$$
$$p(\text{tails}) = 0.5$$

  - Now assume you are making a bet with a friend on such a coin flip
  - If the outcome is heads you win 10 NOK, and if the outcome is tails you lose 10 NOK
  - Let $y(x)$ denote the amount of money you earn in the bet
  - Given your probability assignment $p(x)$ for the coin flip, how much money do you *expect* to earn in this bet?

- Answer:

$$\begin{aligned}\mathrm{E}[y] &= \sum_{x\in\mathcal{D}} y(x)p(x) \\ &= y(\text{heads})p(\text{heads}) + y(\text{tails})p(\text{tails}) \\ &= (10\,\text{NOK}) \times 0.5 \;+\; (-10\,\text{NOK}) \times 0.5 \\ &= 0\,\text{NOK}\end{aligned}$$

- So, unsurprisingly, you can't expect to earn anything on this bet
- This example illustrates an important point: *The expectation value does not necessarily correspond to a high-probability value, or even to a possible value!*
- In our example, the only possible outcomes for y are y = 10 NOK and y = -10 NOK
  * The outcome y = 0 NOK is impossible in a single coin flip
  * But the *expected* y is still 0 NOK

Moments: particularly useful expectation values

Example: moments in physics

- Analogous concept from physics:

$$n\text{-th moment of quantity } Y = \int r^n\,[\text{density of } Y \text{ at } r]\,dr$$

- Example: *moments of mass*
  - Let $\rho(\vec{r})$ be a mass density at position $\vec{r}$ for an object of total mass M
  - The zeroth moment of mass ($n = 0$) is just the *normalisation condition*:

$$M = \int \rho(\vec{r})\,d\vec{r}$$

  - The first moment of mass ($n = 1$), normalised by the total mass M is what we call the *centre of mass*, $\vec{R}$:

$$\vec{R} = \frac{1}{M}\int \vec{r}\rho(\vec{r})\,d\vec{r}$$

  - The second moment of mass ($n = 2$) is the *moment of inertia*, I, describing how spread-out the mass distribution is around a given rotation axis. With $r_\perp$ denoting the distance from

the axis:

$$I = \int r_\perp^2 \rho(\vec{r})\, d\vec{r}$$

Moments of probability distributions

- The n-th moment of a pdf $p(x)$ is the expectation value $\mathrm{E}[x^n]$:

$$\mathrm{E}[x^n] = \int x^n p(x)\, dx$$

- The zeroth moment is just the normalisation condition:

$$\mathrm{E}\Big[x^0\Big] = \mathrm{E}[1] = 1 = \int x^0 p(x)\, dx = \int p(x)\, dx$$

- The first moment, $\mathrm{E}[x]$ or $\langle x \rangle$, is **the mean** of $p(x)$, often denoted μ:

$$\boxed{\mu = \mathrm{E}[x] = \int x p(x)\, dx}$$

  - This is just the average x value, given the pdf $p(x)$
  - It's a sum (integral) of all possible x values, each weighted by its corresponding probability (density) $p(x)$

- In general, moments are relative to some reference point c

$$\mathrm{E}[(x-c)^n] = \int (x-c)^n p(x)\, dx$$

- If we make the particular choice $c = \mu$ we get the **central moments**

$$\mathrm{E}[(x-\mu)^n] = \int (x-\mu)^n p(x)\, dx$$

- The first two central moments are rather uninteresting:

$$n = 0: \quad \mathrm{E}\Big[(x-\mu)^0\Big] = \mathrm{E}[1] = 1$$

$$n = 1: \quad \mathrm{E}\Big[(x-\mu)^1\Big] = \mathrm{E}[x] - \mathrm{E}[\mu] = \mu - \mu = 0$$

- The second central moment is **the variance** of $p(x)$, denoted $\mathrm{Var}(x)$ or σ^2

$$\boxed{\mathrm{Var}(x) = \sigma^2 = \mathrm{E}\Big[(x-\mu)^2\Big] = \int (x-\mu)^2 p(x)\, dx}$$

- The variance is a measure for how spread out the pdf $p(x)$ is, i.e. how much of the probability is associated with x values far away from the expected x value $\mathrm{E}[x] = \mu$

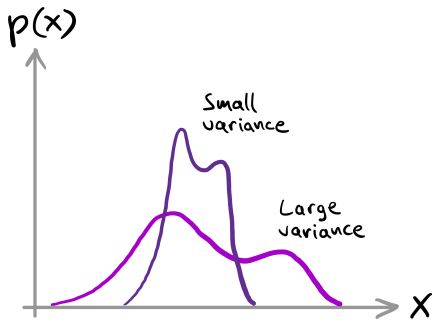


Figure 18: Variance is a measure for how spread out a probability distribution is

- We can express the variance $\mathrm{Var}(x)$ in terms of $\mathrm{E}[x]$ and $\mathrm{E}[x^2]$

$$\begin{aligned}
\mathrm{Var}(x) &= \mathrm{E}\Big[(x-\mu)^2\Big] \\
&= \int (x-\mu)^2 p(x)\, dx \\
&= \int (x^2 - 2x\mu + \mu^2) p(x)\, dx \\
&= \int x^2 p(x)\, dx - \int 2x\mu p(x)\, dx + \int \mu^2 p(x)\, dx \\
&= \mathrm{E}\Big[x^2\Big] - 2\mu \mathrm{E}[x] + \mu^2 \\
&= \mathrm{E}\Big[x^2\Big] - 2\mu^2 + \mu^2 \\
&= \mathrm{E}\Big[x^2\Big] - \mu^2 \\
&= \mathrm{E}\Big[x^2\Big] - (\mathrm{E}[x])^2
\end{aligned}$$

- We note that the variance has units $[x]^2$
- The related quantity in units of $[x]$ is the **standard deviation**, denoted σ

$$\boxed{\sigma = \sqrt{\mathrm{Var}(x)}}$$

- Computation of different means and variances will be important in project 4

Introduction to Monte Carlo methods

- The name refers to the famous *Casino de Monte-Carlo* in Monaco
- The term **Monte Carlo methods** refers to computational methods that in one way or another make use of sampling of random numbers
- Some typical situations in which Monte Carlo methods are used:
  - Working with uncertain/random variables
  - Simulating random/uncertain systems
  - Working with high-dimensional problems (many variables)
  - Numerical integration
- A random variable has a corresponding probability distribution function (pdf)
- For realistic cases (which often involve many variables) it is usually too complicated to work directly with the pdfs — in most cases we don't even have analytical expressions for the pdfs
- Monte Carlo approach: work with **samples from the pdfs** instead of working with the pdfs directly
- For instance we will often
  - make **histograms of samples** to approximate pdfs
  - compute **sample means** to estimate the true expectation values of pdfs
  - compute **sample variances** to estimate the true variances of pdfs

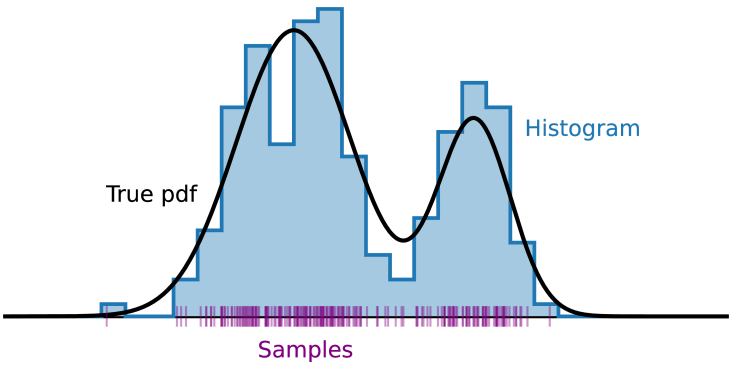


Figure 19: A set of samples, the corresponding histogram and the true pdf

- Basic challenges

1. Given that we are working with deterministic computers, how can we generate (seemingly) random numbers, i.e. numbers drawn from a uniform probability distribution?
2. Assuming we *can* draw random numbers from a uniform distribution, how can we use this to draw random numbers from other probability distributions?
3. How can we maintain sufficient computational efficiency when working in high-dimensional spaces?

– All these topics will be discussed in subsections below

The Monte Carlo approach, broad overview

- Assume $\vec{x}$ is an uncertain/random variable with pdf $p(\vec{x})$
- Assume we have a way of generating samples from $p(\vec{x})$
- The typical main steps of a Monte Carlo approach are then:
  1. Draw a sample $\vec{x}$ from $p(\vec{x})$
  2. Compute other quantities of interest that depend on $\vec{x}$, e.g. $f(\vec{x})$, $g(\vec{x})$ and $h(f(\vec{x}), g(\vec{x}))$
  3. Store the results we are interested in
  4. Repeat from step 1
- The precision in our final estimates increases with the number of Monte Carlo samples we use
- In the end we are left with a set of $\vec{x}$ samples $\{\vec{x}_1, \vec{x}_2, \ldots\}$, as well as the corresponding samples of the computed quantities $\{f_1, f_2, \ldots\}$, $\{g_1, g_2, \ldots\}$ and $\{h_1, h_2, \ldots\}$
- Practical note:
  – Sometimes we only store a subset of the information per sample, to keep file sizes small
  – For instance, if all we are interested in is to make histograms of the f, g and h samples, we don't need to save the $\vec{x}$ samples to file
- Given a set of Monte Carlo samples, we can use them for many different purposes
- Here are some common examples, where N is the number of samples and we use f as our example variable:
  – Create a histogram of all our f samples to approximate the pdf $p_f(f)$
  – Create a histogram using only a specific subset of the f samples, to approximate some conditional pdf

- Estimate some probability or integral, e.g.

$$\mathrm{Prob}(f < 10) = \int_{-\infty}^{10} p_f(f)\, df \approx \frac{N_{f<10}}{N}$$

  where $N_{f<10}$ is the number of our f samples for which $f < 10$

- Approximate the expectation value $\mathrm{E}[f]$ using the **sample mean** $\bar{f}$

$$\mathrm{E}[f] \approx \bar{f} = \frac{1}{N}\sum_{i=1}^{N} f_i$$

- Approximate the variance $\mathrm{Var}(f)$ using the **sample variance** s_f^2

$$\mathrm{Var}(f) \approx s_f^2 = \frac{1}{(N-1)}\sum_{i=1}^{N}(f_i - \bar{f})^2$$

  * Alternatively, approximate the standard deviation $\sigma_f = \sqrt{\mathrm{Var}(f)}$ using the **sample standard deviation** s_f

$$\sigma_f \approx s_f = \sqrt{\frac{1}{(N-1)}\sum_{i=1}^{N}(f_i - \bar{f})^2}$$

- A side note on the uncertainty in using the sample mean $\bar{f}$ to approximate $\mathrm{E}[f]$:
  - The sample mean $\bar{f}$ is a function of the N random samples f_i, so $\bar{f}$ is itself a random variable with an expectation value $\mathrm{E}\left[\bar{f}\right]$ and a variance $\mathrm{Var}\left(\bar{f}\right)$
  - The sample mean $\bar{f}$ is an *unbiased estimator* for the true expectation value $\mathrm{E}[f]$, since the expectation value for $\bar{f}$ satisfies $\mathrm{E}\left[\bar{f}\right] = \mathrm{E}[f]$
  - The *variance of the sample mean*, $\mathrm{Var}\left(\bar{f}\right)$, gets smaller the more f samples that $\bar{f}$ is based on
  - If the N samples are *independent* (which we assume here), $\mathrm{Var}\left(\bar{f}\right)$ is related to the variance of the original variable ($\mathrm{Var}(f)$) as follows:

$$\mathrm{Var}\left(\bar{f}\right) = \frac{\mathrm{Var}(f)}{N}$$

- Or, in terms of standard deviations:

$$\sigma_{\bar{f}} = \frac{\sigma_f}{\sqrt{N}}$$

- This result agrees well with our intuition:
  - If we use many f samples in our computation of $\bar{f}$, we expect that we would get a very similar $\bar{f}$ value if we were to perform the computation again with a second set of f samples
  - If we use few f samples in our $\bar{f}$ computation we expect that we could get quite a different result if we tried again with a second set of f samples
- So, when we use a set of samples to approximate $\mathrm{E}[f]$, we can write our estimate as

$$\mathrm{E}[f] \approx \bar{f} \pm \frac{s_f}{\sqrt{N}}$$

Sampling from low-dimensional pdfs

- As mentioned above, one of the challenges we need to solve in order to use Monte Carlo approaches is this:
  - Assuming we *can* draw random numbers from a uniform distribution, how can we use this to draw random numbers from other probability distributions?
- We will discuss two general methods for solving this problem:
  - Rejection sampling
  - Inverse transform sampling
- In addition to these general methods there are numerous sampling methods developed to efficiently sample from specific, important probability distributions, e.g. the normal (Gaussian) distribution

Rejection sampling

- *A group (gedanken) exercise:*
  - Say I draw some arbitrary probability distribution $p(x)$ on the blackboard, and that the drawing is not very big

  - Then I give each of you a piece of chalk
  - Your task as a group: Without leaving your seats, generate a set of x-samples distributed according to $p(x)$
  - How would you do this?

- *Answer:*
  - *Chuck the chalk!* Without aiming too well, throw the chalk at the blackboard
  - (Let's not actually do this – chalk has a tendency of just exploding and making a real mess when it hits blackboards at high speeds)
  - Let's assume that each chalk hit makes a small mark on the blackboard
    * If a chalk mark is outside the area below the $p(x)$ curve, ignore it
    * If a chalk mark is within the area below the $p(x)$ curve, record its x coordinate
  - If we assume that in the region close to the $p(x)$ drawing the distribution of chalk hits was *close to uniform* in the x and y directions, then the set of recorded x coordinates will be sampled according to $p(x)$
    * The *close to uniform* assumption is why I made the drawing a bit small, and why you should not aim too well
  - This is the basic logic of **rejection sampling**

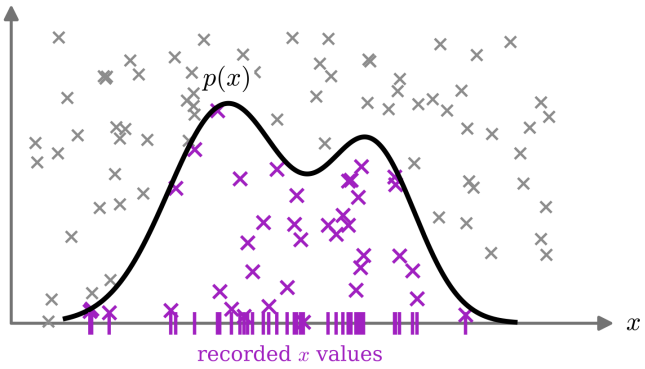


Figure 20: Throwing chalk at a drawing of $p(x)$. If the chalk marks are spread roughly uniformly over the blackboard, the x coordinates of the marks that landed below the $p(x)$ curve (purple) will be distributed according to $p(x)$. The marks above the curve (grey) are ignored.

- Rejection sampling algorithm:

- First define a box $x \in [a, b]$, $y \in [0, h]$ such that the box contains the pdf $p(x)$
  - (Later we will generalise this beyond simple box shapes)
- Then:
  1. Sample $x' \sim U(a, b)$
  2. Sample $y' \sim U(0, h)$
  3. If $y' < p(x')$ then accept x' as a new x sample. If not, reject it.
- Repeat these steps to collect as many x samples as you need

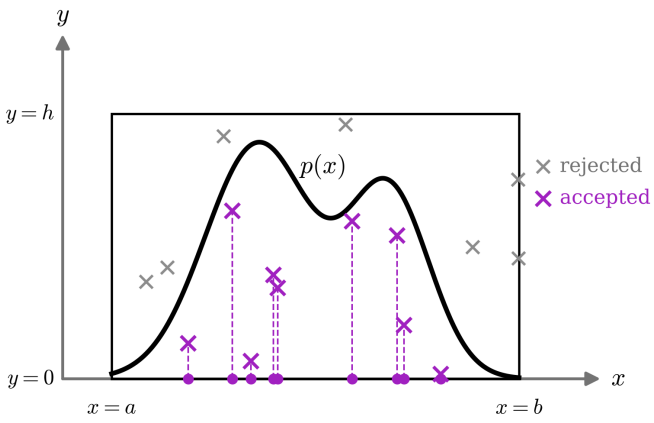


Figure 21: Rejection sampling with a bounding box $x \in [a, b]$, $y \in [0, h]$. The trial points (x', y') are drawn uniformly from the box. Those below the $p(x)$ curve are accepted, and their x' coordinates (purple dots) are our x samples. Those above the curve are rejected.

- This simple method *in principle* works with any arbitrary pdf shape, and the pdf also doesn't have to be properly normalised
  - (The pdf must be bounded, though, and if it has infinitely long tails we have to truncate them to get a box of finite width)
- For a many-dimensional pdf, $p(\vec{x})$, we can modify step 1) to sample a proposal x_i' for each coordinate of $\vec{x}$, using the appropriate range $[a_i, b_i]$ for each coordinate
- But for now, let us focus on the 1D case
- *Question:* In what cases, i.e. for what types of pdfs, will this approach be very inefficient?
- *Answer:* The more strongly peaked the pdf is, the less efficient this approach will be
  - Say we have a pdf with a single high and narrow peak and very long tails

  - Then the area of the bounding box will be much larger than the area below the pdf
  - So when our rejection sampling algorithm generates trial coordinates (x', y') uniformly from the bounding box, almost all such trials will be rejected, i.e. $\mathrm{Prob}(\mathrm{accept}) \approx 0$

- *Question:* In what case would we have $\mathrm{Prob}(\mathrm{accept}) = 1$?
- *Answer:* When our target pdf $p(x)$ is itself a uniform distribution
  - But in this case there is of course no point in using rejection sampling, since our starting assumption was that we can already generate samples from the uniform distribution

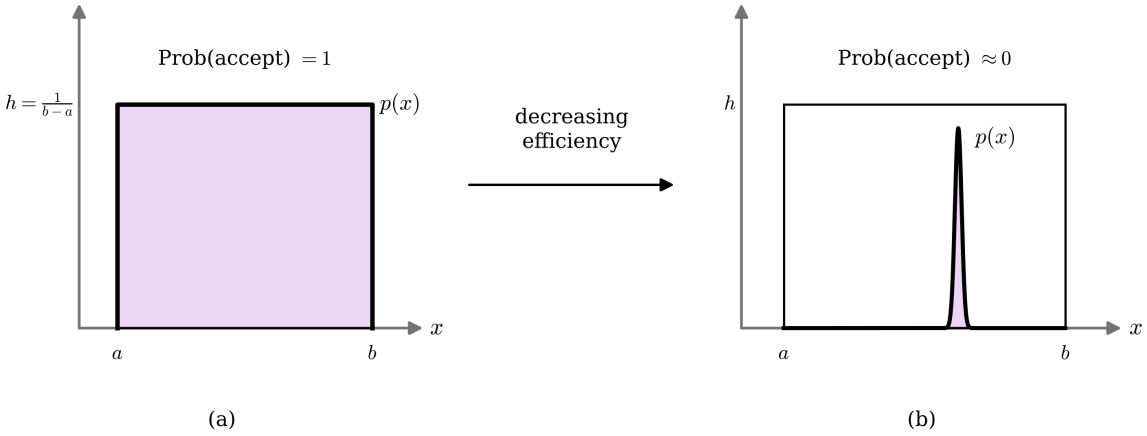


Figure 22: Efficiency of box-shaped rejection sampling. (a) For a uniform pdf the pdf fills the whole box, so every trial point is accepted. (b) For a strongly peaked pdf the area below the curve is a tiny fraction of the area of the box, so almost every trial point is rejected.

- Now let's think about what happens in higher dimensions
- Say we have a 1D pdf $p(x)$ which gives $\mathrm{Prob}(\mathrm{accept}) = 0.5$ for rejection sampling in 1D

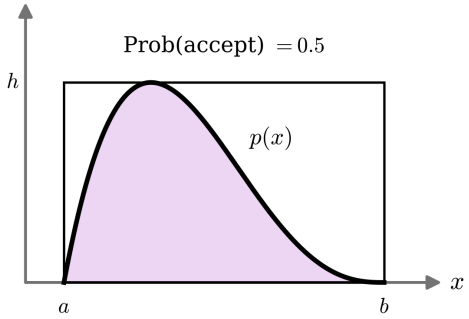


Figure 23: A 1D pdf $p(x)$ for which the area below the curve is half the area of the bounding box, giving $\mathrm{Prob}(\mathrm{accept}) = 0.5$ for rejection sampling.

- Now assume we want to use rejection sampling to generate high-dimensional samples $\vec{x}$ from a joint pdf

$$p_{\vec{x}}(\vec{x}) = p_{\vec{x}}(x_1, x_2, \ldots, x_n) = p(x_1)p(x_2)\ldots p(x_n)$$

  - That is, we have a high-dimensional, joint pdf $p_{\vec{x}}(\vec{x})$ where each coordinate x_i is independent of the others and is distributed according to our 1D pdf $p(x)$ with $\mathrm{Prob}(\mathrm{accept}) = 0.5$

- *Questions:*

  - Will it work to use rejection sampling?
  - Will it work well?

- *Answer:* It will work to use rejection sampling – the accepted $\vec{x}$ samples will be distributed according to $p_{\vec{x}}(\vec{x})$ – but the efficiency drops very quickly (exponentially fast) with dimensionality

$$\begin{aligned}\mathrm{Prob}(\mathrm{accept}\ \vec{x}') &= \mathrm{Prob}(\mathrm{accept}\ x_1') \times \mathrm{Prob}(\mathrm{accept}\ x_2') \times \ldots \times \mathrm{Prob}(\mathrm{accept}\ x_n') \\ &= (0.5)^n\end{aligned}$$

- Example: For $n = 10$ dimensions we get $\mathrm{Prob}(\mathrm{accept}\ \vec{x}') \approx 0.001 = 0.1\%$

  - So if we use naive rejection sampling to generate samples from this pdf $p_{\vec{x}}(\vec{x})$, we expect to have to try around 1000 attempts (each involving generating $10 + 1$ random numbers!) just to get a single $\vec{x}$ sample…

- Another example: If instead each 1D coordinate has $\mathrm{Prob}(\mathrm{accept}\ x_i') = 0.8$, and we have $n = 20$ dimensions, we get $\mathrm{Prob}(\mathrm{accept}\ \vec{x}') = (0.8)^{20} \approx 0.012 \approx 1\%$

  - So even when sampling each 1D pdf on its own would be quite efficient, the efficiency quickly becomes poor in higher dimensions

- This is an example of "the curse of dimensionality" – that methods that work well in low number of dimensions become inefficient or completely useless for higher-dimensional problems

Rejection sampling with non-uniform proposal distribution

- The efficiency of rejection sampling can be improved by using a **non-uniform proposal distribution**

  - This is the generalisation beyond the simple box shape that was mentioned above

  - *A note on terminology:* This method is sometimes referred to as *importance sampling*. But in the statistics literature that term usually refers to a related but different method, where one also samples x from some $q(x)$, but keeps *every* sample and gives it a weight $p(x)/q(x)$ instead of accepting or rejecting it. So here we will stick to the name *rejection sampling* also for the method described below. We will return to importance sampling when we discuss Monte Carlo integration later in the notes

- The idea: Rather than sampling the trial x' values uniformly from the box, $x' \sim U(a, b)$, we sample them from some other pdf $q(x)$
  - $q(x)$ should be a pdf that we already know how to sample from
  - $q(x)$ should be chosen to be as similar to our target pdf $p(x)$ as possible
- We then scale $q(x)$ by a constant M, chosen such that $Mq(x) \geq p(x)$ for all x
  - That is, the curve $Mq(x)$ envelops $p(x)$, in the same way as the top of the box ($y = h$) did in the simple version of rejection sampling

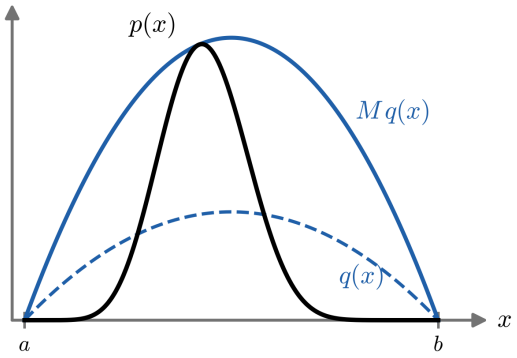


Figure 24: Rejection sampling with a non-uniform proposal distribution. The trial x' values are sampled from the pdf $q(x)$ (dashed) rather than from the uniform distribution $U(a, b)$. The scaled function $Mq(x)$ envelops the target pdf $p(x)$, and plays the role that the top of the bounding box played in the simple version of rejection sampling.

- Rejection sampling algorithm with a non-uniform proposal distribution $q(x)$:
  1. Sample $x' \sim q(x)$ (not $x' \sim U(a, b)$)
  2. Sample $y' \sim U(0, Mq(x'))$
  3. If $y' < p(x')$ then accept x' as a new x sample. If not, reject it.
  - Repeat these steps to collect as many x samples as you need

- An equivalent formulation is to replace steps 2) and 3) with:

  2. Sample $y' \sim U(0,1)$
  3. If $y' < \frac{p(x')}{Mq(x')}$ then accept x' as a new x sample. If not, reject it.

- Note that the simple "box method" is just the special case $q(x) = U(a,b)$, with $M = h(b-a)$ so that $Mq(x) = h$
- Compared to the "box method", this approach more rarely samples x regions where $p(x)$ is low, but it compensates for this by increasing the acceptance probability for such samples
- *Why does this work?* Let's check, using what we learned about joint, conditional and marginal distributions

  - Consider a single trial. The joint probability (density) for proposing the value x *and* accepting it is

$$p(x, \text{acc.}) = q(x)\,\text{Prob}(\text{acc.} \mid x) = q(x)\,\frac{p(x)}{Mq(x)} = \frac{p(x)}{M}$$

  - Marginalising over x gives the total acceptance probability (assuming $p(x)$ and $q(x)$ are normalised):

$$\text{Prob}(\text{acc.}) = \int p(x, \text{acc.})\,dx = \frac{1}{M}\int p(x)\,dx = \frac{1}{M}$$

  - And the distribution of the *accepted* x values is the conditional distribution

$$p(x \mid \text{acc.}) = \frac{p(x, \text{acc.})}{\text{Prob}(\text{acc.})} = \frac{p(x)/M}{1/M} = p(x)$$

    which is exactly what we wanted

  - For the "box method" we have $q(x) = U(a,b)$ and $M = h(b-a)$, so $\text{Prob}(\text{acc.}) = 1/(h(b-a))$, i.e. the area below $p(x)$ (which is 1) divided by the area of the box. This is the ratio we used by hand in the discussion of high-dimensional pdfs above
  - Since we need $Mq(x) \geq p(x)$ everywhere, the smallest possible M is $M = \max_x p(x)/q(x)$, and on average we need M trials per accepted sample

- The more similar $q(x)$ is to $p(x)$, the better the efficiency — since $M = \max_x p(x)/q(x)$ can only be close to 1 if $q(x)$ has nearly the same shape as $p(x)$

  - But if we could sample $x \sim q(x) = p(x)$ directly, we wouldn't need this method in the first place…

- Finally, note that the acceptance test in step 3), *"accept x' with probability $p(x')/(Mq(x'))$"*, has the same structure as the acceptance rule in the Metropolis-Hastings algorithm, which we will discuss later

Inverse transform sampling

- *Recap:* The cumulative distribution function (cdf) is defined by

$$F(x) \equiv \text{Prob}(X \leq x) = \int_{-\infty}^{x} p(x')dx'$$

- So the pdf is the derivative of the cdf:

$$\frac{dF}{dx} = p(x)$$

- Example of a pdf and the corresponding cdf:

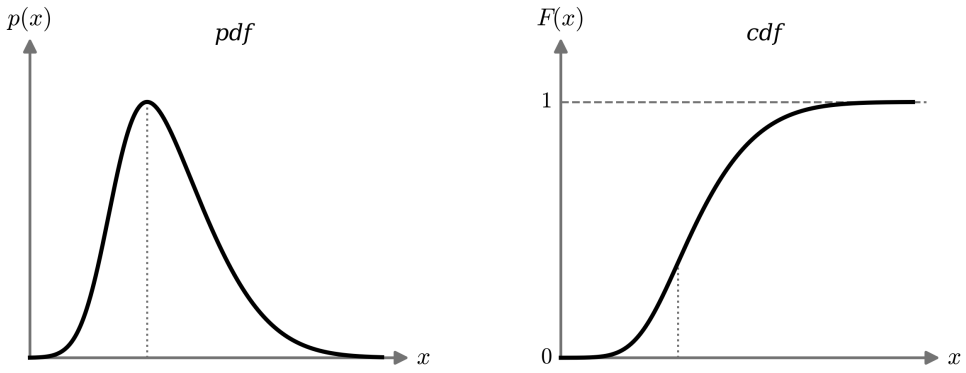


Figure 25: Example of a pdf $p(x)$ and the corresponding cdf $F(x)$. The dotted lines mark the x value where $p(x)$ has its maximum, which is where $F(x)$ is steepest.

- The idea behind **inverse transform sampling**:
  - Assume that we know $y = F(x)$, and that we can find (or at least evaluate) the inverse function F^{-1}
  - Then we can use this to generate x samples from $p(x)$
- Inverse transform sampling algorithm:
  1. Sample $y \sim U(0, 1)$

2. Compute $x = F^{-1}(y)$

– Repeat these steps to collect as many x samples as you need

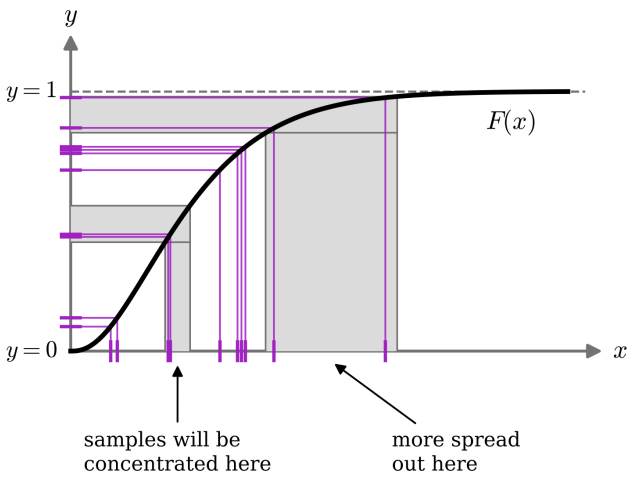


Figure 26: Inverse transform sampling. The y values are sampled uniformly from $U(0, 1)$ and mapped to x values through $x = F^{-1}(y)$. Where $F(x)$ is steep, i.e. where $p(x)$ is large, a wide range of y values is mapped to a narrow range of x values, so the x samples will be concentrated here. Where $F(x)$ is flat, i.e. where $p(x)$ is small, the x samples will be more spread out.

- *Explanation:*

  – In general, if $x \sim p_x(x)$ and y is some *monotonically increasing* function of x, $y = f(x)$, we can find the pdf $p_y(y)$ from the requirement

$$p_y(y)dy = p_x(x)dx$$

  – This is *conservation of probability*: The probability for x to be within a small interval dx around x must equal the probability for y to be within the corresponding interval dy around $y = f(x)$

    * (If f is a *decreasing* function, a positive dx corresponds to a negative dy, so to keep both pdfs positive we should write the requirement as $p_y(y)\,|dy| = p_x(x)\,|dx|$. Since the cdf is a non-decreasing function we don't need to worry about this here.)

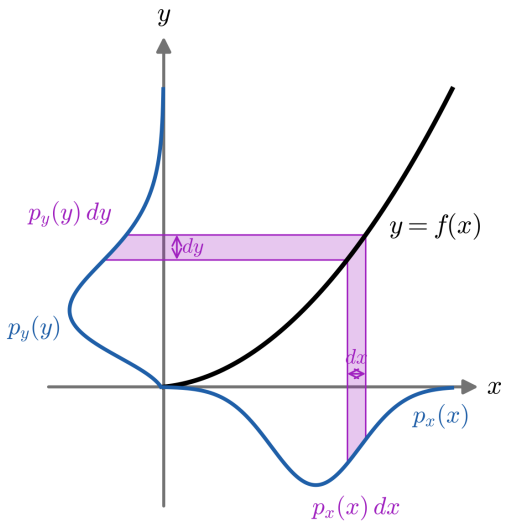


Figure 27: Conservation of probability for a transformation $y = f(x)$: The probability $p_x(x)\,dx$ for x to be in the small interval dx equals the probability $p_y(y)\,dy$ for y to be in the corresponding interval dy.

- Now look at the special case where the function is the cdf for x, i.e. $y = F(x)$
- *Question:* If $x \sim p_x(x)$ and $y = F(x)$, what is $p_y(y)$?
- *Answer:* From conservation of probability we have

$$\begin{aligned} p_x(x)dx &= p_y(y)dy \\ &= p_y(y)\frac{dy}{dx}dx \\ &= p_y(y)\frac{dF}{dx}dx \\ &= p_y(y)p_x(x)dx \end{aligned}$$

  so we must have

$$\boxed{p_y(y) = 1}$$

  which is just the uniform distribution $U(0,1)$
- So if we sample $y \sim U(0,1)$ and compute $x = F^{-1}(y)$, we get an $x \sim p_x(x)$, which was our goal
- *Pro:* No rejection sampling needed (efficiency)
- *Con:* Requires knowledge of the inverse cdf F^{-1}
- *Example:* The **exponential distribution**, which for instance describes the survival time x of an

unstable nucleus with decay rate λ:

$$p(x) = \lambda e^{-\lambda x}, \quad x \in [0, \infty)$$

- The cdf is

$$F(x) = \int_0^x \lambda e^{-\lambda x'} dx'$$

- We can do this integral with the substitution $u = -\lambda x'$, so that $du = -\lambda\, dx'$, and the limits $x' = 0$ and $x' = x$ become $u = 0$ and $u = -\lambda x$:

$$\begin{aligned} F(x) = \int_0^x \lambda e^{-\lambda x'} dx' &= -\int_0^{-\lambda x} e^u du \\ &= -\Big[e^u\Big]_0^{-\lambda x} = -\left(e^{-\lambda x} - 1\right) = 1 - e^{-\lambda x} \end{aligned}$$

- Setting $y = F(x)$ and solving for x gives the inverse

$$x = F^{-1}(y) = -\frac{1}{\lambda}\ln(1 - y)$$

- So to generate samples from $p(x)$: sample $y \sim U(0, 1)$ and compute $x = -\ln(1 - y)/\lambda$
  - (Since $1 - y$ is also uniformly distributed on $(0, 1)$, we can equally well use $x = -\ln(y)/\lambda$)
- This is the kind of transformation that e.g. `std::exponential_distribution` in the <`random`> library performs for you

- Note that this has a similar structure to how random numbers are generated with the <`random`> library in C++:
  1. Use a random number generator to sample from $U(0, 1)$
  2. Use a transformation to obtain a sample from $p(x)$

Generating more samples from a set of existing samples

- *Question:* Say we are given a set of samples from some *unknown* pdf. How can we generate more such samples?

- *Answer:* We can represent the given samples as a histogram, and then use the two methods above with the histogram in place of the pdf:
  - Rejection sampling from the *histogram*, rather than from the pdf
  - Inverse transform sampling using the *cumulative histogram* in place of the cdf

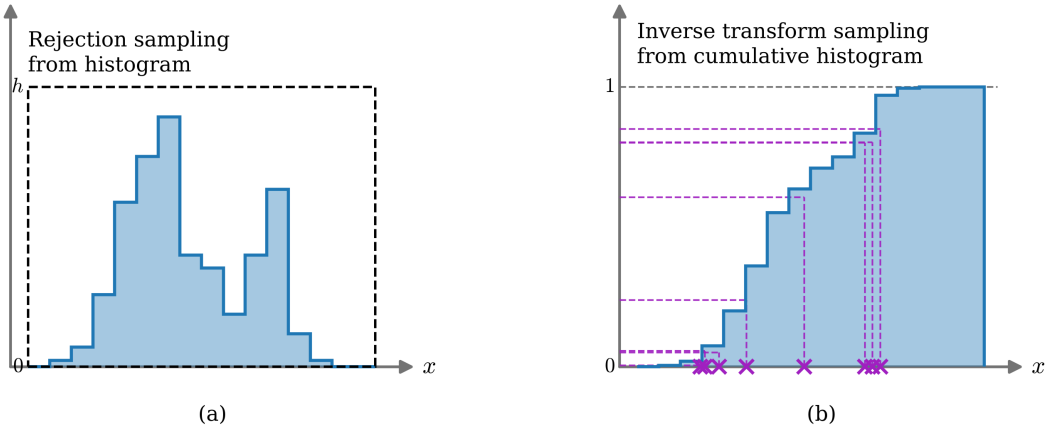


Figure 28: Generating more samples from a set of existing samples. (a) Rejection sampling from the histogram of the existing samples. (b) Inverse transform sampling using the cumulative histogram: y values are sampled from $U(0, 1)$ and mapped to new x samples (purple crosses) through the cumulative histogram.

Sampling from high-dimensional pdfs: Markov chain Monte Carlo

- The previous methods for sampling from pdfs run into problems in high dimensions:
  - *Rejection sampling*: The probability of accepting a proposed new sample will in general decrease exponentially with the number of dimensions, so the method quickly becomes very inefficient
  - *Inverse transform sampling*: When working with high-dimensional pdfs, we in general don't know the cumulative distribution function, let alone its inverse
- So we need another approach
- The approach we will discuss is the famous **Markov chain Monte Carlo** (MCMC) method
- Roughly speaking, Markov chain Monte Carlo alleviates the inefficiency problem of rejection sampling by using the position of the previous accepted sample when proposing the next sample — typically from somewhere in the "neighbourhood" of the previous sample

- We will first discuss what a Markov chain is, and then look at the Markov chain Monte Carlo method

Markov chains

- A **Markov chain** is a random sequence in which *the probability for the next state only depends on the current state*, not the history of states before that.
- That the probability of the next state only depends on the current state is known as the *Markov property*
- Another way of expressing this is to say that a Markov chain is *memoryless*
- More mathematically: If $x_1, x_2, \ldots, x_{i-1}, x_i$ is our current history of states, the Markov property says that

$$\mathrm{Prob}(x_i \to x' \,|\, x_i, x_{i-1}, \ldots, x_2, x_1) = \mathrm{Prob}(x_i \to x' \,|\, x_i)$$

- This just expresses that when we know the current state x_i, the additional information about all the previous states does not alter the probability distribution for what state x' the chain will transition to next
- A Markov chain is fully defined by two pieces of information:
  1. The state space
  2. The transition probabilities between states
  - (To actually *run* a chain we also need to specify a starting state, or a starting distribution across the states)
- For Markov chains we often talk about **walkers**:
  - A walker starts in a given state, and for each iteration its next state is sampled according to the transition probabilities out from its current state
- Next we will look at two simple Markov chain examples

Example 1: A drunkard's walk on the integers

- Let the state space be the integers from 0 to 4

- Given a current integer i, there is a 50% probability of moving to $i+1$ and a 50% probability for moving to $i-1$, modulo 5:

$$\mathrm{Prob}(i \to i+1 \,(\mathrm{mod}\, 5)) = 0.5$$
$$\mathrm{Prob}(i \to i-1 \,(\mathrm{mod}\, 5)) = 0.5$$

- *Question:*
  - Say we start a walker at $i = 3$, then iterate the chain forwards for *a very large number* of iterations, and finally count how many times the walker visited each state (integer). What would this distribution across the states look like?
  - We will see the answer shortly, when we discuss **stationary and limiting distributions** of Markov chains

Example 2: Socks in the duvet cover

- A familiar experience: you wash your socks together with a duvet cover, and when the washing machine stops you find that many of your socks are now inside the duvet cover
- Let's try to describe this as a Markov chain
- Let each cycle of the washing machine count as one step in the Markov chain
- The state space consists of two possible states:
  1. sock inside duvet cover
  2. sock outside duvet cover
- Next we assign some transition probabilities:

$$\mathrm{Prob}(\mathrm{outside} \to \mathrm{outside}) = 0.99, \quad \mathrm{Prob}(\mathrm{outside} \to \mathrm{inside}) = 0.01$$
$$\mathrm{Prob}(\mathrm{inside} \to \mathrm{inside}) = 0.995, \quad \mathrm{Prob}(\mathrm{inside} \to \mathrm{outside}) = 0.005$$

- Whether or not these are reasonable probability assignments is not so important for our example
- *Question:*
  - Similar to the question above: Say we start with 20 socks in the *outside* state, and then run our washing machine for a very long time. Given the transition probabilities chosen above, how many socks should we expect to find inside our duvet cover at the end of the run?
  - (The answer will be given below.)

The transition matrix

- Assuming we are working with a state space with N states, we can collect all the transition probabilities in an $N \times N$ matrix $\mathbf{A}$
  - This matrix is called **the transition matrix**
  - Matrix elements: $(\mathbf{A})_{ij} = \mathrm{Prob}(\text{state } i \to j \text{ in one step})$
- Let a row vector $\vec{\pi}^{(0)} = [\pi_1^{(0)}, \pi_2^{(0)}, \ldots, \pi_N^{(0)}]$ describe how we initially distribute walkers across the state space
  - We normalise $\vec{\pi}^{(0)}$, so we can interpret it as a probability distribution: $\sum_{i=1}^{N} \pi_i^{(0)} = 1$
  - Example: Our starting distribution for the socks-in-duvet-cover example would be $\vec{\pi}^{(0)} = [\pi_{\text{inside}}^{(0)}, \pi_{\text{outside}}^{(0)}] = [0, 1]$
- To find our *expected* distribution of walkers forwards in time we just multiply our current distribution (a row vector) from the right by the transition matrix:

$$\begin{aligned}
\vec{\pi}^{(1)} &= \vec{\pi}^{(0)}\mathbf{A} \\
\vec{\pi}^{(2)} &= \vec{\pi}^{(1)}\mathbf{A} = \vec{\pi}^{(0)}\mathbf{A}^2 \\
&\cdots \\
\vec{\pi}^{(n)} &= \vec{\pi}^{(n-1)}\mathbf{A} = \vec{\pi}^{(0)}\mathbf{A}^n
\end{aligned}$$

- From this we get the following interpretation of the elements of the matrix $\mathbf{A}^n$: $(\mathbf{A}^n)_{ij} = \mathrm{Prob}(\text{state } i \to j \text{ in } n \text{ steps})$

> The fact that we write the vector to the left of the matrix is just due to convention: It is common to define $\vec{\pi}^{(n)}$ as a row vector rather than a column vector, and define the elements of $\mathbf{A}$ accordingly.

- Here we focus on the case of a finite state space. In the case of a continuous space of states $\vec{x}$ we would have the following changes:
  - The transition matrix $\mathbf{A}$ is replaced by a transition probability function $T(\vec{x} \to \vec{x}')$
  - The vector of probabilities $\vec{\pi}^{(n)}$ is replaced by a probability density function $p^{(n)}(\vec{x})$
  - Matrix multiplication is replaced by integration over the entire state space
  - In total:

$$p^{(n)}(\vec{x}') = \int T(\vec{x} \to \vec{x}')\, p^{(n-1)}(\vec{x})\, \mathrm{d}\vec{x}$$

Stationary and limiting distributions

- An important concept for Markov chains is the **stationary distribution** (or **equilibrium distribution**)
  - A probability distribution $\vec{\pi}$ of walkers across the state space ($\pi_i \geq 0$, $\sum_i \pi_i = 1$) is called a *stationary distribution* if it is left unchanged by a step of the chain:
    $$\vec{\pi} = \vec{\pi}\mathbf{A}$$
  - In other words, $\vec{\pi}$ is a normalised row eigenvector of $\mathbf{A}$ with eigenvalue 1
  - Note that this is a property of the transition matrix $\mathbf{A}$ alone — it says nothing about what happens when we run the chain
- Every Markov chain with a *finite* state space has at least one stationary distribution
- So for finite chains the questions are whether it is *unique*, and whether the chain *converges* to it
- (For an *infinite* state space a stationary distribution need not exist, e.g. for the drunkard's walk on *all* the integers, where a walker just keeps diffusing outwards)
- A related, but different, concept is the **limiting distribution**:
  - Start with some distribution $\vec{\pi}^{(0)}$ of walkers and iterate forwards: $\vec{\pi}^{(0)} \to \vec{\pi}^{(1)} \to \vec{\pi}^{(2)} \to \ldots$
  - If $\vec{\pi}^{(n)}$ converges to the *same* distribution $\vec{\pi}^{(\infty)} \equiv \lim_{n\to\infty} \vec{\pi}^{(n)}$ *regardless* of $\vec{\pi}^{(0)}$, we call $\vec{\pi}^{(\infty)}$ the *limiting distribution* of the chain
  - In contrast to a stationary distribution, this is a statement about what happens when we *run* the chain
- How the two concepts are related:
  - A limiting distribution is always a stationary distribution: taking $n \to \infty$ in $\vec{\pi}^{(n+1)} = \vec{\pi}^{(n)}\mathbf{A}$ gives $\vec{\pi}^{(\infty)} = \vec{\pi}^{(\infty)}\mathbf{A}$
  - The converse is *not* true in general. A chain can have a stationary distribution but no limiting distribution, in two ways (examples below):
    1. The chain has *several* stationary distributions, so the limit of $\vec{\pi}^{(n)}$ depends on $\vec{\pi}^{(0)}$
    2. The chain has a *unique* stationary distribution, but $\vec{\pi}^{(n)}$ keeps oscillating and never settles down to it
  - For an *ergodic* chain (see below) the two coincide: there is exactly one stationary distribution, and it is also the limiting distribution. For such chains we can use the two terms interchangeably

- One way to find the stationary distribution(s) of a Markov chain is to solve the eigenvector-eigenvalue problem $\vec{\pi} = \vec{\pi}\mathbf{A}$
  - For the socks-in-duvet-cover example, the two components of the equation $\vec{\pi} = \vec{\pi}\mathbf{A}$ are not independent — they both reduce to $\pi_{\text{inside}} = 2\,\pi_{\text{outside}}$. Combining this with the normalisation condition $\pi_{\text{inside}} + \pi_{\text{outside}} = 1$ gives the answer $[\pi_{\text{inside}}, \pi_{\text{outside}}] = [2/3, 1/3]$
  - This chain is ergodic, so this is also the limiting distribution: whatever we start with, we approach $[2/3, 1/3]$ in the long run
  - So with 20 socks in the washing machine, we should in the long run expect to find around $20 \times 2/3 \approx 13$ socks inside the duvet cover
- For an ergodic chain (see below) the limiting matrix $\lim_{n\to\infty} \mathbf{A}^n$ exists and contains the (unique) stationary distribution $\vec{\pi}$ on every row:

$$\mathbf{A}^\infty \equiv \lim_{n\to\infty} \mathbf{A}^n = \begin{bmatrix} \vec{\pi} \\ \vec{\pi} \\ \vec{\pi} \\ \dots \end{bmatrix} = \begin{bmatrix} \pi_1 & \pi_2 & \pi_3 & \dots \\ \pi_1 & \pi_2 & \pi_3 & \dots \\ \pi_1 & \pi_2 & \pi_3 & \dots \\ \dots & & & \end{bmatrix}$$

  - When combined with the normalisation of the initial distribution $\vec{\pi}^{(0)}$, we see that this limiting matrix $\mathbf{A}^\infty$ implies the expected limit

$$\lim_{n\to\infty} \vec{\pi}^{(0)}\mathbf{A}^n = \vec{\pi}$$

    since

$$\begin{aligned} \vec{\pi}^{(0)}\mathbf{A}^\infty &= \begin{bmatrix} \pi_1^{(0)} & \pi_2^{(0)} & \dots \end{bmatrix} \begin{bmatrix} \pi_1 & \pi_2 & \pi_3 & \dots \\ \pi_1 & \pi_2 & \pi_3 & \dots \\ \pi_1 & \pi_2 & \pi_3 & \dots \\ \dots & & & \end{bmatrix} \\ &= \begin{bmatrix} \pi_1 \left(\pi_1^{(0)} + \pi_2^{(0)} + \dots\right) & \pi_2 \left(\pi_1^{(0)} + \pi_2^{(0)} + \dots\right) & \dots \end{bmatrix} \\ &= \begin{bmatrix} \pi_1 & \pi_2 & \dots \end{bmatrix} \\ &= \vec{\pi} \end{aligned}$$

- We noted above that the matrix $\mathbf{A}^n$ has elements with interpretation $(\mathbf{A}^n)_{ij} = \text{Prob}(\text{state } i \to j \text{ in } n \text{ steps})$
- So, when every row of the limiting matrix $\mathbf{A}^\infty$ is the same it expresses that $(\mathbf{A}^\infty)_{ij} =$

$\text{Prob}(\text{state } i \to j \text{ in } \infty \text{ steps})$ is the same *regardless of initial state* i

  - That is, for an ergodic chain the probability for a walker to end up in state j after a very large number of steps is essentially independent of what state i the walker started from — which is just another way of saying that the chain has a limiting distribution

- Unless $\mathbf{A}$ is too large, a simple way to look for a limiting distribution is to compute $\mathbf{A}, \mathbf{A}^2, \mathbf{A}^3, \ldots$ by repeated multiplication with $\mathbf{A}$, and check whether the powers converge
- Let's try this directly in the Python interpreter for the drunkard's walk example with the integers 0–4 as our state space:

```
>>> import numpy as np
>>> A = np.array([[0., 0.5, 0.0, 0.0, 0.5], [0.5, 0.0, 0.5, 0.0, 0.0], [0.0, 0.5,
    0.0, 0.5, 0.0], [0.0, 0.0, 0.5, 0.0, 0.5], [0.5, 0.0, 0.0,
 0.5, 0.0]])
>>> A
array([[0. , 0.5, 0. , 0. , 0.5],
       [0.5, 0. , 0.5, 0. , 0. ],
       [0. , 0.5, 0. , 0.5, 0. ],
       [0. , 0. , 0.5, 0. , 0.5],
       [0.5, 0. , 0. , 0.5, 0. ]])
>>> An = np.matmul(A, A)
>>> An
array([[0.5 , 0.  , 0.25, 0.25, 0.  ],
       [0.  , 0.5 , 0.  , 0.25, 0.25],
       [0.25, 0.  , 0.5 , 0.  , 0.25],
       [0.25, 0.25, 0.  , 0.5 , 0.  ],
       [0.  , 0.25, 0.25, 0.  , 0.5 ]])
>>> An = np.matmul(An, A)
>>> An
array([[0.   , 0.375, 0.125, 0.125, 0.375],
       [0.375, 0.   , 0.375, 0.125, 0.125],
       [0.125, 0.375, 0.   , 0.375, 0.125],
       [0.125, 0.125, 0.375, 0.   , 0.375],
       [0.375, 0.125, 0.125, 0.375, 0.   ]])
>>> An = np.matmul(An, A)
>>> An
array([[0.375 , 0.0625, 0.25  , 0.25  , 0.0625],
       [0.0625, 0.375 , 0.0625, 0.25  , 0.25  ],
       [0.25  , 0.0625, 0.375 , 0.0625, 0.25  ],
       [0.25  , 0.25  , 0.0625, 0.375 , 0.0625],
       [0.0625, 0.25  , 0.25  , 0.0625, 0.375 ]])
```

(An is now $\mathbf{A}^4$. Keep going, multiplying by A once per step…)

```
>>> for n in range(5, 51):
...     An = np.matmul(An, A)
...
>>> An
array([[0.20001   , 0.19999191, 0.20000309, 0.20000309, 0.19999191],
       [0.19999191, 0.20001   , 0.19999191, 0.20000309, 0.20000309],
       [0.20000309, 0.19999191, 0.20001   , 0.19999191, 0.20000309],
       [0.20000309, 0.20000309, 0.19999191, 0.20001   , 0.19999191],
       [0.19999191, 0.20000309, 0.20000309, 0.19999191, 0.20001   ]])
>>> for n in range(51, 101):
...     An = np.matmul(An, A)
...
>>> An
array([[0.2, 0.2, 0.2, 0.2, 0.2],
       [0.2, 0.2, 0.2, 0.2, 0.2],
       [0.2, 0.2, 0.2, 0.2, 0.2],
       [0.2, 0.2, 0.2, 0.2, 0.2],
       [0.2, 0.2, 0.2, 0.2, 0.2]])
```

- We see that $\mathbf{A}^n$ converges to a matrix with $[0.2, 0.2, 0.2, 0.2, 0.2]$ on every row. So the chain has a limiting distribution, the uniform distribution across the five states, which must then also be a stationary distribution
  - This answers the question for Example 1 above: in the long run the walker visits each state roughly 20% of the time (but see the note below)
- Above we used an *odd* number of states for the drunkard's walk. If you try to use an *even* number of states, e.g. the integers from 0–5, something different happens
  - We get a situation where each state can only return to itself in an even number of steps
  - The distribution $\vec{\pi}^{(n)}$ will then in general depend on whether n is even or odd, no matter how large we make n
    - So the chain has no *limiting* distribution — even though the uniform distribution still satisfies $\vec{\pi} = \vec{\pi}\mathbf{A}$, and thus is a *stationary* distribution of the chain
    - (Try it: the matrix powers $\mathbf{A}^n$ for the 6×6 matrix never converge, but alternate between two matrices)
  - In contrast, in the 0–4 case, a state can return to itself either by an even number of steps (e.g. back and forth) or by an odd number of steps (e.g. going through the cycle once)
  - One way to "fix" the 0–5 case would be to introduce a non-zero probability for staying in the current state
  - We have a short Python code example that demonstrates the case with an even number of states: code_examples/python_MC_stationary_distribution_demo

> *A subtlety*: The question for Example 1 asked about the *fraction of time* a single walker spends in each state, while $\vec{\pi}^{(n)}$ is the *probability distribution for where the walker is* after n steps. For an ergodic chain these agree. For the 0–5 walk they do not: $\vec{\pi}^{(n)}$ never converges, but the time fractions of a long run still converge to the uniform distribution $1/6$. (Try it!) In general, for a finite chain where every state can be reached from every other (an *irreducible* chain, see below), the time fractions converge to the unique stationary distribution even if the chain is periodic.

- The distinction between a stationary and a limiting distribution matters for Markov chain Monte Carlo (below), where we want to construct a chain that converges to a given target distribution p. This takes two ingredients:
  1. *Detailed balance*, which ensures that p is *a stationary* distribution of the chain
  2. *Ergodicity*, which ensures that the chain has only one stationary distribution and converges to it, so that p is also *the limiting* distribution

Detailed balance

- A Markov chain is said to be **time reversible** (or just **reversible**) if it satisfies the following criterion, known as **detailed balance**:

$$p(\text{state } i)\,\text{Prob}(\text{state } i \to j \text{ in one step}) = p(\text{state } j)\,\text{Prob}(\text{state } j \to i \text{ in one step})$$

- Here $\text{Prob}(\text{state } i \to j)$ and $\text{Prob}(\text{state } j \to i)$ are transition probabilities, while $p(\text{state } i)$ and $p(\text{state } j)$ are the probabilities of states i and j in some distribution p across the state space
- If detailed balance holds for a distribution p, then p is a *stationary distribution* of the chain: summing both sides over i gives

$$\sum_i p(\text{state } i)\,\text{Prob}(\text{state } i \to j) = p(\text{state } j) \sum_i \text{Prob}(\text{state } j \to i) = p(\text{state } j)$$

  which is just $\vec{p}\,\mathbf{A} = \vec{p}$ (in the last step we used that the transition probabilities out of state j sum to 1)
  - The converse does not hold: a distribution can be stationary without detailed balance (e.g. the uniform distribution for a drunkard's walk with unequal probabilities for stepping left and right)
  - Detailed balance only makes p *a* stationary distribution. Whether the chain *converges* to p is the separate question of ergodicity, which we turn to next

Ergodicity

- *Note*: There are different conventions in the literature on exactly what the term **ergodicity** is taken to mean. Here we will use the same definition as found in David MacKay's book *Information Theory, Inference, and Learning Algorithms* (2003)
- A Markov chain is **ergodic** if $\vec{\pi}^{(n)} \to \vec{\pi}$ as $n \to \infty$, starting from any $\vec{\pi}^{(0)}$
- In particular, an ergodic chain has a **unique** stationary distribution $\vec{\pi}$, which is also its limiting distribution
  - (The converse does not hold: the drunkard's walk on 0–5 has a unique stationary distribution, but is not ergodic)
- Some reasons why a chain might *not* be ergodic:
  1. The chain might have two or more *closed* subsets of states, i.e. subsets that the chain can never leave once it has entered them, and which therefore can never reach each other. Such a chain is said to be **reducible**. A chain with two or more closed subsets will have multiple stationary distributions (one for each closed subset, plus any mixture of these), so the limit $\lim_{n\to\infty} \vec{\pi}^{(n)}$ will depend on the initial distribution $\vec{\pi}^{(0)}$.
     - The standard way to rule this out is to require that the chain is **irreducible**: that any state can be reached from any other state in a finite number of steps.
  2. The chain might have a **periodic set** — that is, a walker can get trapped in a cycle where $\vec{\pi}^{(n)}$ will never lose its dependence on $\vec{\pi}^{(0)}$, even as we take $n \to \infty$.
     - We saw an example of such dependence for the drunkard's walk on the integers 0–5
     - So another requirement for ergodicity is that the chain is **aperiodic**
- Here's an example of a Markov chain that is *not* ergodic because it is *reducible*:
  - State space: A, B, C, D
  - Transition probabilities:

$$\begin{aligned}
&\mathrm{Prob}(A \to A) = 0.5, \quad \mathrm{Prob}(A \to B) = 0.5 \\
&\mathrm{Prob}(B \to A) = 0.2, \quad \mathrm{Prob}(B \to B) = 0.8 \\
&\mathrm{Prob}(C \to C) = 0.3, \quad \mathrm{Prob}(C \to D) = 0.7 \\
&\mathrm{Prob}(D \to C) = 1.0
\end{aligned}$$

  - We see that $\{A, B\}$ and $\{C, D\}$ are two closed subsets of states: a walker that starts in A or B will forever move between A and B, and a walker that starts in C or D will forever move between C and D

  - So the limit of $\vec{\pi}^{(n)}$ depends on the initial distribution $\vec{\pi}^{(0)}$: the chain has no limiting distribution, and it is not ergodic
    * For instance, a walker starting in A ends up distributed as $[\pi_A, \pi_B, \pi_C, \pi_D] = [2/7, 5/7, 0, 0]$, while a walker starting in C ends up distributed as $[0, 0, 10/17, 7/17]$. Both of these (and any mixture of them) are stationary distributions of the chain

Markov chain Monte Carlo

- **Markov chain Monte Carlo** (MCMC) is a method that produces a Markov chain of samples/events/steps *whose stationary distribution is a given probability distribution*
- In other words: MCMC is a method to generate a set of samples from some chosen pdf $p(x)$ by recording the steps of a Markov chain on x space
- When we say that an MCMC will generate a chain of samples $\{x_1, x_2, \ldots, x_n\}$ distributed according to a pdf $p(x)$, what we mean is that in the limit of an infinitely long chain ($n \to \infty$), we will have

$$\frac{n_{[x,x+dx]}}{n} = p(x)dx$$

  where $n_{[x,x+dx]}$ is the number of steps in the chain where the state was in the range $[x, x+dx]$
- A word on notation:
  - For simplicity I will in the following denote a state simply as x
  - But keep in mind that MCMC is typically used to generate samples in high-dimensional spaces, meaning that we probably should have used the notation $\vec{x}$ instead
- Here are the general steps of an MCMC procedure:
  1. Generate a new candidate state x' using a **proposal pdf** that only depends on the current state x_i
  2. Apply some **acceptance rule** (see below)
     - If accepted: $x_{i+1} = x'$
     - If rejected: $x_{i+1} = x_i$
  3. Repeat from 1.
- In this procedure, it is the acceptance rule that will ensure that the stationary distribution of the Markov chain is our chosen pdf $p(x)$

- Note that at every iteration we do get a next state x_{i+1} — the acceptance rule only determines if the next state should be set equal to the current state x_i or to the candidate state x'
- For the recorded set of samples $\{x_1, x_2, x_3, \ldots\}$ to end up as a set of samples distributed according to our chosen pdf $p(x)$, it is sufficient that
  1. the Markov chain is ergodic
  2. the acceptance rule ensures detailed balance:

$$p(x_i)\,\mathrm{Prob}(x_i \to x') = p(x')\,\mathrm{Prob}(x' \to x_i)$$

- We will now discuss the **Metropolis-Hastings algorithm**, which is a much-used acceptance rule that ensures detailed balance

The Metropolis-Hastings algorithm

- The starting point for Metropolis-Hastings is to write the transition probability $\mathrm{Prob}(x_i \to x')$ as a product of a **proposal probability** $T(x_i \to x')$ and an **acceptance probability** $A(x_i \to x')$:

$$\mathrm{Prob}(x_i \to x') = T(x_i \to x')A(x_i \to x')$$

  - If the proposal pdf is *symmetric*, that is if $T(x_i \to x') = T(x' \to x_i)$, the method is simply called the **Metropolis algorithm**

- The **MCMC procedure with a Metropolis-Hastings acceptance rule** then goes as follows:

1. Sample a candidate x' according to the proposal pdf $T(x_i \to x')$
2. Calculate the acceptance probability $A(x_i \to x')$ as

$$A(x_i \to x') = \min\left(1,\, \frac{p(x')\,T(x' \to x_i)}{p(x_i)\,T(x_i \to x')}\right)$$

   or, if the proposal pdf is symmetric (Metropolis), simply as

$$A(x_i \to x') = \min\left(1,\, \frac{p(x')}{p(x_i)}\right)$$

3. Generate a random number r from $U(0,1)$
   - If $r \leq A(x_i \to x')$, then accept x', i.e. set $x_{i+1} = x'$
   - If $r > A(x_i \to x')$, then reject x', i.e. set $x_{i+1} = x_i$
4. Repeat from 1.

- We note that, for a symmetric proposal pdf, this algorithm will
  - *always* accept a move to a state with higher probability (when $p(x') > p(x_i)$)
  - *sometimes* accept a move to a state with lower probability (when $p(x') < p(x_i)$)
- The probability for accepting a move to a lower-probability state x' is set such that detailed balance is satisfied
  - This is what ensures that the long-run chain of samples $\{x_1, x_2, x_3, \ldots\}$ will be distributed according to our chosen $p(x)$
- We also note that the acceptance rule only depends on $p(x_i)$ and $p(x')$ through the ratio $p(x')/p(x_i)$
  - Any normalisation constant in the pdf $p(x)$ will cancel in this ratio
  - This is important in cases where the normalisation constant in $p(x)$ is unknown or difficult to compute, but the ratio $p(x')/p(x_i)$ is easy to evaluate
  - We will encounter such a case in project 4
- We have freedom to choose the proposal distribution T
  - If we run the chain for long enough, it should theoretically visit all possible states and spend the correct "amount of time" (number of iterations) in each state, as given by $p(x)$

  - However, how long is "long enough" can depend strongly on our choice of proposal distribution T

- Below is an explanation for why we set $A(x_i \to x')$ to the value

$$A(x_i \to x') = \min\left(1, \frac{p(x')\,T(x' \to x_i)}{p(x_i)\,T(x_i \to x')}\right)$$

  - We start from the requirement of detailed balance, which when using the factorisation $\mathrm{Prob}(x_i \to x') = T(x_i \to x')A(x_i \to x')$ takes the form

$$p(x_i)\,T(x_i \to x')\,A(x_i \to x') = p(x')\,T(x' \to x_i)\,A(x' \to x_i)$$

  - The target pdf $p(x)$ is given by the particular problem we are trying to solve with MCMC, and we assume that we have already chosen our proposal pdf (the T's)
  - The only thing left to figure out are what to use as our acceptance probabilities $A(x_i \to x')$ and $A(x' \to x_i)$
  - We can rewrite the detailed balance requirement as

$$\frac{A(x_i \to x')}{A(x' \to x_i)} = \frac{p(x')\,T(x' \to x_i)}{p(x_i)\,T(x_i \to x')}$$

  - This highlights that detailed balance only sets a requirement for the *ratio* of the two acceptance probabilities
  - So we are free to choose one of the A's as we want, as long as we then set the other one according to the ratio above
  - We can use this freedom to maximise the efficiency with which we explore the pdf $p(x)$, by choosing our A's such that we *accept the proposed move to x' as often as possible*
  - That is, we want to set $A(x_i \to x')$ as large as possible, while still satisfying the ratio above
  - Both A's are probabilities, i.e. numbers between 0 and 1
  - So the way to ensure the largest possible $A(x_i \to x')$ consistent with the ratio, is to say that we will set the larger of the two A's equal to 1, and then the other A is set by the ratio above
  - The expression $A(x_i \to x') = \min\left(1, \frac{p(x')\,T(x' \to x_i)}{p(x_i)\,T(x_i \to x')}\right)$ used in the Metropolis-Hastings algorithm corresponds to making this choice
  - To see this, let's first simplify the situation by assuming we have chosen a symmetric proposal pdf, so $T(x_i \to x') = T(x' \to x_i)$

- The detailed balance requirement is then simply

$$\frac{A(x_i \to x')}{A(x' \to x_i)} = \frac{p(x')}{p(x_i)}$$

- First, let's consider the case where the proposed move is to a higher-probability x value, i.e. that $p(x') \geq p(x_i)$
  * Then we have the requirement

$$\frac{A(x_i \to x')}{A(x' \to x_i)} = \frac{p(x')}{p(x_i)} \geq 1$$

  * Here $A(x_i \to x')$ is clearly the larger of the two A's so we set it to

$$A(x_i \to x') = 1$$

  * This choice ensures that we will *always* accept a move from a lower-probability point x_i to a higher-probability point x'
- Second, consider the case where the proposed move is to a lower-probability x value, i.e. when $p(x') < p(x_i)$
  * Then we have the requirement

$$\frac{A(x_i \to x')}{A(x' \to x_i)} = \frac{p(x')}{p(x_i)} < 1$$

  * Here the larger of the two A's is $A(x' \to x_i)$, so we set that to 1

$$A(x' \to x_i) = 1$$

    which means that $A(x_i \to x')$ becomes

$$A(x_i \to x') = \frac{p(x')}{p(x_i)}$$

- Finally, we see that we can summarise these two cases with a single probability assignment on the form

$$A(x_i \to x') = \min\left(1, \frac{p(x')}{p(x_i)}\right)$$

- The general case works in exactly the same way, with $p(x')/p(x_i)$ replaced by the full ratio

$$\frac{p(x')\,T(x' \to x_i)}{p(x_i)\,T(x_i \to x')}$$

Burn-in

- Recap: In the theoretical limit of an infinitely long MCMC run, the resulting chain of samples $\{x_1, x_2, \ldots\}$ will be distributed according to our target pdf $p(x)$:

$$\frac{n_{[x,x+dx]}}{n} = p(x)dx$$

- In this limit, computing e.g. some expectation value $\mathrm{E}[f(x)]$ as the average of all our $f(x)$ samples would give exactly the correct result
- But in practice we can never collect an infinite number of samples
- This means our resulting set of samples will always be somewhat influenced by where we started the chain
- If we happen to start the chain in an x region that according to $p(x)$ should be highly improbable to sample *given the limited number of samples we collect*, then the resulting distribution of samples may be significantly biased towards this region
  - In other words, the resulting set of samples will *not* be a reasonable approximation for $p(x)$
  - Consequently, any expectation values we compute from this set of samples can also be strongly biased by the starting point
- A pragmatic solution is what is called **burn-in**: *to throw away the first set of samples that the chain collects*
- Sometimes you may also see the burn-in referred to as **equilibration** or **warm-up**
- The reasoning behind this is as follows:
  - *If* we have started the chain from a "too improbable" position x, the chain will (probably) begin by mainly evolving towards the higher-probability regions of x space
  - Once the chain has reached this higher-probability region, the samples collected from that time onwards will likely constitute a more reasonable representation of the target pdf $p(x)$
- So, how many of the first samples should be discarded?
- This is difficult to say!
- One practical approach is to plot some time series of your samples and see if some early part of the run has a very different behaviour (typically moving systematically in one direction) compared to the rest of the run

  - Such a time series plot will have the number of iterations along the horizontal axis, but what should the vertical axis be?
  - If x represents points in a one-dimensional space, we'd obviously just plot the x value on the vertical axis
  - But in the one-dimensional case we often wouldn't be using MCMC in the first place…
  - If we are working in a many-dimensional space, you may have to study time series plots for many x coordinates
  - Or, if you are mainly interested in some derived quantity $f(x)$, the time series plot for that quantity is probably the most useful
  - It can also be useful to study the time series of some expectation value $\mathrm{E}[f(x)]$, which at each step of the time series is computed with all the samples collected up to that iteration

- Finally: keep in mind that the burn-in issue becomes less important the longer we can run our MCMC for
- In the limit of infinite samples there is no burn-in issue at all — then all parts of $p(x)$ will be explored in the right proportion
- For an interesting read/rant about burn-in, see this web page: http://users.stat.umn.edu/~geyer/mcmc/burn.html

Correlated samples

- When we are generating samples from some pdf $p(x)$, we usually want **independent samples**
- That is, in the ideal case each sample is generated completely independently from all other samples
- Going back to the methods for low-dimensional sampling, both *rejection sampling* and *inverse transform sampling* are methods that produce independent samples from the given $p(x)$
- But for a Markov chain Monte Carlo, we have seen that a key part is to use the current step x_i to propose the next step x_{i+1}
- So MCMCs typically generate **correlated samples**
  - If I tell you the value of sample x_{1433} you will probably make a more accurate guess for the value of sample x_{1436} than you will if I don't tell you the value of x_{1433}
  - In other words: $\mathrm{Prob}(x_{1436} \,|\, x_{1433}) \neq \mathrm{Prob}(x_{1436})$

  - The width of the proposal pdf matters here: a proposal pdf that suggests large steps gives a lower correlation between samples *when the steps are accepted*
    * But if the proposal pdf is too wide, most proposed steps will be rejected, so the chain stays put at the same x for many iterations — which again increases the correlation between samples (and slows down the exploration of $p(x)$)
    * So there is a sweet spot for the proposal width somewhere in between

- n correlated samples contain less information about $p(x)$ than n independent samples do
  - This makes intuitive sense: if two samples, say x_i and x_{i+1} are correlated, then part of the information about $p(x)$ provided by sample x_{i+1} was already provided by sample x_i
- This is often quantified by computing an **effective sample size**, n_{eff}, based on the n samples the MCMC produced
- The effective sample size expresses the actual information content in the n correlated samples by estimating how many *independent* samples the correlated samples correspond to
- In most cases we will have $n_{\text{eff}} < n$
- So, if we treat our n MCMC samples as if they are independent samples from $p(x)$, we will usually be *underestimating our uncertainties*
  - *Example*: Say we naively estimate some expectation value $\mathrm{E}[f(x)]$ as

    $$\mathrm{E}[f(x)] \approx \bar{f} \pm \frac{s_f}{\sqrt{n}}$$

    where $\bar{f}$ is the sample average, s_f is the sample standard deviation and n is the number of collected MCMC samples
  - In dividing by $\sqrt{n}$ we are assuming that our samples are independent
  - To more accurately estimate our uncertainties, we should divide by $\sqrt{n_{\text{eff}}}$ instead
- The main point to remember: MCMC samples are typically *not* independent, and n_{eff}, not n, is the number to use in uncertainty estimates. For the interested reader, the side note below explains how n_{eff} is computed in practice

> A side note on how to compute the effective sample size:
>
> - Consider a chain of n values $f_i = f(x_i)$, after burn-in

- The sample mean is $\bar{f} = \frac{1}{n}\sum_i f_i$, so its variance is

$$\mathrm{Var}\left(\bar{f}\right) = \frac{1}{n^2}\mathrm{Var}\left(\sum_{i=1}^{n} f_i\right) = \frac{1}{n^2}\left[\sum_{i=1}^{n}\mathrm{Var}(f_i) + \sum_{i\neq j}\mathrm{Cov}(f_i, f_j)\right]$$

  where the covariance $\mathrm{Cov}(f_i, f_j) = \mathrm{E}[(f_i - \mu_f)(f_j - \mu_f)]$, with $\mu_f = \mathrm{E}[f]$, measures how correlated f_i and f_j are

- For independent samples all the covariances vanish, and we get the familiar result $\mathrm{Var}\left(\bar{f}\right) = \sigma_f^2/n$

- For a stationary chain, the covariance of two samples only depends on how many steps apart they are. For two samples k steps apart we write

$$\mathrm{Cov}(f_i, f_{i+k}) \equiv \sigma_f^2\,\rho_k, \qquad \rho_k = \frac{\mathrm{E}[(f_i - \mu_f)(f_{i+k} - \mu_f)]}{\sigma_f^2}$$

  where ρ_k is the **autocorrelation** at **lag** k. We have $\rho_0 = 1$, and for a well-behaved MCMC $\rho_k \to 0$ for large k

- We can now evaluate the two sums in the expression for $\mathrm{Var}\left(\bar{f}\right)$ above:
  - The first sum has n terms, each equal to σ_f^2, so $\sum_i \mathrm{Var}(f_i) = n\sigma_f^2$
  - In the second sum we group the pairs (i, j) by their lag $k = |i - j|$. For a given $k \geq 1$ there are $n - k$ pairs with $j = i + k$ and $n - k$ pairs with $j = i - k$, each contributing $\sigma_f^2\rho_k$. So $\sum_{i\neq j}\mathrm{Cov}(f_i, f_j) = 2\sum_{k=1}^{n-1}(n-k)\,\sigma_f^2\rho_k$
  - Inserting this and dividing by n^2:

$$\mathrm{Var}\left(\bar{f}\right) = \frac{1}{n^2}\left[n\sigma_f^2 + 2\sum_{k=1}^{n-1}(n-k)\,\sigma_f^2\rho_k\right]$$

$$= \frac{\sigma_f^2}{n}\left[1 + 2\sum_{k=1}^{n-1}\left(1 - \frac{k}{n}\right)\rho_k\right]$$

- For a sufficiently long chain, ρ_k has died off long before the lag k becomes comparable to n. Then $(1 - k/n) \approx 1$ for all the terms that matter. A common further approximation is then to extend the sum over k to infinity, since the terms with $k \geq n$ are anyway negligible. This gives

$$\mathrm{Var}\left(\bar{f}\right) \approx \frac{\sigma_f^2}{n}\,\tau, \qquad \tau \equiv 1 + 2\sum_{k=1}^{\infty}\rho_k$$

where τ is the **integrated autocorrelation time**. The point of the approximation is that τ is a property of the chain itself, independent of the number of samples n: the variance of $\bar{f}$ is a factor τ larger than it would be for n independent samples, no matter how long we run the chain

  - For independent samples $\rho_k = 0$ for $k \geq 1$, so $\tau = 1$

- Now define n_{eff} as the number of *independent* samples that would give the same $\mathrm{Var}\left(\bar{f}\right)$, that is, n_{eff} is defined by the relation $\mathrm{Var}\left(\bar{f}\right) \equiv \sigma_f^2 / n_{\text{eff}}$. Comparing this with the expression above, we get

$$n_{\text{eff}} = \frac{n}{\tau}$$

  so τ is the number of MCMC steps per independent sample

- To estimate ρ_k from the chain, we estimate the numerator and denominator in the expression for ρ_k from the available pairs, with $\bar{f}$ in place of μ_f:

$$\hat{\rho}_k = \frac{\sum_{i=1}^{n-k} (f_i - \bar{f})(f_{i+k} - \bar{f})}{\sum_{i=1}^{n} (f_i - \bar{f})^2}$$

  - This is the same expression as r_k in the autocorrelation test in the chapter on random number generation

- We use ρ_k to get an estimate $\hat{\tau}$. The estimated effective sample size is then $\hat{n}_{\text{eff}} = n/\hat{\tau}$, with the sum in $\hat{\tau}$ cut off at some maximum lag K

- *Practical problem*: Why not sum over all lags up to $k = n - 1$?

  - At large lags the true $\rho_k \approx 0$, but $\hat{\rho}_k$ is based on few pairs and fluctuates around zero with size $\sim 1/\sqrt{n}$. Summing many such terms adds noise to $\hat{\tau}$, not information

  - Too small K misses real correlations and overestimates n_{eff}; too large K drowns the estimate in noise

- Common workarounds:

  - *Truncation*: Stop the sum at the first lag where $\hat{\rho}_k$ becomes negative, or (more robust, used in Stan) at the first pair $\hat{\rho}_{2m} + \hat{\rho}_{2m+1}$ with a negative sum

  - *Automatic windowing*: Use a window of a fixed number of autocorrelation times, $K \approx c\,\tau$ with $c \approx 5$–10. Since τ is unknown, choose K self-consistently: increase K from 1 and

> stop at the first K for which $K \geq c\,\hat{\tau}(K)$, where $\hat{\tau}(K)$ is the estimate with the sum cut off at K (used in `emcee`)

Thinning

- **Thinning** refers to a common MCMC practice of *only storing every m-th step in the chain*
- The reason for doing this is typically to
  - save disk space
  - save memory, if samples are not immediately written to file
  - save time, if you have to evaluate some computationally costly $f(x)$ at every saved x sample
- In particular, if the proposal distribution is such that the chain usually makes very small steps, it may be impractical to save every step
- When thinning is used, the degree of correlation between the saved samples is lower compared to if every step in the chain was saved
  - This means that naive error estimates based on $\sqrt{n}$ (see example above) will be less wrong, compared to if thinning was not used
  - However, if it's practically doable, the best approach is usually to save every MCMC step, compute an estimate for the effective sample size n_{eff}, and use this in uncertainty estimates
- In project 4 we will in practice use a form of thinning: Our MCMC will make many tiny steps, but only save every N-th step, where N is the number of spins in a 2D Ising model, and we will ignore the fact that the saved samples are correlated

Physics of Project 4: the Ising model

- We should probably call it *the Lenz-Ising model* (Wilhelm Lenz, Ernst Ising, 1920s)
- Originally a model for ferromagnetism in statistical mechanics, but the same basic idea has since been used to model *many* different phenomena in science
- We will work with the 2D Ising model
- Basic setup of the model:
  - A lattice/grid of spins (variables) that can only take the values -1 (↓) or +1 (↑)

$$\begin{array}{ccccccc} & \vdots & \vdots & \vdots & \vdots & & \\ \ldots & \uparrow & \downarrow & \uparrow & \downarrow & \ldots & \\ \ldots & \uparrow & \uparrow & \downarrow & \downarrow & \ldots & \\ \ldots & \downarrow & \downarrow & \uparrow & \downarrow & \ldots & \\ \ldots & \uparrow & \uparrow & \uparrow & \downarrow & \ldots & \\ & \vdots & \vdots & \vdots & \vdots & & \end{array} \quad \Leftrightarrow \quad \begin{array}{ccccccc} & \vdots & \vdots & \vdots & \vdots & & \\ \ldots & +1 & -1 & +1 & -1 & \ldots & \\ \ldots & +1 & +1 & -1 & -1 & \ldots & \\ \ldots & -1 & -1 & +1 & -1 & \ldots & \\ \ldots & +1 & +1 & +1 & -1 & \ldots & \\ & \vdots & \vdots & \vdots & \vdots & & \end{array}$$

  - Each spin interacts with its nearest neighbours

$$\begin{array}{ccccc} & & \bullet & & \\ & & \updownarrow & & \\ \bullet & \leftrightarrow & \bullet & \leftrightarrow & \bullet \\ & & \updownarrow & & \\ & & \bullet & & \end{array}$$

  - ↑↑ and ↓↓ pairs have lower energy than ↑↓ and ↓↑ pairs
  - The system can exchange heat with the environment (a thermal bath at temperature T)
  - Change in system energy due to heat ↔ spins flip ("thermal fluctuations")
- The main topics for Project 4:
  - We will study the properties of the system at equilibrium, as a function of the chosen temperature T and lattice size
  - Properties of interest:
    * Mean energy
    * Mean magnetisation

    * Heat capacity
    * Magnetic susceptibility
  - We will also determine *the critical temperature* (T_c)
    * If we gradually lower the temperature, T_c is the temperature at which the system undergoes *a phase transition*, from a *disordered, non-magnetised state* to an *ordered, magnetised state*
    * In 1944 Lars Onsager found the exact solution for the 2D Ising model with an infinitely large lattice, from which the exact T_c follows (the value of T_c had already been located by Kramers and Wannier in 1941)
    * We will use our numerical results for T_c for finite-sized lattices to extrapolate towards the infinite limit and compare to Onsager's result

Overview of important variables and functions

- There are many different quantities to keep track of in this project
- Let's introduce them one by one
- Lattice length: L
- Number of spins in lattice: $N = L^2$
- Spin value for a single spin: $s_i \in \{-1, +1\}$
- A *spin configuration* or *microstate* (or simply *state*)
  - This is the state of the entire lattice of spins
  - To keep the notation simple, we will here use vector notation to denote a microstate:

$$\vec{s} = [s_1, s_2, \ldots, s_N]$$

  - In your code it may be more natural to represent the microstate (spin lattice) as a *matrix*, since we are working with a 2D model
  - The N-dimensional state space (the space of all possible $\vec{s}$) contains 2^N possible states, since each individual spin can take two values
- The system energy for a given state $\vec{s}$:

$$E(\vec{s}) = -J \sum_{\langle kl \rangle} s_k s_l$$

  - Here $\sum_{\langle kl \rangle}$ denotes the sum over all neighbouring *spin pairs*, with no double-counting
  - J denotes the coupling constant that sets the interaction strength
  - We will choose $J > 0$
  - The choice $J > 0$, combined with the overall negative sign in the $E(\vec{s})$ expression, means that:
    * An equal-spin interaction (↑↑ or ↓↓) contributes $-J$ to the system energy, i.e. it *lowers* the energy, since $s_k s_l = (\pm 1)(\pm 1) = 1$
    * An unequal-spin interaction (↑↓ or ↓↑) contributes $+J$ to the system energy, i.e. it *raises* the energy, since $s_k s_l = (\pm 1)(\mp 1) = -1$
  - That aligned spins give lower energy is the key modelling choice that makes this a model of ferromagnetism
  - We do not include an external magnetic field
    * If we had done so, we would have additional terms in our $E(\vec{s})$ expression to represent the interaction between the individual spins and the external magnetic field

- The system magnetisation for a given state $\vec{s}$:

$$M(\vec{s}) = \sum_{i=1}^{N} s_i$$

- State *degeneracy*:
  - The *number of different states* $\vec{s}$ that have the same value for some quantity, e.g. $E(\vec{s})$ or $M(\vec{s})$
  - Consider an example with a simple one-dimensional model with only three spins

* All possible states along with their magnetisations:

$$\begin{aligned}
\vec{s}_1 &= [+1,+1,+1], \quad M(\vec{s}_1) = 3 \\
\vec{s}_2 &= [+1,+1,-1], \quad M(\vec{s}_2) = 1 \\
\vec{s}_3 &= [+1,-1,+1], \quad M(\vec{s}_3) = 1 \\
\vec{s}_4 &= [+1,-1,-1], \quad M(\vec{s}_4) = -1 \\
\vec{s}_5 &= [-1,+1,+1], \quad M(\vec{s}_5) = 1 \\
\vec{s}_6 &= [-1,+1,-1], \quad M(\vec{s}_6) = -1 \\
\vec{s}_7 &= [-1,-1,+1], \quad M(\vec{s}_7) = -1 \\
\vec{s}_8 &= [-1,-1,-1], \quad M(\vec{s}_8) = -3
\end{aligned}$$

* We see that:
  - $M = 3$ has a degeneracy of 1
  - $M = 1$ has a degeneracy of 3
  - $M = -1$ has a degeneracy of 3
  - $M = -3$ has a degeneracy of 1

- Probability distribution for $\vec{s}$: *the Boltzmann distribution*

$$p(\vec{s};T) = \frac{1}{Z} e^{-\frac{E(\vec{s})}{k_B T}} = \frac{1}{Z} e^{-\beta E(\vec{s})}$$

  - $\beta = \frac{1}{k_B T}$, where k_B is the Boltzmann constant
  - The normalisation factor Z is the so-called *partition function* (see below)
  - Choosing the Boltzmann distribution as our probability distribution $p(\vec{s};T)$ means that we are studying the system at thermal equilibrium with the environment
  - Keep in mind that $p(\vec{s};T)$ is the pdf for the *states* $\vec{s}$, it is *not* the pdf for energy $E(\vec{s})$
    * Easy to forget, since $\vec{s}$ only appears in $p(\vec{s};T)$ via the energy $E(\vec{s})$
    * If we insert our expression for $E(\vec{s})$ we more clearly see how $p(\vec{s};T)$ is a joint pdf for

the individual components of $\vec{s}$:

$$\begin{aligned} p(\vec{s};T) &= p(s_1, s_2, \ldots, s_N; T) \\ &= \frac{1}{Z} e^{-\beta E(\vec{s})} \\ &= \frac{1}{Z} e^{\beta J \sum_{\langle kl \rangle} s_k s_l} \end{aligned}$$

- The partition function:

$$Z = \sum_{\substack{\text{all possible} \\ \text{states } \vec{s}}} e^{-\beta E(\vec{s})}$$

  - It is called the partition *function* since it is a function of the temperature T (via $\beta = \frac{1}{k_B T}$)
  - For a given temperature, Z serves as the normalisation constant in the pdf $p(\vec{s};T)$
  - It is essentially a weighted count of how many states are thermally accessible at temperature T
  - It can be used to derive how various thermodynamic quantities depend on temperature

- Probability distributions for other quantities
  - In this project we will explore numerically what the pdf for energy, $p_E(E;T)$, looks like
    * We will use MCMC to sample microstates $\vec{s}$ from $p(\vec{s};T)$
    * For each sampled microstate $\vec{s}$ we can compute the corresponding $E(\vec{s})$
    * This $E(\vec{s})$ can then be seen as one sample from the unknown pdf $p_E(E;T)$
  - Similarly, we can compute the magnetisation $M(\vec{s})$ for each sampled microstate, and view these M values as samples from the unknown pdf $p_M(M;T)$

- We will use our E and M samples to compute various expectation values:
  - $\langle E \rangle$, $\langle E^2 \rangle$, $\langle |M| \rangle$, $\langle M^2 \rangle$
  - The reason why we work with the absolute value of the magnetisation, $|M|$:
    * Since we do not have an external magnetic field to pick out a preferred direction, there is an overall and exact "up-down symmetry" in our problem:
      · For every state $\vec{s}$ there is an opposite state $-\vec{s}$ (the state where *all* the individual spins are flipped) that has exactly the same energy, $E(-\vec{s}) = E(\vec{s})$, and hence has the same probability, $p(-\vec{s};T) = p(\vec{s};T)$

  - But this opposite state $-\vec{s}$ would have the *opposite* magnetisation: $M(-\vec{s}) = -M(\vec{s})$
  - So, without an external magnetic field, the expectation value of the signed magnetisation, $\langle M \rangle$, is technically 0 for every temperature T — that is, even at very low temperatures, where there should be overwhelmingly high probability to observe the system in an ordered state, i.e. a state with $|M|$ close to N.
  - By doing the replacement $M \to |M|$ we focus on the question "*How strong* magnetisation do we expect to observe at temperature T?", rather than the question "*What* magnetisation do we expect to observe at temperature T?".
  - This is commonly done for numerical studies of this 2D Ising model. It makes it easier to compare and combine results from individual runs (e.g. repeated runs at the same or similar temperatures), because we don't have to worry about whether the system randomly ended up in a state with mostly -1 spins or a state with mostly $+1$ spins.

- To easily compare results obtained for different lattice sizes, we will often work with energy and magnetisation *per spin*:
  - Our notation:
$$\frac{E}{N} \equiv \epsilon, \qquad \frac{M}{N} \equiv m$$
  - This means we will also be interested in the expectation values
$$\langle \epsilon \rangle = \left\langle \frac{E}{N} \right\rangle = \frac{1}{N} \langle E \rangle$$
$$\langle |m| \rangle = \left\langle \frac{|M|}{N} \right\rangle = \frac{1}{N} \langle |M| \rangle$$

- Heat capacity (C_V) and magnetic susceptibility (χ):
  - We will compute these using the expressions
$$\frac{C_V(T)}{N} = \frac{1}{N}\frac{1}{k_B T^2}\mathrm{Var}(E) = \frac{1}{N}\frac{1}{k_B T^2}\left[\langle E^2 \rangle - \langle E \rangle^2\right] = N\frac{1}{k_B T^2}\left[\langle \epsilon^2 \rangle - \langle \epsilon \rangle^2\right]$$
$$\frac{\chi(T)}{N} = \frac{1}{N}\frac{1}{k_B T}\mathrm{Var}(|M|) = \frac{1}{N}\frac{1}{k_B T}\left[\langle M^2 \rangle - \langle |M| \rangle^2\right] = N\frac{1}{k_B T}\left[\langle m^2 \rangle - \langle |m| \rangle^2\right]$$
    since these expressions are given directly in terms of some simple expectation values

- Note that here we have also included a factor $1/N$, to get heat capacity and magnetic susceptibility per spin

> Some probably more familiar expressions for these quantities are
>
> $$C_V(T) = \frac{\partial \langle E \rangle}{\partial T}$$
>
> $$\chi(T) = \frac{\partial \langle M \rangle}{\partial H}$$
>
> where H is the magnetic field intensity, and the derivative is evaluated at $H = 0$

The basic idea in Project 4

- Choose a lattice size (L) and a temperature (T)
- Use MCMC to sample microstates $\vec{s}$ according to the Boltzmann distribution $p(\vec{s}\,; T)$ and compute derived quantities of interest, such as the corresponding energy samples $E(\vec{s}\,)$ and various expectation values ($\langle E \rangle, \ldots$)
- Repeat for different choices of lattice size and temperature
- Study how the results depend on the lattice size and the temperature, and in particular how the system behaves for temperatures close to the critical temperature T_c
- A word of warning: *Know your types of sums*
  - This project involves computing many different types of sums
  - Make sure you know what you are supposed to be summing over
  - Examples:

$$\sum_{\substack{\text{all spins}\\ \text{in lattice}}} \qquad \sum_{\substack{\text{all neighbouring}\\ \text{spin pairs}}} \qquad \sum_{\substack{\text{all possible}\\ \text{states } \vec{s}}} \qquad \sum_{\text{all } \vec{s} \text{ samples}}$$

$$\sum_{\substack{\text{all possible}\\ E(\vec{s}\,) \text{ values}}} \qquad \sum_{\text{all } E(\vec{s}\,) \text{ samples}} \qquad \ldots$$

Suggested algorithm for Project 4

- As mentioned earlier in the section on *thinning*, for Project 4 we suggest you implement an MCMC algorithm that performs many tiny steps in the space of microstates, but only saves/uses the state at every N-th step for further computations

- One reason for this is purely practical:
  - We *could* have saved/used the state after every single step as a new MCMC sample
  - But we would end up collecting a huge number of almost-identical samples, which would cost more disk space and/or memory
- Another reason why we only use every N-th step is that the collected samples are then less correlated with each other
  - This means we can more reasonably ignore complications like *autocorrelation* and *effective sample size* when we compute variances and error estimates
- We'll use the following terminology:
  - *A step*: Flip a single spin in our 2D lattice and accept/reject the move to this new microstate
  - *A cycle*: Attempt N single-spin flips
- So, *a single MCMC cycle* consists of N *steps*
- The total number of MCMC cycles = the number of pdf samples we collect/use
- Here's the suggested algorithm:

1. Choose a temperature (T) and a lattice size ($L \times L = N$ = number of spins)
2. Choose the number of MCMC cycles (n_{cycles}) to perform
3. Choose an initial state ($\vec{s}$) for the system
4. For each MCMC cycle:
   a) For each step within the cycle:
      i. From the current state $\vec{s}$, generate a new candidate state $\vec{s}\,'$ by picking a random spin in the lattice (use a uniform pdf) and flipping it
      ii. Compute the ratio $\frac{p(\vec{s}\,')}{p(\vec{s})}$
      iii. Generate a random number $r \sim U(0,1)$ and accept the move $\vec{s} \to \vec{s}\,'$ if $r \leq \frac{p(\vec{s}\,')}{p(\vec{s})}$ (otherwise keep the current state $\vec{s}$)
   b) After N steps, use the resulting state $\vec{s}$ to compute any derived quantities of interest, e.g. the current energy and magnetisation
   c) Store all the numbers you need

- A few comments to the suggested algorithm above:
  - Our procedure for generating $\vec{s}'$ (select a single spin at random and flip it) means that our proposal distribution $T(\vec{s} \rightarrow \vec{s}')$ is a uniform distribution on all the "neighbour states" of the current state $\vec{s}$, and it is symmetric, $T(\vec{s} \rightarrow \vec{s}') = T(\vec{s}' \rightarrow \vec{s})$, so we can use the simple Metropolis acceptance rule
  - For every tiny step $\vec{s} \rightarrow \vec{s}'$ we attempt we will need the ratio $\frac{p(\vec{s}')}{p(\vec{s})}$, so you should think about how you can evaluate this ratio as efficiently as possible
  - In particular, note that we just need the *ratio* $\frac{p(\vec{s}')}{p(\vec{s})}$, not the individual $p(\vec{s})$ and $p(\vec{s}')$

Parallel computing

- "History":
  - Rough model for program runtime:

$$\begin{aligned}[\text{runtime for program}] &\approx [\#\text{ instructions in program}] \times \frac{[\#\text{ cycles}]}{[\text{instruction}]} \times \frac{[\text{time}]}{[\text{cycle}]} \\ &= [\#\text{ instructions in program}] \times \frac{[\#\text{ cycles}]}{[\text{instruction}]} \times \frac{1}{f_{\text{proc}}}\end{aligned}$$

    where f_{proc} is the processor frequency, i.e. the number of cycles per time
  - So as f_{proc} increased, programs would run faster without changes to the code itself
  - Transistor power consumption when switching: $P \propto CV(f)^2 f$
  - Up to around 2004, shrinking transistors $\rightarrow$ lower capacitance (C) and supply voltage (V) and higher frequency (f) without increasing power consumption (*Dennard scaling*)
  - But when the transistors got sufficiently small, V could no longer be reduced this way (leakage currents and other practical challenges)
  - So then increasing f would increase power consumption per transistor
  - So eventually, increasing f_{proc} runs into problems with overheating, etc.
  - Therefore, from ~2004, performance increase has come mainly from shift to **parallel computing** and **multicore processors**
- The challenge: Now increased performance also requires changes on the software side! Codes must distribute the tasks and/or data across multiple threads or processes!
  - We need to learn some basic **parallelisation**
- **Parallelisation** vs **concurrency**
  - We should try to not confuse parallelisation with concurrency:
  - Let's consider an example with two tasks:
    * Task 1: Debugging your code for project 4
    * Task 2: Get your passport renewed at the police office
  - **Parallelisation**: The two tasks are being worked on at the same time
    * Example: Your friend works on debugging the code (task 1) while you are at the police office to get your new passport (task 2)

  - **Concurrency**: Two tasks are both in progress over the same period, but not necessarily being worked on at the same time, e.g. by switching between them
    * Example: You have brought your laptop to the police office, so while you are waiting in line (idle time on task 2) you do a bit of debugging (work on task 1)
  - We are focusing on *parallelisation*

- There are two main approaches to parallelisation: **shared memory** and **distributed memory**
- 1) **Shared memory** / **threading**:
  - Typically used for parallelisation within a single computer/node
  - A single process at some point splits into multiple threads, performs some task in parallel across the threads, and then combines the threads back to a single process
  - All threads by default share the same memory (technically: *address space*), designated to the one initial process
  - Example of this approach: `OpenMP` and `<thread>`
    * We will see examples of how to use `OpenMP` for shared-memory parallelisation
- 2) **Distributed memory** / **message-passing**:
  - Can be used on a single computer/node or across multiple computers/nodes
    * Much used when working with supercomputers (clusters with many nodes)
  - Example of this approach: `MPI` (Message Passing Interface)
  - Launch multiple processes, each with its own private memory, and the processes can communicate and share the workload by explicitly sending messages back and forth
  - Typical case:
    * MPI launches multiple independent instances of the same program, each process has a designated process number (the **rank**)
    * The program contains a bunch of if-else blocks that determine what each individual process should do
  - Example logic:
    * If rank == 0:
      · Send message with new input to all other processes
      · Wait to receive replies with the new results
    * Else:

        - Wait for message from rank 0
        - Perform computation using the input in the received message
        - Send message back to rank 0 with the new result

- Sometimes useful to combine distributed-memory and shared-memory parallelisation in a single program: two-level parallelisation
  - Example:
    * Outer-level tasks parallelised using MPI
    * Each MPI process can at some point split into multiple threads to perform some inner-level task
  - Whether or not such two-level parallelisation is beneficial depends on the nature of the problem
    * Often most useful to simply use all the available CPUs to parallelise the outer-level tasks across as many processes as possible
- Important: Parallelisation always comes with some **overhead**
  - Spawning threads, sending messages, organising the inputs/outputs, etc.
  - Parallelisation is only useful if $t_{\text{task}} > t_{\text{overhead}}$
  - Also, parallelisation can make the code more complex
    * Need to consider the increased human time spent on coding/debugging when deciding whether or not to parallelise a given code

Examples of parallelisation in C++ using OpenMP:

- See the example programs here:

  github.com/anderkve/FYS3150/tree/master/code_examples/omp_parallelization

Speedup from parallelisation

- We can define the speedup from parallelisation as follows:

$$\text{speedup} = \frac{\text{[time spent with single thread/process]}}{\text{[time spent with } n \text{ threads/processes]}} = \frac{T_1}{T_n}$$

- Ideal case for n threads:

$$T_n = \frac{T_1}{n} \quad \Longrightarrow \quad \text{speedup} = \frac{T_1}{\frac{T_1}{n}} = n$$

  - In most cases we will not get ideal speedup
  - In rare cases we may actually get *better* than the "ideal speedup". This can happen because each core has a small, very fast memory (the *cache*): when the work is split across n cores, each core's $1/n$ share of the data may fit in its cache even though the full data set did not, so less time is spent waiting for data from the slower main memory

- *Important reminder:* A clever algorithm with less speedup from parallelisation can still be a much better choice than a simple algorithm with better parallelisation speedup
  - Example: Find the maximum point of a complicated, high-dimensional "black-box" function, where we do not know the gradients
    * *Approach 1:* Random sampling. Ideal parallelisation speedup ("embarrassingly parallel problem")
    * *Approach 2:* Some adaptive sampler, e.g. differential evolution. Needs communication and synchronisation, so less than ideal speedup.
    * For such a problem approach 2 is typically way more computationally efficient than approach 1.

An upper bound on speedup: Amdahl's law

- A task takes time T_1 on a single thread/process
- The fraction of this time that is spent inside perfectly parallelisable code: f
  - Non-parallelisable time fraction: $1 - f$
- For a single thread/process we can decompose T_1:

$$T_1 = (1 - f)T_1 + fT_1$$

- For n threads/processes, with ideal parallelisation:

$$T_n = (1 - f)T_1 + f\frac{T_1}{n}$$

- Speedup:

$$\text{speedup} = \frac{T_1}{T_n} = \frac{T_1}{(1-f)T_1 + f\frac{T_1}{n}} = \frac{1}{(1-f) + \frac{f}{n}}$$

  - This result for the speedup is known as **Amdahl's law**

- In the limit of many threads/processes:

$$\lim_{n\to\infty} \frac{T_1}{T_n} = \frac{1}{1-f}$$

  - Examples:
    * 99% of a task is parallelisable: largest possible speedup factor is 100
    * 80% of a task is parallelisable: largest possible speedup factor is 5

- Remember: f represents a fraction of the total runtime with a single process, not e.g. the fraction of the code length. So if e.g. our task requires us to increase the number of iterations of a parallelised loop, then this would correspond to increasing f, making parallelisation more beneficial

GPU parallelisation

- So far we have talked about parallelisation across CPU cores. But today much of the computing power in computational science comes from **graphics processing units (GPUs)**
  - The fastest supercomputers in the world get most of their computing power from GPUs
  - GPUs are also what made the current wave of machine learning possible
- CPU vs GPU, roughly:
  - A **CPU** has a small number of powerful cores (typically something like 4–64). Each core is built to run one sequence of instructions as fast as possible, and different cores can do completely different things
  - A **GPU** has thousands of simple cores, running at a lower frequency. The cores work in groups, where all cores in a group execute the *same sequence of instructions* at the same time, each on its own piece of data
  - A GPU is thus the extreme version of the trend from the "History" above: many simple cores rather than a few fast ones
- A GPU is efficient for **data-parallel** problems, where the same operation is applied to a large number of data elements:

  - Matrix and vector operations
  - Updating all the points on a grid (e.g. the PDE solver in project 5)
  - Evaluating the same function for a large number of inputs
- A GPU is *not* efficient when
  - different threads need to run different code for long stretches (the cores in a group must then take turns, with the rest idle). A short `if`/`else`, like an accept/reject step, costs little
  - the problem is small, or a single chain of steps where each step depends on the previous one
    * The Ising model in Project 4 is a single such chain, but we could still utilise a GPU: we can run thousands of *independent* chains at once, one per core. Each chain executes the same instructions (pick a spin, compute ΔE, draw a random number, accept/reject), just on their own lattice with their own random numbers.
- The main overhead: the GPU has its own memory, so data must be copied back and forth between CPU memory and GPU memory over a relatively slow connection
  - This transfer is often the bottleneck, so the goal is to keep the data on the GPU and do as much computation as possible per transfer
  - Amdahl's law applies here too: the parts of the code that stay on the CPU limit the total speedup

Random number generation

- We have seen that being able to generate samples x from some probability distribution $p(x)$ can be incredibly useful
- But our starting assumption has always been that we are able to generate samples (numbers) from a uniform (flat) distribution
- How can we do this, given that our computers are deterministic?
- This is the purpose of a **random number generator** (RNG)
- Old-school ways of generating random numbers: dice, coin flips, …
- Hardware RNGs can be used to generate "actually random" numbers, i.e. series of numbers that we can't predict

  - Generate numbers based on some unpredictable feature of the physical environment, e.g. thermal noise

- But we will focus on the concept of a **pseudorandom number generator** (PRNG): a *deterministic algorithm* that produces numbers that *are predetermined, but that still appear random (unpredictable)*
- PRNG algorithms are initialised by a starting number, called the **seed**
  - Two PRNG runs that start from the same seed will produce the same sequence of numbers (great for reproducibility of simulations, MCMC runs, etc.)
- Desired properties of pseudorandom number generators:
  1. Produce numbers that are distributed according to the standard uniform distribution, $U(0,1)$
  2. Negligible correlations between numbers
  - Knowing a previous number shouldn't help you in guessing the next number — unless you actually know the algorithm, of course
  3. The **period** before the sequence of numbers repeats should be as long as possible (more on that below)
  4. The algorithm should be computationally cheap
- A classic PRNG algorithm is the **Linear Congruential Generator** (LCG), which dates back to the 1950s
- The algorithm is as follows

$$\boxed{N_{i+1} = (aN_i + c) \bmod (m)}$$

  where we have the following parameters:

  - a (the *multiplier*): $0 < a < m$
  - c (the *increment*): $0 \leq c < m$
  - m (the *modulus*): $0 < m$
  - N_0 (the *seed*): $0 \leq N_0 < m$

> A side note on the modulo operator:
>
> - The modulo operator returns the integer remainder
> - *Examples*:
>
> $$13 \bmod (2) = 1$$
> $$17 \bmod (5) = 2$$
> $$8 \bmod (8) = 0$$
> $$16 \bmod (17) = 16$$

- To get a number x_i on $[0, 1)$ we simply do $x_i = N_i/m$
- How good the generator is depends crucially on the choice of parameters: a, c, m, and potentially N_0
  - There is a lot of research on the parameter choices for LCGs and other PRNG algorithms
- Some silly examples, for illustration:
  - *Example 1*:
    - Parameters: $m = 9$, $a = 2$, $c = 0$, $N_0 = 1$
    - Sequence: $1, 2, 4, 8, 7, 5, 1$ (repeats)
    - Period: 6
  - *Example 2*:
    - Parameters: $m = 9$, $a = 2$, $c = 0$, $N_0 = 3$
    - Sequence: $3, 6, 3$ (repeats)
    - Period: 2
  - *Example 3*:
    - Parameters: $m = 9$, $a = 4$, $c = 1$, $N_0 = 0$
    - Sequence: $0, 1, 5, 3, 4, 8, 6, 7, 2, 0$ (repeats)
    - Period: 9
- A more realistic example:
  - Parameters: $m = 2^{32} = 4\,294\,967\,296$, $a = 1\,664\,525$, $c = 1\,013\,904\,223$
  - These are the parameters of the "quick and dirty" generator `ranqd1` in the book *Numerical Recipes*

- It's recommended to look up the period of a PRNG that you plan to use for serious work
  - Some famous PRNGs have had surprisingly short periods, meaning the results are no longer random-looking when a sufficiently large number of samples are needed
- The period is not the only concern:
  - Question: What happens for the case $m = \text{some large number}$, $a = 1$, $c = 1$?
  - Answer: We simply get a "modulo counter"
    * Sequence: $N_0, N_0 + 1, N_0 + 2, \ldots$
  - The period may be long, but does this sequence look random/unpredictable? No!
  - Famous real-world example of hidden correlations: the *RANDU* algorithm (IBM, 1960s)
    * The samples looked uniformly distributed in 1D and 2D
    * But when used to generate points in a 3D space, all the points fall on just 15 parallel planes
    * This algorithm was used in many applications and research papers before this problem was discovered…
- There are collections of statistical randomness tests that are used to test PRNG algorithms
  - The common idea: compute some quantity from a long sequence of numbers, for which we know both the expected value *and* the expected statistical fluctuations if the numbers were truly random. If the observed value is too far off, the generator fails the test
  - Some simple examples:
    * *Frequency test*: Fill a histogram of the numbers with k bins. Each bin should get about n/k counts, with fluctuations of order $\sqrt{n/k}$. This tests the uniform distribution
    * *Serial test*: The same, but with a 2D histogram of consecutive pairs (x_i, x_{i+1}), a 3D histogram of triples (x_i, x_{i+1}, x_{i+2}), and so on. This is the test RANDU fails in 3D
      · (Try it: make a scatter plot of (x_i, x_{i+1}) for an LCG with a small modulus m. The points fall on a few straight lines)
    * *Autocorrelation test*: The correlation between x_i and x_{i+k} should be zero for every lag k. From the sequence we estimate it as

$$r_k = \frac{\sum_{i=1}^{n-k}(x_i - \bar{x})(x_{i+k} - \bar{x})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

where $\bar{x}$ is the average of all the x_i. For truly random numbers r_k fluctuates around zero with a size of order $1/\sqrt{n}$

  - Any PRNG will fail *some* test if you look hard enough (it is deterministic, after all), but good modern generators pass the standard test suites
  - Famous sets of tests: *Diehard tests* and *TestU01*

- Other examples of PRNG algorithms:
  - **Permuted Congruential Generator** (PCG)
    * PCG: LCG + a small permutation of the bits of the LCG output number
    * Still a short, fast method, can achieve better properties than LCG alone
    * The default generator in NumPy (`numpy.random.default_rng()`) is a PCG, while the older `numpy.random.rand()`-style functions use the Mersenne Twister (see below)
- The **Mersenne Twister** algorithm:
  - A very commonly used algorithm
  - Developed in 1997 by Makoto Matsumoto and Takuji Nishimura
  - Available from the <`random`> library in `C++11`
  - The standard version is known as `MT19937` and has a period of $2^{19937} - 1$, which is more than 10^{6001}
- Note that there are *lots* of PRNG algorithms out there!
- A common pitfall when using PRNGs in parallelised code
  - If the PRNGs on different threads/processes use the same seed, they will generate identical sets of samples!
  - In that case the result would be equivalent to many identical copies of a single run, instead of many independent runs (which is probably what you want)
  - A trick to give each thread a different seed while still starting from a single seed:
    * Start by choosing a base seed
    * Then use a PRNG with this base seed to generate a series of random seeds, one for each thread
    * Then each thread will have its own seed, but the results are still perfectly reproducible from just the base seed, as long as you run with the same number of threads

Numerical integration: high-dimensional integrals

- We will discuss two classes of methods for numerical integration:
  1. **Monte Carlo integration**, for high-dimensional integrals
  2. **Deterministic algorithms**, for low-dimensional integrals
- This chapter covers Monte Carlo integration. The deterministic algorithms are covered in the next chapter

Monte Carlo integration

- Consider a 1D integral

$$I = \int_a^b f(x)\,dx$$

- From calculus we have the definition of the **average function value** f_{avg} on $x \in [a, b]$:

$$f_{\text{avg}} = \frac{1}{b-a}\int_a^b f(x)\,dx \quad \Rightarrow \quad I = (b-a)\,f_{\text{avg}}$$

- Now consider the **expectation value** for f, given that $x \sim U(a, b)$, that is

$$p(x) = \begin{cases} \frac{1}{b-a} & x \in [a, b] \\ 0 & \text{else} \end{cases}$$

- From probability theory we have

$$\mathrm{E}[f] = \langle f \rangle = \mu_f = \int_a^b f(x)\,p(x)\,dx = \frac{1}{b-a}\int_a^b f(x)\,dx = f_{\text{avg}}$$

- So the average f value is the expected $f(x)$ value when x follows a uniform distribution:

$$\boxed{I = (b-a)\,\langle f \rangle}$$

  - This is the basis for Monte Carlo integration (here in 1D)

- We can estimate $\langle f \rangle$ using the **sample mean** $\bar{f}$:
  - Draw N samples $x_i \sim U(a, b)$
  - Compute

$$\langle f \rangle \approx \bar{f} = \frac{1}{N} \sum_{i=1}^{N} f(x_i)$$

- Our estimate of the integral is then

$$I \approx \hat{I} = (b-a)\, \bar{f} = (b-a)\, \frac{1}{N} \sum_{i=1}^{N} f(x_i)$$

Error estimation

- We want the variance in $\hat{I}$: $\sigma_{\hat{I}}^2$
  - Remember that $\hat{I}$ is a random variable, since $\bar{f}$ is
- Start from the **sample variance** for the f samples:

$$s_f^2 = \frac{1}{N-1} \sum_{i=1}^{N} \left(f_i - \bar{f} \right)^2$$

  - This is an unbiased estimator for the true variance $\sigma_f^2 = \mathrm{Var}(f)$, that is $\mathrm{E}\left[s_f^2 \right] = \sigma_f^2$
- The **variance of the mean**, $\sigma_{\bar{f}}^2$, is then

$$\boxed{\sigma_{\bar{f}}^2 = \frac{\sigma_f^2}{N} \approx \frac{s_f^2}{N}}$$

  - This follows from the N samples being *independent*, see the side note below

> A side note on where the variance of the mean comes from:
>
> - The general expression for the variance of a sum of random variables $Y_1, Y_2, \ldots, Y_N$ is
>
> $$\mathrm{Var}\left(\sum_{i=1}^{N} Y_i \right) = \sum_{i=1}^{N} \mathrm{Var}(Y_i) + \sum_{i \neq j} \mathrm{Cov}(Y_i, Y_j)$$
>
> where $\mathrm{Cov}(Y_i, Y_j) = \mathrm{E}[(Y_i - \mathrm{E}[Y_i])(Y_j - \mathrm{E}[Y_j])]$ is the *covariance* of Y_i and Y_j

- For *independent* variables all the covariances vanish, so the variance of the sum is simply the sum of the variances
- The $f_i = f(x_i)$ are independent random variables, since the x_i are independent samples, and they all have the same true variance σ_f^2. Using also $\mathrm{Var}(cY) = c^2\,\mathrm{Var}(Y)$ (see the next side note), we get

$$\mathrm{Var}\left(\bar{f}\right) = \mathrm{Var}\left(\frac{1}{N}\sum_{i=1}^{N} f_i\right) = \frac{1}{N^2}\sum_{i=1}^{N}\mathrm{Var}(f_i) = \frac{1}{N^2}\,N\sigma_f^2 = \frac{\sigma_f^2}{N}$$

- This is the exact result. In practice we don't know the true variance σ_f^2, so we replace it with our estimate for it, the sample variance s_f^2, giving $\sigma_{\bar{f}}^2 \approx s_f^2/N$

- Since $\hat{I} = (b-a)\,\bar{f}$, we have

$$\sigma_{\hat{I}}^2 = (b-a)^2\,\sigma_{\bar{f}}^2$$

$$\sigma_{\hat{I}}^2 = (b-a)^2\,\frac{\sigma_f^2}{N}$$

$$\boxed{\sigma_{\hat{I}} = (b-a)\,\frac{\sigma_f}{\sqrt{N}} \approx (b-a)\,\frac{s_f}{\sqrt{N}}}$$

A side note on where the factor $(b-a)^2$ comes from:

- Let $Y(x) = c\,x$, with c a constant. Then

$$\begin{aligned}\mathrm{Var}(Y) &= \mathrm{E}\left[Y^2\right] - \mathrm{E}[Y]^2 \\ &= \mathrm{E}\left[c^2x^2\right] - \mathrm{E}[c\,x]^2 \\ &= c^2\left(\mathrm{E}\left[x^2\right] - \mathrm{E}[x]^2\right)\end{aligned}$$

$$\mathrm{Var}(Y) = c^2\,\mathrm{Var}(x)$$

- Our final result is

$$\boxed{I \approx \hat{I} \pm \sigma_{\hat{I}} = \frac{(b-a)}{N}\sum_{i=1}^{N} f_i \;\pm\; (b-a)\,\frac{s_f}{\sqrt{N}}}$$

Going to d dimensions

- Now go to d dimensions:

$$I = \int_{a_1}^{b_1} dx_1 \int_{a_2}^{b_2} dx_2 \cdots \int_{a_d}^{b_d} dx_d \; f(x_1, x_2, \ldots, x_d) = \int_V f(\vec{x})\, d\vec{x}$$

- *Nothing* changes in the derivation above, except $(b-a) \to V$
- **Basic Monte Carlo integration**:
  - Sample N points $\vec{x}_i$ uniformly on V
  - Evaluate the function, $f(\vec{x}_i) \equiv f_i$

$$\boxed{I \approx \frac{V}{N} \sum_{i=1}^{N} f_i}$$

  - With uncertainty estimate:

$$s_f^2 = \frac{1}{N-1} \sum_{i=1}^{N} \left(f_i - \bar{f}\right)^2, \qquad s_f = \sqrt{s_f^2}$$

$$\boxed{I \approx \frac{V}{N} \sum_{i=1}^{N} f_i \; \pm \; \frac{V s_f}{\sqrt{N}}}$$

- Observations:
  1. The error scales as $\mathcal{O}\left(\frac{1}{\sqrt{N}}\right) = \mathcal{O}\left(\frac{1}{N^{1/2}}\right)$ with the number of samples N, *independent of the number of dimensions d!*
     - In all dimensions, doubling the number of samples will reduce the error by a factor $\frac{1}{\sqrt{2}}$
  2. If the variance in the f values is small (small σ_f), the error on I is small
     - Makes sense: small $\sigma_f \leftrightarrow$ slowly varying function $f(\vec{x}) \leftrightarrow$ need few samples to estimate $\langle f \rangle$ fairly well
     - Extreme limit: A flat function ($\sigma_f = 0$) $\to$ need only 1 sample to integrate

When to use Monte Carlo integration?

- Example: compare to the **trapezoidal rule** (linear interpolation between grid points)
  - Step size h in each dimension
  - (Global) error: $\mathcal{O}\left(h^2\right)$

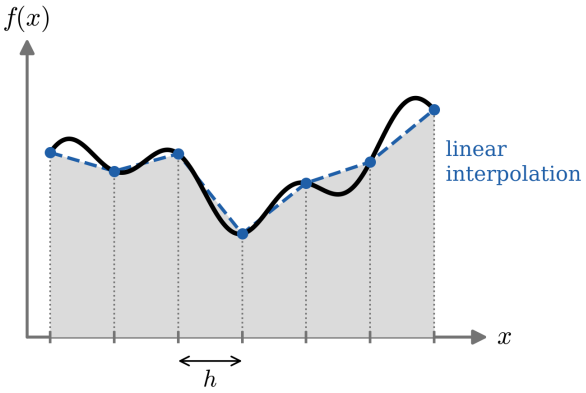


Figure 29: The trapezoidal rule: the function $f(x)$ is approximated by linear interpolation (dashed) between grid points separated by the step size h, and the integral is approximated by the total area of the resulting trapezoids (grey).

- Number of points:

$$N = N_{x_1} N_{x_2} N_{x_3} \cdots N_{x_d} \sim \left(\frac{1}{h}\right)^d = \frac{1}{h^d} \quad \Rightarrow \quad h \sim \frac{1}{N^{1/d}}$$

- Error scaling with number of points:

$$\mathcal{O}\left(h^2\right) \sim \mathcal{O}\left(\frac{1}{N^{2/d}}\right)$$

- Monte Carlo error scaling: $\mathcal{O}\left(\frac{1}{N^{1/2}}\right)$
  - We want the error to decrease quickly as a function of N, so we want a large denominator

- So Monte Carlo integration is *preferable* over the trapezoidal rule when

$$N^{1/2} > N^{2/d}$$

$$\frac{1}{2} > \frac{2}{d}$$

$$d > 2 \cdot 2$$

$$d > 4$$

  - The second factor 2 in $d > 2 \cdot 2$ comes from the $\mathcal{O}\left(h^2\right)$ error for the trapezoidal rule

- In general, for an $\mathcal{O}\left(h^k\right)$ method, Monte Carlo integration is preferred when

$$\boxed{d > 2k}$$

  - E.g. $d > 8$ for Simpson's rule ($k = 4$)

Importance sampling

- Consider a function $f(x)$ that is strongly peaked in a small part of the integration interval $[a, b]$

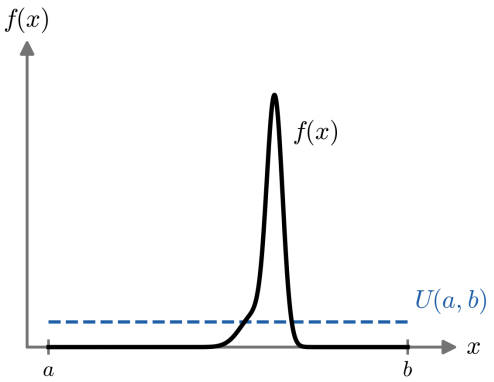


Figure 30: A function $f(x)$ that is strongly peaked in a small part of the interval $[a, b]$, compared to the uniform pdf $U(a, b)$ (dashed) that basic Monte Carlo integration draws its samples from. Most samples will land where $f(x) \approx 0$.

- "Vanilla" Monte Carlo integration will be inefficient in this case

  - Most samples will not contribute to the estimate of I

- Can we use a different pdf than the uniform one? Let's sample x from some pdf $q(x)$ of our choice:

$$I = \int_a^b f(x)\,dx = \int_a^b \frac{f(x)}{q(x)} \underbrace{q(x)}_{\text{pdf}}\,dx \equiv \int_a^b w(x)\,q(x)\,dx = \langle w \rangle_{q(x)}$$

- So

$$I \approx \frac{1}{N}\sum_i w_i \qquad (\text{if } x_i \sim q(x))$$

- Since $w(x) = \frac{f(x)}{q(x)}$ is a flatter function than $f(x)$, the sample variance is smaller for the w_i than for the f_i

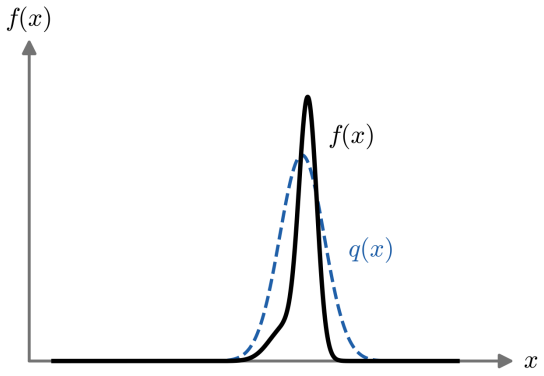


Figure 31: A sampling pdf $q(x)$ (dashed) that resembles the function $f(x)$. The ratio $w(x) = f(x)/q(x)$ is then a flatter function than $f(x)$.

- So

$$\boxed{I \approx \frac{1}{N}\sum_{i=1}^{N} w_i \;\pm\; \frac{s_w}{\sqrt{N}}}$$

with $w(x) = \frac{f(x)}{q(x)}$ will be a better estimate of I if the sampling distribution $q(x)$ resembles $f(x)$, i.e. such that $w(x)$ is as flat as possible

  - This result holds unchanged in d dimensions, with $x \to \vec{x}$ and $q(\vec{x})$ a pdf on the integration region V. Since $q(\vec{x})$ is normalised, there is no volume factor V in the estimate

    * For the special case of the uniform pdf $q(\vec{x}) = 1/V$ we get $w_i = V f_i$, and we recover the basic Monte Carlo estimate $I \approx \frac{V}{N} \sum_i f_i$

- *Connection to rejection sampling:* This is the "keep every sample, but weight it" method that was mentioned in the terminology note in the section on rejection sampling with a non-uniform proposal distribution
  - In both methods we draw samples from a convenient pdf $q(x)$ that resembles the function we are actually interested in, rather than from the uniform distribution
  - In rejection sampling we correct for the mismatch between the proposal pdf $q(x)$ and the target pdf $p(x)$ by *throwing away* some of the samples: a sample x' is accepted with probability $p(x')/(Mq(x'))$, i.e. the ratio of the two functions
  - In importance sampling we keep *all* the samples, and instead correct for the mismatch between $q(x)$ and $f(x)$ by *weighting* each sample with the ratio of the two functions, $w_i = f(x_i)/q(x_i)$. This is why the w_i are called **importance weights**
- It can be difficult to find a suitable pdf $q(x)$!

Numerical integration: low-dimensional integrals

- A common term for numerical integration is **quadrature**
  - Historically, "quadrature" meant computing the area of a region by constructing a square with the same area
- There are *many* methods for numerical integration. We will look at two families:
  - **Newton-Cotes quadrature**: constant step size h
    * Examples: the *trapezoidal rule* and *Simpson's rule*
    * These are well-known methods, so we will focus on implementation approaches
  - **Gaussian quadrature**: not constant step size
- General considerations for any method:
  - How many evaluations of $f(x)$ do we need?
  - How should we distribute the evaluations along the x axis? (step size, …)
  - How do we interpolate $f(x)$ at the top of the rectangle-like areas?
  - In the end, all these methods are just "fancy Riemann sums"

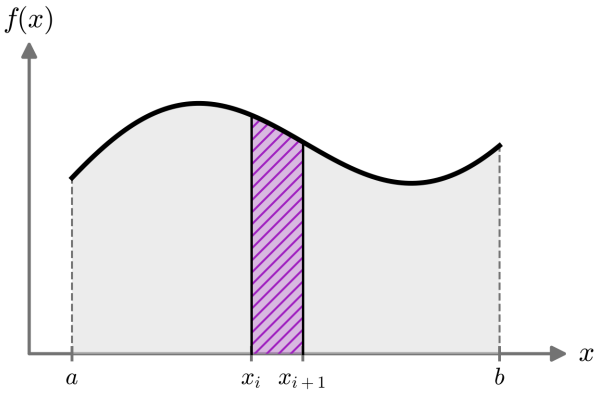


Figure 32: The integral $I = \int_a^b f(x)\,dx$ is the area below $f(x)$. The numerical methods split this area into rectangle-like strips, like the one between x_i and x_{i+1}, and differ in how they treat the top of each strip.

Background: Lagrange's interpolation formula

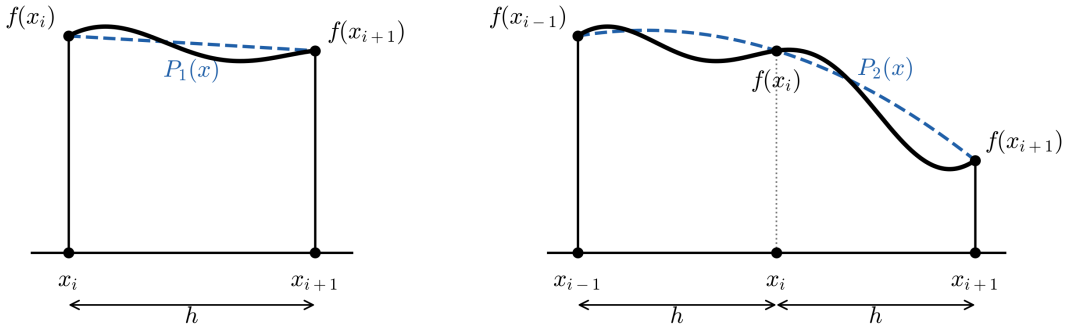
- The N-th order polynomial defined by $N+1$ known points (x_i, y_i) is given by **Lagrange's interpolation formula**:

$$\boxed{P_N(x) = \sum_{i=0}^{N} \left(\prod_{k \neq i} \frac{x - x_k}{x_i - x_k} \right) y_i}$$

- Example: The second-order polynomial going through (x_0, y_0), (x_1, y_1) and (x_2, y_2) is

$$P_2(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}\,y_0 + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}\,y_1 + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}\,y_2$$

- We can use such polynomials to interpolate the tops of the integration rectangles



Trapezoidal rule
(a)

Simpson's rule
(b)

Figure 33: Interpolating $f(x)$ at the top of the integration strips. (a) A first-order polynomial $P_1(x)$ through two points gives the trapezoidal rule. (b) A second-order polynomial $P_2(x)$ through three points gives Simpson's rule.

Trapezoidal rule

- Approximate $f(x)$ using a first-order polynomial on each interval $[x_i, x_{i+1}]$, where $x_{i+1} = x_i + h$:

$$I = \int_{x_i}^{x_{i+1}} f(x)\,dx = \int_{x_i}^{x_{i+1}} \left[P_1(x) + \mathcal{O}\left(h^2 \frac{d^2 f}{dx^2}\right)\right] dx = h\left[\tfrac{1}{2}f(x_i) + \tfrac{1}{2}f(x_{i+1})\right] + \mathcal{O}\left(h^3 \frac{d^2 f}{dx^2}\right)$$

  - The $\mathcal{O}\left(h^3\right)$ term is the **local error**, i.e. the error from a single step. We will get a **global error** of $\mathcal{O}\left(h^2\right)$

- Extended integration region $a = x_0$, $b = x_{n-1}$, using n points:

$$\boxed{I = \int_{x_0}^{x_{n-1}} f(x)\,dx = h\left[\tfrac{1}{2}f(x_0) + f(x_1) + \ldots + f(x_{n-2}) + \tfrac{1}{2}f(x_{n-1})\right] + \mathcal{O}\left(h^2\right)}$$

- Denoting the trapezoidal estimate above by T_h, the actual error term is

$$I - T_h = -\frac{(b-a)}{12} h^2 f''(\xi) \quad \text{for some } \xi \in [a, b]$$

  - We don't know ξ, but if we know that $|f''(x)| \leq M$ for some constant M, we have a *bound* for the error:

$$|I - T_h| \leq \frac{(b-a)}{12} h^2 M$$

- The error scales as $\mathcal{O}\left(h^2\right) = \mathcal{O}\left(1/n_{\mathrm{1D}}^2\right)$ in one dimension, and as $\mathcal{O}\left(h^2\right) = \mathcal{O}\left(1/n_{d\mathrm{D}}^{2/d}\right)$ in d dimensions (with $h_x = h_y = h_z = \ldots = h$)

  - Total number of points required to have a step size h in each dimension, assuming the same integration range $[a, b]$ in every dimension:

$$\text{1D:} \quad n_{\mathrm{1D}} = \frac{b-a}{h} + 1 \simeq \frac{b-a}{h}$$

$$\text{2D:} \quad n_{\mathrm{2D}} = n_{\mathrm{1D}}^2 \simeq \frac{(b-a)^2}{h^2}$$

$$d\text{D:} \quad n_{d\mathrm{D}} = n_{\mathrm{1D}}^d \simeq \frac{(b-a)^d}{h^d} \quad \Rightarrow \quad h \simeq \frac{b-a}{n_{d\mathrm{D}}^{1/d}}$$

Efficient implementation

- Consider $I = \int_a^b f(x)\,dx$, and let us apply the trapezoidal rule for a sequence of step sizes, each half of the previous one
- Step size $h_1 = b - a$, two points $x_0 = a$ and $x_1 = b$:

$$T_1 = h_1 \left[\tfrac{1}{2} f(x_0) + \tfrac{1}{2} f(x_1)\right]$$

- Step size $h_2 = \frac{1}{2} h_1$, three points x_0, x_1, x_2:

$$\begin{aligned} T_2 &= h_2 \left[\tfrac{1}{2} f(x_0) + f(x_1) + \tfrac{1}{2} f(x_2)\right] \\ &= h_2 \left[\tfrac{1}{2} f(x_0) + \tfrac{1}{2} f(x_2)\right] + h_2 f(x_1) \\ &= \tfrac{1}{2} h_1 \left[\tfrac{1}{2} f(x_0) + \tfrac{1}{2} f(x_2)\right] + h_2 f(x_1) \\ &= \tfrac{1}{2} T_1 + h_2 f(x_1) \end{aligned}$$

  - Only one new function evaluation, at the new point x_1 (3 points in total)

- Step size $h_3 = \frac{1}{2} h_2$, five points $x_0, \ldots, x_4$:

$$T_3 = \tfrac{1}{2} T_2 + h_3 \left[f(x_1) + f(x_3)\right]$$

- Two new function evaluations (5 points in total)

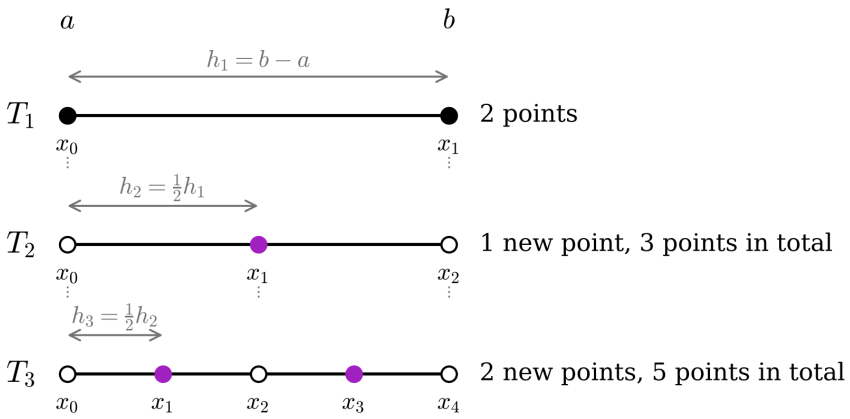


Figure 34: The trapezoidal rule with successive halving of the step size. The points from the previous step are reused (open circles), and only the new midpoints (purple) require new function evaluations.

- In general:

$$\boxed{T_{j+1} = \tfrac{1}{2}T_j + h\left[f(x_1) + f(x_3) + \ldots + f(x_{n-2})\right]}$$

  where h is the step size for T_{j+1}, $n = 2^j + 1$ is the total number of points, and the x_i with odd i are the new points, i.e. the midpoints of the intervals used for T_j

- Algorithm:

  1. Set the desired relative accuracy ϵ
  2. Compute T_1
  3. Compute T_2
  4. While $\left|\frac{T_{j+1}-T_j}{T_j}\right| > \epsilon$: compute the next T_{j+1} and compare with T_j

Adaptive division of the integration range

- We can also use an **adaptive division of the integration range**, with more points where the function varies quickly

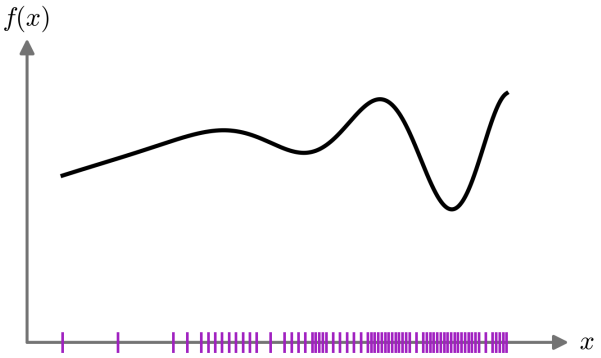


Figure 35: Adaptive division of the integration range: the points (purple) are placed densely where $f(x)$ varies quickly and sparsely where it is smooth.

- Example: Recursive comparison between T_2 and T_1 on each interval, i.e. check $\frac{T_2 - T_1}{T_1}$
  - T_1 uses the two end points of the interval, T_2 also uses the midpoint
  - If the two estimates agree to the desired accuracy: stop, and keep the result for this interval
  - If not: split the interval in two halves, and repeat the check on each half

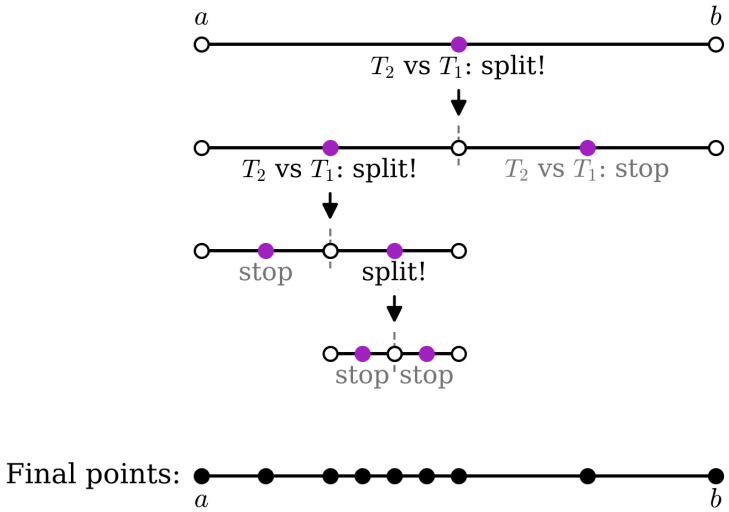


Figure 36: Recursive comparison of T_2 and T_1. An interval is split in two if the two estimates disagree, and the two halves are then checked in the same way. The final set of points is dense where splitting was needed and sparse elsewhere.

- This can be implemented as a *function calling itself* (recursion). See our code example `python_numerical_integration_low_dim/trapezoidal_recursive.py` for a simple demo.

Simpson's rule

- Approximate $f(x)$ with a **second-order polynomial** $P_2(x)$ in each interval of **three points**:

$$\int_{x_i}^{x_{i+2}} f(x)\,dx = h\left[\tfrac{1}{3}f(x_i) + \tfrac{4}{3}f(x_{i+1}) + \tfrac{1}{3}f(x_{i+2})\right] + \mathcal{O}\left(h^5\frac{d^4f}{dx^4}\right)$$

  - The weights come from integrating Lagrange's interpolation formula for $P_2(x)$ across the two steps
  - Surprise! We might naively have expected an $\mathcal{O}\left(h^4\right)$ local error: h^3 from the leading Taylor expansion term that $P_2(x)$ ignores, times h from the integration. But this term vanishes, and the local error is $\mathcal{O}\left(h^5\right)$. The reason is the symmetry of the three-point formula about x_{i+1}: the ignored f''' term is antisymmetric about the midpoint and integrates to zero, so the error expansion contains only odd powers of h

- Simpson's rule can be seen as a weighted sum of two applications of the trapezoidal rule:

$$\boxed{S = \tfrac{4}{3}T_{j+1} - \tfrac{1}{3}T_j}$$

  - The error of the trapezoidal rule contains only *even* powers of h: $T_h = I + c_2h^2 + c_4h^4 + \ldots$, where $c_2, c_4, \ldots$ depend on f but not on h
  - Going from step size $2h$ (T_j) to step size h (T_{j+1}) reduces the h^2 term by a factor 4 and the h^4 term by a factor 16, so

$$\tfrac{4}{3}T_{j+1} - \tfrac{1}{3}T_j = I + \tfrac{1}{3}(4-4)\,c_2h^2 + \tfrac{1}{3}(4-16)\,c_4h^4 + \ldots = I - 4c_4h^4 + \ldots$$

  - The combination is chosen to cancel the $\mathcal{O}\left(h^2\right)$ term. With no h^3 term to take its place, the global error of S is $\mathcal{O}\left(h^4\right)$, i.e. the local error is $\mathcal{O}\left(h^5\right)$

- Extended integration region using n points (n is odd):

$$\boxed{\begin{aligned} I = \int_{x_0}^{x_{n-1}} f(x)\,dx = h\Big[&\tfrac{1}{3}f(x_0) + \tfrac{4}{3}f(x_1) + \tfrac{2}{3}f(x_2) + \tfrac{4}{3}f(x_3) + \ldots \\ &+ \tfrac{2}{3}f(x_{n-3}) + \tfrac{4}{3}f(x_{n-2}) + \tfrac{1}{3}f(x_{n-1})\Big] + \mathcal{O}\left(h^4\right) \end{aligned}}$$

  where $\mathcal{O}\left(h^4\right)$ is the global error

  - We just patch together three-point regions side by side: an even number of steps, an odd number of points

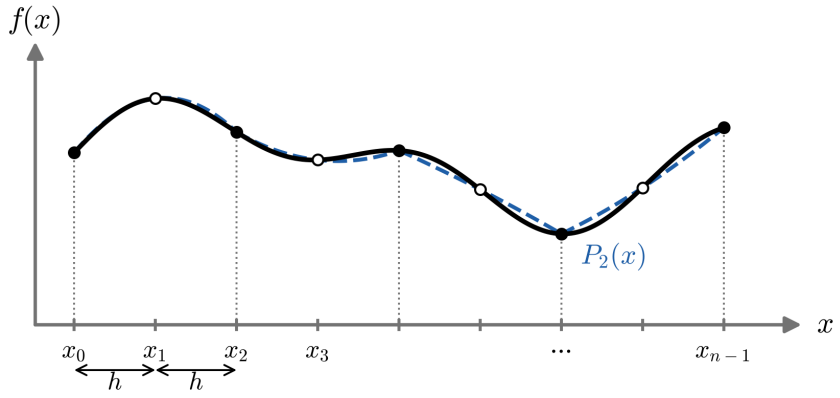


Figure 37: Composite Simpson's rule: three-point regions, each with its own second-order polynomial $P_2(x)$ (dashed), are patched together side by side. This requires an even number of steps, i.e. an odd number of points.

- A useful approach:
  1. Use a trapezoidal method that computes both T_j and T_{j+1}
  2. Improve the final estimate via $S = \frac{4}{3}T_{j+1} - \frac{1}{3}T_j$
- This can be combined with the smart trapezoidal implementations above, e.g. the recursive one
- (The connection between Simpson's rule and the RK4 method for initial value problems was discussed in the section *RK4 and Simpson's rule for integration*)

Higher-degree polynomials and Runge's phenomenon

- We can use higher-degree polynomials, but we don't necessarily gain much. An example of the problem:
- **Runge's phenomenon**: high-degree polynomial interpolation with equally spaced points leads to "ringing", i.e. the polynomials oscillate

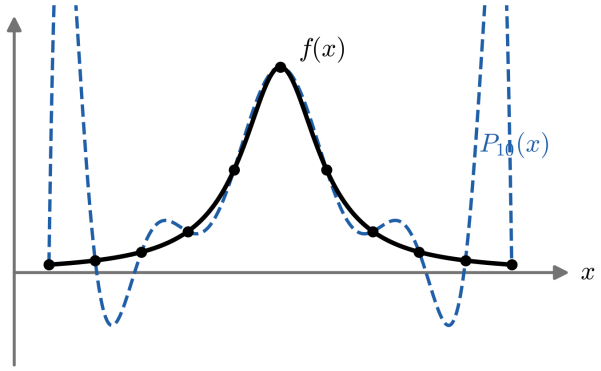


Figure 38: Runge's phenomenon: a degree-10 polynomial $P_{10}(x)$ (dashed) interpolating $f(x) = 1/(1 + 25x^2)$ at 11 equally spaced points oscillates strongly near the ends of the interval.

- We end up having to use a small h to combat this, so we don't gain much from going to higher order

Gaussian quadrature

- We will not discuss Gaussian quadrature in detail, but it is important to know about this class of methods
- All the methods above can be written in the form

$$I = \int_a^b f(x)\,dx \approx \sum_{i=0}^{n-1} \omega_i f(x_i)$$

  with **weights** ω_i and **grid points** x_i

  - Example, trapezoidal rule: $\omega_i = \frac{h}{2}, h, h, \ldots, h, \frac{h}{2}$
  - Example, Simpson's rule: $\omega_i = \frac{h}{3}, \frac{4h}{3}, \frac{2h}{3}, \ldots, \frac{4h}{3}, \frac{h}{3}$
- Now consider a *single* interval with n points. In the Newton-Cotes methods the point positions x_i are fixed (equally spaced), so we only have the n weights ω_i to adjust
  - With n free parameters we can make the rule exact for the n functions $1, x, x^2, \ldots, x^{n-1}$, i.e. for any polynomial $P_{n-1}(x)$
  - This is how the trapezoidal rule ($n = 2$) and Simpson's rule ($n = 3$) were constructed above. The composite versions are then exact for $P_1(x)$ and $P_2(x)$ on each subinterval, no matter how many subintervals we use

- **Gaussian quadrature**: Allow varying both the weights *and* the point positions. This gives $2n$ parameters to adjust
  - Result: With n points we can integrate any polynomial $P_{2n-1}(x)$ exactly
  - Works best if $f(x)$ is well approximated by a polynomial $P_{2n-1}(x)$ (or lower order)
- The point positions x_i are found as the roots of **orthogonal polynomials**
- *Example: the two-point rule on* $[-1, 1]$
  - With $n = 2$ we have four parameters, $\omega_0, \omega_1, x_0, x_1$. So we can demand that the rule

$$\int_{-1}^{1} f(x)\,dx \approx \omega_0 f(x_0) + \omega_1 f(x_1)$$

    is exact for the four functions $f = 1, x, x^2, x^3$, and hence for any polynomial up to degree $2n - 1 = 3$
  - This gives four equations for the four unknowns:

$$f = 1: \quad \omega_0 + \omega_1 = \int_{-1}^{1} dx = 2$$

$$f = x: \quad \omega_0 x_0 + \omega_1 x_1 = \int_{-1}^{1} x\,dx = 0$$

$$f = x^2: \quad \omega_0 x_0^2 + \omega_1 x_1^2 = \int_{-1}^{1} x^2\,dx = \frac{2}{3}$$

$$f = x^3: \quad \omega_0 x_0^3 + \omega_1 x_1^3 = \int_{-1}^{1} x^3\,dx = 0$$

  - Since both the integral and the quadrature sum are linear in f, a rule that is exact for $1, x, x^2$ and x^3 is automatically exact for every linear combination of them, that is, for every polynomial up to degree 3. The weights and points are fixed once and for all
  - We can simplify this using symmetry. The interval $[-1, 1]$ is symmetric about $x = 0$, so nothing distinguishes left from right. It is therefore natural to look for a rule with the same symmetry: equal weights, $\omega_0 = \omega_1 \equiv \omega$, and mirrored points, $x_0 = -t$ and $x_1 = t$. (It turns out that this is in fact the only solution)

- With this choice the equations for the odd functions x and x^3 are satisfied automatically, since $\omega(-t) + \omega t = 0$ and $\omega(-t)^3 + \omega t^3 = 0$. The two remaining equations determine ω and t:

$$f = 1: \quad 2\omega = 2 \quad \Rightarrow \quad \omega = 1$$

$$f = x^2: \quad 2\omega t^2 = \frac{2}{3} \quad \Rightarrow \quad t = \frac{1}{\sqrt{3}}$$

- So the two-point Gauss rule is

$$\int_{-1}^{1} f(x)\,dx \approx f\left(-\tfrac{1}{\sqrt{3}}\right) + f\left(\tfrac{1}{\sqrt{3}}\right)$$

  which is exact for every polynomial up to degree 3, using only two function evaluations

- Test on $\int_{-1}^{1} e^x\,dx = e - e^{-1} = 2.3504$: the two-point Gauss rule gives 2.3427 (error 0.3%). The trapezoidal rule with the same two function evaluations gives 3.0862 (error 31%), and Simpson's rule with three evaluations gives 2.3621 (error 0.5%)
- As stated above, the points are roots of an orthogonal polynomial: $\pm 1/\sqrt{3}$ are the roots of the Legendre polynomial of degree 2, $\frac{1}{2}(3x^2 - 1)$
- For a general interval $[a, b]$, substitute $x = \frac{a+b}{2} + \frac{b-a}{2}\,t$ with $t \in [-1, 1]$, which also multiplies the weights by $\frac{b-a}{2}$

Topics in project 5

> For the 2026 edition of this course there is no Project 5. We will still cover most of the same material, but the lecture notes concerning Project 5 will be updated.

- Physics case: the double-slit experiment in quantum physics
  - Technically: solve the Schrödinger equation in 2+1 dimensions for a given time-independent potential
- Partial differential equations (PDEs)
  - Classification
  - Some schemes for solving PDEs:
    * Forward difference scheme
    * Backward difference scheme
    * Crank-Nicolson
- Work with complex numbers in C++

Physics of project 5: the double-slit experiment

- The single-particle double-slit experiment is one of the most famous experiments in the history of physics
- Goal of this project: solve the relevant differential equation, the Schrödinger equation, with a potential that models a double-slit experiment configuration
- For an introduction to the Schrödinger equation and the notation we will use, see the Project 5 page: anderkve.github.io/FYS3150/book/projects/project5

Introduction to partial differential equations

- A partial differential equation (PDE): a differential equation in more than one variable
  - In physics these variables often represent time and space coordinates, but can be anything
- Solving PDEs numerically is a *huge* topic

  - Many different PDEs are relevant across different fields of science, technology, medicine, economics, …
  - Many specialised approaches have been developed tailored to solve specific PDEs

- Some examples of PDEs in physics:
  - The wave equation in (2+1) dimensions:

$$\frac{\partial^2 u}{\partial t^2} = v^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

  - The diffusion equation in (2+1) dimensions:

$$\frac{\partial u}{\partial t} = D \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

  - Poisson's equation in 2 dimensions:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x, y)$$

  - The single-particle Schrödinger equation in 2+1 dimensions:

$$i\frac{\partial^2 u}{\partial t^2} = -\frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2} + v(x, y, t)u$$

  - We note that all these equations are *second-order* and *linear* PDEs

- To find the solution $u(x, t)$ of some PDE (here assuming 1+1 dimensions) we will in general need to worry about *both* initial and boundary conditions, e.g.

$$\begin{aligned} u(x, t = 0) &= \ ? \\ \left.\frac{\partial u}{\partial x}\right|_{(x,t=0)} &= \ ? \\ u(x = x_{\min}, t) &= \ ? \\ u(x = x_{\max}, t) &= \ ? \end{aligned}$$

Classification of second-order, linear PDEs

- Here we look at PDEs of two variables

  - The classification scheme can be extended to more variables

- The classification scheme focuses on *which second derivatives appear in the equation*, and how they "interact" with each other
  - This determines how information "propagates" in our solution, i.e. which points in the solution can affect each other
  - As the information flow is different in different classes of PDEs, we typically use different numerical schemes to solve different types of PDEs
- Start from this general form of the PDE:

$$A\frac{\partial^2 u}{\partial x^2} + B\frac{\partial^2 u}{\partial x \partial y} + C\frac{\partial^2 u}{\partial y^2} + \left[\text{terms with } u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial y}\right] = g(x, y)$$

  - Note that we could e.g. have used $u(x, t)$ instead of $u(x, y)$ — it's just two variables of some type
- We define the *discriminant* Q:

$$Q \equiv B^2 - 4AC$$

- Classification:
  - *Hyperbolic* PDEs: $Q > 0$
  - *Parabolic* PDEs: $Q = 0$
  - *Elliptic* PDEs: $Q < 0$
- The terminology is borrowed from the classification of conic sections, which in Cartesian coordinates have graphs described by the general quadratic equation

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$$

  and can be classified according to the value of the discriminant $Q = B^2 - 4AC$
- When discussing some different numerical schemes, we will focus on examples from parabolic PDEs (the diffusion equation)

Example of a hyperbolic PDE: the wave equation

- Wave equation in 1+1 dimensions

$$v^2 \frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial t^2} = 0$$

- Comparing to the general equation above, we get $A = v^2$, $B = 0$ and $C = -1$
- Discriminant

$$Q = B^2 - 4AC = 4v^2 > 0$$

  - So the wave equation is a hyperbolic PDE

- Let's see how information "propagates" through the solution of a hyperbolic PDE
  - Consider a specific point (x', t') in the domain of the solution $u(x, t)$
    * The *domain of dependence*: all (x, t) points where a change in $u(x, t)$ can affect the solution value $u(x', t')$ at the point (x', t')
    * The *domain of influence*: all (x, t) points whose solution value $u(x, t)$ can be affected by a change in $u(x', t')$

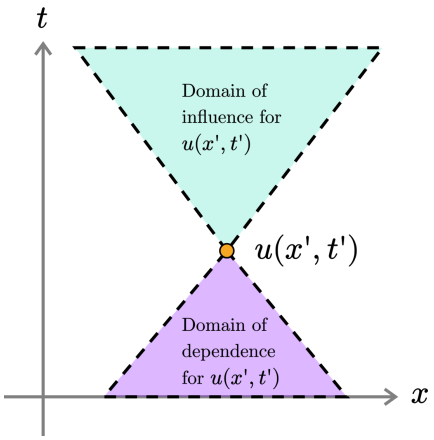


Figure 39: Domain of dependence and domain of influence for the solution at $u(x', t')$, given a hyperbolic PDE.

- This diagram makes intuitive sense when we remember that the parameter v in the wave equation represents the wave propagation speed, which limits which points in the domain can influence each other

- To those who have worked with the theory of special relativity: This diagram should remind you of a spacetime diagram with a past and future light cone
  - (In quantum field theory, which combines quantum mechanics with special relativity, the equations of motion are all wave equations)
- The solution point $u(x', t')$ depends on initial conditions, e.g.

$$u(x, t = 0) = f(x)$$
$$\left.\frac{\partial u}{\partial t}\right|_{t=0} = g(x)$$

Example of a parabolic PDE: the diffusion equation

- The diffusion equation in 1+1 dimensions

$$D\frac{\partial^2 u}{\partial x^2} - \frac{\partial u}{\partial t} = 0$$

- Comparing to the general equation above, we get $A = D$, $B = 0$ and $C = 0$
- Discriminant

$$Q = B^2 - 4AC = 0$$

  - So the diffusion equation is a parabolic PDE
- Domain of dependence and domain of influence for the solution point $u(x', t')$:

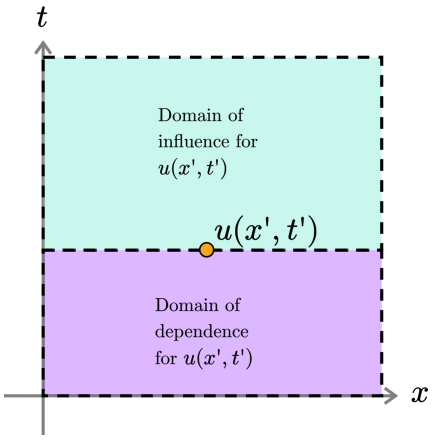


Figure 40: Domain of dependence and domain of influence for the solution at $u(x', t')$, given a parabolic PDE.

- The solution point $u(x', t')$ depends on initial and boundary conditions, e.g.

$$u(x, t = 0) = f(x)$$
$$u(x = x_{\min}, t) = g(t)$$
$$u(x = x_{\max}, t) = h(t)$$

Example of an elliptic PDE: the Poisson equation

- The Poisson equation in two space dimensions

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x, y)$$

- Comparing to the general equation above, we get $A = 1$, $B = 0$ and $C = 1$
- Discriminant

$$Q = B^2 - 4AC = -4 < 0$$

  - So the Poisson equation is an elliptic PDE

- Domain of dependence and domain of influence for the solution point $u(x', y')$:

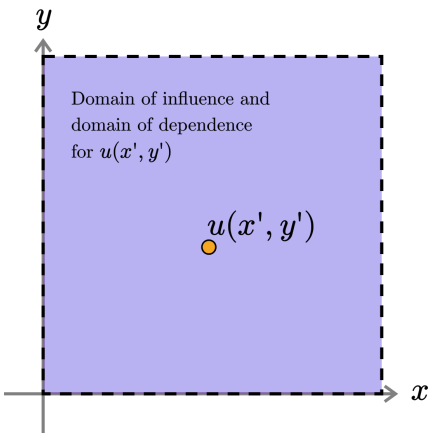


Figure 41: Domain of influence and domain of dependence for the solution at $u(x', y')$, given an elliptic PDE.

- Here the solution at (x', y') can depend on the solution at all other points (x, y), and likewise, the solution at (x', y') can impact the solution at all other points (x, y)

- The solution point $u(x', y')$ depends on boundary conditions on a closed contour bounding the domain, e.g.

$$
\begin{aligned}
u(x = x_{\min}, y) &= f(y) \\
u(x = x_{\max}, y) &= g(y) \\
u(x, y = y_{\min}) &= h(x) \\
u(x, y = y_{\max}) &= k(x)
\end{aligned}
$$

Discretised partial derivatives

- Before we look at some different numerical schemes to solve PDEs, let's list some useful expressions for discretised partial derivatives
- When discussing PDEs we will generally use Δx, Δy and Δt for the step sizes
- Notation:

$$
\begin{aligned}
u(x_i, y_j) &\equiv u_{ij} \\
u(x_i, y_j, t_n) &\equiv u_{ij}^n \\
\left.\frac{\partial^2 u}{\partial x^2}\right|_{(x_i, y_j)} &\equiv \left(\frac{\partial^2 u}{\partial x^2}\right)_{ij}
\end{aligned}
$$

- In the expressions below we will just use space coordinates, to keep the notation as simple as possible
- **First partial derivatives**:

$$
\begin{aligned}
\left(\frac{\partial u}{\partial x}\right)_{ij} &= \frac{u_{i+1,j} - u_{i,j}}{\Delta x} + \mathcal{O}\left(\Delta x\right), \quad \text{(forward difference)} \\
\left(\frac{\partial u}{\partial x}\right)_{ij} &= \frac{u_{i,j} - u_{i-1,j}}{\Delta x} + \mathcal{O}\left(\Delta x\right), \quad \text{(backward difference)} \\
\left(\frac{\partial u}{\partial x}\right)_{ij} &= \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x} + \mathcal{O}\left((\Delta x)^2\right), \quad \text{(central difference)}
\end{aligned}
$$

  - Similar expressions for $\frac{\partial u}{\partial y}$, where instead the index i is kept unchanged

- **Second partial derivatives**:

$$
\left(\frac{\partial^2 u}{\partial x^2}\right)_{ij} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{(\Delta x)^2} + \mathcal{O}\left((\Delta x)^2\right)
$$

  - …and a similar expression for $\frac{\partial^2 u}{\partial y^2}$
  - An expression for the mixed partial second derivative can be found by applying the central difference expression twice:

$$\begin{aligned}\left(\frac{\partial^2 u}{\partial x \partial y}\right)_{ij} &= \left(\frac{\partial}{\partial x}\frac{\partial u}{\partial y}\right)_{ij} \\ &= \frac{\left(\frac{\partial u}{\partial y}\right)_{i+1,j} - \left(\frac{\partial u}{\partial y}\right)_{i-1,j}}{2\Delta x} + \mathcal{O}\left((\Delta x)^2\right) \\ &= \frac{u_{i+1,j+1} - u_{i+1,j-1} - u_{i-1,j+1} + u_{i-1,j-1}}{4\Delta x \Delta y} + \mathcal{O}\left((\Delta x)^2, (\Delta y)^2\right)\end{aligned}$$

Some schemes for PDEs

- Out of the many, many numerical schemes for solving different types of PDEs, we will consider three schemes:
  - Forward Time Central Space (FTCS)
  - Backward Time Central Space (BTCS)
  - Crank-Nicolson (C-N)
- These are commonly used to solve parabolic PDEs (like the heat equation), but also have other applications
  - For instance, C-N is often used to solve the Schrödinger equation, as we will do
- They are all "time-steppers", i.e. methods that assume that the solution has a direction of evolution
  - You would typically not use these for a static, boundary-value problem

The forward difference scheme, a.k.a. the explicit scheme

- Also known as **Forward Time Central Space** (FTCS)
- To illustrate this scheme let's consider the diffusion equation in 1+1 dimensions

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2}$$

- For simplicity we will set D = 1
- Remember our notation: subscript for the space index and superscript for the time index, $u(x_i, t_n) \equiv u_i^n$
- As the FTCS name suggests, this scheme uses the forward difference expression for $\frac{\partial u}{\partial t}$
- So after discretisation and approximation, the diffusion equation takes the form

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}$$

- Collect the terms for the next time step $(n+1)$ on the left-hand side
- Define notation: $\alpha \equiv \frac{\Delta t}{(\Delta x)^2}$

$$\boxed{u_i^{n+1} = (1-2\alpha)u_i^n + \alpha\Big[u_{i+1}^n + u_{i-1}^n\Big]} \qquad (\star)$$

  or, in a form more useful to see the matrix structure:

$$\boxed{u_i^{n+1} = \alpha u_{i-1}^n + (1-2\alpha)u_i^n + \alpha u_{i+1}^n}$$

- This scheme is **explicit**: We can obtain u_i^{n+1} (the next time step at position x_i) without having to solve a system of equations.
  - All we need to do is evaluate the expression on the right-hand side of Eq. $(\star)$, using the solution at the current time step (n), which we know
- The *computational stencil* (a.k.a. the *calculational molecule*):

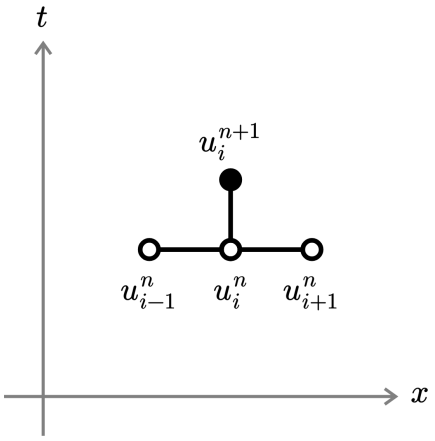


Figure 42: The computational stencil for the forward difference scheme in 1+1 dimensions

- If we use vector notation to collect the solutions at different x points, $\vec{u}^n \equiv [u_1^n, u_2^n, \ldots]$, we can write Eq. ($\star$) on matrix form:

$$\boxed{\begin{aligned}\vec{u}^{n+1} &= \mathbf{A}\vec{u}^n \\ &= (\mathbf{I} - \alpha\mathbf{B})\vec{u}^n\end{aligned}}$$

  where $\mathbf{B}$ is a simple tridiagonal matrix with $(-1, 2, -1)$ form, or in other words, $\mathbf{A}$ is a tridiagonal matrix with the signature $(\alpha, 1-2\alpha, \alpha)$

- That this is an explicit scheme is here expressed by the fact that to get $\vec{u}^{n+1}$ we only need to perform a matrix multiplication — no need to solve a matrix equation
  - (But here there is also no need to perform the full matrix multiplication, since $\mathbf{A}$ is a tridiagonal matrix, so the matrix multiplication would involve a bunch of unnecessary 0-multiplications. In this case it would be more efficient to just work on component form, rather than on matrix form.)
- The forward difference scheme is only **conditionally stable**
- We will later discuss stability analysis for PDEs
- Then we will see that the forward difference scheme is only a stable scheme for solving the diffusion equation if the following **criterion for stability** is satisfied:

$$\boxed{\alpha = \frac{\Delta t}{(\Delta x)^2} \leq \frac{1}{2}}$$

- Or, with $D \neq 1$ in the diffusion equation, the criterion becomes

$$\boxed{\alpha = \frac{D\Delta t}{(\Delta x)^2} \leq \frac{1}{2}}$$

- So if we want to use this scheme to solve the diffusion equation with high spatial resolution (small Δx), we need to use *very small* time steps:

$$\Delta t \leq \frac{(\Delta x)^2}{2D}$$

  - Some examples, assuming $D = 1$:

$$\Delta x = 0.1 \;\Rightarrow\; \Delta t \leq 0.005$$
$$\Delta x = 0.01 \;\Rightarrow\; \Delta t \leq 0.00005$$

- **Accuracy** of the forward difference scheme: global error from truncation is $\mathcal{O}\left((\Delta x)^2\right) + \mathcal{O}\left(\Delta t\right)$

The backward difference scheme, a.k.a. the implicit scheme

- Also known as **Backward Time Central Space** (BTCS)
- We are still considering the diffusion equation in 1+1 dimensions

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2}$$

  with $D = 1$
- We now use the backward difference expression for $\frac{\partial u}{\partial t}$
- Our diffusion equation then becomes

$$\frac{u_i^n - u_i^{n-1}}{\Delta t} = \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}$$

- Collect the terms for time step n on the left-hand side to get

$$\boxed{u_i^n(1+2\alpha) - \alpha\Big[u_{i+1}^n + u_{i-1}^n\Big] = u_i^{n-1}} \qquad (\star)$$

or, in a form more useful to see the matrix structure:

$$\boxed{-\alpha u_{i-1}^n + (1+2\alpha)u_i^n - \alpha u_{i+1}^n = u_i^{n-1}}$$

- As before we have used $\alpha = \frac{\Delta t}{(\Delta x)^2}$
- Here we have *three* unknown variables (u_{i-1}^n, u_i^n, u_{i+1}^n) and only one known variable (u_i^{n-1})
- We cannot solve this equation in isolation
  - We need the *full system* of equations to have N equations with N unknowns
- Thus, the backward difference scheme is an **implicit** scheme: Need to solve a system of equations to get the diff. eq. solution at the next time step
- To get the matrix form of this scheme, just write out the equations for
  - $i = 1$
  - $i = 2$
  - and so on …
- This gives a familiar tridiagonal system of equations:

$$\boxed{\mathbf{A}\vec{u}^n = \vec{u}^{n-1}}$$

with $\mathbf{A} = \text{tridiag}(-\alpha, 1+2\alpha, -\alpha)$, or

$$\boxed{(\mathbf{I} + \alpha\mathbf{B})\vec{u}^n = \vec{u}^{n-1}}$$

with $\mathbf{B} = \text{tridiag}(-1, 2, -1)$

- The computational stencil:

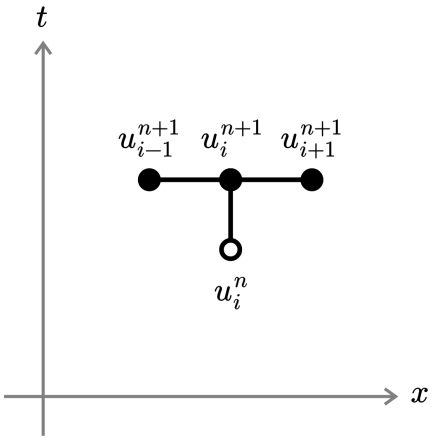


Figure 43: The computational stencil for the backward difference scheme in 1+1 dimensions

- To find the solution to our PDE we can then solve $\mathbf{A}\vec{u}^n = \vec{u}^{n-1}$ for $vecu^n$ at every time step, using e.g. an algorithm for solving tridiagonal matrix equations (Project 1)
- **Stability**: The backward difference scheme is an **unconditionally stable** scheme for solving our PDE
  - That is, there is no requirement on the choice of Δt and Δx that must be satisfied for the scheme to be stable
- **Accuracy**: The global error from truncation is $\mathcal{O}\left((\Delta x)^2\right) + \mathcal{O}\left(\Delta t\right)$ (same as the forward difference scheme)

The Crank-Nicolson scheme

Introduction: Can we find a scheme that is second-order in both time and space?

- We would like a scheme with accuracy $\mathcal{O}\left((\Delta x)^2\right) + \mathcal{O}\left((\Delta t)^2\right)$
- **Richardson scheme** (1920): Use the central difference expression for $\frac{\partial u}{\partial t}$,

$$\left(\frac{\partial u}{\partial x}\right)_i^n = \frac{u_i^{n+1} - u_i^{n-1}}{2\Delta t} + \mathcal{O}\left((\Delta t)^2\right)$$

- Using this in our diffusion equation we get

$$u_i^{n+1} = u_i^{n-1} + 2\alpha\left[u_{i+1}^n - 2u_i^n + u_{i-1}^n\right]$$

- This scheme has $\mathcal{O}\left((\Delta x)^2\right)$ accuracy
- …but it is **unconditionally unstable**
- (Also, it involves three time steps: $n-1, n, n+1$, so it will not be self-starting)
- **Dufort-Frankel scheme** (1953): Makes the Richardson scheme stable by using the replacement

$$u_i^n \to \frac{1}{2}\left[u_i^{n+1} + u_i^{n-1}\right]$$

- This gives the following scheme to solve our diffusion equation:

$$u_i^{n+1} = \frac{1}{1+2\alpha}\left[(1-2\alpha)u_i^{n-1} + 2\alpha\left[u_{i+1}^n + u_{i-1}^n\right]\right]$$

- This scheme is
  - explicit
  - stable
  - second-order accurate in both time and space
- But it also has some severe drawbacks:
  - It "leapfrogs" in time, since u_i^n is absent
  - It involves three time steps, so it is not self-starting
  - It **needs** $\Delta t \propto (\Delta x)^2$ to ensure **consistency**
    - (*Reminder*: Consistency is the requirement that the discretised equation recovers the original differential equation in the limit $\Delta t \to 0$ and $\Delta x \to 0$)

Crank-Nicolson

- Let's use a shorthand notation for our diffusion equation:

$$\frac{\partial u}{\partial t} = F(x,t)$$

- Now consider different discretised versions based on different choices for the discretised time derivative:
  - Forward difference:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = F_i^n$$

  - Backward difference, written with time steps n and $(n+1)$:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = F_i^{n+1}$$

    (note the $(n+1)$ on the right-hand side)

- By introducing a parameter $\theta \in [0, 1]$ we can now write down a linear combination of these two equations:

$$\boxed{\frac{u_i^{n+1} - u_i^n}{\Delta t} = \theta F_i^{n+1} + (1-\theta) F_i^n}$$

- This is known as *the θ rule* or *the θ scheme*
- If we set $\theta = 0$ we get the forward difference scheme, if we take $\theta = 1$ we get the backward difference scheme
- If we take $\theta = \frac{1}{2}$ we get the (original) **Crank-Nicolson scheme**, due to John Crank and Phyllis Nicolson (1947):

$$\boxed{\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{1}{2}\left[F_i^{n+1} + F_i^n\right]} \qquad (\star)$$

Explanation of the second-order accuracy in time

- Now, you might wonder how this scheme can give us $\mathcal{O}\left((\Delta t)^2\right)$ accuracy in time — after all, the left-hand side of Eq. $(\star)$ looks just like the forward difference expression for the time derivative?
  - The answer lies in how the right-hand side of our differential equation is evaluated at two time steps
  - The symmetric combination of F_i^n and F_i^{n+1} corresponds to using the trapezoidal integration rule when evolving our solution across one time step, $u_i^n \to u_i^{n+1}$
  - Let's see how this works. Assuming that we know $u(x_i, t_n)$ exactly, we can write down an exact expression for $u(x_i, t_{n+1})$:

$$u(x_i, t_{n+1}) = u(x_i, t_n) + \int_{t_n}^{t_{n+1}} \left.\frac{du}{dt}\right|_{x_i,t} dt$$

- Since u is just a function of the two independent coordinates x and t, we have

$$\frac{du}{dt} = \frac{\partial u}{\partial t} + \frac{\partial u}{\partial x}\frac{\partial x}{\partial t} = \frac{\partial u}{\partial t}$$

- And from the diffusion equation we are trying to solve, we know that

$$\left.\frac{du}{dt}\right|_{x_i,t} = F(x_i, t)$$

- So we can express $u(x_i, t_{n+1})$ exactly as

$$u(x_i, t_{n+1}) = u(x_i, t_n) + \int_{t_n}^{t_{n+1}} F(x_i, t)dt$$

- Let us next express the integral as [trapezoidal approximation] + [error term]. The trapezoidal rule for integration across a single step of size Δt has a $\mathcal{O}\left((\Delta t)^3\right)$ error, so we get

$$u(x_i, t_{n+1}) = u(x_i, t_n) + \Delta t\frac{1}{2}\Big[F(x_i, t_n) + F(x_i, t_{n+1})\Big] + \mathcal{O}\left((\Delta t)^3\right)$$

- We see that this *exact* expression for $u(x_i, t_{n+1}) = u_i^{n+1}$ is on the form

$$u(x_i, t_{n+1}) = \text{[Crank-Nicolson approximation]} + \mathcal{O}\left((\Delta t)^3\right)$$

- So the Crank-Nicolson scheme has $\mathcal{O}\left((\Delta t)^3\right)$ *local* error in time, leading to a $\mathcal{O}\left((\Delta t)^2\right)$ *global* error in time

Application to the diffusion equation in 1+1 dimensions

- Let us now apply the Crank-Nicolson scheme to our diffusion equation (still with $D = 1$):

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$$

- We then get

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{1}{2}\left[\frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2} + \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}\right]$$

- Collect terms for time step $(n+1)$ on the left and terms for time step n on the right, and use our familiar notation $\alpha = \frac{\Delta t}{(\Delta x)^2}$
- Then we get

$$\boxed{-\alpha u_{i-1}^{n+1} + (2+2\alpha)u_i^{n+1} - \alpha u_{i+1}^{n+1} = \alpha u_{i-1}^n + (2-2\alpha)u_i^n + \alpha u_{i+1}^n}$$

- In Project 5 you will derive a similar type of expression for the case of the Schrödinger equation in 2+1 dimensions
- Here we are only working with 1+1 dimensions, so nothing fancy is needed to put this into matrix form – just start writing out the equations for $i = 1$, $i = 2$, …, to get

$$\boxed{\mathbf{A}\vec{u}^{n+1} = \mathbf{B}\vec{u}^n}$$

where

$$\mathbf{A} = 2\mathbf{I} + \alpha\mathbf{T}$$
$$\mathbf{B} = 2\mathbf{I} - \alpha\mathbf{T}$$

with $\mathbf{T} = \text{tridiag}(-1, 2, -1)$

- So, at each time step n we can find the next $\vec{u}^{n+1}$ using a two-step procedure:
  1. Multiply: $\mathbf{B}\vec{u}^n \equiv \vec{b}$
  2. Solve $\mathbf{A}\vec{u}^{n+1} = \vec{b}$ for $\vec{u}^{n+1}$
- So the Crank-Nicolson scheme is clearly an **implicit** scheme
- **Accuracy**: $\mathcal{O}\left((\Delta x)^2\right) + \mathcal{O}\left((\Delta t)^2\right)$
- **Stability**: Stable for all choices of Δt and Δx
- Also, the scheme only involves two time steps (compared to three time steps in the Richardson and Dufort-Frankel schemes)
- The computational stencil in 1+1 dimensions:

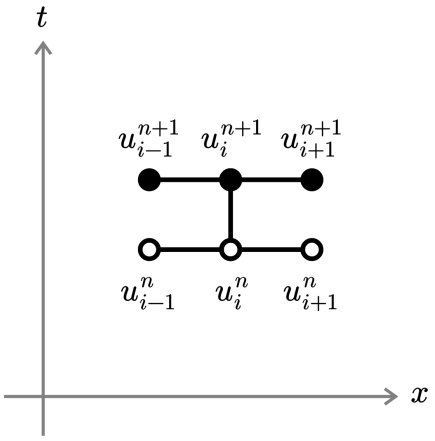


Figure 44: The computational stencil for the Crank-Nicolson scheme in 1+1 dimensions

Crank-Nicolson matrix structure in 2+1 dimensions

- Consider again the case of 1+1 dimensions
- At a given time step, the solution across all of space fits naturally in a vector representation $\vec{u}$, since points only have neighbours in space along one dimension
- This gives rise to a simple tridiagonal structure for the matrices $\mathbf{A}$ and $\mathbf{B}$
  - Another way to see this is to note that, in the 1+1 dim. case, putting the solutions at each time step into vectors does not pull apart neighbouring points in the computational stencil:

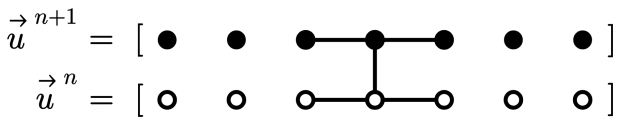


Figure 45: In 1+1 dimensions, the entire solution at each time step can neatly be represented as a vector, without breaking apart any neighbouring pairs in the computational stencil

- Now let us go to the 2+1 dimensional case (as in Project 5)
- The computational stencil in 2+1 dimensions consists of two crosses in the xy plane, connected in the time direction

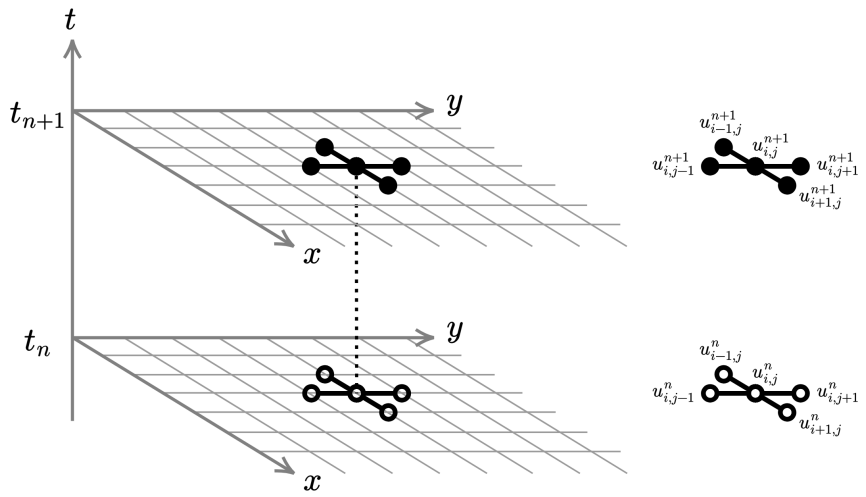


Figure 46: The computational stencil for the Crank-Nicolson scheme in 2+1 dimensions

- We still want to express the scheme using the general matrix-vector formulation as before:

$$\boxed{\mathbf{A}\vec{u}^{n+1} = \mathbf{B}\vec{u}^{n}}$$

- But now we will have to think more carefully about:
  - How we put the solution at each time step n into a vector $\vec{u}^n$
  - What structure the matrices $\mathbf{A}$ and $\mathbf{B}$ will have
- Consider the solution at time step n: We can always go from a 2D representation (u^n_{ij}) to a 1D representation (u^n_k) — all we need is a rule that maps each index pair (i, j) to a single, unique index k
- But any way we do this, we will necessarily break apart some neighbouring pairs when going to the 1D representation
- To illustrate this, consider the example of mapping a matrix to a vector by going through the matrix one column at a time, and just put each column below the next to get a vector

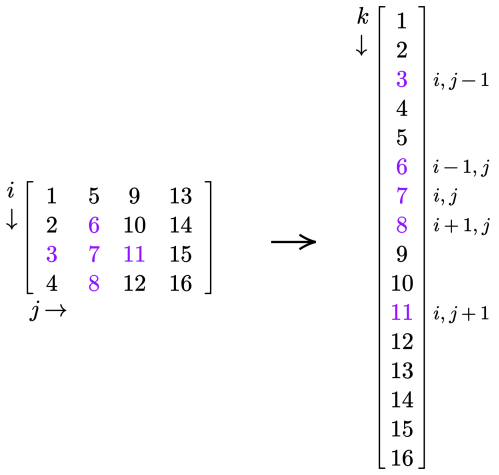


Figure 47: Example of how going from a 2D representation (with two indices i and j) to a 1D representation (with index k) breaks apart pairs that were neighbours in the 2D representation

- Notice how row-neighbours in the matrix then end up far apart in the resulting vector
- Thus, points connected by our computational stencil end up far apart in our vector representation
- As a consequence, the matrices $\mathbf{A}$ and $\mathbf{B}$ will no longer have the simple tridiagonal structure they had in the case of 1+1 dimensions
- Here is an illustration of the pattern of non-zero elements in the matrices $\mathbf{A}$ and $\mathbf{B}$:

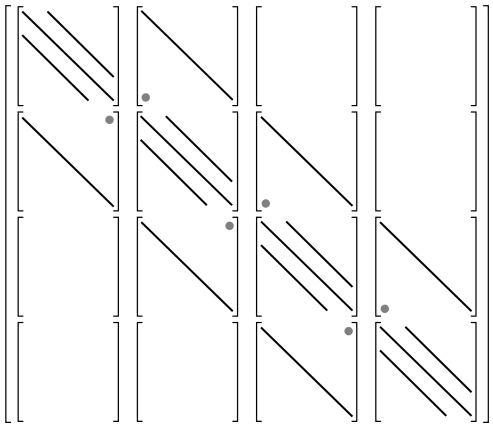


Figure 48: Example of the structure of non-zero elements in the matrices $\mathbf{A}$ and $\mathbf{B}$ in the Crank-Nicolson scheme for 2+1 dimensions

- The diagonal lines and grey dots represent the (possibly) non-zero elements, all other elements are zero
  - The elements at the grey dots depend on the boundary conditions. If e.g. using Dirichlet

boundary conditions, these elements will be zero

- Notice how the matrix structure can be seen as consisting of multiple sub-matrices
- When working with two space dimensions, our matrices and vectors quickly become very large
  - Say we use M grid points in each space direction, including the boundary points
  - This gives $(M-2)$ internal points in each space direction
  - That means our xy plane has $(M-2)^2$ internal grid points, so our vector $\vec{u}^n$ will contain $(M-2)^2$ elements
  - And it means that the matrices $\mathbf{A}$ and $\mathbf{B}$ will be of size $(M-2)^2 \times (M-2)^2$, so contain $(M-2)^4$ elements!
    * For the illustration above we assumed $(M-2) = 4$, leading to a matrix structure with a 4×4 set of submatrices, where each submatrix has size 4×4
- We also note that $\mathbf{A}$ and $\mathbf{B}$ mostly consist of zeros — they are very sparse matrices
  - So when solving the matrix equation at each time step, we may benefit from using a method that can make use of this sparsity to speed up the computation
- If we extend the Crank-Nicolson scheme to more space dimensions, we would again get larger vectors and larger matrices, now with even more off-diagonal bands of non-zero elements.